Algorithms for VLSI Design Automation

Sabih H. Gerez
University of Twente, Department of Electrical Engineering, The Netherlands

JOHN WILEY & SONS
Chichester · New York · Weinheim · Brisbane · Singapore · Toronto

Copyright © 1999 by John Wiley & Sons Ltd,
Baffins Lane, Chichester,
West Sussex PO19 1UD, England

National 01243 779777
International (+44) 1243 779777

e-mail (for orders and customer service enquiries): cs-books@wiley.co.uk

Visit our Home Page On http://www.wiley.co.uk
or
http://www.wiley.com

Reprinted June 2000

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except under the terms of the Copyright, Designs and Patents Act 1988 or under the terms of a licence issued by the Copyright Licensing Agency, 90 Tottenham Court Road, London, W1P 9HE, UK, without the permission in writing of the publisher.

Other Wiley Editorial Offices

John Wiley & Sons, Inc., 605 Third Avenue,
New York, NY 10158-0012, USA

Wiley-VCH Verlag GmbH, Pappelallee 3,
D-69469 Weinheim, Germany

Jacaranda Wiley Ltd, 33 Park Road, Milton,
Queensland 4064, Australia

John Wiley & Sons (Asia) Pte Ltd, 2 Clementi Loop #02-01,
Jin Xing Distripark, Singapore 129 809

John Wiley & Sons (Canada) Ltd, 22 Worcester Road,
Rexdale, Ontario M9W 1L1, Canada

Library of Congress Cataloging-in-Publication Data

Gerez, Sabih H.
Algorithms for VLSI design Automation / Sabih H. Gerez – Draft ed.
p. cm.
Includes bibliographical references and index.
ISBN 0-471-98489-2
1. Integrated Circuits –Very large scale integration – Design and construction – Data processing. 2. Computer-aided design. 3. Algorithms. I. Title.
TK7874.75.G47 1998
621.39′5 — dc21 98-39574
CIP

British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library.

ISBN 0 471 98489 2

Produced from PostScript files supplied by the author.
Printed and bound in Great Britain by Bookcraft (Bath) Ltd.
This book is printed on acid-free paper responsibly manufactured from sustainable forestry, in which at least two trees are planted for each one used in paper production.

Contents

I Preliminaries 1

1 Introduction to Design Methodologies . . . 3
1.1 The VLSI Design Problem . . . 3
1.2 The Design Domains . . . 5
1.3 Design Actions . . . 7
1.4 Design Methods and Technologies . . . 8
1.5 Bibliographic Notes . . . 9

2 A Quick Tour of VLSI Design Automation Tools . . . 11
2.1 Algorithmic and System Design . . . 11
2.2 Structural and Logic Design . . . 13
2.3 Transistor-level Design . . . 15
2.4 Layout Design . . . 15
2.5 Verification Methods . . . 17
2.6 Design Management Tools . . . 18
2.7 Bibliographic Notes . . . 19

3 Algorithmic Graph Theory and Computational Complexity . . . 21
3.1 Terminology . . . 22
3.2 Data Structures for the Representation of Graphs . . . 24
3.3 Computational Complexity . . . 26
3.4 Examples of Graph Algorithms . . . 29
3.4.1 Depth-first Search . . . 30
3.4.2 Breadth-first Search . . . 32
3.4.3 Dijkstra's Shortest-path Algorithm . . . 34
3.4.4 Prim's Algorithm for Minimum Spanning Trees . . . 37
3.5 Bibliographic Notes . . . 39
3.6 Exercises . . . 40

4 Tractable and Intractable Problems . . . 41

4.1 Combinatorial Optimization Problems 41
4.2 Decision Problems 43
4.3 Complexity Classes 45
4.4 NP-completeness and NP-hardness 46
4.5 Consequences 49
4.6 Bibliographic Notes 50
4.7 Exercises 51

5 General-purpose Methods for Combinatorial Optimization 53
5.1 The Unit-size Placement Problem 54
5.2 Backtracking and Branch-and-bound 55
5.2.1 Backtracking 56
5.2.2 Branch-and-bound 59
5.3 Dynamic Programming 62
5.4 Integer Linear Programming 65
5.4.1 Linear Programming 65
5.4.2 Integer Linear Programming 67
5.5 Local Search 69
5.6 Simulated Annealing 71
5.7 Tabu Search 73
5.8 Genetic Algorithms 75
5.9 A Few Final Remarks on General-purpose Methods 78
5.10 Bibliographic Notes 79

II Selected Design Problems and Algorithms 81

6 Layout Compaction 83
6.1 Design Rules 83
6.2 Symbolic Layout 85
6.3 Problem Formulation 86
6.3.1 Applications of Compaction 86
6.3.2 Informal Problem Formulation 86
6.3.3 Graph-theoretical Formulation 87
6.3.4 Maximum-distance Constraints 90
6.4 Algorithms for Constraint-graph Compaction 91
6.4.1 A Longest-path Algorithm for DAGs 91
6.4.2 The Longest Path in Graphs with Cycles 92
6.4.3 The Liao-Wong Algorithm 93
6.4.4 The Bellman-Ford Algorithm 95
6.4.5 Discussion: Shortest Paths, Longest Paths and Time Complexity 96
6.5 Other Issues 97
6.6 Bibliographic Notes 99

6.7 Exercises 99

7 Placement and Partitioning 101
7.1 Circuit Representation 102
7.2 Wire-length Estimation 105
7.3 Types of Placement Problem 106
7.4 Placement Algorithms 108
7.4.1 Constructive Placement 108
7.4.2 Iterative Improvement 109
7.5 Partitioning 112
7.5.1 The Kernighan-Lin Partitioning Algorithm 112
7.6 Bibliographic Notes 117
7.7 Exercises 118

8 Floorplanning 119
8.1 Floorplanning Concepts 121
8.1.1 Terminology and Floorplan Representation 121
8.1.2 Optimization Problems in Floorplanning 124
8.2 Shape Functions and Floorplan Sizing 125
8.3 Bibliographic Notes 130
8.4 Exercises 131

9 Routing 133
9.1 Types of Local Routing Problems 133
9.2 Area Routing 134
9.3 Channel Routing 138
9.3.1 Channel Routing Models 139
9.3.2 The Vertical Constraint Graph 140
9.3.3 Horizontal Constraints and the Left-edge Algorithm 143
9.3.4 Channel Routing Algorithms 146
9.4 Introduction to Global Routing 150
9.4.1 Standard-cell Layout 151
9.4.2 Building-block Layout and Channel Ordering 153
9.5 Algorithms for Global Routing 154
9.5.1 Problem Definition and Discussion 155
9.5.2 Efficient Rectilinear Steiner-tree Construction 157
9.5.3 Local Transformations for Global Routing 163
9.6 Bibliographic Notes 163
9.7 Exercises 166

10 Simulation 167
10.1 General Remarks on VLSI Simulation 167
10.2 Gate-level Modeling and Simulation 169
10.2.1 Signal Modeling 170

10.2.2 Gate Modeling . . . 171
10.2.3 Delay Modeling . . . 171
10.2.4 Connectivity Modeling . . . 172
10.2.5 Compiler-driven Simulation . . . 173
10.2.6 Event-driven Simulation . . . 176
10.3 Switch-level Modeling and Simulation . . . 180
10.3.1 Connectivity and Signal Modeling . . . 181
10.3.2 Simulation Mechanisms . . . 183
10.4 Bibliographic Notes . . . 191
10.5 Exercises . . . 192

11 Logic Synthesis and Verification . . . 195
11.1 Introduction to Combinational Logic Synthesis . . . 195
11.1.1 Basic Issues and Terminology . . . 195
11.1.2 A Practical Example . . . 199
11.2 Binary-decision Diagrams . . . 201
11.2.1 ROBDD Principles . . . 201
11.2.2 ROBDD Implementation and Construction . . . 206
11.2.3 ROBDD Manipulation . . . 208
11.2.4 Variable Ordering . . . 215
11.2.5 Applications to Verification . . . 217
11.2.6 Applications to Combinatorial Optimization . . . 219
11.3 Two-level Logic Synthesis . . . 222
11.3.1 Problem Definition and Analysis . . . 222
11.3.2 A Heuristic Based on ROBDDs . . . 225
11.4 Bibliographic Notes . . . 230
11.5 Exercises . . . 232

12 High-level Synthesis . . . 235
12.1 Hardware Models for High-level Synthesis . . . 235
12.1.1 Hardware for Computations, Data Storage, and Interconnection 236
12.1.2 Data, Control, and Clocks . . . 238
12.2 Internal Representation of the Input Algorithm . . . 239
12.2.1 Simple Data Flow . . . 239
12.2.2 Conditional Data Flow . . . 241
12.2.3 Iterative Data Flow . . . 243
12.2.4 Data-flow Graph Representation . . . 245
12.3 Allocation, Assignment and Scheduling . . . 247
12.3.1 Goals and Terminology . . . 247
12.3.2 A Detailed Example . . . 248
12.3.3 Optimization Issues . . . 251
12.4 Some Scheduling Algorithms . . . 253
12.4.1 ASAP Scheduling . . . 253
12.4.2 Mobility-based Scheduling . . . 254

12.4.3 Force-directed Scheduling 256
12.4.4 List Scheduling 259
12.5 Some Aspects of the Assignment Problem 261
12.5.1 Optimization Issues 261
12.5.2 Graph Theoretical Problem Formulation 262
12.5.3 Assignment by Interval and Circular-arc Graph Coloring . . 264
12.5.4 Assignment by Clique Partitioning 265
12.6 High-level Transformations 266
12.7 Bibliographic Notes 271
12.8 Exercises 273

III Appendices **275**

Appendix A CMOS Technology 277
A.1 The MOS Transistor and CMOS Logic Design 278
A.2 Transistor Layout in CMOS and Related Issues 282
A.3 Bibliographic Notes 285

Appendix B About the Pseudo-code Notation 287
B.1 Data Structures and Declarations 288
B.2 C-language Constructs 289
B.3 Pseudo-code Constructs 292
B.4 Bibliographic Notes 294

Appendix C List of Acronyms 295

References 297

Index 315

3

Algorithmic Graph Theory and Computational Complexity

A *graph* is a mathematical structure that describes a set of objects and the connections between them. The use of graphs may facilitate the mathematical formulation, analysis, and solution of a problem. A road map can, for example, be represented by a graph: points in a plane represent cities and line segments between these points show those cities connected by a direct road. Graphs are often encountered in the field of design automation for integrated circuits, both when dealing with entities that naturally look like a network (e.g. a circuit of transistors) as well as in more abstract cases (e.g. precedence relations in the computations of some algorithm, see Chapter 12).

Algorithmic graph theory, as opposed to *pure* graph theory, emphasizes the design of algorithms that operate on graphs, instead of concentrating on mathematical properties of graphs and theorems expressing those properties. The distinction between the two is not very sharp, however, and algorithmic graph theory certainly benefits from results in pure graph theory.

Computational complexity refers to the time and memory required by a certain algorithm as function of the size of the algorithm's input. The concept applies to algorithms in general and is not restricted to graph algorithms.

The theory that is presented in this chapter consists of the minimum knowledge that will be of help to understand the applications of graph theory to specific problems in the field of design automation for VLSI. After studying this chapter, the reader is supposed to be able to understand the description of graph algorithms, analyze their computational complexity, slightly modify them, and write simple programs that implement graph algorithms (if already sufficiently experienced in programming). The chapter starts with the terminology of graph theory, followed by a discussion of elementary data structures for the representation of graphs. An introduction to the theory of computational complexity follows next. The chapter concludes with a small selection of graph algorithms.

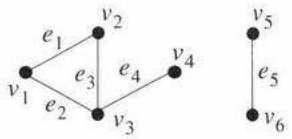


Figure 3.1 An example of a graph.

3.1 Terminology

A graph is characterized by two sets: a *vertex* set V and an *edge* set E. It is customary to denote a graph by $G(V, E)$, where G refers to the graph itself. Not everybody uses the same terms when discussing graphs: a vertex is also called a *node* and an edge is called an *arc* or a *branch.*

The sets V and E fully characterize a graph. Figure 3.1 shows a graph where $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ and $E = \{e_1, e_2, e_3, e_4, e_5\}$. The two vertices that are joined by an edge are called the edge's *endpoints.* An edge can be identified by its two endpoints $u, v \in V$, in which case the notation (u, v) is used. In the example of Figure 3.1, $e_1 = (v_1, v_2)$, $e_2 = (v_1, v_3)$, etc. When $e = (u, v)$, it is said that e is *incident with* u (and also with v). The vertices u and v such that $(u, v) \in E$, are called *adjacent.*

When one removes vertices and/or edges from a given graph G, one gets a *subgraph* of G. In this process, the following rule should be obeyed: removing a vertex implies the removal of all edges connected to it. More formally: given a graph $G(V, E)$, a subgraph induced by a set $V_s \subset V$ is the graph $G_s(V_s, E_s)$, with $E_s = \{(v_i, v_j) | v_i \in V_s \wedge v_j \in V_s \wedge (v_i, v_j) \in E\}$.

A graph $K(V_k, E_k)$ for which the edge $(v_i, v_j) \in E_k$ for every pair of vertices $v_i, v_j \in V_k$ is called a *complete* graph. A subgraph that is complete, and that is not contained in a larger complete subgraph, is called a *clique.* The graph of Figure 3.1 has three cliques identified by the vertex sets $\{v_1, v_2, v_3\}$, $\{v_3, v_4\}$ and $\{v_5, v_6\}$. Some authors call *any* complete subgraph a clique and one that is not contained in a larger one a *maximal clique.*

The *degree* of a vertex is equal to the number of edges incident with it. An edge (u, u), i.e. one starting and finishing at the same vertex, is called a *selfloop.* Two edges of the form $e_1 = (v_1, v_2)$ and $e_2 = (v_1, v_2)$, i.e. having the same endpoints, are called *parallel edges.* A graph without selfloops or parallel edges is called a *simple* graph. In most contexts, only simple graphs are discussed and therefore the term *graph* is used instead of simple graph. A graph without selfloops but with parallel edges is called a *multigraph.*

If the vertex set V of a graph is the union of two disjoint sets V_1 and V_2 and all edges of this graph exclusively connect a vertex from V_1 with a vertex V_2, the graph is called *bipartite.* Such a graph is often denoted by $G(V_1, V_2, E)$. An example is given in Figure 3.2, where the two types of vertices have been colored black and white respectively.

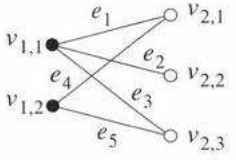


Figure 3.2 An example of a bipartite graph.

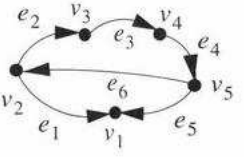


Figure 3.3 An example of a directed graph.

A graph that can be drawn on a two-dimensional plane without any of its edges intersecting is called *planar*. Otherwise, the graph is called *nonplanar.*

A sequence of alternating vertices and edges, starting and finishing with a vertex, such that an edge $e = (u, v)$ is preceded by u and followed by v in the sequence (or vice versa), is called a *path*. In Figure 3.1 v_1, e_1, v_2, e_3, v_3 is an example of a path. The *length* of a path equals the number of edges it contains. Note that a path can have a length of zero. A path, of which the first and last vertices are the same and the length is larger than zero, is called a *cycle* (sometimes also: *loop* or *circuit*). A path or a cycle not containing two or more occurrences of the same vertex is a *simple* path or cycle (except, of course, for the first and last vertices of a cycle). The adjective 'simple' is often omitted when all paths and cycles in some context are known to be simple.

Two vertices u and v are called *connected* if there is a path starting at u and finishing at v. In Figure 3.1 v_1 and v_4 are connected but v_1 and v_5 are not. If all pairs of vertices in a graph are connected, the graph is called a *connected graph.* Otherwise, the graph is *disconnected.* The graph of Figure 3.1 is disconnected. It consists of two *connected components:* a connected component is a subgraph induced by a maximal subset of the vertex set, such that all pairs in the set are connected (a "maximal subset" is the largest possible subset obeying some property).

Sometimes a *direction* is associated with the edges in a graph. Such a graph is called a *directed* graph or *digraph.* An example of a directed graph is shown in Figure 3.3. A graph whose edges do not have directions is consequently called an *undirected graph.* The terminology and notation presented above have to be extended for directed graphs. In the notation for an edge $e = (u, v)$, u and v cannot be freely interchanged anymore. The edge e is directed from u to v. It is said that e is *incident from* u and *incident to* v. The *in-degree* of a vertex is equal to the number of edges incident to it; the *out-degree* of an edge is equal to the number of edges incident from

it.

A *path* for a directed graph is defined in the same way as for an undirected graph: in the sequence of alternating vertices and edges, an edge does not need to be directed from the vertex preceding it to the vertex following it. However, if all the edges are directed in this way, the path is called a *directed path*. In the directed graph of Figure 3.3, v_2, e_1, v_1, e_5, v_5 is an example of a path and v_2, e_2, v_3, e_3, v_4 is an example of a directed path. In an analogous way, the terms *cycle* and *directed cycle* can be defined. In the graph of Figure 3.3, $v_2, e_1, v_1, e_5, v_5, e_6, v_2$ is an example of a cycle and $v_2, e_2, v_3, e_3, v_4, e_4, v_5, e_6, v_2$ is an example of a directed cycle. In many discussions related to directed graphs, often only directed paths and directed cycles are relevant and the adjective 'directed' is often omitted when no confusion is possible.

Two vertices u and v in a directed graph are called *strongly connected* if there is both a directed path from u to v and a directed path from v to u. The property of being 'strongly connected' partitions the directed graph into *strongly connected components*. In each such component, all possible pairs of vertices are strongly connected and no other vertex can be added to it that is strongly connected to all those already in the component. In the example of Figure 3.3, there are two strongly connected components: $\{v_1\}$ and $\{v_2, v_3, v_4, v_5\}$. If there is a path between two vertices u and v, the vertices are called *weakly connected*. The property of being 'weakly connected' analogously partitions the graph into *weakly connected components*. In Figure 3.3, there is only one weakly connected component consisting of the entire graph.

Depending on the problem that one wants to solve by means of graphs, a graph can be extended to carry more information. One of the most common extensions is to assign *weights* to the edges of a graph. For example, if a graph is used to represent a road map with the vertices representing cities and edges representing the roads between cities, an edge weight might represent the distance between the two cities connected by a road. A graph of this type is called an *edge-weighted* graph. In some applications, it might be necessary to associate a weight with a vertex; such a graph is called a *vertex-weighted* graph.

The terminology mentioned in this section is just a basic set of terms. More graph theoretical terms will be introduced in other chapters, in the context of specific problems from the field of VLSI design automation.

3.2 Data Structures for the Representation of Graphs

If one wants to implement graph algorithms, one of the first issues to be settled is how to represent the graph in a computer. In other words, one should choose a suitable *data structure* for graphs. There is no optimal data structure that should be used in all algorithms. Different algorithms have different requirements.

One of the most straightforward ways to represent graphs is by means of an *adjacency matrix*. If the graph $G(V, E)$ has n vertices, an $n \times n$ matrix A is used. $A_{ij} = 1$ if $(v_i, v_j) \in E$, and $A_{ij} = 0$ if $(v_i, v_j) \notin E$. As the edges do not have

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

(a) (b)

Figure 3.4 The adjacency matrices for the graphs of Figure 3.1 (a) and Figure 3.3 (b).

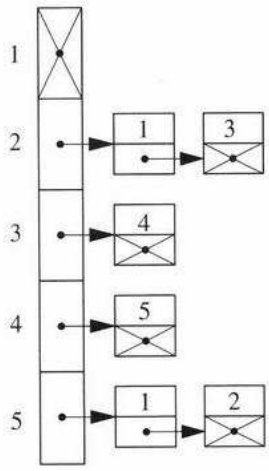


Figure 3.5 The the adjacency list representation of the graph given in Figure 3.3.

directions in undirected graphs, the adjacency matrices of such graphs are symmetric. This data structure is illustrated in Figure 3.4, where the adjacency matrices for the graphs of Figures 3.1 and 3.3 are given.

When using the adjacency matrix, testing whether two given vertices are connected or not can be performed in constant time, i.e. the time to perform the test does not depend on the size of the graph. However, finding all vertices connected to a given vertex requires inspection of a complete row and a complete column of the matrix. This is not very efficient when most of the entries in the matrix are equal to zero.

The *adjacency list* representation is a better choice in such a case. It consists of an array that has as many elements as the number of vertices in the graph. So, an array element identified by an index i corresponds with the vertex v_i. Each array element points to a *linked list* that contains the indices of all vertices to which the vertex corresponding to the element is connected. Figure 3.5 illustrates the adjacency list representation for the graph of Figure 3.3.

```
struct vertex {
  int vertex_index;
  struct edge *outgoing_edges;
};

struct edge {
  int edge_index;
  struct vertex *from, *to;
  struct edge *next;
};
```

Figure 3.6 The data structure for a graph representation with explicit vertex and edge structures.

If an algorithm operates on edges rather than vertices and needs to efficiently identify the vertices that an edge connects, a data structure that is built from explicit structures (records in the *Pascal* jargon), can be used for both vertices and edges. The description in C of this data structure is given in Figure 3.6. It is assumed that edges as well as vertices are identified by an integer number. This number can also be the index in an array pointing to all vertices or all edges. It is supposed that the graph is directed and that it is only required to traverse the graph according to the edge directions. Each vertex points to the list of edges incident from it by means of the member (field in Pascal) `outgoing_edges` in the `vertex` structure and the member `next` in the `edge` structure. Each edge points to the two vertices that it connects by means of the members `to` and `from`. This data structure is illustrated in Figure 3.7 for a small directed graph.

The choice of a suitable data structure in the implementation of a graph algorithm can be very important and may directly affect the computational effort necessary to solve some problem. This section has presented the most common data structues for graphs. Given the characteristics of the algorithm to be implemented, one should carefully consider which of the data structures is most appropriate and how these basic data structures should be modified and extended to satisfy the requirements of the algorithm.

3.3 Computational Complexity

A main concern for the design of an algorithm is how the algorithm will perform. How fast will it run when implemented on a computer? How much of the computer's memory will it claim? The theory of computational complexity tries to answer these questions without having to deal with specific hardware and also without providing absolute values in seconds and bytes. Instead, the behavior of an algorithm is characterized by mathematical functions of the algorithm's "input size".

The *input size* or *problem size* of an algorithm is related to the number of symbols necessary to describe the input. Suppose that a sorting algorithm has to sort a list of n words, each consisting of at most 10 letters, then the input size is bounded by $10n$.

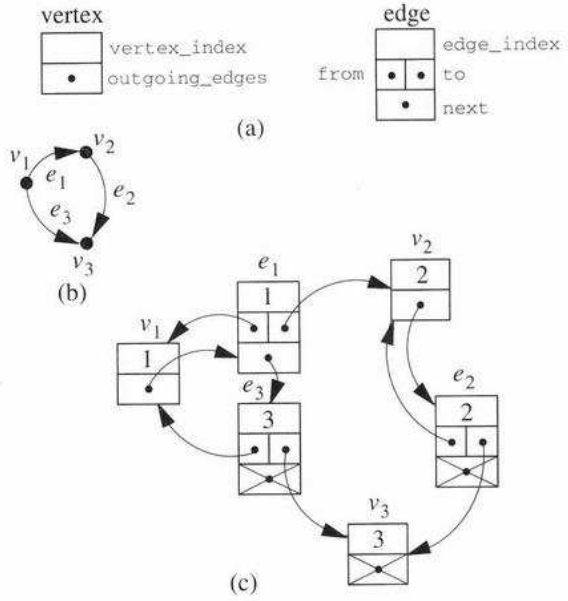


Figure 3.7 The visualization of the `vertex` and `edge` structures (a), a small directed graph (b) and its representation with a data structure built from those structures (c).

Constant factors are not relevant in this context (one wants to avoid technical details such as e.g. the number of bits to code a letter; using the ASCII code that uses 8 bits for each letter, one might as well claim that the input size of the problem is bounded by $80n$). Therefore, constants are eliminated: in the sorting problem the input size is then defined to be n.

A few more examples: an algorithm that takes as input a single natural number n of arbitrary size, has an input size of $\log n$, as the number of symbols in a *reasonable* encoding grows logarithmically with n (a reasonable encoding is e.g. binary or decimal notation; an unreasonable encoding is e.g. unary notation, that uses n bits to encode number n). The input size of a graph algorithm operating on a graph $G(V, E)$ is characterized by two parameters: the size of the vertex set $|V|$ and the size of the edge set $|E|$.

Two types of computational complexity are distinguished: *time complexity*, which is a measure for the time necessary to accomplish a computation, and *space complexity* which is a measure for the amount of memory required for a computation. Space complexity is often given less importance than time complexity (although an algorithm that is using more memory than what is available will not run at all). Because most of the statements made on time complexity in the rest of this text hold as well for space complexity, space complexity will not receive any special attention. In the rest of this text, sometimes "time complexity" will simply be called "complexity".

Before defining the time complexity of an algorithm, it is necessary to introduce

the notion of the *order* of a function. Given two functions f and g that map a natural number n to some positive value, it is said that $f(n) = \mathcal{O}(g(n))$ (pronounced as "$f(n)$ is *big O* of $g(n)$" or as "$f(n)$ is of order at most $g(n)$") if two constants n_0 and K can be found, such that the following proposition is true:

$$\forall n \geq n_0 : f(n) \leq K \cdot g(n)$$

Note that the equals sign in $f(n) = \mathcal{O}(g(n))$ actually indicates set membership, viz. of the set all functions that are $\mathcal{O}(g(n))$. The notation is maintained for historical reasons. Examples:

$$\begin{aligned} \tfrac{1}{3}n^2 &= \mathcal{O}(n^2) \\ 0.02n^2 + 127n + 1923 &= \mathcal{O}(n^2) \\ 3n \log n + n &= \mathcal{O}(n \log n) \\ 5n^3 &= \mathcal{O}(\tfrac{1}{5}n^3) \\ 5n^3 &= \mathcal{O}(n^4) \\ 5n^3 &= \mathcal{O}(n^3 + n^2) \end{aligned}$$

Although the last three equalities are correct, they are not used to describe computational complexity. One chooses the order expression to be as simple and as sharp as possible.

The big-O notation describes the upper bound of a function. If one wants to describe a lower bound, the *big-Omega* notation is used: $f(n) = \Omega(g(n))$ (pronunciation: "$f(n)$ is big Omega $g(n)$" or "$f(n)$ is of order at least $g(n)$") if $g(n) = \mathcal{O}(f(n))$. The fact that two functions have the same *rate of growth* is indicated by the *big-Theta* notation: $f(n) = \Theta(g(n))$ (pronunciation: "$f(n)$ is big Theta $g(n)$" or "$f(n)$ is of order exactly $g(n)$") if $f(n) = \mathcal{O}(g(n))$ and $g(n) = \mathcal{O}(f(n))$. For most algorithms, and especially those that are discussed in this text, the big-O notation is sufficient for the description of their computational complexities.

The duration of a computation is expressed in *elementary computational steps*. This is a simple computation that does not depend on the input size, e.g. the addition, multiplication or comparison of two 32 bit integers, or the access of a record when a pointer to it is available.

The time complexity of a computation is a function that gives the number of elementary computational steps executed for inputs of a specific size. Normally, it is not only the size of the input that determines the number of computational steps: conditional constructs in the algorithm are the reason that the time required by the algorithm is different for different inputs of the same size. Therefore one works with the *worst-case* time complexity, assuming that the condition that requires the largest number of computational steps will be true for an input of a given size.

Apart from the worst-case time complexity, there are other time complexity measures. For example, the *average-case* time complexity is the expected value of the computation time for a given distribution of the distinct inputs of the algorithm. Actually, average-case time complexity has a higher practical value than worst-case time complexity. Its analysis is, however, more complex. For this reason, discussions

on time complexity in the rest of this book will be limited to worst-case time complexity.

Only the order of an algorithm's time complexity is normally relevant. One e.g. says that an algorithm operates in $\mathcal{O}(n^2)$ time. Depending on the magnitude of the input size, a number of different criteria can be used for qualifying an algorithm:

1. *Polynomial* vs. *exponential* order. As an exponential function grows faster than any polynomial and the exponents of a polynomial tend to be small, an algorithm with a polynomial time complexity is to be preferred over an exponential algorithm. Actually, this distinction corresponds to the one between *tractable* and *intractable* problems that is the topic of Chapter 4.

2. *Linear* vs. *quadratic* order. Suppose that the input size of an algorithm is determined by the number of transistors in a circuit and that the algorithm has to be applied to VLSI circuit containing some 10^6 transistors. Then, running an algorithm with a linear time complexity is feasible on a computer with a realistic speed, but an algorithm with quadratic time complexity is not. Between these categories are the functions with complexities $\mathcal{O}(n \log n)$, $\mathcal{O}(n \log \log n)$, $\mathcal{O}(n \log^2 n)$, etc. As logarithmic functions grow very slowly, algorithms with this type of time complexities are also acceptable in the above context.

3. *Sublinear* order. When the input of an algorithm is structured in some way, an algorithm might find the solution to some problem without processing all input elements separately. For example, finding an element when the input is already sorted and is available in an array can be done by "binary search" and requires $\mathcal{O}(\log n)$ time. An extreme case is when an algorithm's computation is completely independent of the input size: one says that the algorithm operates in constant time and its complexity is written as $\mathcal{O}(1)$.

A final issue to be mentioned is how to compute an algorithm's time complexity. This can be done in different ways. Direct analysis can lead to an upper bound to the number of computational steps, or the time complexity of the whole can be derived from the complexities of its parts. For example, the time complexity of a loop construct is the time complexity of the loop's body multiplied by the number of times the loop is executed (or an upper bound for this number) and the time complexity of an "if statement" is the sum of the time complexities of its "then" and "else" parts. The rule for multiplication of $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2)$ is: $f_1 \times f_2 = \mathcal{O}(g_1 \times g_2)$. And for addition of $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2)$: $f_1 + f_2 = \mathcal{O}(g_1 + g_2)$, followed by a simplification if either of g_1 or g_2 is of a lower order than the other.

3.4 Examples of Graph Algorithms

In this section a number of relatively simple graph algorithms are discussed. The goal of their presentation is twofold: to become acquainted with graph algorithms as such and to apply the theory of computational complexity given above.

```
/* Given is the graph G(V, E) */

struct vertex {
  ...
  int mark;
};

dfs(struct vertex v)
{
  v.mark ← 0;
  "process v";
  for each (v, u) ∈ E {
    "process (v, u)";
    if (u.mark)
      dfs(u);
  }
}

main ()
{
  for each v ∈ V
    v.mark ← 1;
  for each v ∈ V
    if (v.mark)
      dfs(v);
}
```

Figure 3.8 A pseudo-code description of the depth-first search algorithm `dfs`.

3.4.1 Depth-first Search

In many graph algorithms, one needs to traverse the graph in one way or the other and "do something" with the nodes and/or edges that one encounters during the traversal. One systematic way of doing this is by means of *depth-first search*. Another systematic way is by means of *breadth-first search* as is discussed in Section 3.4.2. Depending on the application, both search methods may be equivalent or only one of them may be appropriate. What is actually being done at each vertex and/or edge visited is not specified here. In this respect, the descriptions contain "generic actions" to be filled by the applications. The description in pseudo-code of a version of depth-first search for *directed* graphs is given in Figure 3.8, where the function that performs the depth-first search is called `dfs`.

The goal is to visit all vertices only once. This is achieved by introducing a member `mark` in the `vertex` structure. This member is initialized with the value 1 and given the value 0 when the vertex is visited. As the value is never restored to 1 and only the vertices whose `mark` members have value 1 are visited, it is guaranteed that each vertex is visited at most once. Note that an actual implementation does not necessarily need to use a `struct vertex` data structure. What the presentation

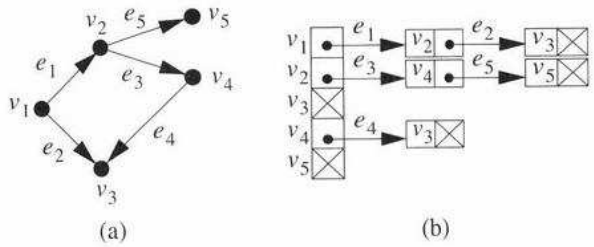


Figure 3.9 A directed graph (a) and its adjacency-list representation (b).

dfs(v_1) $\rightarrow e_1 = (v_1, v_2)$ $\rightarrow e_2 = (v_1, v_3)$	dfs(v_2) $\rightarrow e_3 = (v_2, v_4)$ $\rightarrow e_5 = (v_2, v_5)$	dfs(v_4) $\rightarrow e_4 = (v_4, v_3)$ dfs(v_5)	dfs(v_3)

Figure 3.10 The different steps of the depth-first search algorithm applied to the graph of Figure 3.9(a).

in Figure 3.8 tries to convey is that somehow a "mark" *attribute* for a vertex should be provided in the data structures used (a convenient alternative would be to use an array to hold the mark information).

The function `dfs` is a recursive function that takes a vertex as its argument. It "processes" the vertex (the generic vertex action) and then inspects all its outgoing edges one by one. After "processing the edge" (the generic edge action), the vertex to which this edge is incident is used for a recursive call of `dfs`, unless the vertex had already been visited. In the main program, the function is applied to each vertex of the graph to account for the fact that not all vertices may be reachable from a single vertex.

An adjacency-list representation is appropriate in this situation as it gives direct access to the outgoing edges of each vertex. Using this data structure, the analysis of the time complexity of depth-first search is rather simple. It was already mentioned that each vertex is visited exactly once. Since the outgoing edges of a vertex are only visited when the vertex itself is visited, all edges are also visited exactly once (an edge is an outgoing edge of one vertex only). Assuming that the generic vertex and edge actions have a constant time complexity, this leads to a time complexity of $\mathcal{O}(n + |E|)$ for depth-first search, where $n = |V|$.

The idea of depth-first search is illustrated using the graph of Figure 3.9(a) represented by means of its adjacency list shown in Figure 3.9(b). Figure 3.10 shows the evolution of the algorithm applied to the graph. In the figure, the recursion depth increases from left to right. The edges processed as a result of a call to `dfs` at a specific recursion depth are shown in the same column as the function call (preceded

by an arrow). Time increases from left to right and from top to bottom. From the figure, it can be concluded that the vertices are visited in the order v_1, v_2, v_4, v_3 and v_5 while the edges are visited in the order e_1, e_3, e_4, e_5 and e_2. Because all vertices are reachable from v_1, the first vertex for which `dfs` is called, there is only a single call to `dfs` at the lowest level of recursion.

The last remark directly indicates a straightforward application of depth-first search: the identification of vertices reachable from a specific vertex $v_s \in V$ for a graph $G(V, E)$. If the main program of Figure 3.8 is modified such that it consists of a single call of `dfs` with argument v_s after setting all `mark` attributes to 1, exactly those vertices that are reachable from v_s will be marked 0 after termination of the function call.

3.4.2 *Breadth-first Search*

As was mentioned in Section 3.4.1, *breadth-first search* is an alternative to depth-first search for systematically visiting all vertices of a graph. In this section a breadth-first search algorithm will be presented for a situation similar to the one in Section 3.4.1, viz. for directed graphs represented by an adjacency list. A pseudo-code description is provided in Figure 3.11, where the function that performs the actual breadth-first search is called `bfs`.

The central element in the description is the *FIFO queue* (first-in first-out queue) Q. Objects can be added and removed from such a queue in such a way that the order in which objects are removed is identical to the order in which the objects were originally added. The exact implementation is not very relevant here (see Section 3.5 for pointers to the literature). It is sufficient to know that the call `shift_in(q,o)` adds an object `o` to the queue `q`, that `shift_out(q)` removes the oldest object from the queue `q` and that the empty queue is denoted by `()` in pseudo-code. Besides, one should know that adding and removing objects from a FIFO queue can be done in constant time.

The main difference with depth-first search is that calling the `dfs` function recursively with a new vertex adjacent to the current vertex has been replaced by the addition of the new vertex to the FIFO queue. This results in the fact that the two methods visit the vertices and edges in a different order, although all vertices and edges are eventually visited by both methods. Using a similar reasoning as in Section 3.4.1 based on the `mark` attribute of a vertex and the same assumptions for the complexity of the generic actions, it is easy to prove that the algorithm visits all vertices and all edges of a graph exactly once leading to a time complexity of $\mathcal{O}(n + |E|)$. Actually, changing the behavior of the queue from FIFO to LIFO (last in first out) changes the breadth-first algorithm into the depth-first algorithm (see Exercise 3.2).

The application of `bfs` to the graph of Figure 3.9(a) is illustrated in Figure 3.12. Each line of the figure corresponds to one iteration of the `do` loop in the pseudo-code description. For each iteration, the contents of the FIFO queue (the leftmost object is the oldest), the vertex processed and the edges processed are given. Note that the

```
/* Given is the graph G(V, E) */

struct vertex {
  ...
  int mark;
};

bfs(struct vertex v)
{
  struct fifo *Q;
  struct vertex u, w;
  Q ← ();
  shift_in(Q, v);
  do { w ← shift_out(Q);
       "process w";
       for each (w, u) ∈ E {
         "process (w, u)";
         if (u.mark) {
           u.mark ← 0;
           shift_in(Q, u);
         }
       }
  } while (Q ≠ ())
}

main ()
{
   for each v ∈ V
     v.mark ← 1;
   for each v ∈ V
     if (v.mark) {
       v.mark ← 0;
       bfs(v);
     }
}
```

Figure 3.11 A pseudo-code description of the breadth-first search algorithm `bfs`.

Q	w	edges processed
(v_1)	v_1	$e_1 = (v_1, v_2), e_2 = (v_1, v_3)$
(v_2, v_3)	v_2	$e_3 = (v_2, v_4), e_5 = (v_2, v_5)$
(v_3, v_4, v_5)	v_3	-
(v_4, v_5)	v_4	$e_4 = (v_4, v_3)$
(v_5)	v_5	-

Figure 3.12 The different steps of the breadth-first search algorithm applied to the graph of Figure 3.9(a).

edges processed determine the vertices added to the queue. From the figure it can be concluded that the vertices are visited in the order v_1, v_2, v_3, v_4, v_5 while the edges are visited in the order e_1, e_2, e_3, e_5 and e_4.

In Section 3.4.1 it was mentioned that depth-first search could be used to find all vertices connected to a specific vertex $v_s \in V$ of a graph $G(V, E)$. Breadth-first search could be used for the same purpose. However, breadth-first search has an additional property. First all vertices adjacent to v_s are visited. These are the vertices connected to v_s with a path of length 1. The vertices visited next are reachable from v_s through a path of length 2, etc. It is therefore not difficult to see that the vertices are visited in the order of their *shortest path* from v_s. Breadth-first search can indeed be turned into a shortest-path algorithm by adding some bookkeeping statements and substituting appropriate code for the generic vertex and edge actions (see Exercise 3.3).

The shortest-path problem becomes more complex, however, if the length of the path between two vertices is not simply the number of edges in the path. This is the case in edge-weighted graphs, where the length of a path is defined as the sum of the edge weights of the edges in the path. Such a graph could e.g. model a group of cities and the distances between these cities (the graph can be constructed by taking a vertex for each city, an edge for each pair of cities that have a direct connection, and edge weights corresponding to the distance between adjacent cities). This version of the shortest-path problem is the topic of the next section.

3.4.3 Dijkstra's Shortest-path Algorithm

Suppose that a weighted directed graph $G(V, E)$ is given with the edge weights $w(e)$, $w(e) > 0$, for each edge $e \in E$. The problem addressed in this section is finding the *shortest path* from a source vertex $v_s \in V$ to a target vertex $v_t \in V$. The solution discussed is an algorithm proposed by Dijkstra in 1959 and nowadays known as "Dijkstra's shortest-path algorithm".

How should one solve the shortest-path problem for an edge-weighted directed graph? It looks a good idea to start at vertex v_s and visit its adjacent vertices as both depth-first and breadth-first search do. The problem is, of course, that a single edge is not always the shortest connection between two adjacent vertices. There may exist a path (or multiple paths) between two vertices going through several edges of which the total weight is lower than the weight of the edge connecting the two vertices directly. A straightforward way is to enumerate all possible paths between v_s and v_t and then select the shortest one. This is not very efficient. It is for example possible to construct graphs whose sizes grow linearly with some parameter k, while the number of paths grows exponentially. An example of a family of graphs with this property is shown in Figure 3.13: it has $2k + 2$ vertices (k pairs of vertices together with v_s and v_t) and $4k$ edges, while the number of possible paths from v_s to v_t is 2^k (at each of the first k vertical positions one can choose between two edges to proceed to the right).

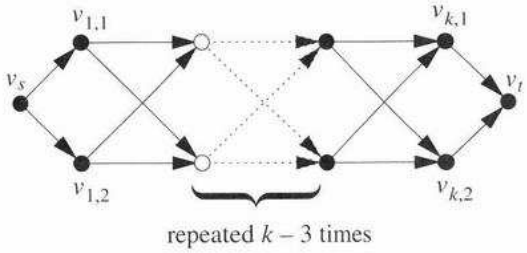


Figure 3.13 A family of graphs with an exponentially growing number of paths from v_s to v_t.

It turns out that there are more clever ways of computing the shortest path than merely enumerating all paths. This is e.g. the case for Dijkstra's shortest-path algorithm. As it will be shown later, its time complexity is a polynomial function of n (the number of vertices) and $|E|$ in spite of the fact that the number of paths may grow exponentially with n and $|E|$. This is due to the fact that many nonoptimal paths are directly eliminated while the algorithm proceeds.

A description of the algorithm in pseudo-code is given in Figure 3.14. This algorithm actually only computes the *length* of the shortest path (the computation of the shortest path itself is left as an exercise to the reader, see Exercise 3.4). In its main loop, the vertices of the set V are transferred one by one to a set T. The vertex that is selected for transfer, has the property that the shortest-path length to it is known at that moment (i.e. it will not change in the next iterations). Therefore, if the vertex to be transferred corresponds to v_t, the algorithm can stop.

The crucial element of the algorithm is to select the vertices in the correct order. This ordering results from the minimal value for the vertex attribute `distance` (similar to the descriptions of depth-first and breadth-first search, the use of `struct vertex` with member `distance` in the pseudo-code does not necessarily imply an implementation with `vertex` structures). In the first iteration, the `distance` attribute of a vertex $v \in V$ is equal to the edge weight $w((v_s, v))$ for all edges incident from v_s. Clearly, if the vertex with minimal `distance` value is selected, say vertex u, there cannot be a shorter path from v_s to u than the one through the single edge (v_s, u) (all edge weights are positive, so any path from v_s to u via an intermediate vertex is longer). Directly after the transfer of u from V to T, the `distance` attributes of the vertices at the endpoints of the edges incident from u are updated in the case that a path via u is shorter than the direct connection from v_s. Continuing this reasoning leads to the conclusion that the algorithm indeed computes shortest paths (see also Exercise 3.5). In general, it can be remarked that the value of the `distance` attribute of some vertex in V is equal to the length of the shortest path starting in v_s and passing only through the vertices in T.

To illustrate its effects, Dijkstra's shortest-path algorithm has been applied on a small graph shown in Figure 3.15. The growth of the set T and the subsequent values of the `distance` attributes of the vertices in the graph have been listed in

```
struct vertex {
  ...
  int distance;
};

dijkstra(set of struct vertex V, struct vertex v_s, struct vertex v_t)
{
  set of struct vertex T;
  struct vertex u, v;
  V ← V \ {v_s};
  T ← {v_s};
  v_s.distance ← 0;
  for each u ∈ V
    if ((v_s, u) ∈ E)
      u.distance ← w((v_s, u))
    else u.distance ← +∞;
  while (v_t ∉ T) {
    u ← "u ∈ V, such that ∀v ∈ V : u.distance ≤ v.distance";
    T ← T ∪ {u};
    V ← V \ {u};
    for each v "such that (u, v) ∈ E and v ∈ V"
      if (v.distance > w((u, v)) + u.distance)
        v.distance ← w((u, v)) + u.distance;
  }
}
```

Figure 3.14 A pseudo-code description of Dijkstra's shortest-path algorithm.

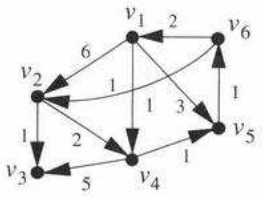


Figure 3.15 A small edge-weighted directed graph.

Figure 3.16. An underlined `distance` value indicates that the associated vertex is transferred from V to T at that moment. From the figure it can be seen that the $v_t = v_2$ is reached after 5 iterations and that continuing for one iteration more computes the lengths of the shortest paths for all vertices in the graph.

The time complexity of Dijkstra's algorithm depends on the exact type of data structures used. In a straightforward implementation, selecting the next element to be included in set T (the first statement in the `while` loop of Figure 3.14) requires $\mathcal{O}(n)$ time, where $n = |V|$, because all vertices in V have to be inspected in order to

iteration	T	v_i.distance, for $i =$ 1	2	3	4	5	6
1	$\{v_1\}$		6	∞	$\underline{1}$	3	∞
2	$\{v_1, v_4\}$		6	6		$\underline{2}$	∞
3	$\{v_1, v_4, v_5\}$		6	6			$\underline{3}$
4	$\{v_1, v_4, v_5, v_6\}$		$\underline{4}$	6			
5	$\{v_1, v_4, v_5, v_6, v_2\}$			$\underline{5}$			
6	$\{v_1, v_4, v_5, v_6, v_2, v_3\}$						

Figure 3.16 The evolution of the `distance` attributes in Dijkstra's algorithm when applied to the graph of Figure 3.15; $v_s = v_1$ and $v_t = v_2$.

find the one with minimal `distance` attribute. As the loop is executed $\mathcal{O}(n)$ times, this results in a contribution to the overall time complexity of $\mathcal{O}(n^2)$. The body of the "for each" loop inside the while loop is executed $|E|$ times in total, as all edges are visited exactly once, viz. after the vertex from which they are incident is added to set T. This gives a contribution of $\mathcal{O}(|E|)$ to the overall time complexity, assuming that the edges incident from a node are directly accessible (e.g. because of an adjacency list representation). In a simple graph there are at most n^2 edges and therefore the worst-case time complexity of $\mathcal{O}(n^2 + |E|)$ can be simplified to $\mathcal{O}(n^2)$.

3.4.4 Prim's Algorithm for Minimum Spanning Trees

A *tree* is a connected graph without cycles. A *spanning tree* of a connected graph $G(V, E)$ is a subgraph of G that is a tree and contains all vertices of V. So, one gets a spanning tree by removing edges from E until all cycles in the graph have disappeared while all vertices remain connected. Unless the graph is already a tree, a graph has several spanning trees, all of which have the same number of edges (equal to the number of vertices minus one). In the case of edge-weighted undirected graphs, it is often interesting to find the spanning tree with the least total edge weight, also called the *tree length*. This problem is the *minimum spanning tree* problem. The problem is e.g. encountered when one has a set of cities and wants to build a railroad system of minimal total length that connects all cities by means of pairwise connections, or in layout design for integrated circuits, when it is necessary to connect a set of terminals with the minimal total wire length[1]. In this section an algorithm originally published by Prim in 1957 will be presented.

The description of the algorithm in pseudo-code is given in Figure 3.17. The algorithm starts with an arbitrary vertex which is considered the initial tree (all the vertices are part of the spanning tree!). In the main loop, edges are added to the tree one by one (adding an edge automatically adds a new vertex as well) until the tree has become a spanning tree and contains all the vertices. Both the partial tree that is

[1] Actually, one often needs to find the minimal *Steiner tree*; this type of tree and a way to find an approximate solution are discussed in Section 4.5, where also its relation to the spanning tree is explained.

```
struct vertex {
  ...
  int distance;
  struct edge *via_edge;
};
struct edge {
  ...
};

prim(set of struct vertex V)
{
  set of struct edge F;
  set of struct vertex W;
  struct vertex u;
  u ← "any vertex from V";
  V ← V \ {u};
  W ← {u};
  F ← ∅;
  for each v ∈ V
    if ((u, v) ∈ E) {
      v.distance ← w((u, v));
      v.via_edge ← (u, v);
    }
    else v.distance ← +∞;
  while (V ≠ ∅) {
    u ← "u ∈ V, such that ∀v ∈ V : u.distance ≤ v.distance";
    W ← W ∪ {u};
    V ← V \ {u};
    F ← F ∪ {u.via_edge};
    for each v "such that (u, v) ∈ E"
      if (v.distance > w((u, v))) {
        v.distance ← w((u, v));
        v.via_edge ← (u, v);
      }
  }
}
```

Figure 3.17 The pseudo-code description of Prim's minimum spanning tree algorithm.

being constructed as well as the final spanning tree are graphs. Their vertex and edge sets are represented by the variables W and F respectively.

Here again, the data structures used are left vague intentionally. Even the use of separate records for vertices and edges is not necessarily required. They are only mentioned in order to be able to deal with abstract sets of vertices and edges. Besides, it is assumed that a vertex has the attributes `distance` and `via_edge`. The first one is used to register the shortest distance from a vertex not yet in the tree to a vertex already selected. The second one represents the edge through which this shortest

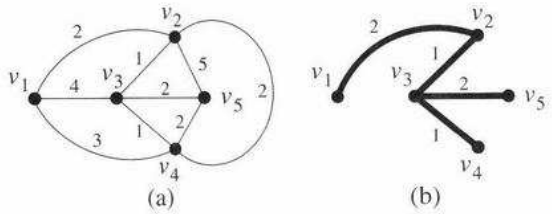


Figure 3.18 A small edge-weighted undirected graph (a) and its minimum spanning tree (b).

iteration	u	v_i.distance, v_i.via_edge, for $i =$ 1	2	3	4	5
0	v_1		2, (v_1, v_2)	4,(v_1, v_3)	3,(v_1, v_4)	$+\infty$,?
1	v_2			1, (v_2, v_3)	2,(v_2, v_4)	5,(v_2, v_5)
2	v_3				1, (v_3, v_4)	2,(v_3, v_5)
3	v_4					2, (v_3, v_5)
4	v_5					

Figure 3.19 The evolution of Prim's algorithm when applied to the graph of Figure 3.18.

connection is made. In each iteration, the vertex with the minimal `distance` value is added to the tree together with the edge of its `via_edge` attribute. Then the values of the `distance` and `via_edge` attributes are updated for the vertices not yet in the tree.

Figure 3.18(a) shows an example of a graph the minimal spanning tree of which should be determined. The result found by Prim's algorithm is depicted in Figure 3.18(b). Figure 3.19 shows the steps leading to this result. For each iteration, the figure shows the vertex u selected for inclusion in the set W and the values of the `distance` and `via_edge` attributes for the vertices not yet in W.

In a straightforward implementation, Prim's algorithm will require $\mathcal{O}(n^2)$ time, if n is the number of vertices. The main `while` loop is executed n times and within the loop finding the vertex with lowest `distance` value requires $\mathcal{O}(n)$ time. This worst-case time complexity can be improved by using more sophisticated data structures (see Section 3.5).

3.5 Bibliographic Notes

Several books entirely dedicated to algorithmic graph theory are available. Examples are: [Gib85] and [McH90]. Many books that deal with algorithms in general pay attention to graph theory in one or more chapters. Examples are: [Sed88] (this book uses Pascal as the language to describe algorithms; a version of the same book that uses C also exists [Sed90]), [Cor90] and [Mor91]. In these last books one can also find more details about the implementation of a FIFO queue necessary

for the implementation of breadth-first search. A discussion on the order notation, its standardization and its history can be found in [Knu76]. Dijkstra's shortest-path algorithm can be found in almost any book that is entirely or partially dedicated to algorithmic graph theory. The original publication is [Dij59]. It is possible to improve the worst-case time complexity of Dijkstra's algorithm from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n + |E|)$ by using more sophisticated data structures [Cor90]. Prim's algorithm for the minimum spanning tree is described in most of the books mentioned above. The original publication is [Pri57]. A discussion on how to improve the worst-case time complexity with respect to the description in Figure 3.17 can e.g. be found in [Mor91].

3.6 Exercises

3.1 Consider the graph data structure with "explicit edge and vertex representation" as shown in Figure 3.7. This data structure gives direct access to the outgoing edges of each vertex. Suppose that an algorithm also needs direct access to the incoming edges of a vertex. How should the data structure be modified to provide this direct access? Draw the representation of the graph of Figure 3.7(a) using this new data structure. How should the data structure be modified for the representation of undirected graphs? Draw the representation of the graph of Figure 3.1 using the modified data structure.

3.2 Suppose that the FIFO queue in the pseudo-code of Figure 3.11 is replaced by a queue with LIFO (last in first out) behavior. Show that the algorithm now more or less behaves as depth-first search by applying the modified algorithm to the graph of Figure 3.9.

3.3 Consider the shortest-path problem for directed graphs in which the length of a path is the number of edges in the path. It can be solved by the breadth-first search method described in Figure 3.11. What actions should be performed by the functions that "process" a vertex and an edge in order to actually compute the shortest path from any start vertex v_s to all other vertices in the graph?

3.4 Indicate how the description of Dijkstra's algorithm in Figure 3.14 should be modified when one wants to find the shortest path itself instead of merely the length of the shortest path. Pay special attention to the data structure for the representation of the path.

3.5 Prove that the algorithm given in Figure 3.14 correctly computes the length of the shortest path between v_s and v_t. Hint: use induction on the size of set T.

3.6 Prove the correctness of the algorithm given in Figure 3.17.

5

General-purpose Methods for Combinatorial Optimization

In VLSI design automation one encounters many combinatorial optimization problems. Most of them are intractable in the sense of Chapter 4. For others, polynomial solutions are known. In Chapter 4, it was mentioned that the algorithm designer has three possibilities when confronted with an intractable problem.

The first option is to try to solve the problem exactly if the problem size is sufficiently small using an algorithm that has an exponential (or even a higher order) time complexity in the worst case. This can be done in many different ways, all of which have the same theoretical worst-case time complexity, although some are clearly preferable in practice. The simplest way to look for an exact solution is *exhaustive search:* it simply visits all points in the search space in some order and retains the best solution visited. Other methods only visit part of the search space, albeit the number of points visited may grow exponentially (or worse) with the problem size. The first sections of this chapter are dedicated to methods in this category, viz. *backtracking with branch-and-bound, dynamic programming* and *integer-linear programming.* (Note that the term *programming* does not refer here to the act of writing code in order to implement a computation on a computer, but to the mathematical formulation of the problem.) One can call these methods "general purpose" in the sense that they are applicable to almost any combinatorial optimization problem as opposed to the "special purpose" methods that only work for specific problems.

Approximation algorithms and heuristics are the other two options to tackle intractable problems. General-purpose approximation algorithms do not exist: guarantees in finding a solution, the cost of which is within a certain margin of the optimal cost, can only be given by involving problem-specific issues in the analysis of the algorithm. The second part of this chapter consists of sections dealing with general-purpose heuristics that do not guarantee an optimal solution: *local search, tabu search, simulated annealing* (under theoretical conditions that are impossible to satisfy in practice, this method finds an exact solution), and *genetic algorithms.*

It is remarkable that some general-purpose methods for combinatorial optimization find their inspiration in other domains than mathematics and computer science.

Simulated annealing was inspired by physics, whereas evolution theory was a source of inspiration for *genetic algorithms*.

The goal of this chapter is to provide some additional background for the discussion of the combinatorial optimization problems that are found in different places in VLSI design automation as explained in various chapters of this text. Special-purpose algorithms are described in the appropriate chapters, but algorithms that are applicable in more than one place can better be presented centrally. No attempt has been made to achieve completeness for general-purpose methods and the selection is mainly based on the occurrence of the methods in CAD for VLSI. The methods are presented rather informally, sometimes at the expense of mathematical precision.

The very first issue discussed in this chapter is, however, not an optimization method but a simplified CAD problem that has been called the *unit-size placement problem*. It can be tackled by most of the general-purpose methods to be introduced. By defining the problem at this stage, it will be possible to use it for the illustration of the different methods besides more mathematical problems such as TSP.

5.1 The Unit-size Placement Problem

An instance of the placement problem is defined by a set of *cells* and a description of how these cells should be interconnected. Cells are small subcircuits the internal layout of which is known. The interconnections to be made are specified by *nets*. A net can be seen as a set of cells that share the same electrical signal. The goal of placement is to assign a location to each cell such that the total chip area occupied is minimized. As the number of cells is not modified by placement, minimizing the area amounts to avoiding empty space and keeping the wires that will realize the interconnections as short as possible.

In this section, only a "toy version" of the placement problem is defined. More realistic versions of the problem are presented in Chapter 7. The toy problem will be called *unit-size placement* for the simple reason that all cells in the circuit are supposed to have a layout with dimensions 1×1 (measured in some abstract length unit). Furthermore it can be assumed that the only positions on which a cell can be put on the chip are the grid points of a grid created by horizontal and vertical lines with unit-length separation. A nice property of unit-size placement is that the assignment of distinct coordinate pairs to each cell guarantees that the layouts of the cells will not overlap. If the range of coordinates available in two dimensions is fixed in advance, the only contribution to the cost function will come from the area occupied by wiring.

Figure 5.1(a) shows the specification of a circuit description consisting of 7 cell instances A to F, and 8 nets n_1 to n_8. For each net, the specification gives the cells to which the net is connected. Such a description is called a *netlist*. A possible placement of this circuit on a unit-size grid is given in Figure 5.1(b).

The best way to evaluate the quality of a solution for unit-size placement is to route all nets and measure the extra area necessary for wiring. In the unit-size placement model, as much area as required can be created by pulling apart adjacent rows or

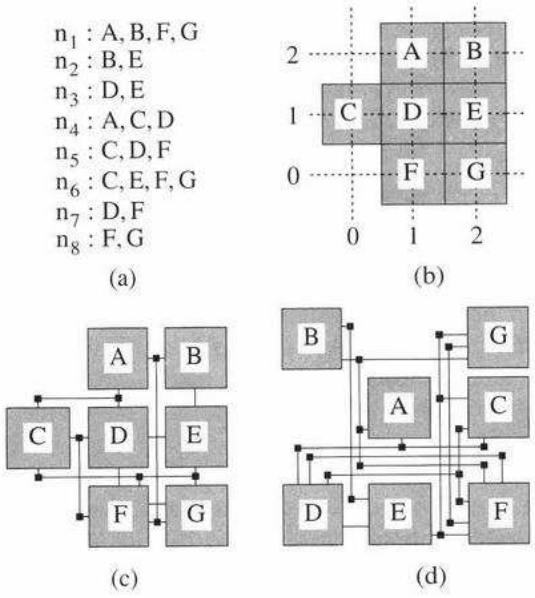


Figure 5.1 An example netlist for the unit-size placement problem (a), a possible placement solution (b), the routing for this placement (c), and the routing for a placement of worse quality (d).

columns. This is shown in Figure 5.1(c) for the placement of Figure 5.1(b). The solution presented is just a sketch. For a real solution, one needs to know to which specific port of a cell's layout a net should connect. A bad placement will have longer connections which normally will lead to more routing *tracks* between the cells and therefore to a larger chip area. This is illustrated in Figure 5.1(d).

Most versions of the routing problem are intractable and even heuristics require a considerable amount of computation time. Therefore, solving the routing problem is an expensive way to evaluate the quality of a placement. This is especially the case if many tentative placements have to be evaluated in an algorithm that tries to find a good one. An alternative used in most placement algorithms is to only *estimate* the wiring area. The first approximation made is to estimate the total wire length instead of wiring area. At this stage, it is sufficient to know that techniques for wire length estimation exist that require the position of all cells and the netlist as their only inputs. They will be discussed in detail in Section 7.2.

5.2 Backtracking and Branch-and-bound

This section presents two techniques for finding the optimal solution of a combinatorial optimization problem that are quite similar: backtracking and branch-and-bound. Actually, branch-and-bound is a refinement of backtracking. Before explaining these techniques some general remarks have to be made.

In Chapter 4, an instance I of a combinatorial optimization problem was defined by a pair (F, c), with F the "set of feasible solutions" (also called the "search space" or "solution space") and c a cost function assigning a real number to each element in F. Suppose that each feasible solution can be characterized by an n-dimensional vector $\mathbf{f} = [f_1 f_2 \ldots f_n]^T$ and each f_i $(1 \leq i \leq n)$ can assume a finite number of values, called the *explicit constraints* of f_i. Besides, the values assigned to the different components of $\mathbf{f}$ may sometimes not be chosen independently. In such a case one speaks of *implicit constraints*. Consider e.g. a combinatorial optimization problem related to some graph $G(V, E)$ in which a path with some properties is looked for. One can then associate a variable f_i with each edge $e_i \in E$, whose value is either 1 to indicate that the corresponding edge is part of the path or 0 to indicate the opposite. The explicit constraints then state that $f_i \in \{0, 1\}$ for all i. The implicit constraints say that the edges selected by the variables should form a path.

Note: the specification of a solution by means of a fixed-length vector is not always the most appropriate choice for a combinatorial optimization problem. In the example above, a more natural representation of a path is given by a sequence of vertices and edges with a variable length (cf. the definition of a path in Section 3.1). However, the notions presented here do not change essentially if a variable-length encoding of the solution is chosen instead of a fixed-length one.

Assuming the fixed-length vector representation of a solution, one defines a *partial solution* by leaving one or more elements of the vector unspecified. One might as well say that a partial solution corresponds to a subspace of the solution space. A partial solution will be denoted by $\tilde{\mathbf{f}}$.

5.2.1 *Backtracking*

The principle of using *backtracking* for an *exhaustive search* of the solution space is to start with an initial partial solution in which as many variables as possible are left unspecified, and then to systematically assign values to the unspecified variables until either a single point in the search space is identified or an implicit constraint makes it impossible to process more unspecified variables. In the first case, the cost of the feasible solution found can be computed. In both cases, the algorithm continues by going back to a partial solution generated earlier and then assigning a next value to an unspecified variable (hence the name "backtracking"). The pseudo-code of such a backtracking algorithm is given in Figure 5.2. It is assumed that all variables f_i have type `solution_element`. The partial solutions are generated in such a way that the variables f_i are specified for $1 \leq i \leq k$ and are unspecified for $i > k$. Partial solutions having this structure will be denoted by $\tilde{\mathbf{f}}^{(k)}$. Note that $\tilde{\mathbf{f}}^{(n)}$ corresponds to a fully-specified solution (a member of the set of feasible solutions). The global array `val` corresponds to the vector $\tilde{\mathbf{f}}^{(k)}$. The value of f_k is stored in `val`$[k-1]$. So, the values of array elements with index greater than or equal to k are meaningless and should not be inspected. The procedure `cost(val)` is supposed to compute the cost of a feasible solution using the cost function c. It is obviously only called when $k = n$, i.e. when a solution is fully specified. The

```
float best_cost;
solution_element val[n], best_solution[n];

backtrack(int k)
{
  float new_cost;
  if (k = n) {
    new_cost := cost(val);
    if (new_cost < best_cost) {
      best_cost := new_cost;
      best_solution := copy(val);
    }
  }
  else
    for each (el ∈ allowed(val, k)) {
      val[k] := el;
      backtrack(k + 1);
    }
}

main ()
{
  best_cost := ∞;
  backtrack(0);
  report(best_solution);
}
```

Figure 5.2 The pseudo-code of an algorithm for an exhaustive search by means of backtracking.

procedure `allowed(val, k)` returns a set of values allowed by the explicit and implicit constraints for the variable f_{k+1} given $\tilde{\mathbf{f}}^{(k)}$. The best solution found is stored in the global array `best_solution` and it is reported at the end of the search process.

Consider the graph version of the *traveling salesman problem* (TSP) introduced in Chapter 4, in which the number of vertices in a graph $G(V, E)$ equals $n - 1$. A solution can be specified by a sequence of n vertices such that an edge between subsequent vertices exists, the first vertex in the sequence equals the last one and the first $n - 1$ vertices are distinct. The sequence can be characterized by n variables f_i ($1 \leq i \leq n$), one for each position in the sequence. The value of f_i is the vertex at position i in the sequence. Because any vertex $v \in V$ in the cycle can be taken as the first and last element of the sequence, one can select any of the vertices in the graph and state that f_1 and f_n can only have value v. These are the explicit constraints for f_1 and f_n. The explicit constraints for the other variables are the remaining vertices of V ($V \setminus \{v\}$). The implicit constraints are that any sequence specified by

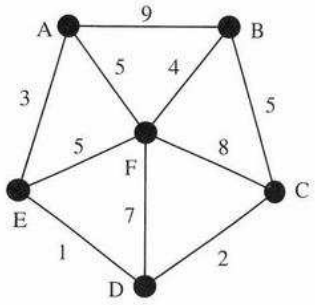


Figure 5.3 An example traveling salesman problem instance.

the variables should form a path[1]. Figure 5.3 shows an example problem instance. The six vertices have been labeled A to F. The cost of traveling between two adjacent vertices is given by a weight along the edge connecting the two vertices. It will be assumed that $f_1 = f_7 = A$ and that the other variables take any of the values in the set $\{B, C, D, E, F\}$.

Consider as an other example the *unit-size placement* problem defined in Section 5.1. If there are n cells to be placed, a solution can be described by $2n$ variables f_1 to f_{2n}, where f_{2i-1} corresponds to the x-coordinate of cell i and f_{2i} to its y-coordinate ($1 \leq i \leq n$). The explicit constraints say that values should be chosen in the available range of coordinates (x or y). The implicit constraints disallow that two cells are assigned the same coordinate pair.

The subsequent generations of (partial) solutions by the backtracking algorithm can be visualized by nodes in a tree, called the *search tree*[2]. The search tree for the example of Figure 5.3 is shown in Figure 5.4. A *tree* is a directed graph here. It is normally drawn in such a way that all edges have an implicit direction from top to bottom. The single node at the highest level, Level 1, is called the *root* of the tree. The nodes *incident from* the node (connected to it at the next lower level) are called its *children*. Every level of the tree corresponds to a variable. The children at level $k+1$ of a node at level k correspond to the partial solutions obtained by specifying a value for f_{k+1}. So, a node at level k represents a partial solution $\tilde{\mathbf{f}}^{(k)}$. Nodes without children are called *leaf nodes*. Note that each node except for the root has exactly one incoming edge. The other endpoint of this edge is called the node's *parent*. Removing this incoming edge for a node $\tilde{\mathbf{f}}^{(k)}$ gives a *subtree* having $\tilde{\mathbf{f}}^{(k)}$ as its root. Each path in the tree starting at the root and ending in a leaf node corresponds to either a fully

[1] One could say that a variable that can only assume a single value is not a real variable and claim that a feasible solution could be described with $n-2$ instead of n variables. However, the description chosen here makes it e.g. easier to exclude by means of implicit constraints a path that contains all vertices but does not form a cycle.

[2] In order not to get confused with the vertices of e.g. the graph of the TSP example, the vertices of the search tree will be called *nodes* in this section.

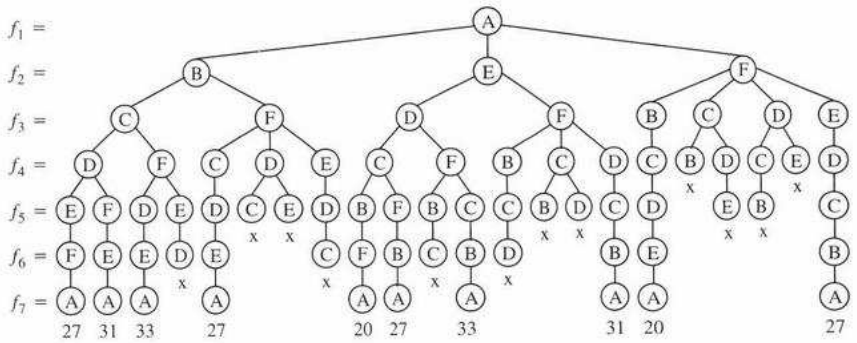


Figure 5.4 The search tree obtained by the exhaustive search backtracking algorithm for the example of Figure 5.3.

specified solution of which the cost can be computed (indicated with the cost value in the figure) or to a state in the backtracking procedure from which no new (partial) solution can be generated due to the fact that no legal assignment to the next variable is possible (indicated with an 'x' in the figure). Note that each tour is found twice, the second of each pair of equivalent solutions being the reverse of the first (this can be avoided by an extra implicit constraint, e.g. putting that City B should always precede City C in each solution; such a constraint will reduce the search space and, therefore, speed up the search). Obviously, the optimal tour visits the vertices in the order A, E, D, C, B, F, and A (or in the reverse order) and has a total length of 20.

5.2.2 *Branch-and-bound*

Normally, it is not necessary to visit all (partial) solutions that the backtracking procedure generates. Let $D(\tilde{\mathbf{f}}^{(k)})$ denote the set of fully-specified solutions in the subtree with root $\tilde{\mathbf{f}}^{(k)}$. Information about a certain partial solution $\tilde{\mathbf{f}}^{(k)}$, $1 \leq k < n$, at a certain level can indicate that any fully-specified solution $\tilde{\mathbf{f}}^{(n)} \in D(\tilde{\mathbf{f}}^{(k)})$ derived from it can never be the optimal solution. This conclusion is based on the estimation of the lower bound of the cost of all solutions in $D(\tilde{\mathbf{f}}^{(k)})$. The function that estimates this cost lower bound will be denoted by $\tilde{c}$. If inspection of $\tilde{\mathbf{f}}^{(k)}$ can guarantee that all of the solutions in $D(\tilde{\mathbf{f}}^{(k)})$ have a higher cost than some solution already found earlier during the backtracking, none of the children of $\tilde{\mathbf{f}}^{(k)}$ need any further investigation. One says that the node in the tree corresponding to $\tilde{\mathbf{f}}^{(k)}$ can be *killed*. The modification of the backtracking algorithm that provides in killing partial solutions is called *branch-and-bound*. One also says that one can *prune* the search tree by removing the subtree having $\tilde{\mathbf{f}}^{(k)}$ as its root.

The pseudo-code describing the branch-and-bound version of backtracking is presented in Figure 5.5. The main recursive procedure is called `b_and_b`. It is quite similar to the procedure `backtrack` of Figure 5.2 with the difference that the procedure `lower_bound_cost` is called to get a lower bound of the partial solution

```
float best_cost;
solution_element val[n], best_solution[n];

b_and_b(int k)
{
  float new_cost;
  if (k = n) {
    new_cost := cost(val);
    if (new_cost < best_cost) {
      best_cost := new_cost;
      best_solution := copy(val);
    }
  }
  else if (lower_bound_cost(val,k) ≥ best_cost)
        /* No action, node is killed. */
  else
    for each (el ∈ allowed(val, k)) {
      val[k] := el;
      b_and_b(k + 1);
    }
}

main ()
{
   best_cost := ∞;
   b_and_b(0);
   report(best_solution);
}
```

Figure 5.5 The pseudo-code of the branch-and-bound algorithm.

based on the function $\tilde{c}$. A next level of recursion is entered only if the node in the search tree cannot be killed.

An important issue is how to define $\tilde{c}$ such that it is useful. It is quite natural to compose it out of two terms:

$$\tilde{c}(\tilde{\mathbf{f}}^{(k)}) = \tilde{g}(\tilde{\mathbf{f}}^{(k)}) + \tilde{h}(\tilde{\mathbf{f}}^{(k)})$$

where $\tilde{g}(\tilde{\mathbf{f}}^{(k)})$ is a function that is computed given the specified variables of $\tilde{\mathbf{f}}^{(k)}$ and $\tilde{h}(\tilde{\mathbf{f}}^{(k)})$ is a term that is based on the unspecified variables of $\tilde{\mathbf{f}}^{(k)}$. Taking the traveling salesman problem as an example, $\tilde{g}(\tilde{\mathbf{f}}^{(k)})$ can give the length of the path fixed by the specified variables and $\tilde{h}(\tilde{\mathbf{f}}^{(k)})$ can give a lower bound on the remaining tour length. One possibility is to define $\tilde{h}(\tilde{\mathbf{f}}^{(k)}) = 0$ for all partial solutions. This certainly satisfies the requirement that all final solutions in the set $D(\tilde{\mathbf{f}}^{(k)})$ have a cost higher than the estimated one. However, this definition does not help much to reduce the search space. A more effective possibility is to use the length of the *minimum spanning tree* (see Section 3.4.4) as a lower bound. Given a sequence of vertices that represents

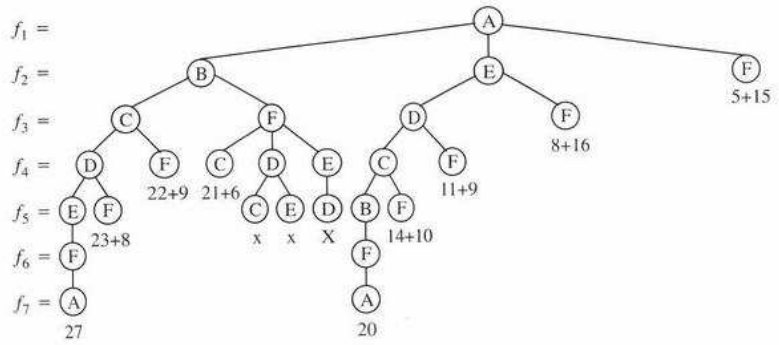


Figure 5.6 The search tree for the TSP example of Figure 5.3 when branch-and-bound is used.

a partial solution, one knows that the fully-specified solution derived from it will somehow connect the vertices not yet in the sequence with the first and last vertex. Call the set consisting of the vertices not in the sequence augmented by the first and last one of the sequence the *remaining* vertices. A path that connects all remaining vertices and that starts at the last one already in the sequence and finishes at the first vertex already in the sequence, will turn the partial solution into a fully-specified one. Note that such a path is a spanning tree in the subgraph induced by the remaining vertices. So, the length of the minimum spanning tree in this subgraph is a lower bound on the length of the required path.

The branch-and-bound search tree for the TSP example of Figure 5.3 that is obtained using a function $\tilde{h}(\tilde{\mathbf{f}}^{(k)})$ based on the minimal spanning tree, is shown in Figure 5.6. In the figure, a label '$a + b$' means that the node is killed due to the fact that the sum exceeds the cost of the best known solution up to that moment. The term a is the value of $\tilde{g}(\tilde{\mathbf{f}}^{(k)})$ and b is the value of $\tilde{h}(\tilde{\mathbf{f}}^{(k)})$. A label 'X' means that the subgraph induced by the remaining nodes is not connected and that a spanning tree does not exist. Clearly, as a consequence, a solution to TSP cannot be found in the subtree of that node. Note that the number of nodes in the search tree is 72 for the exhaustive search version and 27 for the branch-and-bound version.

The application of branch-and-bound search to the unit-size placement problem is more complex. As was mentioned before and will be discussed in more detail in Section 7.2, the cost function of this problem computes an estimate of the wire length on the basis of the positions of the cells. In a partial solution, only a subset of the cells has a fixed position. This makes it necessary to use more advanced estimation techniques that give lower bounds for the lengths of the wires connected to cells without a fixed position.

An essential point is that the function $\tilde{c}(\tilde{\mathbf{f}}^{(k)})$ that computes the lower bound of the solutions in $D(\tilde{\mathbf{f}}^{(k)})$ should, in general, be easier to compute than the mere traversal of the subtree at $\tilde{\mathbf{f}}^{(k)}$ in order to have some computational gain. There is some preservation of the required effort here: more sophisticated functions can estimate

tighter lower bounds allowing the killing of nodes in an earlier stage, but they will normally require a higher computational effort themselves.

In this section only a simple version of backtracking (with and without branch-and-bound) has been presented. It traverses the search tree in a *depth-first* fashion (see also Section 3.4.1). In depth-first search, one focuses on a single partial solution of which a child at the next level is generated. Only when a solution has been killed or has been fully specified, does the algorithm go back to a higher level. The algorithm only keeps track of a single partial solution at a time.

Alternative ways to explore the search tree become possible if one is prepared to use *queues* to store intermediate nodes of the search tree. The principle is to evaluate *all* children of a node (the first node to be evaluated is, of course, the root) and store them in a queue. Then the elements already stored are removed one by one from the queue while their children are added to it. The partial solutions in the queue can be processed in different orders: first-in-first-out (FIFO), last-in-first-out (LIFO) and least-cost (LC). Using a FIFO queue will result in *breadth-first* search, while using a LIFO queue will amount to *depth-first* search. LC uses the function $\tilde{c}(\tilde{\mathbf{f}}^{(k)})$ to have an estimated cost of all elements in the queue and then first generates the children of the partial solution with minimal cost. Depending on the problem and its typical instances, using any of the search strategies just mentioned may lead to significant improvements in the time necessary to obtain an optimal solution.

More variations are possible. Instead of having the *static* search tree discussed above, in which partial solutions are generated by assigning values to the f_i in order of increasing index i, one can have *dynamic* search trees, both for depth-first and breadth-first search. In a dynamic search tree, those parameters are assigned a value first that will either lead to a fast detection of a solution or an early killing of a large subtree.

5.3 Dynamic Programming

Dynamic programming is a technique that systematically constructs the optimal solution of some problem instance by defining the optimal solution in terms of optimal solutions of smaller size instances. Suppose that some problem instance can be characterized by some "complexity parameter" p (this parameter is not necessarily the input size). Dynamic programming can be applied to such a problem if there is a rule to construct the optimal solution for $p = k$ from the optimal solutions of instances for which $p < k$. Given such a rule and the fact that the lowest complexity problem instances (those instances for which, say, $p = 0$ or $p = 1$) are easy to obtain, the optimal solutions for instances with any complexity can be constructed. The fact that an optimal solution for a specific complexity can be constructed from the optimal lower complexity problems only, is essential for dynamic programming. In this way, many parts of the search space can be discarded without visiting them. This idea is called the *principle of optimality*.

The explanation just given may be a little abstract. It will, therefore, be illustrated by applying dynamic programming to two example problems. The first problem is the shortest-path problem as solved by Dijkstra's algorithm and the second is TSP.

Following its presentation in Section 3.4.3, the goal in the shortest-path problem is to find the shortest path from a source vertex $v_s \in V$ to a destination vertex $v_t \in V$ in a directed graph $G(V, E)$ where the distance between two vertices $u, v \in V$ is given by the edge weight $w((u, v))$. Actually, no new shortest-path algorithm will be presented here, but Dijkstra's algorithm will be "reinterpreted" as an application of dynamic programming. The variable names used refer to those in the pseudo-code of Figure 3.14.

An instance of the shortest-path problem is extended with a parameter p for the purpose of dynamic programming. If $p = k$, the optimization goal becomes: find the shortest path from v_s to all other vertices in the graph considering paths that only pass through the first k closest vertices to v_s. Once the problem has been formulated in this way, Dijkstra's algorithm becomes an almost obvious solution to the shortest-path problem. The optimal solution for the instance with $p = 0$ is found in a trivial way by assigning the edge weight $w((v_s, u))$ to the `distance` attribute of all vertices $u \in V$. Suppose that the optimal solution for $p = k$ is known and that the k closest vertices to v_s have been identified and transferred from V to T. Then, solving the problem for $p = k + 1$ is simple: transfer the vertex u in V having the lowest value for its `distance` attribute from V to T and update the value of the `distance` attributes for those vertices remaining in V. A crucial observation is, of course, that the shortest path length from v_s to u has now been found. By the assumption, it is known that the `distance` attribute of u gives the length of the shortest path that passes through any of the vertices in T. Suppose now that the actual shortest path to u also passes through some vertex $w \in V$; then the shortest path length of u would at least equal w.`distance` $+ w((w, u))$. This leads to a contradiction, however, because all edge weights are positive and w.`distance` $>$ u.`distance` (otherwise w would have been selected instead of u). Note that the reasoning just presented is a correctness proof of the algorithm (see also Exercise 3.5).

There is a single optimal solution for each value of p in the dynamic programming interpretation of Dijkstra's algorithm. One only needs to keep the optimal solution for $p = k$ for the computation of the optimum for $p = k + 1$. This solution is kept in the `distance` attributes of the vertices. In general, many optimal solutions for $p = k$ have to be stored such that they can be consulted when constructing solutions for the instances with $p = k + 1$. Said more precisely, additional parameters may be necessary to distinguish multiple instances of the problem for the same value of p. This is the case in the dynamic programming approach to the graph version of the traveling salesman problem discussed below.

As was the case in Section 5.2, the graph considered is an undirected graph $G(V, E)$, with edge weights $w((u, v))$ for an edge $(u, v) \in E$. Some vertex $v_s \in V$ should be chosen as the starting point of the tour (as any optimal tour visits all vertices, the actual choice for v_s is irrelevant). The introduction of a parameter p in TSP creates a set of problem instances derived from the original instance. If $p = k$,

the goal becomes to find the shortest paths to any $v \in V$ ($v \neq v_s$) starting from v_s that go through exactly k intermediate vertices.

In the following, $C(S, v)$ will denote the shortest path length from v_s to v exactly passing through the intermediate vertices given by the set S. The shortest tour length for the original instance of TSP is then given by $C(V \setminus \{v_s\}, v_s)$. The rule for constructing solutions with $p = k + 1 = |S|$ from solutions with $p = k$ is:

$$C(S, v) = \min_{m \in S} [C(S \setminus \{m\}, m) + w((m, v))] \tag{5.1}$$

Clearly, for each value of k, $C(S, v)$ has to be computed for all possible S and all possible v. The construction of the solution starts with the values $C(\emptyset, v)$ which equals the weight $w((v_s, v))$ when $(v_s, v) \in E$, and ∞ otherwise.

In the process of arriving at the final solution almost all subsets of V have to be considered (several times). This leads to a time complexity that is at least an exponential function of the number of vertices (a set with n elements has 2^n subsets). This is not surprising given the NP-completeness of TSP. However, the dynamic programming approach may still be interesting because it does not visit all points in the search space due to the elimination of all suboptimal solutions of size k when computing the solutions for size $k + 1$. It is comparable to the branch-and-bound approach in this sense, which also had an exponential worst-case time complexity in spite of the fact that it eliminates many parts of the search space.

This dynamic programming approach for TSP will now be illustrated for the example graph of Figure 5.3 where the choice $v_s = A$ has been made. First the values for $|S| = 0$ have to be computed:

$$\begin{aligned} C(\emptyset, B) &= 9 \\ C(\emptyset, C) &= \infty \\ C(\emptyset, D) &= \infty \\ C(\emptyset, E) &= 3 \\ C(\emptyset, F) &= 5 \end{aligned}$$

The next step is to compute all possible values for $|S| = 1$ applying the construction rule given in Equation (5.1). Below a selection of the 20 values to be computed is given (5 values for S and 4 for v not contained in S, when computing $C(S, v)$):

$$\begin{aligned} C(\{B\}, C) &= C(\emptyset, B) + w((B, C)) = 9 + 5 = 14 \\ C(\{B\}, F) &= C(\emptyset, B) + w((B, F)) = 9 + 4 = 13 \\ C(\{F\}, B) &= C(\emptyset, F) + w((F, B)) = 5 + 4 = 9 \end{aligned}$$

There are 30 values to be computed for $|S| = 2$. One of them is computed below:

$$\begin{aligned} C(\{B, F\}, C) &= \min [C(\{B\}, F) + w((F, C)), C(\{F\}, B) + w((B, C))] \\ &= \min [13 + 8, 9 + 5] = 14 \end{aligned}$$

This result indicates that the shortest path from A to C that passes through B and F has a length of 14 (by keeping track of the choices that led to this minimum, one could also report that the shortest path goes first to F and then to B). By continuing this process for ever larger sizes of $|S|$, one eventually computes $C(\{B, C, D, E, F\}, A)$ and obtains the solution of the example TSP instance.

The application of dynamic programming for the other example problem considered in this chapter, viz. unit-size placement, is not so obvious. It is not clear how the optimal placement of $k + 1$ cells should be defined in terms the optimal solutions of all subproblems with k cells in such a way that parts of the search space are eliminated. An application of dynamic programming to a problem that occurs in a channel routing algorithm is presented in Section 9.3.4.

5.4 Integer Linear Programming

Integer linear programming (ILP) is a specific way of casting a combinatorial optimization problem in a mathematical format. More precisely, many combinatorial optimization problems can relatively easily be *reduced* to ILP. This does not help from the point of view of computational complexity as ILP is NP-complete itself. However, ILP formulations for problems from the field of VLSI design automation are often encountered due to the existence of "ILP solvers". ILP solvers are software packages that accept any instance of ILP as their input and generate an exact solution for the instance. As a consequence of the structure of an ILP formulation, the solver does not need any knowledge of the problem. The only effort to be made is to find the right reduction to correctly translate all features of the problem into the ILP format. This section actually only deals with the format and the translation process. The techniques used inside an ILP solver are not explained. It will seldom be necessary to implement an ILP solver because powerful solvers are available both commercially and in the public domain.

There are several reasons why ILP is useful in CAD for VLSI. First, the input sizes of the problems involved may be small enough for an ILP solver to find a solution in reasonable time. One then has an easy way of obtaining exact solutions, compared to techniques such as branch-and-bound (by the way, an ILP solver may use branch-and-bound itself). It may also be that one needs the exact solutions of some benchmark circuits in order to judge the merits of some problem-specific heuristics. Excessive run times may be acceptable in such a case. Finally, an ILP formulation may be a source of inspiration for the development of problem-specific heuristics.

5.4.1 Linear Programming

Integer linear programming is a special variant of *linear programming* (LP) and a basic understanding of LP is necessary in order to understand ILP. LP will be introduced by means of an example. Suppose that a factory produces two different food products P_1 and P_2 that are composed of the same two ingredients I_1 and I_2 in

different proportions. P_1 is made of a_{11} units of I_1 and a_{21} units of I_2. Similarly, P_2 is made of a_{12} units of I_1 and a_{22} units of I_2. The factory sells one unit of P_1 for price c_1 and one unit of P_2 for c_2. The goal is to maximize the daily sales, given that the company can receive at most b_1 units of I_1 and b_2 units of I_2 per day. The demand is such that all production will be sold. The unknowns in this problem are x_1 and x_2, the quantities of P_1 and respectively P_2 to be produced daily. Stated mathematically, the goal is to maximize:

$$c_1x_1 + c_2x_2$$

while satisfying the constraints:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 &\leq b_1 \\ a_{21}x_1 + a_{22}x_2 &\leq b_2 \\ x_1 &\geq 0 \\ x_2 &\geq 0 \end{aligned}$$

Such a formulation is called a *linear program.* In general a linear program can be formulated in terms of a vector $\mathbf{x}$ of variables, a cost vector $\mathbf{c}$ and a matrix $\mathbf{A}$ and a vector $\mathbf{b}$ that determine the constraints ($\mathbf{A}$ is not necessarily square!). The goal is either to maximize or minimize:

$$\mathbf{c}^T\mathbf{x}$$

while satisfying:

$$\begin{aligned} \mathbf{Ax} &\leq \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0} \end{aligned}$$

Note that the expressions for the cost and constraints are *linear* in $\mathbf{x}$. The problem can be changed from a minimization problem to a maximization problem by simply multiplying $\mathbf{c}$ by -1. Similarly, one can convert a constraint containing a "less than or equal" sign to one containing containing a "greater than or equal" sign by multiplying by -1 the appropriate elements in $\mathbf{A}$ and $\mathbf{b}$. The formulation just given is called the *canonical form* of LP. Although all variables are apparently restricted to be positive, this restriction can easily be overcome by replacing a variable x_i, that may assume negative values, by the difference $x_j - x_k$ of two new variables x_j and x_k that are both restricted to be positive.

Apart from the canonical form, LP also has a *standard form* in which the constraints look like:

$$\begin{aligned} \mathbf{Ax} &= \mathbf{b} \\ \mathbf{x} &\geq \mathbf{0} \end{aligned}$$

Here, the constraints to be satisfied are expressed as equations rather than inequalities. Inequalities can be converted to equations by adding to or subtracting from each

equation a new variable, a so-called *slack* variable. For example, $a_{11}x_1 + a_{12}x_2 \leq b_1$ becomes $a_{11}x_1 + a_{12}x_2 + x_3 = b_1$. Conversely, an equation can be replaced by a pair of inequalities when going from the standard form to the canonical form. One can for example replace $a_{11}x_1 + a_{12}x_2 = b_1$ by the two inequalities $a_{11}x_1 + a_{12}x_2 \leq b_1$ and $a_{11}x_1 + a_{12}x_2 \geq b_1$.

So-called "LP solvers" are available to solve LP problems. It is outside the scope of this text to present the algorithms used in LP solvers, but some interesting remarks can be made about them. It is possible to solve LP problems by a polynomial-time algorithm called the *ellipsoid algorithm*. This algorithm is, however, outperformed in practice by the *simplex algorithm* which has an exponential worst-case time complexity. LP is therefore one of the few problems for which an exponential algorithm is often preferable above a polynomial one. This illustrates the importance of average-case time complexity in practice.

5.4.2 *Integer Linear Programming*

ILP is a variant of LP in which the variables are restricted to be integers. Although it seems to be a minor modification, this restriction makes ILP NP-complete. The techniques used for finding solutions of LP are in general not suitable for ILP. It does, for example, not help to treat an instance of ILP as an instance of LP and then to round the results: the solution obtained may not be the optimal one or even a feasible one. Therefore, other techniques that more explicitly deal with the integer values of the variables should be used. As was mentioned before, however, solution techniques will not be discussed here.

In many situations, the integer variables are restricted further to assume either of the values zero or one. This variant of ILP is called *zero-one integer linear programming*. A way to obtain a zero-one ILP formulation for the TSP in graphs is presented below.

Consider a graph $G(V, E)$ where the edge set E contains k edges: $E = \{e_1, e_2, \ldots, e_k\}$. The ILP formulation requires a variable x_i for each edge $e_i \in E$. The variable x_i can either have the value 1, which means that the corresponding edge e_i has been selected as part of the solution, or the value 0 meaning that e_i is not part of the solution. Given the fact that the weight of an edge e_i is denoted by $w(e_i)$, the cost function can be written as:

$$\sum_{i=1}^{k} w(e_i)x_i \tag{5.2}$$

In addition, the following restriction applies:

$$x_i \in \{0, 1\}, \quad i = 1, 2, \ldots, k \tag{5.3}$$

In the optimal solution, only those x_i that correspond to edges in the optimal tour have a value 1. The constraints should enforce that the edges selected actually form

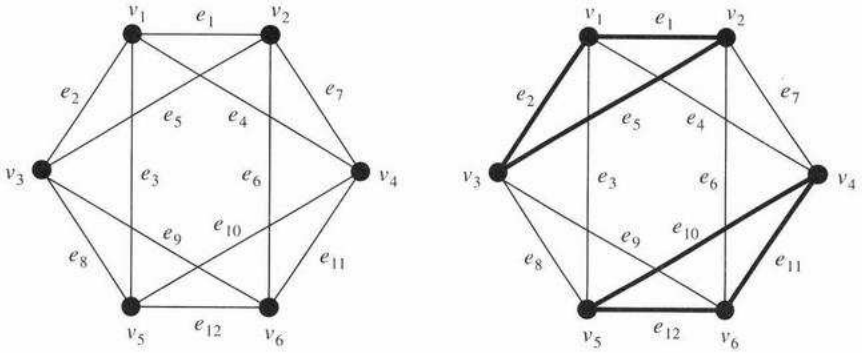


Figure 5.7 A graph to illustrate the ILP formulation of TSP (a) and an illegal solution consisting of two tours (b).

a tour that visits all the vertices. A necessary (but not sufficient) condition to achieve this is that exactly two edges of those incident to a vertex are selected. In the example of Figure 5.7(a), this means that the following constraints should be satisfied:

$$\begin{aligned} v_1 &: x_1 + x_2 + x_3 + x_4 = 2 \\ v_2 &: x_1 + x_5 + x_6 + x_7 = 2 \\ v_3 &: x_2 + x_5 + x_8 + x_9 = 2 \\ v_4 &: x_4 + x_7 + x_{10} + x_{11} = 2 \\ v_5 &: x_3 + x_8 + x_{10} + x_{12} = 2 \\ v_6 &: x_6 + x_9 + x_{11} + x_{12} = 2 \end{aligned} \tag{5.4}$$

These constraints are, however, not enough to enforce a tour as they do not exclude that ILP finds a solution that consists of multiple disjoint tours. An example of such an illegal solution that satisfies the constraints is shown in Figure 5.7(b). More constraints are necessary to avoid these illegal solutions. These additional constraints can be obtained by looking at bipartitions of the vertex set: some subset $V_1 \subset V$ and its complement $V_2 = V \setminus V_1$. A tour that visits all vertices in the graph should pass through at least two of the edges that connect a vertex in V_1 with a vertex V_2. If e.g. the subset $\{v_1, v_2, v_3\}$ is considered in the graph of Figure 5.7, it can be stated that a tour should pass through at least two of the vertices e_3, e_4, e_6, e_7, e_8 and e_9. Such a condition prevents the occurrence of the illegal solution shown in Figure 5.7(b).

So, an additional set of constraints should be generated systematically to prevent solutions containing multiple tours based on the subsets of the vertex set. A number of observations can be made in this respect:

- All relevant subsets V_1 should be taken into consideration in order to prevent illegal solutions.

- Both V_1 and V_2 should contain at least three vertices (no multiple tours are otherwise possible). Also, all vertices in any subgraph induced by either V_1 or V_2 should have at least a degree two. Otherwise, no multiple subtours covering all vertices are possible and it is therefore not necessary to have extra constraints to avoid them.

Taking these observations into account leads to the following relevant partitions and corresponding constraints for the graph of Figure 5.7:

$$\begin{aligned} &\{v_1, v_2, v_3\} + \{v_4, v_5, v_6\} : x_3 + x_4 + x_6 + x_7 + x_8 + x_9 \geq 2 \\ &\{v_1, v_3, v_5\} + \{v_2, v_4, v_6\} : x_1 + x_4 + x_5 + x_9 + x_{10} + x_{12} \geq 2 \\ &\{v_1, v_2, v_4\} + \{v_3, v_5, v_6\} : x_2 + x_3 + x_5 + x_6 + x_{10} + x_{11} \geq 2 \end{aligned} \tag{5.5}$$

In summary, Equations (5.2) through (5.5) define the ILP formulation of TSP for the graph of Figure 5.7. The formulation given is neither in the canonical form nor in the standard form, but it can easily be converted to any of the two forms using the "tricks" mentioned in Section 5.4.1.

An interesting issue is the size of the ILP formulation with respect to the size of the problem instance. In the example above, the number of variables is equal to the number of edges. The number of constraints of the type presented in Equation (5.4) is equal to the number of vertices. The number of constraints of the type given in Equation (5.5), however, can grow exponentially as the number of subsets of V equals $2^{|V|}$. If additional *real-valued* variables are allowed in the formulation, an alternative set of constraints can be constructed whose size is a polynomial function of the number of vertices. Such a formulation will not be presented here (see the Bibliographic Notes at the end of the chapter for pointers to the literature). Linear programming that contains both integer as well as real variables is called *mixed integer linear programming*.

ILP can be used for a large range of combinatorial optimization problems. An important reason why ILP *cannot* directly be used is a nonlinear cost function. The unit-size placement problem is an example of such a problem. Its cost function is an estimation of the wire length and there is no obvious way to relate e.g. cell locations to wire lengths using linear operators (the estimation methods mentioned in Section 7.2 use operations like squaring, minimum, maximum and absolute value).

5.5 Local Search

Local search is a general-purpose optimization method that works with fully-specified solutions $f \in F$ of a problem instance (F, c). It makes use of the notion of a *neighborhood* $N(f)$ of a feasible solution f, which consists of a subset of F that is "close" to f in some sense. More formally, a neighborhood is a function that assigns a number of feasible solutions to each feasible solution: $N : F \rightarrow 2^F$. Here, 2^F denotes the *power set* of F, the set of all its subsets. Any $g \in N(f)$ is called a *neighbor* of f (with respect to the neighborhood N).

```
local_search()
{
  struct feasible_solution f;
  set of struct feasible_solution G;

  f ← initial_solution();
  do {
     G ← {g|g ∈ N(f), c(g) < c(f)};
     if (G ≠ ∅)
        f ← "any element of G";
  } while (G ≠ ∅);
  "report f";
}
```

Figure 5.8 The pseudo-code description of local search.

Consider for example the unit-size placement problem in which each cell has to be assigned two coordinates with the goal of minimizing the estimated wire length. The neighborhood of a feasible solution f could consist of those solutions in F that can be obtained from f by exchanging the coordinates of two cells. Supposing that the instance has n cells, each feasible solution has a neighborhood of $\frac{n(n-1)}{2}$ elements, while the size of the complete search space is in the order of $n!$. More complex neighborhoods could be obtained by selecting three or more cells and interchanging their coordinates. In an extreme case, a neighborhood involving all n cells could be defined, which means that the neighborhood of each feasible solution $f \in F$ consists of the complete search space F. In such a case, as will become clear later, local search becomes equivalent to exhaustive search.

The principle of local search is to subsequently visit a number of feasible solutions in the search space, each solution being in the neighborhood of the previous one. Such a transition from one solution to the next is called a *move* or a *local transformation*. One starts with a feasible solution f and then moves to some $g \in N(f)$, such that $c(g) < c(f)$. This is repeated until some feasible solution is found that is cheaper than all its neighbors. The pseudo-code presenting this process is given in Figure 5.8. It is assumed that feasible solutions can be represented by the data structure `feasible_solution`. The function `initial_solution` has the task to generate a feasible solution. This could be done randomly or by some problem-specific heuristic. The moves that transform the initial solution into the final one are executed in the body of the `do` loop. In a practical implementation, it is not always necessary to generate the complete set G of all cheaper neighbors of f, unless one is specifically interested in selecting the cheapest one as the next value of f. A strategy that moves to the first cheaper neighbor encountered is called *first improvement* and the one moving to the cheapest one in G is called *steepest descent*. The latter strategy clearly requires a larger computational effort than the former one. It may still be interesting because of the belief that the final solution will be reached after fewer iteration steps or that a better quality solution will be found. Neither of these

is guaranteed, though.

The effectiveness of local search depends on the "shape" of the cost function c. If the function has a single minimum, it will be found sooner or later. However, most of the interesting combinatorial optimization problems have cost functions with many minima, most of which are local, while only one (or a few) are global. So, local search has the property that it can get stuck in a local minimum. This disadvantage cannot be fully avoided unless the extreme neighborhoods that coincide with the whole search space are considered. In general, the larger the neighborhoods considered, the larger is the part of the search space explored and the higher is the chance of finding a solution with good quality. However, the investigation of larger neighborhoods also requires a larger computational effort.

One can generalize local search in several ways. One possibility is to repeat the search a number of times with different initial solutions. In this way, it becomes more likely that larger parts of the search space will be explored and multiple minima will be encountered. Another possibility is to adapt the size of the neighborhood during local search based on properties of the feasible solution that one is visiting.

In order to circumvent the main disadvantage of local search, viz. the fact that it will get stuck in a local minimum, one should be able to move to a solution with a higher cost, by means of so-called *uphill moves*. However, this should be done in a way that still guarantees convergence to some solution. The methods *simulated annealing* and *tabu search*, to be discussed in the next sections, are examples of methods operating in this way.

5.6 Simulated Annealing

Simulated annealing (sometimes also called *statistical cooling*) performs a computation that is analogous to a physical process. In the process concerned, a material is first heated up to a temperature that allows all its molecules to move freely around (the material becomes liquid), and is then cooled down very slowly. The freedom of movement for the molecules decreases gradually until all the molecules take a fixed position. At the end of the process, the total energy of the material is minimal provided that the cooling is very slow. The idea of applying this analogy to combinatorial optimization was first published by Kirkpatrick *et al.* in an article that included applications from VLSI design automation (placement and global routing) as examples.

The analogy with the physical model has the following points of correspondence with a combinatorial optimization problem:

- The energy corresponds to the cost function.
- The movement of the molecules corresponds to a sequence of moves in the set of feasible solutions.
- The temperature corresponds to a control parameter T which controls the acceptance probability for a move from $f \in F$ to $g \in N(f)$. A good move, i.e. a

```
int accept(struct feasible_solution f, g)
{
  float Δc;

  Δc ← c(g) − c(f);
  if (Δc ≤ 0)
    return 1;
  else return (e^(−Δc/T) > random(1));
}

simulated_annealing()
{
  struct feasible_solution f, g;
  float T;

  f ← initial_solution();
  do {
     do {
        g ← "some element of N(f)";
        if (accept(f, g))
           f ← g
     while (!thermal_equilibrium());
     T ← new_temperature(T);
  while (!stop());
  "report f";
}
```

Figure 5.9 A pseudo-code description of simulated annealing.

move for which $c(g) \leq c(f)$, is always accepted irrespective of the value of T. A bad move, for which $c(g) > c(f)$, is accepted with a probability $e^{\frac{-\Delta c}{T}}$, where $\Delta c = c(g) - c(f)$. So, for high values of T nearly all bad moves are accepted, while hardly any bad move is accepted when T is low. (The Boltzmann distribution in statistical mechanics states that the number of molecules N_1 with energy level ϵ_1 divided by the number of molecules N_0 with energy level ϵ_0 equals $e^{\frac{-\Delta\epsilon}{kT}}$, where $\Delta\epsilon = \epsilon_1 - \epsilon_0$, k is the Boltzmann constant and T is the absolute temperature.)

The algorithm itself consists of an outer loop in which the temperature is gradually lowered and an inner loop in which the configuration is randomly perturbed by moves that are either accepted or rejected. The inner loop should be executed a number of times large enough to reach "thermal equilibrium" before going back to the outer loop. This is shown in the pseudo-code description given in Figure 5.9. The strategy for accepting or rejecting a move is represented by the function `accept`, in which the function `random(`k`)` generates a real-valued random number between 0 and k with a uniform distribution. (Check that the code of `accept` correctly implements the acceptance strategy described above.) The function `thermal_equilibrium`

should only return a value unequal to zero if the inner loop has been executed a "sufficient" number of times. This number is normally a function of the problem instance size. The function `new_temperature` computes a new, lower, value for the temperature to be used for the next execution of the inner loop. In practice, this is often implemented by a multiplication of T by a constant between 0 and 1. The function `stop`, finally, decides about the termination of the search. One possibility is to stop when none of the moves in the inner loop has been accepted. Note that simulated annealing may visit an optimal solution and then move away from it. It is therefore wise to record the best solution in a separate variable and report its value at the end of the search instead of the final value of f.

The combination of the functions `thermal_equilibrium`, `new_temperature` and `stop` realizes a strategy for simulated annealing, which is called the *cooling schedule*. Theoretical analysis shows that the cooling schedule can be chosen in such a way that the probability of finding the global minimum becomes equal to one. However, these schedules imply an infinite number of moves before stopping. In practice, one is interested in finding a solution as fast as possible and uses cooling schedules that cannot guarantee an optimal solution.

Simulated annealing has been applied with varying success to almost any combinatorial optimization problem in VLSI design automation. Especially in the case of the placement problem (see Chapter 7), it has proved itself as one of the best methods, if not the overall best one. On the other hand, when good tailor-made heuristics are available, they are often preferable to simulated annealing: they obtain better quality results in a fraction of the time needed for simulated annealing. An adapted version of the Kernighan-Lin algorithm for graph partitioning (see Section 7.5.1), for example, clearly outperforms simulated annealing when applied to circuit partitioning (a problem that is somewhat more complex than graph partitioning).

5.7 Tabu Search

Simulated annealing allows many uphill moves at the beginning of the search and gradually decreases their frequency. In this way, a convergence mechanism is imposed to the search. The *tabu search* method, on the other hand, does not directly restrict uphill moves throughout the search process. Given a *neighborhood subset* $G \subseteq N(f)$ of a feasible solution f, the principle of tabu search is to move to the cheapest element $g \in G$ *even when* $c(g) > c(f)$. In order to avoid a circular search pattern, a so-called *tabu list* containing the k last visited feasible solutions is maintained. Transitions to these solutions are prohibited (they are *taboo*, hence the name of the method). This only helps, of course, to avoid cycles of length $\leq k$.

A pseudo-code description of tabu search is given in Figure 5.10. In the description, the tabu list is represented by the FIFO (first-in first-out) queue Q of length k. The variable b is used to store the best solution encountered in the search process. The function `stop` decides when to terminate the search, e.g. when no improvement on the best solution is found in the last n iterations.

```
tabu_search()
{
  struct feasible_solution f, g, b;
  set of struct feasible_solution G;
  "k-element FIFO queue of" feasible_solution Q;

  Q ← "empty";
  b ← initial_solution();
  f ← initial_solution();
  do {
     G ← "some subset of N(f) such that ∀s ∈ Q, s ∉ G";
    if (G ≠ ∅) {
       g ← "cheapest element of G";
       "shift g into Q";
       f ← g;
       if (c(f) < c(b))
         b ← f;
     }
  }
  while (G ≠ ∅ or stop());
  "report b";
}
```

Figure 5.10 A pseudo-code description of tabu search.

Consider as an example an instance of the unit-size placement problem, where 10 000 cells have to be placed on grid points identified by 100 x-coordinates and 100 y-coordinates. Suppose that a move selects two cells and exchanges their places. For such a problem, the quantity of data to be stored for each solution kept in the tabu list is quite high: the 10 000 coordinate pairs that define a feasible solution. The data to be stored can be drastically reduced by e.g. only storing the two cells that have been moved to arrive at the solution, which implies that these two cells will not be moved during the next k iterations. The price for the storage reduction is a reduction of the search space: many feasible solutions that would not given rise to a circular search pattern will also remain out of consideration as long as the cells are kept in the taboo list.

The tabu search method described above is the most elementary form of the many variations possible. Actually, tabu search should be seen as a large family of optimization methods offering (and sometimes requiring) extensive adaptation to the combinatorial optimization problem at hand. An unsatisfactory aspect of tabu search is that no theoretical analysis exists to guide the choice for the parameter k (the tabu list size) and the stopping criterion: they should be determined empirically.

5.8 Genetic Algorithms

In this section a simple version of search by *genetic algorithms* is presented. Many variations on this version are possible and some of them will be mentioned at the end of this section.

As in local search and its related methods, a genetic algorithm also works with fully specified solutions f included in the set of feasible solutions F. However, instead of repetitively transforming a single current solution into a next one by the application of a move, the algorithm simultaneously keeps track of a set P of feasible solutions, called the *population*. In an iterative search process, the current population $P^{(k)}$ is replaced by the next one $P^{(k+1)}$ using a procedure that is characteristic for genetic algorithms.

In order to generate a feasible solution $f^{(k+1)} \in P^{(k+1)}$, two feasible solutions $f^{(k)}$ and $g^{(k)}$, called the *parents* of the *child* $f^{(k+1)}$, are first selected from $P^{(k)}$. $f^{(k+1)}$ is generated in such a way that it *inherits* parts of its "properties" from one parent and the other part from the second parent by the application of an operation called *crossover*. First of all, this operation assumes that all feasible solutions $f \in F$ can be encoded by a fixed length vector $\mathbf{f} = [f_1 f_2 \ldots f_n]^T \equiv f$ as was the case for the backtracking algorithm discussed in Section 5.2. In their basic form, genetic algorithms use bit strings to represent feasible solutions. This not only implies that the number of vector elements n is fixed, but that the number of bits to represent the value of each element f_i $(1 \leq i \leq n)$ is fixed as well. The string of bits that specifies a feasible solution in this way, is called a *chromosome*. Consider an instance of the unit-size placement problem with 100 cells and a 10×10 grid. As 4 bits are necessary to represent one coordinate value (each value is an integer between 1 and 10) and 200 coordinates (100 coordinate pairs) specify a feasible solution, the chromosomes of this problem instance have a length of 800 bits.

It is important to be aware of the distinction between a feasible solution (in biological terms, the *phenotype*) and its encoding as a chromosome (its *genotype*). In the rest of this section, however, an $f \in F$ that is formally the phenotype, is sometimes used to denote the genotype as well in order to avoid the introduction of a more complex notation.

Given two chromosomes, a crossover operator will use some of the bits of the first parents and some of the second parent to create a new bit string representing the child. A simple crossover operator works as follows:

- Generate a random number r between 1 and the length l of the bit strings for the problem instance.
- Copy the bits 1 through $r - 1$ from the first parent and the bits r through l from the second parent into the bit string for the child. Sometimes, it is customary to generate a second child using the same r, now reversing the roles of both parents when copying the bits.

The generation of a pair of children $f^{(k+1)}$ and $g^{(k+1)}$ from a pair of parents $f^{(k)}$ and $g^{(k)}$ is illustrated in Figure 5.11. The example also illustrates a complication

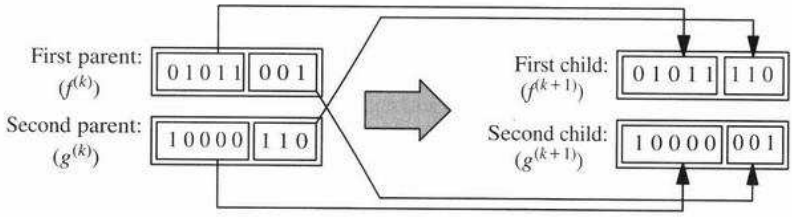


Figure 5.11 The generation of a pair of children by crossover.

that arises in genetic algorithms due to the encoding of the solution as a bit string. Suppose that the bit strings of the example represent the coordinates of the placement problem on a 10×10 grid mentioned earlier, now with only a single cell to place (an artificial problem). The bit string for a feasible solution is then obtained by concatenating the two 4-bit values of the coordinates of the cell. So, $f^{(k)}$ is a placement on position (5, 9) and $g^{(k)}$ one on position (8, 6). The children generated by crossover represent placements at respectively (5, 14) and (8, 1). Clearly, a placement at (5, 14) is illegal: it does not represent a feasible solution as coordinate values cannot exceed 10.

One solution to this complication is to use strings that consist of more sophisticated data structures than single bits. A chromosome could then be represented as a vector of e.g. integers or characters. In this way, the complication mentioned in the previous paragraph cannot occur. The combination of the chromosome representation and the crossover operator for generating new feasible solutions, leads, however, to more complications. Consider e.g. the traveling salesman problem for which each of the feasible solutions can be represented by a permutation of the cities (at least in those versions of the problem where there is a direct connection between each pair of cities, such as in the Euclidean version). Two example chromosomes for a six-city problem instance with cities c_1 through c_6 could then look like "$c_1c_3c_6c_5c_2c_4$" and "$c_4c_2c_1c_5c_3c_6$". In such a situation, the application of the crossover operator as described for binary strings is very likely to produce solutions that are not feasible. Performing crossover on the two chromosomes just mentioned by cutting them after the second city would lead to the illegal solution "$c_1c_3c_1c_5c_3c_6$" (or "$c_4c_2c_6c_5c_2c_4$") that is not a permutation of all cities in the problem instance.

This behavior can be avoided by using special-purpose crossover operators tailored for chromosomes that represent permutations. One such operator is called *order crossover*. This operator copies the elements of the first parent chromosome until the point of the cut into the child chromosome. The remaining part of the child is composed of the elements missing in the permutation in the *order* in which they appear in the second parent chromosome. Consider again the chromosomes "$c_1c_3c_6c_5c_2c_4$" and "$c_4c_2c_1c_5c_3c_6$" cut after the second city. Then the application of order crossover would lead to the child "$c_1c_3c_4c_2c_5c_6$" (reversing the roles of the first and second parents would lead to: "$c_4c_2c_1c_3c_6c_5$").

The aspects of genetic algorithms that have been introduced until now do not

```
genetic()
{
  int pop_size;
  set of struct chromosome pop, newpop;
  struct chromosome parent1, parent2, child;

  pop ← ∅;
  for (i ← 1; i ≤ pop_size; i ← i + 1)
    pop ← pop ∪ {"chromosome of random feasible solution"};
  do {
     newpop ← ∅;
     for (i ← 1; i ≤ pop_size; i ← i + 1) {
       parent1 ← select(pop);
       parent2 ← select(pop);
       child ← crossover(parent1, parent2);
       newpop ← newpop ∪ {child};
     }
     pop ← newpop;
  } while (!stop());
  "report best solution";
}
```

Figure 5.12 The pseudo-code description of a genetic algorithm.

lead to any optimization. One needs to favor good solutions above bad solutions in some way. This is done by giving a stronger preference to parents with a lower cost when selecting pairs of parents to be submitted to the crossover operator. So, the better the cost of some feasible solution, the higher the chances that it will be selected for reproduction. Children generated in this way combine "good" features of their parents and lead to improved solutions. The reverse, children that mainly inherit "bad" features of their parents, can also happen. Those children are, however, unlikely to survive the transition to the next generation.

The pseudo-code description of search by genetic algorithms presented in Figure 5.12 summarizes the concepts introduced above and gives some more details as well. Note that the description in the figure deals with chromosomes that are manipulated, not the feasible solutions themselves. In the main loop of the code, two parents at a time are selected and used to generate one new child in the new population. The function `select` is responsible for the selection of feasible solutions from the current population favoring those that have a better cost. The function `crossover` actually generates a new child from two parent chromosomes. The function `stop` decides when to terminate the search, e.g. when there has been no improvement on the best solution in the population during the last m iterations, where m is a parameter of the algorithm.

A final main feature of genetic algorithms is *mutation*. This is a phenomenon also encountered in nature, viz. that errors can be made during the copying of a chromosome from a parent to the child (this happens inside the procedure `crossover` in

the version described here). Mutation helps to avoid getting stuck in a local minimum.

Many variations are possible to the scheme presented above. Examples are:

- One can work with more sophisticated crossover operators, e.g. operators that make multiple cuts in a chromosome.
- One can copy some members of the population entirely to the new generation instead of generating new children from them.
- Instead of distinguishing between the populations $P^{(k)}$ and $P^{(k+1)}$, one could directly add a new child to the population and simultaneously remove some "weak" member of the population. This allows for the coexistence of different generations as is often the case in nature.

5.9 A Few Final Remarks on General-purpose Methods

Quite a variety of general-purpose methods for combinatorial optimization has been presented in this chapter. Even more methods exist, such as *neural networks* and *simulated evolution*, to name just two methods that have sometimes successfully been applied to problems in VLSI design automation. Yet another method, based on the *satisfiability* problem, is discussed in Chapter 11 in the context of Boolean function manipulation.

The advantage of using a general-purpose method for some problem is clearly in the reduction in algorithm design time as compared to development time for a special-purpose method. Sometimes, a general-purpose method directly leads to a powerful solution method that cannot be outperformed by any special-purpose method. This is e.g. the case for placement by simulated annealing. It may, however, be that the general-purpose method gives unsatisfactory results. This is often due to the loss of problem-specific knowledge that occurs when trying to reformulate a specific problem in terms of the general-purpose method. Genetic algorithms, for example, require a feasible solution to be represented by a chromosome which is a linear data structure, whereas a more complex data structure may be a more natural representation of the solution. Or, it may not always be easy to find "moves" for local search that do not create illegal solutions, due to the complexity of the representation on which the moves have to operate.

Sometimes good results can be obtained by a *hybrid* algorithm that has both general-purpose and problem-specific parts. One may e.g. have a problem-specific heuristic that can be controlled by means of some parameters (different solutions for a problem instance will be generated for different parameter values) and that has a low time complexity. Such a heuristic should have the property that any interesting part of the search space, and especially the part that contains the optimal solution, can be reached by the correct choice of the parameter values. Besides, the parameter values themselves should be simple data structures, such as integers or the permutations of a finite set. A general-purpose method can then be used to find those

parameter values that will lead to a good solution by repetitively calling the heuristic with different values. What has been achieved is that the heuristic has replaced the complex data structures required to adequately represent a feasible solution, by a simple data structure suitable for a successful application of the general-purpose method. One can also say that the heuristic has become a sophisticated cost function that calculates the cost of a feasible solution represented by the parameters.

5.10 Bibliographic Notes

A book that presents general-purpose heuristics for combinatorial optimization, including many of those presented in this chapter is [Ree95].

Backtracking and branch-and-bound are clearly explained in [Hor78]. Some of the terminology used here in the explanation of this topic also originates from this book. The idea of using the minimum spanning tree length as a lower bound for the unspecified part of the partial solution of a traveling salesman problem is mentioned in [Sed88].

Dynamic programming is a common technique that is discussed in many books on combinatorial optimization. A short but clear presentation is e.g. given in [Pap82]. The application of dynamic programming to TSP as presented in this chapter is based on this source.

Many books and review papers can be found on linear and integer linear programming. A readable introduction to these topics as well as some other issues in combinatorial optimization is given in [Fou84]. Other examples are [Hu69], [Pap82], [Nem88], and [Nem89]. These sources pay attention to ILP formulations of TSP including the one that uses mixed integer linear programming. All formulations are based on directed graphs as opposed to the version for undirected graphs discussed in Section 5.4.2. The NP-completeness of ILP is mentioned in [Gar79].

The discussion on local search given in this chapter is based on [Pap82]. The seminal paper on simulated annealing including its application to VLSI design automation is [Kir83]. A theoretical study for the optimal cooling schedule is provided in [Haj88]. Readers interested in an extended analysis of the convergence of simulated annealing are referred to books dedicated to the topic, like [Laa87], [Aar89] and [Ott89]. One of the best results of simulated annealing in VLSI design automation has been obtained by the Timberwolf placement program as reported in [Sec85] and [Sun95]. The comparison between the Kernighan-Lin algorithm and simulated annealing for the placement problem, as reported in this chapter, is made in [Nah89]. The book [Won88] is entirely dedicated to applications of simulated annealing to VLSI layout design problems. The book [Ban94] that deals with parallel algorithms for VLSI design automation, also covers a number of layout applications of simulated annealing.

A review of tabu search and possible variations is given in [Glo93]. A successful application of tabu search in VLSI design automation has been reported for the high-level synthesis scheduling problem in [Ame94].

The presentation of genetic algorithms in this chapter roughly follows the description given in [Gol89]. Other books on the topic are [Dav91] and [Mic92a]. Genetic algorithms have been applied to various combinatorial optimization problems arising in VLSI design automation. Interesting results have e.g. been reported for the standard-cell placement problem [Coh87, Sha90]. The results obtained are comparable in quality to simulated annealing [Sec85], but at the expense of longer run times. It is remarkable, though, that the number of feasible solutions considered by the genetic algorithm reported in [Sha90] is 10 to 50 times less than in simulated annealing (so the computational effort per feasible solution is relatively high).

More information on the use of neural networks for combinatorial optimization can be found in [Hop86], [Ram88], [Bou90], and [Phi95]. Examples of the application of simulated evolution to VLSI design automation are presented in [Kli89] and [Lin89]. The application of hybrid genetic algorithms to high-level synthesis has been reported in [Hei95] and [Hei96]. They have later been extended for *multiprocessor scheduling* in [Bon97].

6

Layout Compaction

As mentioned in Chapter 1, one of the goals in VLSI design is to minimize the area of the final circuit. Design decisions at all levels of abstraction have a consequence for the area of the final circuit. At the lowest level, the level of the mask patterns for the fabrication of the circuit, a final optimization can be applied to remove redundant space. This optimization is called *layout compaction.*

The main topic of this chapter is the so-called *constraint-graph compaction* method. Before introducing it, some attention is paid to the notions of *design rules* and *symbolic layout.* Then, the compaction problem is formulated in graph-theoretical terms for the purpose of constraint-graph compaction and algorithms to solve the problem are presented. The chapter concludes with a short presentation of related topics that cannot be covered in depth.

6.1 Design Rules

The mask patterns that are used for the fabrication of an integrated circuit have to obey certain restrictions on their shapes and sizes. These restrictions are called the *design rules.* Sticking to the design rules decreases the probability that the fabricated circuit will not work due to shortcircuits, disconnections in wires, parasitics, etc.

The shape of the patterns is often restricted to *rectilinear polygons*, i.e. polygons that are made of horizontal and vertical segments only (see Figure 6.1(a)). Some technologies also allow 45-degree segments in polygons, segments that are parallel

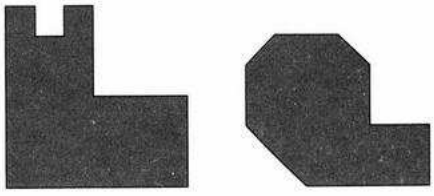


Figure 6.1 A rectilinear polygon (a) and one with 45-degree segments (b).

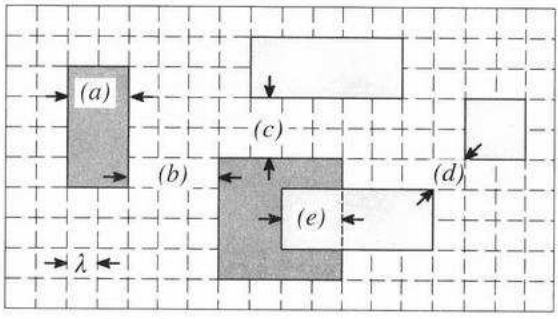


Figure 6.2 Minimum-distance design rules on a lambda grid: minimum width (a), minimum separation (b, c and d) and minimum overlap (e).

to the lines $y = x$ or $y = -x$ on an x-y plane (see Figure 6.1(b)). In the rest of this text, for reasons of simplicity, the mask patterns will be assumed to consist of rectangles only (a rectilinear polygon can be decomposed into a set of rectangles). There are design rules for layout elements located in the same fabrication layer and rules for elements in different layers. If patterns in two specific layers are constrained by one or more design rules, the layers are said to *interact*. For example, polysilicon and diffusion are interacting layers as their overlapping creates a transistor, whereas polysilicon and metal form noninteracting layers (if one ignores parasitic capacitances). Design rules can be quite complex. However, most of them can be expressed as *minimum-distance* rules.

As the minimum feature size that can be realized on a chip is subject to continual change (from several microns a few years ago to a few tenths of microns nowadays), distances are often expressed in integer multiples (or small fractions) of a relative length unit, the λ, rather than absolute length units. In this way, designers can deal with simple expressions independent of actual length values. This means that all mask patterns are drawn along the lines of a so-called *lambda grid* as shown in Figure 6.2.

The most common types of minimum-distance rules are:

- *Minimum width:* a pattern in a certain layer cannot be narrower than a certain distance (see Figure 6.2(a)).
- *Minimum separation:* two patterns belonging to the same (see Figure 6.2(b)) layer or to different but interacting layers (see Figure 6.2(c)) cannot be positioned closer to each other than a certain distance; this is also true when the rectangles are diagonally separated (see Figure 6.2(d)).
- *Minimum overlap:* a pattern in one layer located on top of a pattern in another interacting layer, should have a minimal overlap (see Figure 6.2(e)).

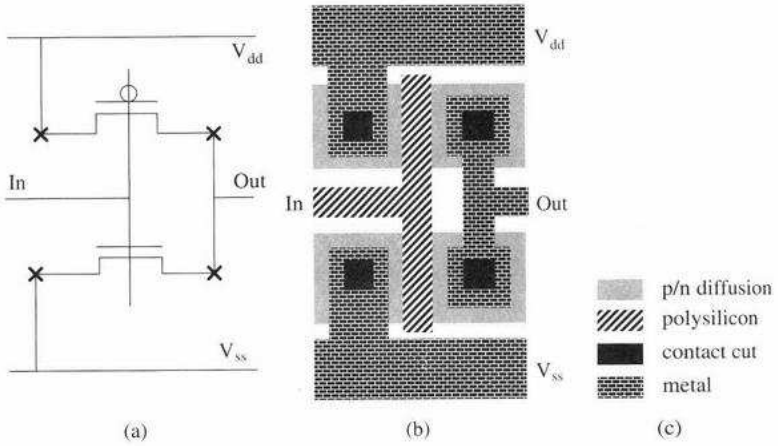


Figure 6.3 The symbolic layout of a CMOS inverter (a), its geometric layout (b) and a legend explaining the layers in the geometric layout (c).

6.2 Symbolic Layout

The existence of design rules makes the design of layout very cumbersome; there are many design rules and a human designer can overlook one of them very easily when fixing the position of a certain rectangle. Although special tools, so-called *design-rule checkers*, exist for detecting this type of mistakes, the mistake might not be easy to correct, requiring one to reposition many other rectangles in the neighborhood of the wrong one. As a remedy to this problem, it has been proposed not to design the *geometry* of the layout directly, but only to fix its *topology* instead. In this context, geometry means that the coordinates of the rectangles are absolute (or in multiples of λ), and topology means that only relations between layout elements, such as "to the left of" or "below", are known. In the rest of this chapter, a distinction will be made between *geometric* or *mask* layout on one hand, and *symbolic* or *topological* layout on the other hand.

The entities of which a symbolic layout consists are, not surprisingly, symbols: there is a symbol for a transistor, a wire in a specific layer, a contact cut, etc. In a symbolic layout, the interconnections of these symbols fix the topology of the circuit: the relative positions of transistors, the shapes of wires, etc. are known. The symbolic layout of a simple CMOS inverter is given in Figure 6.3(a).

Symbolic layout can normally be created interactively on a graphics computer screen, by means of a *symbolic layout editor* or it can be specified in textual form, by means of a formal *layout language*.

The advantages of symbolic layout can only be exploited if tools exist for automatically converting symbolic layout into geometric layout. Such a tool should e.g. accept some description of Figure 6.3(a) together with a *technology file* that contains all design rule information for the target technology, and produce the layout of Figure 6.3(b) (the shading patterns used to identify the different layers are given in

Figure 6.3(c)). The conversion from symbolic to geometric layout is one of the applications of layout compaction. Some other applications are mentioned in the next section.

6.3 Problem Formulation

6.3.1 Applications of Compaction

Layout compaction can be applied in four situations, of which the first two have already received some attention.

- Converting symbolic layout to geometric layout.
- Removing redundant area from geometric layout.
- Adapting geometric layout to a new technology. A new technology means that the design rules have changed; as long as the new and old technologies are compatible (e.g. both are CMOS technologies with the same mask layers), this adaptation can be done automatically, e.g. by means of so-called *mask-to-symbolic extraction*. In such a case geometric layout in the old technology is converted to a symbolic layout and then the design rules of the new technology are used for the generation of the new geometric layout.
- Correcting small design rule errors. If there are methods to put layout elements closer to each other to remove redundant space, it is plausible to assume that pulling layout elements apart when they are too close to each other can be done similarly. This is true as long as the layout with design-rule errors is topologically correct: the relative ordering of the rectangle edges in interacting layers should be the same as in the correct design.

6.3.2 Informal Problem Formulation

As mentioned before, a layout is considered to consist of rectangles. However, not all rectangles are the same. Basically, the rectangles can be classified into two groups: *rigid* rectangles and *stretchable* rectangles. Rigid rectangles correspond to transistors and contact cuts whose length and width are fixed (as a consequence of the technology or the wishes of the designer). When they are moved during a compaction process, their lengths and widths do not change. Stretchable rectangles correspond to *wires*. In principle the width of a wire cannot be modified (a designer may have reasons to make a wire wider than strictly necessary in order to decrease its resistance). The length of a wire, however, can be changed by compaction (the wire connecting two contact cuts, for example, should become shorter if the contact cuts are moved closer to each other).

Layout is essentially two-dimensional and layout elements can in principle be moved both horizontally and vertically for the purpose of compaction. When *one-dimensional compaction* tools are used, the layout elements are only moved along

one direction (either vertically or horizontally). This means that the tool has to be applied at least twice: once for horizontal and once for vertical compaction. *Two-dimensional compaction* tools move layout elements in both directions simultaneously. Theoretically, only two-dimensional compaction can achieve an optimal result. However, this type of compaction is NP-complete. On the other hand, one-dimensional compaction can be solved optimally in polynomial time as is shown later on in this chapter. Actually, repeated one-dimensional compaction can be considered a straightforward but valuable heuristic for two-dimensional compaction.

This idea is illustrated in Figure 6.4. The layout in the figure consists of nine squares labeled A to I, each with a size of $2\lambda \times 2\lambda$. These squares are rigid and are interconnected by wires of the same material. The design rules are such that the minimum separation between rectangles of the same material is 1λ. Figure 6.4(a) shows the starting situation with redundant spacing that covers an area of $11\lambda \times 11\lambda$. If the layout is first compacted horizontally from right to left and then vertically from top to bottom, the $8\lambda \times 11\lambda$ solution of Figure 6.4(c) is obtained. If the layout is first compacted vertically from top to bottom and then horizontally from right to left, the $11\lambda \times 8\lambda$ solution of Figure 6.4(e) is obtained. The optimal solution that has an area of $8\lambda \times 8\lambda$ is shown in Figure 6.4(f). In order to obtain this solution from the starting position in one pass, the layout elements must be moved in two dimensions. Note that the optimal result could be obtained if the solution of Figure 6.4(c) is compacted from left to right and then once more from top to bottom. More complex examples can be constructed, however, in which iterated one-dimensional compaction will never obtain the optimal result of two-dimensional compaction.

As mentioned above, two-dimensional compaction is NP-complete and exact as heuristic algorithms to solve the problem can be quite complex. Most practical compaction tools are based on repeated one-dimensional compaction and therefore algorithms for this case will be presented in the rest of this chapter. For this purpose, the compaction problem is first formulated in graph-theoretical terms in the next two sections.

6.3.3 *Graph-theoretical Formulation*

In one-dimensional, say horizontal, compaction a rigid rectangle can be represented by one x-coordinate (of its center, for example) and a stretchable one by two (one for each of the endpoints). Figure 6.5 shows some rectangles that are horizontally stretchable.

For the purpose of the algorithms to be explained, it is assumed that there are n distinct x-coordinates. They will be indicated as $x_1, x_2, \ldots, x_n$. A minimum-distance design rule between two rectangle edges can now be expressed as an inequality:

$$x_j - x_i \geq d_{ij} \tag{6.1}$$

In the example of Figure 6.5, assuming that the minimum width for the layer concerned is a and the minimum separation is b, the following (and many other)

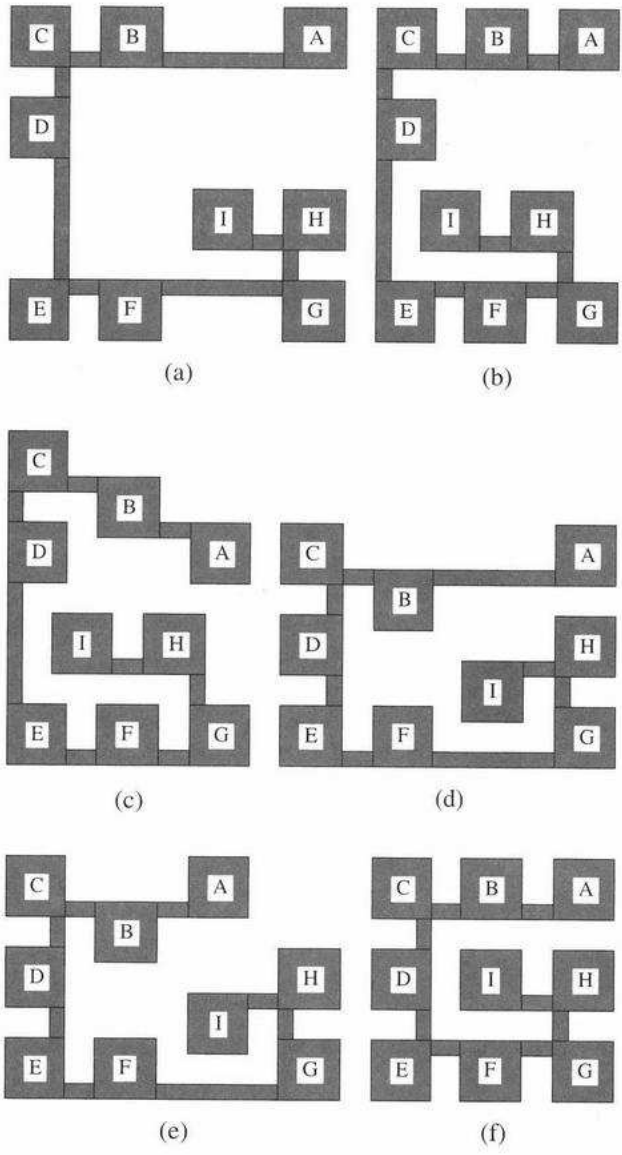


Figure 6.4 A layout with redundant space (a); the compaction result after horizontal compaction (b) followed by vertical compaction (c); the result after vertical (d) followed by horizontal compaction (e); the optimally compact result (f).

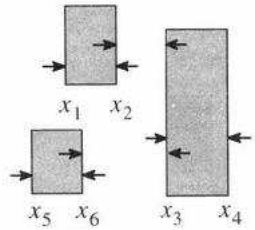


Figure 6.5 The pairs of variables associated with horizontally stretchable rectangles.

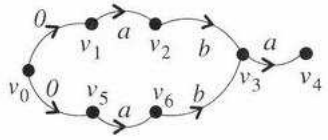


Figure 6.6 The constraint graph for the example of Figure 6.5.

inequalities are valid:

$$x_2 - x_1 \geq a$$
$$x_3 - x_2 \geq b$$
$$x_3 - x_6 \geq b$$

It is now possible to represent all these inequalities in a so-called *constraint graph* $G(V, E)$, constructed in the following way:

- The vertex set V is composed by associating a vertex v_i with each variable x_i that occurs in an inequality.
- The edge set E is composed of edges (v_i, v_j) with weight $w((v_i, v_j)) = d_{ij}$ for each inequality $x_j - x_i \geq d_{ij}$.
- There is a source vertex v_0, located at $x = 0$. So, there are $n + 1$ vertices in total: $v_0, v_1, \ldots, v_n$. All layout elements are assumed to have a positive x-coordinate. This is incorporated in the graph by edges from the source vertex to those vertices that do not have any other vertices constraining them at the left.

The constraint graph for the example of Figure 6.5 has been given in Figure 6.6.

A constraint graph derived from only minimum-distance constraints has no cycles (why?). It is called a *directed acyclic graph*, often denoted by the abbreviation *DAG*. The following observation can be made: the length of the longest path from the source vertex v_0 to a specific vertex v_i in a the constraint graph $G(V, E)$ gives the minimal x-coordinate x_i associated to that vertex. By taking the longest path to v_i, one makes sure that all inequalities in which x_i participates are satisfied. So, computing the lengths of the longest paths to *all* vertices in the constraint graph

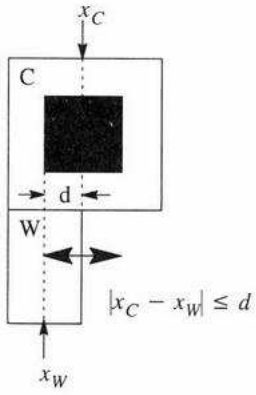


Figure 6.7 Maintaining a wire connected to a contact cut gives rise to a maximum distance constraint.

results in a solution for the one-dimensional compaction problem. An algorithm to compute these longest paths efficiently is given in Section 6.4.1. Prior to the presentation of that algorithm, a more general case of the compaction problem is formulated in the next section.

6.3.4 Maximum-distance Constraints

Some situations necessitate the limitation of the maximum distance between the coordinates of layout elements. An example of such a situation is a connectivity constraint: moving an element too far away with respect to another can break an electrical connection. An illustration for the case of a wire segment connected to a contact cut is given in Figure 6.7. The situation in the figure gives rise to two constraints: $x_C - x_W \leq d$ and $x_W - x_C \leq d$.

Maximum-distance constraints can in general be written as:

$$x_j - x_i \leq c_{ij}$$

where $c_{ij} \geq 0$. This can also be written as:

$$x_i - x_j \geq -c_{ij}$$

The last inequality has the same form as Inequality (6.1) and can be represented in the constraint graph by an edge (v_j, v_i) with weight $d_{ij} = -c_{ij}$. The addition of this type of edges can create cycles in the constraint graph. In the presence of cycles, the solution of the compaction problem still amounts to computing the lengths of the longest paths. However, the problem is more difficult than finding the longest path in a DAG as will become clear below.

```
longest-path(G)
{
  for (i ← 1; i ≤ n; i ← i + 1)
    p_i ← "in-degree of v_i";
  Q ← {v_0};
  while (Q ≠ ∅) {
    v_i ← "any element from Q";
    Q ← Q \{v_i};
    for each v_j "such that" (v_i, v_j) ∈ E {
      x_j ← max(x_j, x_i + d_ij);
      p_j ← p_j − 1;
      if (p_j ≤ 0)
        Q ← Q ∪ {v_j};
    }
  }
}

main ()
{
  for (i ← 0; i ≤ n; i ← i + 1)
    x_i ← 0;
  longest-path(G);
}
```

Figure 6.8 A longest-path algorithm for DAGs.

6.4 Algorithms for Constraint-graph Compaction

In this section, algorithms for constraint-graph compaction are presented. This type of compaction amounts to finding the longest path in a directed graph as was discussed above. First an algorithm for the simpler case of the longest path in a DAG is given. Then two different algorithms for graphs with cycles are explained.

6.4.1 A Longest-path Algorithm for DAGs

The longest-path problem for DAGs can be solved efficiently by an algorithm that is quite similar to breadth-first search (see Section 3.4.2). A description in pseudo-code of an algorithm that computes the longest paths from the source vertex v_0 of a graph $G(V, E)$ to any other vertex is shown in Figure 6.8. A variable p_i is associated with each vertex v_i to keep count of the edges incident to v_i that have already been processed. Because the graph is acyclic, once all incoming edges have been processed, the longest path to v_i is known. Then v_i is included in a set Q. It will be taken out later on to traverse the edges incident from it in order to propagate the longest-path values to the vertices at their endpoints. The data structure used to implement Q is left open here. Any data structure that is able to implement the semantics of a "set" can be used. The two actions to be supported are: to add

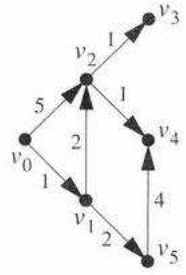


Figure 6.9 A directed acyclic graph.

Q	p_1	p_2	p_3	p_4	p_5	x_1	x_2	x_3	x_4	x_5
"not initialized"	1	2	1	2	1	0	0	0	0	0
$\{v_0\}$	0	1	1	2	1	1	5	0	0	0
$\{v_1\}$	0	0	1	2	0	1	5	0	0	3
$\{v_2, v_5\}$	0	0	0	1	0	1	5	6	6	3
$\{v_3, v_5\}$	0	0	0	1	0	1	5	6	6	3
$\{v_5\}$	0	0	0	0	0	1	5	6	7	3
$\{v_4\}$	0	0	0	0	0	1	5	6	7	3

Figure 6.10 The evolution of the variables in the longest-path algorithm for DAGs when applied to the graph of Figure 6.9.

a new element and to remove an arbitrary element. Note: the initialization of the variables x_i has been put outside the procedure body as this makes it possible to use `longest-path` as part of another algorithm that will be presented later.

The longest-path algorithm presented has a time complexity $\mathcal{O}(|E|)$. This is easy to see: all edges in the graph are visited exactly once during the execution of the inner for-each loop.

Consider the DAG given in Figure 6.9. The evolution of the variable values throughout the different iterations of the while loop is given in Figure 6.10. The value given for the set Q is the value at the beginning of the iteration. The first element is removed. The remaining elements in the same row are the updated values after processing this first element.

6.4.2 *The Longest Path in Graphs with Cycles*

Before presenting algorithms that compute the longest path in graphs with cycles, some general remarks on this case will be made in this section. Two cases can be distinguished:

1. The graph only contains *negative* cycles, i.e. the sum of the edge weights along any cycle is negative.

```
count ← 0;
for (i ← 1; i ≤ n; i ← i + 1)
  x_i ← −∞;
x_0 ← 0;

do { flag ← 0;
     longest-path(G_f);
     for each (v_i, v_j) ∈ E_b
       if (x_j < x_i + d_ij) {
         x_j ← x_i + d_ij;
         flag ← 1;
       }
     count ← count +1;
     if (count > |E_b| && flag)
       error("positive cycle")
}
while (flag);
```

Figure 6.11 The Liao-Wong compaction algorithm.

2. The graph contains *positive* cycles.

The problem for graphs with positive cycles is NP-hard. However, a constraint graph with positive cycles corresponds to a layout with conflicting constraints (in which e.g. the minimum distance for two coordinates exceeds the maximum one). Such a layout is called *overconstrained* and is impossible to realize. So, the best to be done in such a case is to detect the existence of positive cycles. They can be detected in polynomial time. In fact, the two algorithms to be described for the longest-path problem for graphs with negative cycles can detect the existence of positive cycles (but not localize them). The two algorithms presented next, the Liao-Wong algorithm and the Bellman-Ford algorithm, are not the only algorithms known for the computation of longest paths in directed graphs. They have been selected because they have successfully been applied to layout compaction.

6.4.3 The Liao-Wong Algorithm

Liao and Wong have proposed an algorithm that partitions the edge set E of the constraint graph $G(V, E)$ into two sets E_f and E_b. The edges in E_f have been obtained from the minimum-distance inequalities and are called *forward* edges. The edges in E_b correspond to maximum-distance inequalities and are called *backward* edges (they create cycles by going backward).

The pseudo-code for the algorithm is given in Figure 6.11. The main idea is to start with $G_f(V, E_f)$ which is acyclic and to which the DAG longest-path algorithm of Figure 6.8 can be applied. Then the backward edges are considered and modifications to the minimal x-coordinates are made, followed by a call to the DAG longest-path algorithm to propagate the effects of the modifications through the forward

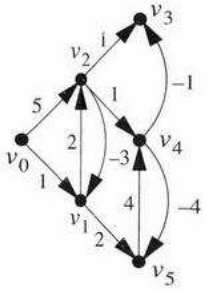


Figure 6.12 A directed graph with "backward" edges.

Step	x_1	x_2	x_3	x_4	x_5
Initialize	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
Forward 1	1	5	6	7	3
Backward 1	2	5	6	7	3
Forward 2	2	5	6	8	4
Backward 2	2	5	7	8	4
Forward 3	2	5	7	8	4
Backward 3	2	5	7	8	4

Figure 6.13 The evolution of the distances for the graph of Figure 6.12 as computed by the Liao-Wong algorithm.

edges. This process (modifications due to backward edges followed by propagation) is repeated until the values of the minimal x-coordinates stabilize or a maximum number of iterations equal to $|E_b|$ have been performed. In the last case, a positive cycle exists. The execution of the algorithm is then terminated by calling the function `error` that prints the error message provided as an argument and then performs a "long jump" to an appropriate environment in the software.

In order to understand that the algorithm is correct, one should realize that at the kth iteration of the do loop, the values of the x_i represent the longest-paths going through all forward edges and possibly k backward edges. As a longest-path contains each edge at most once, the algorithm should terminate after at most $|E_b|$ iterations in the absence of positive cycles.

As the DAG longest-path algorithm has a time complexity of $\mathcal{O}(|E_f|)$ and is called at most E_b times, the Liao-Wong algorithm has a time complexity of $\mathcal{O}(|E_b| \times |E_f|)$. This makes the algorithm interesting in cases when the number of backward edges is relatively small.

An example graph that will be used for the illustration of the algorithm is given in Figure 6.12. The graph has been obtained from the graph of Figure 6.9 by adding to it three backward edges (recognizable by their negative weights). The evolution of the Liao-Wong algorithm applied to this graph is shown in Figure 6.13. The figure shows

```
for (i ← 1; i ≤ n; i ← i + 1)
  x_i ← −∞;
x_0 ← 0;
count ← 0;
S_1 ← {v_0};
S_2 ← ∅;
while (count ≤ n && S_1 ≠ ∅) {
   for each v_i ∈ S_1
     for each v_j "such that" (v_i, v_j) ∈ E
       if (x_j < x_i + d_ij) {
         x_j ← x_i + d_ij;
         S_2 ← S_2 ∪ {v_j}
       }
   S_1 ← S_2;
   S_2 ← ∅;
   count ← count + 1;
}
if (count > n)
  error("positive cycle");
```

Figure 6.14 The Bellman-Ford algorithm.

the values of the found distances after the "forward" and "backward" steps in each iteration (respectively, the results obtained by the DAG longest-path algorithm and the update steps). No more changes are found in the third iteration and the algorithm stops. Note that the result obtained after the first "forward" step is equal to the final result of Figure 6.9.

6.4.4 The Bellman-Ford Algorithm

An alternative to the Liao-Wong algorithm is the *Bellman-Ford* algorithm. The algorithm does not discriminate between forward and backward edges. It is comparable to the longest-path algorithm for DAGs with the difference that several iterations through the graph are necessary before the lengths of the longest paths have been computed.

The pseudo-code for the algorithm is given in Figure 6.14. One way of looking at the algorithm is to see it as repeated wave front propagation: S_1 contains the current wave front and S_2 is the one for the next iteration. As in the Liao-Wong algorithm, if there are more than n iterations, where n is the number of vertices in the graph $G(V, E)$, it can be concluded that the graph has positive cycles. Informally, this can be seen as follows: after k iterations, the algorithm has computed the longest-path values for paths going through $k - 1$ intermediate vertices. If there are no cycles, the algorithm should terminate after at most n iterations, as the longest path to a vertex can go through at most $n - 1$ vertices.

The time complexity of the Bellman-Ford algorithm is $\mathcal{O}(n \times |E|)$ as each iteration

S_1	x_1	x_2	x_3	x_4	x_5
"not initialized"	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$\{v_0\}$	1	5	$-\infty$	$-\infty$	$-\infty$
$\{v_1, v_2\}$	2	5	6	6	3
$\{v_1, v_3, v_4, v_5\}$	2	5	6	7	4
$\{v_4, v_5\}$	2	5	6	8	4
$\{v_4\}$	2	5	7	8	4
$\{v_3\}$	2	5	7	8	4

Figure 6.15 The evolution of the distances for the graph of Figure 6.12 as computed by the Bellman-Ford algorithm.

visits all edges at most once and there are at most n iterations. If the graph is dense, i.e. the number of edges is $\mathcal{O}(n^2)$, this would mean a worst-case time complexity of $\mathcal{O}(n^3)$. However, under assumptions that are realistic for compaction, the *average* time complexity turns out to be $\mathcal{O}(n^{1.5})$.

The evolution of the Bellman-Ford algorithm when applied to the graph of Figure 6.12, that was also used for the illustration of the Liao-Wong algorithm, is shown in Figure 6.15. In each row, the first column gives the contents of the set S_1 at the beginning of the algorithm and the remaining entries of the row show the distance value after having processed all elements of S_1. Not surprisingly, the final result is the same as the one obtained by the Liao-Wong algorithm.

6.4.5 *Discussion: Shortest Paths, Longest Paths and Time Complexity*

Until now different versions of shortest and longest-path problems have been presented in this text. In order to eliminate possible confusion, a short discussion follows here that shows the relationships between the different algorithms presented as well as the type of graphs on which they operate.

Consider first a DAG. The longest-path problem for a DAG can be solved with the algorithm given in Figure 6.8. If one wants to solve the shortest-path problem in a DAG, a slightly modified version of the algorithm can be applied by initializing all path lengths to $+\infty$ and taking the minimum instead of the maximum in the inner loop. One can also apply the longest-path algorithm without modification if it operates on a "transformed" graph where all weights have been multiplied with –1. The multiplication by –1 of the longest path lengths found gives the shortest path lengths.

The situation is more complicated when the graph is cyclic. If it only contains negative cycles, the longest-path problem can be solved in polynomial time. Analogously, if the graph only contains positive cycles, the shortest-path problem can be solved in polynomial time (actually, the original version of the Bellman-Ford algorithm is for shortest paths).

The longest-path problem in graphs with positive cycles and analogously the shortest-path problem in graphs with negative cycles is NP-hard (see Exercise 4.1).

Note that Dijkstra's algorithm for shortest paths (see Section 3.4.3) only works when all edge weights are positive. Therefore, it does not operate correctly if the edge weights are first multiplied by –1 in an attempt to adapt the algorithm for a longest-path computation.

6.5 Other Issues

There follows a short discussion of a number of additional issues that are important for compaction but that cannot be dealt with in detail in this text:

- *Noncritical layout elements*
  Consider horizontal, right-to-left, one-dimensional compaction using a constraint graph as in the text above. The width of the layout after compaction is determined by the vertex v that has the largest longest-path length. The longest path starting at the source vertex v_0 and finishing at v is called the *critical path* of the layout (there may be more than a single critical path). In the case of the graph of Figure 6.9 for example, v_4 receives the largest coordinate after compaction and the critical path consists of the vertices v_0, v_2, v_1, v_5 and v_4. The layout elements corresponding to these vertices cannot be moved further to the right without making the total width of the layout larger than strictly necessary. This is, however, not the case with the layout element corresponding to the remaining vertex v_3: it can be moved one length unit to the right (to position 8) without affecting the minimal width of the layout. In general, all layout elements not on the critical path have some *mobility* or *freedom*. The interval within which a layout element can be moved can easily be found by performing both a right-to-left and a left-to-right compaction and taking the longest path lengths found as the interval boundaries. Simply assigning each layout element its leftmost or rightmost possible value does not exploit the mobility of the noncritical elements. It may be that a subsequent compaction step in a perpendicular dimension will achieve better results if the layout elements are assigned a coordinate that is not on the boundary of this interval. Finding the best position for these noncritical layout elements is a difficult task (an optimal algorithm would actually solve the two-dimensional compaction problem that is known to be NP-complete) but some heuristics could always be applied to exploit mobility.

- *Automatic Jog Insertion*
  In Section 6.3.2 layout elements were partitioned in rigid and stretchable rectangles. Considering a wire as a rectangle that is merely stretchable in one dimension, does not exploit all the possible freedom that one has with wires. One such possibility is the insertion of *jogs*. This is the splitting of a wire in segments such that these segments can be moved with respect to each other. This situation can lead to a reduction of the layout width as illustrated in Figure 6.16. Compaction algorithms capable of inserting jogs are quite common. They especially introduce jogs to reduce the critical-path length.

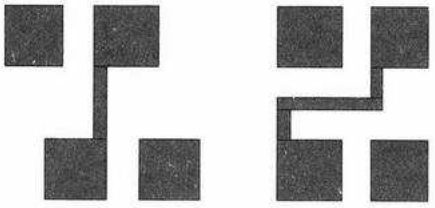


Figure 6.16 A simple compacted layout before (a) and after (b) the introduction of jogs.

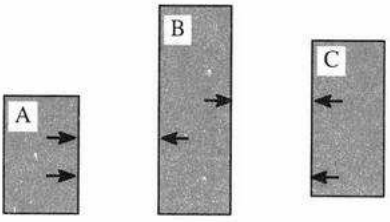


Figure 6.17 The redundancy of the minimal-distance constraint between rectangles A and C.

- *Constraint Generation*
  The algorithms discussed above operate on a constraint graph. However, compaction has to be applied on a layout. Therefore, an efficient algorithm is necessary to convert a layout into a constraint graph. In a straightforward algorithm, assuming horizontal compaction, one could inspect all pairs of layout elements and generate a weighted edge between them, if they overlap when projected on the vertical axis. When the original layout consists of n rectangles, this will result in an algorithm with a time complexity of $\mathcal{O}(n^2)$. There are two problems with this approach. First, too many rectangle pairs are inspected in the generation process and second, the resulting graph is likely to have many redundant edges (remember that the time complexity of compaction algorithms depends on the number of edges in the constraint graph). An edge (v_i, v_j) in a graph $G(V, E)$ is called *redundant* if there also exist edges (v_i, v_k) and (v_k, v_j), while $w((v_i, v_j)) \leq w((v_i, v_k)) + w((v_k, v_j))$. This has been illustrated in Figure 6.17, that shows three rectangles A, B, and C located in the same layer. The minimum-distance constraints for the rectangle pairs (A,B) and (B,C) make the minimum-distance constraint for pair (A,C) redundant.

- *Hierarchy*
  It is important to note that compaction will seldom be applied to the layout of a complete integrated circuit. As was mentioned in Chapter 1, the physical domain can be described hierarchically, such that small groups of transistors form a cell, groups of cells form modules, etc. If special attention is paid to cell boundaries, it

is normally sufficient to apply compaction to cells only.

6.6 Bibliographic Notes

The reader that is interested in knowing more about compaction or in an alternative presentation can consult textbooks dedicated to physical VLSI design automation such as [Len90], [She93], [Sai95] or [Sar96]. Reviews on the compaction problem include [Cho85], [Mly86], [Boy88], [Wol88]. [Hil89] presents the results of a layout generation project that included symbolic layout, a formal *layout language* to describe it and compaction methods.

More on design rules can be found in textbooks on CMOS VLSI design such as [Wes93] but also in [Uye95], which is a more practice-oriented book on CMOS circuit and layout design. One of the early sources to introduce the symbol λ for the minimal feature size was [Mea80].

An example of a symbolic layout editor is described in [Cro88]. More on *mask-to-symbolic extraction* can e.g. be found in [Beu90].

A proof that two-dimensional compaction is NP-complete is given in [Sch83b]. The fact that it is an NP-complete problem justifies the use of simulated annealing as reported in [Mos87]. A interesting compaction method that allows some lateral movement during one-dimensional compaction is reported in [Séq87].

A one-dimensional compaction method that has some similarities with constraint-graph compaction but has not been discussed here is *virtual-grid compaction*. It was introduced by Weste [Wes81] and has the property that layout elements placed by a designer on the same horizontal or vertical "virtual grid" line remain on the same line during compaction.

The Liao-Wong algorithm is explained in [Lia83]. The application of the Bellman-Ford algorithm, which is well-known in graph theory (see e.g. [Law76], [Cor90] or [McH90]), to the compaction problem has been pointed out by Schiele [Sch83a]. The pseudo-code in the text is based on Schiele's version of the algorithm. The average time complexity result of $\mathcal{O}(n^{1.5})$ has also been reported by Schiele.

More on constraint generation can be found in the textbooks and review papers mentioned above. The best algorithm for the constraint-graph generation problem has been proposed by Doenhardt and Lengauer [Doe87, Len90]. It has a time complexity of $\mathcal{O}(n \log n)$, while it only generates irredundant constraints.

6.7 Exercises

6.1 Write a recursive procedure for computing the longest-path lengths based on the algorithm of Figure 6.8 that does not store vertices to be processed explicitly in a data structure: so, the new procedure should not contain the variable Q.

6.2 The three algorithms for longest-path computations presented in this chapter (in Figures 6.8, 6.11 and 6.14) all compute longest-path lengths rather than longest paths. How should these algorithms be modified to compute the longest paths themselves? Do the modifications that you propose affect the time complexity of the algorithms?

7

Placement and Partitioning

Placement is a very common problem in VLSI design. The input to the problem is the structural description of a circuit. Such a description consists of a list of design subentities (hardware subparts) and their interconnection patterns that together specify the complete circuit. At the moment that the placement problem has to be solved, the layouts of these subentities should be available. The goal of placement is to determine the location of these layouts on the chip such that the total resulting chip area is minimal, while the layouts of the subentities do not overlap and sufficient space is left for wiring. The wiring should realize exactly the interconnections specified in the structural description. The determination of the wiring patterns on chip forms the *routing* problem that is discussed in Chapter 9. During placement, it is sufficient to estimate the area occupied by wiring.

A "toy" version of the *placement* problem, viz. the *unit-size placement* problem, has already been introduced in Chapter 5. There, some indications were given on how to solve this problem using general-purpose optimization methods. In this chapter, two versions of the placement problem that are more realistic than unit-size placement, viz. *standard-cell placement* and *building-block placement*, are first defined. Most of the general-purpose optimization methods can also be used for these versions of the placement problem. The emphasis in this chapter is on problem-specific solution methods for placement. They can be subdivided in the categories *constructive* and *iterative placement* and are discussed separately.

Before dealing with the placement problem and possible solutions, however, attention is paid to the representation of an electric circuit such that a placement program (and many other design automation tools) can access the circuit easily. Some possibilities are then explained for the estimation of the wiring area.

The *partitioning* problem deals with splitting a network into two or more parts by cutting connections. Although this problem has major importance as a problem on its own in the field of VLSI design automation, it is treated here together with placement because solution methods for the partitioning problem can be used as a subroutine for some type of placement algorithms. Only one partitioning algorithm is presented here in detail, viz. the one proposed by Kernighan and Lin.

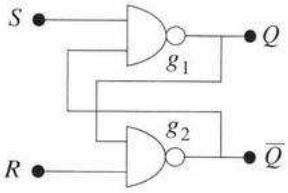


Figure 7.1 The schematics of an RS-latch.

7.1 Circuit Representation

The structural description of an electric circuit is a central issue in design automation. It is the input of tools like placement, simulation, etc., while it is the output of e.g. tools for logic and high-level synthesis. Consider the schematics shown in Figure 7.1 that indicate how an *RS-latch* can be constructed from two NAND gates. In the schematics, one can distinguish the two NAND gates g_1 and g_2, two *input terminals* S and R, two *output terminals* Q and $\overline{Q}$ and the wires connecting the gates and the terminals. Besides, a complete description of the circuit should indicate to which specific input or output a wire is connected. Obviously, a *data model* of an electric circuit (the organization of the data structures that represent it) should correctly deal with all issues mentioned for the example.

The data model proposed here consists of the three structures *cell*, *port* and *net*. A *cell* is the basic building block of a circuit. A NAND gate is an example of a cell. The point at which a connection between a wire and a cell is established is called a *port*. So, a cell has one or more ports. The wire that electrically connects two or more ports is a *net*. So, a set of ports is associated with each net and a port can only be part of a single net. These notions can be expressed as data structure definitions in pseudo-code, as shown in Figure 7.2.

A cell in a circuit is an *instance* of a *master* cell. The master contains all information that all cells of a specific type, e.g. all NAND gates, have in common. The term instance refers to each occurrence of the cell in the circuit. One property stored in the master is, of course, the name of the cell: "NAND". Another property is a list of its inputs and outputs. Still other properties could be related to electrical properties, such as the switching delay or layout properties, such as width and height. Note that the information stored in masters originates from a *library*, either a library of predesigned cells such as standard cells, or a library of cells designed by the designer.

For each occurrence of a cell in the circuit there is a separate instance. It might be useful to uniquely identify each cell instance with a name (e.g. for easy communication in a design team). For this reason each `cell` structure has an attribute `id`. Finally, each cell instance needs to have access to its ports, identified here by the sets `in_ports` and `out_ports`. The ports have been partitioned into inputs and outputs. This may not always be relevant, but it surely is the case in e.g. the context of simulation where the cells have to be processed in the order of signal flow.

```
struct cell {
  struct cell_master *cell_type; /* Access to cell type, e.g. NAND
                                    gate and other generic properties */
  char id[ ]; /* A string that uniquely identifies the cell, e.g. g1 */
  set of struct port in_ports, out_ports;
};

struct port {
  struct port_master *port_type; /* Access to generic port information */
  char id[ ]; /* Unique identification */
  struct cell *parent_cell; /* To which cell does this port belong? */
  struct net *connected_net; /* To which net is this port connected? */
};

struct net {
  char id[ ]; /* Unique identification */
  set of struct port joined_ports; /* Ports connected by the net */
};
```

Figure 7.2 Data structure definitions for circuit representation.

What has been said about instances and masters for cells also applies to ports. The information to be stored in a port master could be the name of the port (e.g. "carry-in"), whether the port is an input or an output, etc. The attributes `parent_cell` and `connected_net` in the `port` structure identify respectively the cell instance to which the port instance belongs and the net to which the instance is connected.

The structure `net`, finally, is quite simple. For each net, it should give access to the ports to which the net is connected.

Any electric circuit communicates with the external world in one way or the other through its terminals (see S, R, etc. in Figure 7.1). These terminals cannot be directly incorporated in the data model just presented. For a consistent modeling, pseudo-cells called *input cells* and *output cells* are introduced. An input cell has a single port through which it sends a signal to the circuit and an output cell has a single port through which it receives a signal from the circuit. In Figure 7.3(a) the RS-latch of Figure 7.1 is shown once more, now explicitly showing the cells, pseudo-cells and ports. Input cells are shown using an "arrow" symbol pointing to its port and output cells using the same symbol, but now pointing away from its port. Dashed lines show the cell boundaries. Ports are indicated by small squares located on the cell boundaries.

It is quite straightforward to derive a *graph model* of an electric circuit from the data model just presented. The graph will have three distinct sets of vertices: a cell set, a port set and a net set. There will be two edge sets: one for edges connecting cells with ports and one for edges connecting nets with ports. Note that edges never connect vertices of the same type and neither do they connect nets with cells. One could call such a graph a *tripartite graph*. This graph model of the RS-latch circuit

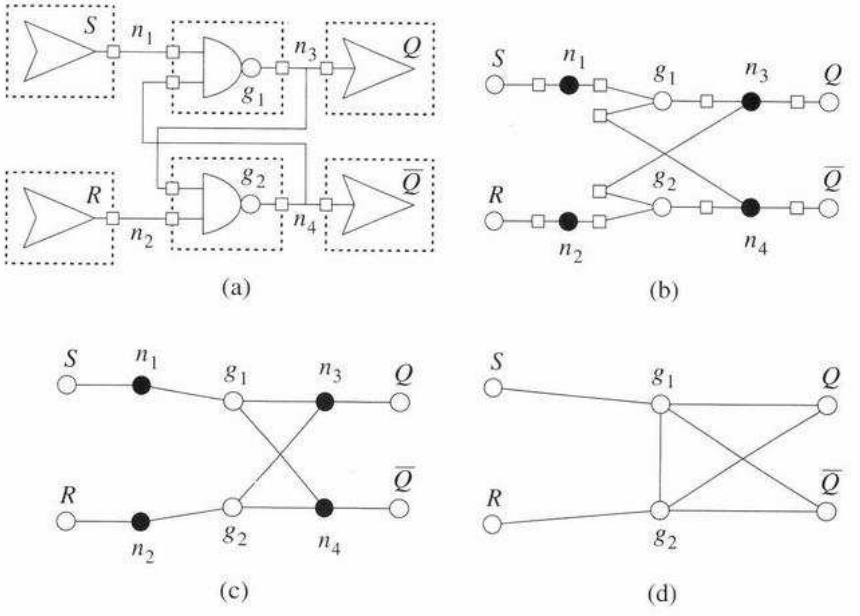


Figure 7.3 The representation of the RS-latch using the cell-port-net data model (a), the tripartite the graph model (b), the bipartite graph model (c) and the clique model (d).

is shown in Figure 7.3 (b). In the figure, white circles represent cells, black circles represent nets and squares represent ports.

In some applications, such as the placement problem to be discussed in this chapter, it is not always important to distinguish the ports of a cell. In such a case all ports can be merged with their associated cell vertices. The result is a *bipartite graph* consisting of cell and net vertices and a single edge set that links nets with cells. The representation of the RS-latch example using this graph model is shown in Figure 7.3(c). This model is equivalent to the *hypergraph* model of a circuit. A hypergraph consists of vertices and *hyperedges*, where hyperedges connect two or more vertices instead of exactly two as is the case in a common graph. Clearly, in a hypergraph model, the vertices represent the cells and the hyperedges the nets.

The model can be simplified even further by omitting the explicit representation of nets. Only the set of cell vertices remains in the graph. For each set of cells that are connected to the same net, edges are created between each pair of cells in this set. So, there is a *clique* in the graph (see Section 3.1) for each net in the circuit. The representation of the RS-latch using this clique model is shown in Figure 7.3(d). Clearly, the graph models discussed here differ in the accuracy with which they model an electric circuit. It is up to the designer of CAD software to chose the most suitable one for the problem at hand.

As was discussed in Chapter 1, the notions of *hierarchy* and *abstraction* are very important in VLSI design. The complexity of VLSI circuits can only be mastered

through the use of hierarchy and abstraction. The data model presented here does not directly support hierarchy, but can easily be modified to do so. A complete circuit can e.g. be converted into a single cell by assigning the circuit's contents to a new *master*. In this process, the input and output pseudo-cells become the ports of the new cell (see Exercise 7.1). Applying these ideas to the circuit of the RS-latch would e.g. result in a new master cell "RS-latch" that can be used as a component in a larger circuit.

The data model presented in this section is a general one that can be used by several applications. Each application will require additional information to be stored in the model. A simulator e.g. will extend the `net` data structure to store the value of the signal carried by the net. Layout information will be added to a `cell` data structure by a placement program, etc. The use of *object-oriented* programming techniques could be of help here as extending the basic data structures and the functions operating on them can be done without modifying the basic code itself.

7.2 Wire-length Estimation

As mentioned at the end of Section 5.1, the estimation of the total wire length is used to evaluate the quality of placement. A wire-length *metric* is applied to each net, resulting in a length estimate per net. The total wire length estimation is then obtained by summing the individual estimates. The total wiring area (the cost function for the unit-size placement problem) can then be derived from this length by assuming a certain wire width and a wire separation distance.

All metrics refer to a cell's coordinates. In the unit-size placement problem, these are the coordinates of the unit-size grid as shown e.g. in Figure 5.1(b). In more general placement problems, one will need a finer grid to accurately indicate the location of a cell. A cell's coordinates can then be the coordinates of its center. Some common metrics are:

- *Half perimeter:* This metric computes the smallest rectangle that encloses all terminals of a net and takes the sum of the width and height of the rectangle as an estimation of the wire length. The estimation is exact for two- and three-terminal nets and gives a lower bound for the wire length of nets with four or more terminals.

- *Minimum rectilinear spanning/Steiner tree:* The two types of tree were introduced in Section 4.5. In both cases, a wiring pattern is computed that interconnects all terminals of the net. Remember that the spanning tree is relatively easy to compute, whereas the minimum Steiner tree problem is NP-complete. The minimum Steiner tree always has a length shorter than or equal to the spanning tree length which means that the latter is an upper bound for the former. For both types of tree, when computing the length between two points (x_1, y_1) and (x_2, y_2), one can either use the *rectilinear* or *Manhattan* distance defined as $|x_1 - x_2| + |y_1 - y_2|$ or the *Euclidean* distance defined as $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- *Squared Euclidean distance:* This method is meant for the clique-model representation of an electric circuit (see Section 7.1). As nets are not explicitly present in this model, the total cost is obtained by summing over the cells rather than over the nets. The cost of a placement is then defined as:

$$\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\gamma_{ij}[(x_i - x_j)^2 + (y_i - y_j)^2]$$

Here, n denotes the number of vertices (cells), γ_{ij} is the edge weight of the edge between vertex v_i and v_j, and x_i, x_j, y_i, and y_j are the x-coordinates and y-coordinates of the corresponding cells. The factor $\frac{1}{2}$ just expresses that all vertex pairs are counted twice, but could obviously be omitted without affecting the optimization result obtained using this metric. Also the fact that the squared distance instead of the distance itself is used here, does not affect the results of optimization. γ_{ij} is zero if there is no edge between the vertices v_i and v_j. For nets that are connected to many cells, its value can be made lower to express that not all pairwise connections in the clique representing the net will be realized during routing. Also, the fact that the two cells v_i and v_j are both connected to more than one net can be translated in a higher value of γ_{ij}.

7.3 Types of Placement Problem

The most common placement problems are *standard-cell placement* and *building-block* placement. They are introduced in this section.

As was mentioned in Section 1.4, *standard cells* are predesigned small circuits (e.g. simple logic gates, flip-flops, etc.) for a specific fabrication process that are stored in a library. For each cell, the library contains behavioral data (e.g. Boolean equations, delay parameters) that can be used by a simulator, and also layout data (e.g. size, positions of terminals, etc.) to be used by placement and routing programs. The designer combines the standard cells into the desired circuit by interconnecting them in an appropriate way.

Standard cells obey some restrictions for their layout. Connections that are shared by all or most cells, like e.g. power and clock connections, cross the cells from left to right at fixed locations. These connections are sometimes called the *logistic signals* as opposed to *logic signals*. Signals related to the specific I/O of the cell have to leave the cell either at the top or the bottom. This is illustrated in Figure 7.4. In a standard-cell design, the cells are collected into *rows* separated by wiring or routing *channels*. Within one row, the logistic signals automatically realize continuing connections due to their fixed positions. It is said that the cells *abut* horizontally or that they are connected by horizontal *abutment*. Figure 7.5 shows a small arrangement of cells obeying the standard-cell layout style.

In full-custom design where designers have the freedom to give arbitrary (but often rectangular) shapes to their cells or where *macro cells* are used that consist of regular parameterized layout structures, the cells need wiring space all around. The layout

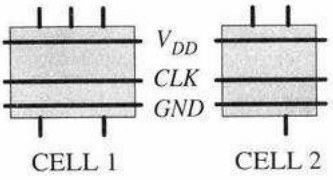


Figure 7.4 Standard cells: logistic signals like power and clock cross the cell at fixed positions while I/O signals specific to a cell have terminals at the top and bottom.

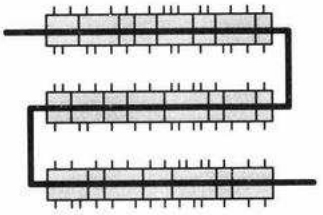


Figure 7.5 An example of standard cell placement: there are three rows separated by wiring channels, while the logistic signals connecting to all cells have symbolically been represented by a thick line.

style that supports designs with these type of cells is called *building-block layout* (or sometimes also *general-cell layout*). Figure 7.6 shows a stylized representation of a placement in building-block layout.

Apart from the standard-cell and building-block layout styles, a combination of both is also common. One e.g. uses standard cells for the implementation of the logic functions of a design together with a large RAM module for the memory. Such a module occupies far less area than an equivalent memory built of standard cells only (in a standard-cell implementation of the memory, much more space is occupied by wiring).

Clearly, the placement problem for standard cells or building blocks is more complex than the *unit-size* placement problem discussed in Section 5.1. One obvious difference is that *moves* that exchange two cells as encountered in many general-purpose algorithms (see Chapter 5) are not always possible due to the size difference

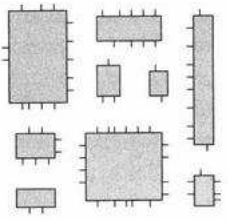


Figure 7.6 An example of the placement of cells in the building-block layout style.

of cells. This is one of the issues discussed in the next section.

7.4 Placement Algorithms

Placement algorithms can be grouped into two categories:

- *constructive placement:* the algorithm is such that once the coordinates of a cell have been fixed they are not modified anymore;
- *iterative placement:* all cells have already some coordinates and cells are moved around, their positions are interchanged, etc. in order to get a new (hopefully better) configuration.

Most placement algorithms contain both approaches: an *initial placement* is obtained in a constructive way and attempts are made to increase the quality of the placement by *iterative improvement.* Note: this searching strategy does not apply to placement only, but is applicable to many combinatorial optimization problems.

Below, separate attention is given to the constructive and iterative placement approaches.

7.4.1 Constructive Placement

There are many ways to perform constructive placement. Two important methods are based on *min-cut partitioning* and *clustering.* Both methods are essentially *partitioning* methods which divide the circuit in two or more subcircuits of a given size while minimizing the number of connections between the subcircuits. More information on partitioning will be given in Section 7.5.

The basic idea of min-cut placement is to split the circuit into two subcircuits of more or less equal size while minimizing the number of nets that are connected to both subcircuits (a net that is connected to both subcircuits is "cut", hence the name "min-cut"). The two subcircuits obtained will each be placed in separate halves of the layout (upper and lower halves or left and right halves). Because the number of nets crossing from one half to another has been minimized, one can assume that the number of long wires crossing from one half of the chip to the other has been minimized as well. This type of *bipartitioning* is recursively applied to the subcircuits until some criterion is satisfied (e.g. until all subcircuits only contain one cell). Also, after each bipartitioning stage, the area available for the subcircuit concerned is bisected and a decision is made on the direction of the bisection (horizontal or vertical) and on which of the two smaller subcircuits obtained will be assigned to which half. The idea is illustrated in Figure 7.7, that shows three stages of min-cut placement. For each stage, the netlist status is shown at the top and the placement at the bottom. Note that a clique model is used for the netlist where parallel edges indicate multiple connections between the cells.

There are two important tasks in min-cut placement: the partitioning of the graph and the assignment of the partitions to relative layout positions. An algorithm for

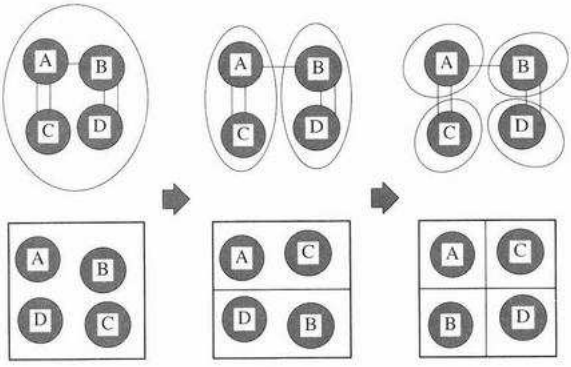


Figure 7.7 The recursive bipartitioning of a circuit leading to a min-cut placement.

partitioning will be presented in Section 7.5.1. The second task can be based on different heuristics. One such heuristic is to look at the parts of the circuit that already have a fixed position (either because the placement of these parts is already fixed or because they are connected to the inputs or outputs of the chip that are located at the chip's periphery). The decision can then be based on keeping the connections to these fixed parts as short as possible. Consider the example of Figure 7.7: after the first bipartitioning, there are no fixed parts and the cells A and C are assigned to the upper half of the layout, while the cells B and D are assigned to the lower half. In the next stage, due to the lack of fixed cells, A is assigned to the upper right and C to the upper left (the decision to assign them both to the upper half had already been taken before). Now, when taking decisions for the cells B and D, one can see that it is preferable to put B at the lower left rather than the lower right because of its connections with A.

Min-cut placement is a *top-down* method: one starts with the whole circuit and ends with small subcircuits. A *bottom-up* method for initial placement is *clustering*. Here one starts with a single cell and finds one or more cells that share many nets with it. These cells are all taken together to form a cluster. More cells are added to the cluster in this way until the cluster contains the whole circuit. As was the case in the bipartitioning-based approach, each time after a cell has been selected into a cluster, a decision should be taken on its location. Again, the connectivity of a cell with those already in the cluster can guide the decision. The idea is illustrated in Figure 7.8.

7.4.2 Iterative Improvement

Iterative improvement is a method that perturbs a given placement by changing the positions of one or more cells and evaluates the result. If the new cost is less than the old one, the new placement replaces the old one and the process continues. If the new situation is worse than the old one, the perturbed situation may or may not be

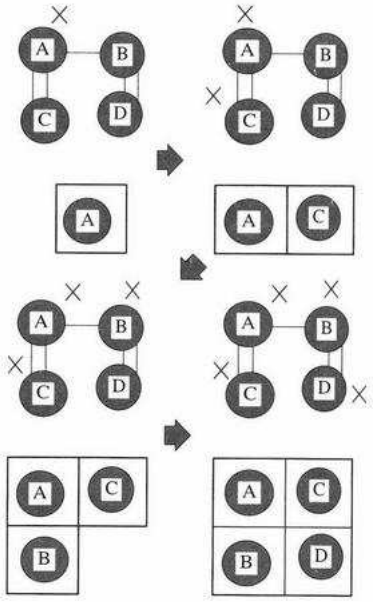


Figure 7.8 Placement by clustering (the cells that are already part of the cluster are marked by a cross).

```
iterative_improvement()
{
  s ← initial_configuration();
  c ← cost(s);
  while (!stop()) {
      s' ← perturb(s);
      c' ← cost(s');
      if (accept(c, c'))
        s ← s';
  }
}
```

Figure 7.9 A generic algorithm for iterative improvement.

accepted depending on the exact search method used. A general framework for an iterative improvement algorithm is given in Figure 7.9.

The behavior of the algorithm depends on the definitions of the functions `initial_configuration`, `cost`, `stop`, `perturb` and `accept`. `initial_configuration` generates an initial placement and `cost` computes the costs of a given placement. The function `stop` decides when the main loop in iterative improvement should be terminated. This function can be very simple, counting a fixed number of iterations, or arbitrarily more complex. The Boolean function

`accept(`c, c'`)` normally simply implements the test $c' < c$, which means that perturbations that decrease the cost function (downhill moves), are always accepted. An appropriate choice for the functions mentioned can make the algorithm operate as some of the general-purpose optimization methods discussed in Chapter 5, such as *local search*, *simulated annealing* or *tabu search*.

The function `perturb` computes a new feasible solution s' from a given feasible solution s. This function is easy to implement for the case of *unit-size* placement (see Section 5.1), as was indicated in several places in Chapter 5: any exchange of cells or the displacement of a cell to an empty slot will result in a feasible solution. Perturbation of a feasible solution for standard cell or building-block placement is more complex due to the inequality of the cell sizes. Different approaches are possible:

- One can allow that cells in a feasible solution overlap and make the overlap part of the cost function to be minimized. So, the cost function becomes some weighted sum of the estimated wire length and the overlap cost. This will direct a placement algorithm towards solutions with little or no overlap. Any overlap that remains can be eliminated by pulling apart the cells in the final layout (at the expense of a larger overall chip area).

- One can eliminate overlaps directly after each move by shifting an appropriate part of the cells in the layout. In general, this is a computation-intensive operation as the coordinates of many cells in the layout have to be recomputed as well as the estimated wire lengths of all nets that connect to the displaced cells. It is, however, feasible for standard cell placement, because overlaps can occur in one dimension only. This means that at most half of the cells in a row need to be shifted (when inserting a cell at a position without sufficient space, shift to the side that contains the smallest number of cells).

- One can resort to more sophisticated encoding methods for feasible solutions, instead of the mere assignment of coordinates to cells. These methods store the "relative position" of the cells with respect to each other. One could think e.g. of representations similar to the *slicing tree* to be discussed in Chapter 8. Applying transformations on these encodings to find neighboring solutions will always result in placements without overlap. However, the price paid for feasible solutions that are free of overlap is again a computational one, because the absolute coordinates of each cell and the net length estimates have to be recomputed after each move.

Apart from random perturbations that are characteristic for optimization methods such as simulated annealing, one could use more domain-specific methods for placement by means of iterative improvement. *Force-directed placement* is an example of such a method. It assumes that cells that share nets, feel an attractive "force" from each other. The goal is to reduce the total force in the network. To achieve this goal one can compute the "center of gravity" of a cell, the position where the cell feels a force zero, given the positions of all the cells to which it is connected.

Consider the "clique representation" of a netlist by means of an edge-weighted graph $G(V, E)$ as discussed in Section 7.1. The weight of an edge $(i, j) \in E$ is given by γ_{ij} ($\gamma_{ij} = 0$ when $(i, j) \notin E$). Then the center of gravity (x_i^g, y_i^g) of a cell i is defined as:

$$x_i^g = \frac{\sum_j \gamma_{ij} x_j}{\sum_j \gamma_{ij}}$$

$$y_i^g = \frac{\sum_j \gamma_{ij} y_j}{\sum_j \gamma_{ij}}$$

So, the center of gravity is found by computing the weighted average of the coordinates of those cells connected to i. An example of a perturbation is then to move a cell to a legal position close to its center of gravity and if there is another cell at that position to move that cell to some empty location or to its own center of gravity (which creates a chain of moves until the last cell in the chain is put in an empty location).

7.5 Partitioning

The direct motivation for paying attention here to the partitioning problem originates from min-cut placement. However, partitioning is itself an important problem in the field of VLSI design automation. It shows up e.g. when a large circuit has to be implemented with multiple chips and the number of pins on the IC packages necessary for interchip communication should be minimized.

Many versions of the partitioning problem exist and many algorithms for each version. A famous algorithm was published by Kernighan and Lin in 1970. It will be discussed in the next section. The section on Bibliographic Notes at the end of this chapter provides pointers to other algorithms.

7.5.1 The Kernighan-Lin Partitioning Algorithm

The model assumed by the algorithm is as follows: there is an edge-weighted undirected graph $G(V, E)$; the graph has $2n$ vertices ($|V| = 2n$); an edge $(a, b) \in E$ has a weight γ_{ab}; if $(a, b) \notin E$, $\gamma_{ab} = 0$. The problem is to find two sets A and B, subject to $A \cup B = V$, $A \cap B = \emptyset$, and $|A| = |B| = n$, which minimizes the *cut cost* defined as follows:

$$\sum_{(a,b) \in A \times B} \gamma_{ab}$$

In other words, the goal is to minimize the total weight of the edges cut by the partitioning of V into the sets A and B. Note that the algorithm assumes that the clique model has been used for the representation of nets (see Section 7.1).

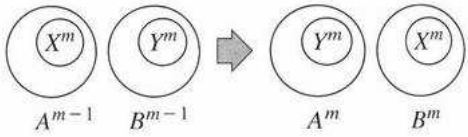


Figure 7.10 The interchange of subsets in the Kernighan-Lin algorithm.

The first thing to remark is that the problem is NP-complete and that the algorithm proposed by Kernighan and Lin is a heuristic that turns out to be rather successful. The principle of the algorithm is to start with an initial partition consisting of the sets A^0 and B^0 which, in general, will not have a minimal cut cost. In an iterative process, subsets of both sets are isolated and interchanged. In iteration number m, the set isolated from A^{m-1} will be denoted by X^m and the set isolated from B^{m-1} will be denoted by Y^m. The new sets, A^m and B^m are then obtained as follows:

$$A^m = (A^{m-1} \setminus X^m) \cup Y^m$$
$$B^m = (B^{m-1} \setminus Y^m) \cup X^m$$

This idea is illustrated in Figure 7.10. The iteration goes on until no improvement in the cut cost is possible.

An important issue in the algorithm is the construction of the sets X^m and Y^m. Note that for any nonoptimal partition, there are subsets X and Y that will lead to an optimal partition in one interchange step. The difficulty of the problem arises, of course, from the fact that these subsets cannot be identified easily. Therefore, the algorithm makes an attempt to find suitable subsets, interchanges them and then tries to make a new attempt, until the attempt does not lead to an improvement of the cut cost. In this context, each exchange of subsets is called a *pass*. The total number of passes needed turns out not to be dependent on the problem size n: most examples reported in the literature do not need more than 4.

The construction of the sets X^m and Y^m is based on *external* and *internal* costs for vertices in the sets A^{m-1} and B^{m-1}. The external cost E_a of $a \in A^{m-1}$ is defined as follows:

$$E_a = \sum_{y \in B^{m-1}} \gamma_{ay}, \quad a \in A^{m-1}$$

So, the external cost for vertex $a \in A^{m-1}$ is a measure for the *pull* that the vertex experiences from the vertices in B^{m-1}. In a similar way, the external cost E_b for a vertex $b \in B^{m-1}$ and the internal costs I_a and I_b can be defined:

$$E_b = \sum_{x \in A^{m-1}} \gamma_{bx}, \quad b \in B^{m-1}$$

$$I_a = \sum_{x \in A^{m-1}} \gamma_{ax}, \quad a \in A^{m-1}$$

$$I_b = \sum_{y \in B^{m-1}} \gamma_{by}, \quad b \in B^{m-1}$$

The difference between internal and external costs gives an indication about the desirability to move the vertex: a positive value shows that the vertex should be better moved to the opposite set, a negative value shows a preference to keep the vertex in its current set. The differences for the vertices in both sets are given by the variables D_a and D_b:

$$D_a = E_a - I_a, \quad a \in A^{m-1}$$
$$D_b = E_b - I_b, \quad b \in B^{m-1}$$

Now the gain in the cut cost, Δ, resulting from the interchange of two vertices can be expressed as:

$$\Delta = D_a + D_b - 2\gamma_{ab}, \quad a \in A^{m-1}, \; b \in B^{m-1}$$

The last term is a correction for a possible edge between a and b, which will continue to cross the cut after swapping the vertices.

The pseudo-code code of the Kernighan-Lin algorithm is shown in Figure 7.11. As mentioned earlier, each execution of the outer loop is a new pass. In the main inner loop, the `for` loop with iteration variable i, the subsets to be interchanged are constructed element by element. In each iteration of this loop, the pair $(a_i, b_i) \in A^{m-1} \times B^{m-1}$ giving the best improvement for the cut cost is selected. The vertices are then "locked", meaning that they cannot be selected once more in the inner loop. They are candidates to be included in the subsets. Actually, it is pretended that they have already been interchanged, and the differences between external and internal costs of the unlocked vertices are therefore updated (check the correctness of the expressions that update the values of D_x and D_y).

It is important to realize that the best cut cost improvement leading to the selection of a pair (a_i, b_i) may be negative. Once all vertices have been locked, the pairs are investigated in the order of selection: the actual subsets to be interchanged correspond to the sequence of pairs (starting with $i = 1$) giving the best improvement. So, pairs in the sequence may have negative cost improvements as long as the pairs following them compensate for it. Such a situation would e.g. occur when the exchange of two clusters of tightly connected vertices results in an improvement, while the exchange of individual vertices from each cluster does not improve the cut cost.

The algorithm will be illustrated using the example graph given in Figure 7.12. The graph consists of the vertices v_1 to v_8. The initial partition consists of the two sets $\{v_2, v_3, v_6, v_7\}$ and $\{v_1, v_4, v_5, v_8\}$ with cut cost 14 as shown in Figure 7.12(a). The evolution of the algorithm in the first pass is shown in Figure 7.13. Each row in the figure corresponds to a value of the loop variable i as is shown in the first column. The values under a column headed by a vertex name v show the subsequent values of the variable D_v (the difference between the external cost E_v and the internal cost

```
initialize(A^0, B^0);
m ← 1;
do { for each a ∈ A^{m-1}
       "compute D_a";
     for each b ∈ B^{m-1}
       "compute D_b";
     for (i ← 1; i ≤ n; i ← i + 1) {
       "find unlocked vertices a_i ∈ A^{m-1}, b_i ∈ B^{m-1} such that
        Δ_i = D_{a_i} + D_{b_i} − 2γ_{a_i b_i} is maximal";
       "lock a_i and b_i";
       for each "unlocked" x ∈ A^{m-1}
         D_x ← D_x + 2γ_{x a_i} − 2γ_{x b_i};
       for each "unlocked" y ∈ B^{m-1}
         D_y ← D_y − 2γ_{y a_i} + 2γ_{y b_i};
     }
     "find a k such that Σ_{i=1}^{k} Δ_i is maximal";
     G ← Σ_{i=1}^{k} Δ_i;
     if (G > 0) {
       X^m ← {a_1, ..., a_k};
       Y^m ← {b_1, ..., b_k};
       A^m ← (A^{m-1} \ X^m) ∪ Y^m;
       B^m ← (B^{m-1} \ Y^m) ∪ X^m;
       "unlock all vertices in A^m and B^m";
       m ← m + 1
     }
   }
while (G > 0);
```

Figure 7.11 The pseudo-code description of the Kernighan-Lin algorithm.

I_v). The variables that are locked at a specific stage of the inner loop are underlined and the value Δ_i corresponding to the selected pair is given in the last column.

Once the inner loop has been traversed, the value k has to be determined such that the sum $\sum_1^k \Delta_i$ is maximal. In this case, the value of k is 1 and the subsets of vertices to be exchanged are $\{v_2\}$ and $\{v_4\}$ (note that $k = 3$ is optimal as well). This leads to the new sets $A^1 = \{v_3, v_4, v_6, v_7\}$ and $B^1 = \{v_1, v_2, v_5, v_8\}$ as is shown in Figure 7.12(b). The cut cost has now decreased from 14 to 8.

The evolution of the variables in the second pass is shown in Figure 7.14. The conclusion of the second iteration is that the maximal gain corresponds to the value of $k = 2$, which means that the vertex subsets to be exchanged are $\{v_3, v_4\}$ and $\{v_5, v_8\}$. The exchange of these subsets leads to $A^2 = \{v_1, v_2, v_3, v_4\}$ and $B^2 = \{v_5, v_6, v_7, v_8\}$ which is the optimal solution for the example. The cut cost now equals 2. Clearly, a third pass will not lead to an improvement and the algorithm will stop. The final partition is illustrated in Figure 7.12(c). Note that the Kernighan-Lin algorithm will not always find the optimal solution: it is just a powerful heuristic.

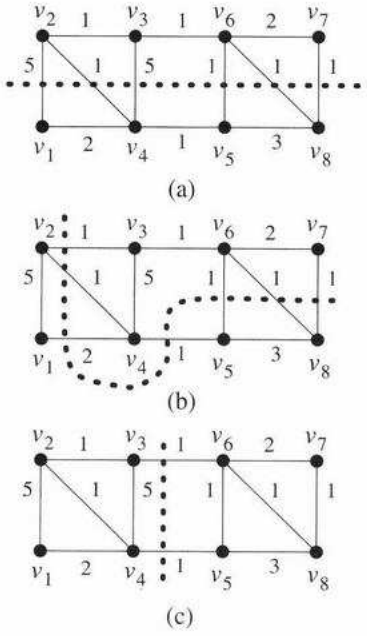


Figure 7.12 Different steps in the application of the Kernighan-Lin algorithm: the initial partition (a), the situation after the first iteration (b) and the final solution (c).

	A^0				B^0				
i	v_2	v_3	v_6	v_7	v_1	v_4	v_5	v_8	Δ_i
1	5	3	−1	−1	3	3	−1	−3	6
2		−5	−1	−1	−3		−1	−1	−2
3		−5	1		−3			3	2
4		−3			−3				−6

Figure 7.13 The first pass of the Kernighan-Lin algorithm applied to the graph of Figure 7.12.

	A^1				B^1				
i	v_3	v_4	v_6	v_7	v_1	v_2	v_5	v_8	Δ_i
1	−5	−1	−1	−1	−3	−3	−1	−1	−2
2	5		−3	−3	−7	−5	3		8
3			−3	−3	−7	−7			−10
4				1		3			4

Figure 7.14 The second pass of the Kernighan-Lin algorithm applied to the graph of Figure 7.12.

The time complexity of the algorithm is determined by the inner loop that is executed n times (remember that the number of executions of the outer loop was not problem-dependent). Finding the best pair of vertices to be locked next requires the pairwise comparison of elements each with at most n elements. Therefore, the number of comparisons is $\mathcal{O}(n^2)$ and the total time complexity becomes $\mathcal{O}(n^3)$. In practical situations (but not in the worst case) sorting the elements of A^{m-1} and B^{m-1} according to their difference values will limit the number of comparisons to the first few elements of the sorted lists. As sorting can be done in $\mathcal{O}(n \log n)$ time, this would lead to an overall time complexity of $\mathcal{O}(n^2 \log n)$. A better time complexity can be achieved by the use of more sophisticated data structures and a modified search strategy (see also the Bibliographic Notes).

Note that the Kernighan-Lin algorithm can be seen as a local search with variable neighborhood (see Section 5.5): the number of elements exchanged between the sets A^{m-1} and B^{m-1} depends on the feasible solution that one is visiting.

7.6 Bibliographic Notes

Many of the textbooks that deal with physical design automation, such as [Len90], [She93], [Sai95] and [Sar96], pay quite some attention to the placement problem. In addition, general information on the placement problem can be obtained through the review papers [Got86], [Bra87] and [Pre88].

Electric circuit representation as presented in this chapter is not so often discussed in the literature, as it is quite straightforward and because different applications have different requirements. Circuit representations that are e.g. appropriate for placement or partitioning are paid attention to in [Len90] and [Alp95]. What has been presented here, is an *internal* data structure, meant to reside in the computer's memory while an algorithm is processing it. A related issue is the *external* representation meant to be stored on file, usually in a human-readable format, that can be shared by many applications. Such a format is EDIF (Electronic Design Interchange Format), that has been standardized by ANSI [Kah92a]. It deals with the notions of cells, ports and nets and many, many more issues that are relevant for real-world CAD.

The wire-length metrics mentioned in the text and some others are listed in [Shi88], [Len90] and [Sai95].

The term *logistic signal* (for power and clock wires) was coined by Spaanenburg [Spa85]. An example of a min-cut algorithm for placement is given in [Bre77]. Examples of clustering algorithms can be found in [Ake82].

As reported in [Sun95] simulated annealing is probably the best algorithm currently known for placement, especially for standard-cell placement. Already the first publication on simulated annealing as a general-purpose optimization method [Kir83] presented results applied to the placement problem. Also, the introductory paper on simulated annealing [Rut89] uses the placement problem as the main illustration of the method. The Timberwolf system [Sec85] is famous for its good results in standard-cell placement by simulated annealing. The version discussed in [Sun95] directly eliminates overlaps after each move. A "rectangle packing" method based

on simulated annealing that is applicable to the building-block placement problem is reported in [Mur95]; this method uses an "encoding" for feasible solutions that excludes overlaps.

Apart from simulated annealing, the application of other general-purpose optimization methods to placement has been reported, including genetic algorithms [Coh87, Sha90] and simulated evolution [Kli89].

More information on the partitioning problem can be found in textbooks dealing with physical design automation, such as [Len90], [She93], [Sai95] and [Sar96]. Also, a number of review papers are available: [Don88] is rather mathematics oriented, [Alp95] is a very extended and detailed review, while [Joh96] mainly concentrates on the new developments in the years 1994-1996.

The original description of the Kernighan-Lin algorithm is given in [Ker70]. The NP-completeness of the version of partitioning problem as solved by the Kernighan-Lin algorithm is mentioned in [Gar79]. A modified version of the algorithm that deals with an explicit net representation (the bipartite-graph model) is given in [Sch72]. Algorithms that have the same objectives as the Kernighan-Lin algorithm but a better time complexity have been reported in [Fid82] and [Kri84]. However, the quality of the results obtained is often worse than the result of the Kernighan-Lin algorithm due to the fact that fewer possibilities for the exchange of subsets are investigated. The use of the method presented in [Dut93], on the other hand, does not affect the search space of the original Kernighan-Lin algorithm, but significantly improves the time complexity by using more sophisticated data structures and a better search strategy.

7.7 Exercises

7.1 Indicate how the data model for the representation of electric circuits, as presented in Section 7.1, can be extended for the description of hierarchically specified circuits. Which modifications are necessary for the data structure definitions given in Figure 7.2?

7.2 Apply the metrics mentioned in Section 7.2 to the two placements given in Figure 5.1. Do they consistently predict that the solution of Figure 5.1(c) is better than the one of Figure 5.1(d)?

7.3 The Kernighan-Lin algorithm as described in the text partitions the vertex set of a graph with $2n$ vertices in two sets of exactly n elements. How can one use the algorithm for a vertex set with an odd number of elements $2n + 1$ that is to be partitioned into sets of n and $n + 1$ elements? What about a situation in which the two sets do not need to have the same size but may differ by at most m elements?

8

Floorplanning

Often in VLSI design layout aspects are dealt with in a bottom-up fashion. The designer either uses cells from a library or designs his/her cells and subsequently composes the overall layout of the chip by means of placement and routing. This often results in poor utilization of the chip area due to large portions of the chip that are occupied by wiring.

Of course, only a well-conceived design methodology can result in final designs of high quality. When looking for such a methodology, it was obvious that the structured design methods developed for software systems (e.g. the hierarchical partitioning of the software in small procedures) could also be used for systems to be implemented in hardware. However, hardware design is harder, as a hardware system has to be embedded in two or three dimensions (a VLSI circuit can be said to be realized in two dimensions only as a relatively few number of layers constitute the third dimension). Software has an arbitrary number of dimensions: there are no limitations in the number of procedures that one procedure can call. The hardware analog of a procedure call is the exchange of signals between modules. When the number of interconnections between modules increases, it becomes harder to design a satisfactory circuit.

These insights have led to the so-called *floorplan-based design methodology*. This top-down design methodology advocates that layout aspects should be taken into account in all design stages. When the three design domains of VLSI are considered (see Section 1.2), a synthesis step from the behavioral domain to the structural domain should be followed by synthesis step from the structural domain to the physical domain. Note that at higher levels of abstraction, due to the lack of detailed information, only the relative positions of the subblocks in the structural description can be fixed. The illustration of the floorplan-based design methodology on the Y-chart is given in Figure 8.1. Taking layout into account in all design stages also gives early feedback: structural synthesis decisions can immediately be evaluated for their layout consequences and corrected if necessary. The presence of (approximate) layout information allows for an estimation of wire lengths. From these lengths, one can derive performance properties of the design such as timing and power consumption. They both increase when the wire lengths grow, as a consequence of the parasitic capacitances between the wires and the substrate.

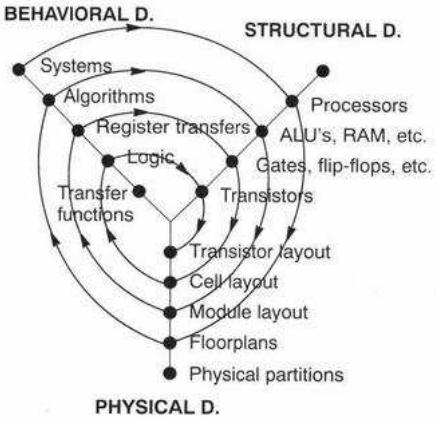


Figure 8.1 The visualization of the floorplan-based design methodology on Gajski's Y-chart.

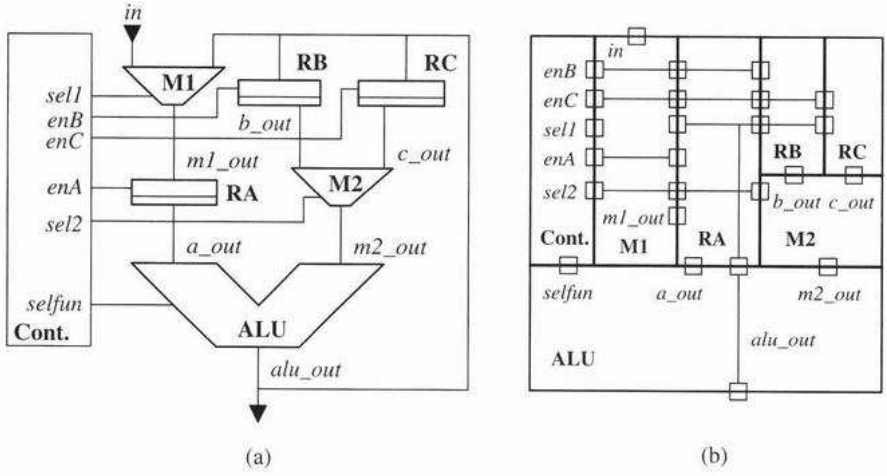


Figure 8.2 A structural description of some circuit (a) and a possible floorplan (b).

It is easy to deal with layout when structural detail at the lowest abstraction is available: one knows the exact number of transistors in the circuit and the way they are interconnected. At the moment that this type of structural information is not fully available, one can estimate the area to be occupied by the various subblocks and, together with a precise or estimated interconnection pattern, try to allocate distinct regions of the integrated circuit to the specific subblocks. This process is called *floorplanning*. The idea is illustrated in Figure 8.2, where a hardware structure has tentatively been mapped on a floorplan. Figure 8.2(a) shows the schematic of some circuit at the register-transfer level that consists of an ALU (arithmetic logic unit),

three registers, two multiplexers and a controller (see Section 12.1 for an explanation of the symbols used in the figure). Figure 8.2(b) shows a possible floorplan annotated with the names of the cells and nets of the schematic. Terminal positions (ports) are indicated by small squares on a cell boundary. *Feedthrough wires,* wires that cross a cell without the wire necessarily carrying a signal relevant to the cell, are indicated by lines connecting the terminals. A real-life floorplan will contain more information than shown in the figure such as the bit width of the wires (control signals only require a single wire, but data signals may e.g. be 16-bit wide). Note that functionally equivalent subblocks (registers, multiplexers) have different shapes and terminal positions. This is one of the main characteristics of floorplan-based design: one chooses the shape and terminal positions such that they fit best with the original structure, and assumes that there is a way to design the module satisfying the chosen shape and terminal positions.

The rest of this chapter is organized in two main parts. First some concepts related to floorplanning are explained followed by a discussion of "shape functions" and "floorplan sizing".

8.1 Floorplanning Concepts

8.1.1 Terminology and Floorplan Representation

The layout of an integrated circuit or, more precisely, its floorplan can be represented hierarchically: cells are built from other cells, except for those cells that are at the lowest level of the hierarchy. These lowest-level cells are called *leaf cells.* Cells that are made from leaf cells are called *composite cells.* Composite cells can contain other composite cells as well, which makes it possible to have an arbitrary number of levels in the hierarchy. The direct subcells of a composite cell are called its *children.* Conversely, every cell, except for the one representing the complete circuit, has a *parent* cell.

For the sake of simplicity, both leaf cells and composite cells are assumed to have a rectangular shape. If the children of all composite cells can be obtained by bisecting the cell horizontally or vertically, the floorplan is called a *slicing floorplan.* So, in a slicing floorplan a composite cell is made by combining its children horizontally or vertically (putting them next to each other from left to right or stacking them on top of each other). A natural way to represent a slicing floorplan is by means of a *slicing tree.* The leaves of this tree correspond to the leaf cells. Other nodes correspond with horizontal and vertical composition of the children nodes. The floorplan presented in Figure 8.2 is slicing; its slicing tree is given in Figure 8.3. In the figure a node labeled with an 'H' was obtained by horizontal composition and a node labeled with a 'V' by vertical composition. The ordering of a composite node's children is relevant: 'from left to right' in horizontal composition and 'from bottom to top' in vertical composition.

Not all floorplans are slicing. For example, in the so-called *wheel* or *spiral* floorplan presented in Figure 8.4, the children of the given cell cannot be obtained

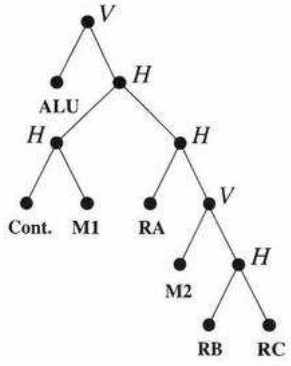


Figure 8.3 The slicing tree for the floorplan of Figure 8.2.

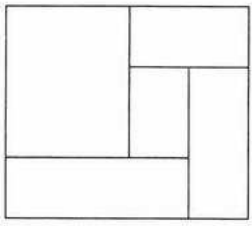


Figure 8.4 The "wheel" floorplan.

by bisections. Note that a composite cell needs to be composed of at least five cells in order not to be slicing. One can derive new composition operators from the wheel floorplan and its mirror image and use them in combination with the horizontal and vertical composition operators in a *floorplan tree.* A floorplan that can be described in this way is called a *floorplan of order 5* because the operators in the floorplan tree have at most 5 operands. Figure 8.5(a) gives an example of a floorplan of order 5 while the corresponding floorplan tree is given in Figure 8.5(c). Figure 8.5(b) shows the locations of the children cells by means of numeric labels that are also used in the tree of Figure 8.5(c). Clearly, a slicing floorplan can also be called a floorplan of *order 2*, because its floorplan tree only consists of the horizontal and vertical composition operators which have two operands.

Higher order floorplans that require operators with more than 5 operands also exist. In an extreme situation, a floorplan with n cells will require operators of order n to describe it and the floorplan tree will have a single parent cell having all leaf cells as its children. However, *hierarchical* descriptions using floorplan trees are seldom used for floorplans of order higher than 5. A representation mechanism that can deal with any floorplan is the *polar graph* which actually consists of two directed graphs: the horizontal polar graph and the vertical one. These graphs can be constructed by identifying the longest possible line segments that separate the cells in the floorplans.

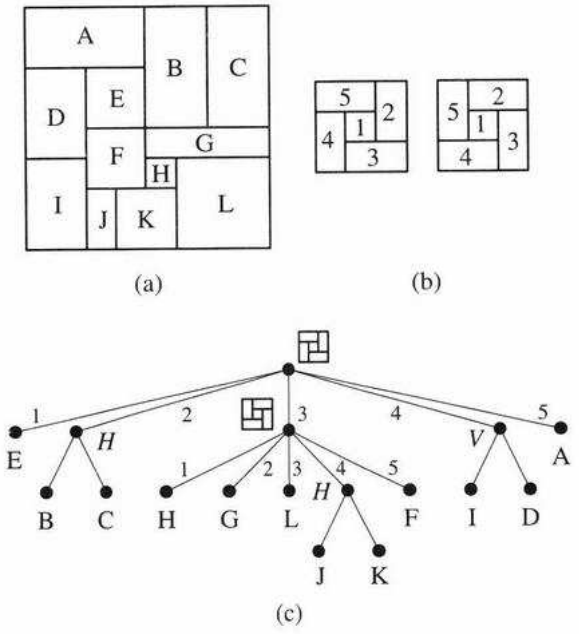


Figure 8.5 A floorplan of order 5 (a), the characterization of the operators used (b) and the floorplan tree (c).

The horizontal segments are used as vertices in the horizontal polar graph and the vertical segments as the vertices in the vertical polar graph. Each cell is represented by an edge in the polar graph. In the horizontal one, there will be an edge directed from the line segment that is the cell's top boundary to the line segment that is its bottom boundary. In the vertical one, a similar idea is used where the edge direction is from the left boundary to the right one. Figure 8.6 gives an example of the polar graph representation.

When two cells that need to be electrically connected have their terminals in the right order and separated correctly, the cells can simply be put against each other without the necessity for a routing channel in between them. Such cells are said to *abut*. The creation of a composite cell by means of the abutment of two cells is shown in Figure 8.7.

Ideally, all composite cells are created by abutment and no routing channels are used in a floorplan-based design methodology. This requires the existence of *flexible* cells, leaf cells that are generated by a cell generation program that is able to generate a certain function in a given region with predefined terminal positions. Besides, flexible cells should be able to accommodate *feedthrough* wires. Feedthrough wires are a consequence of the absence of routing channels (see Figure 8.2 for an illustration). On the other hand, floorplan-based design does not exclude the existence of routing channels. The channels can be taken care of by

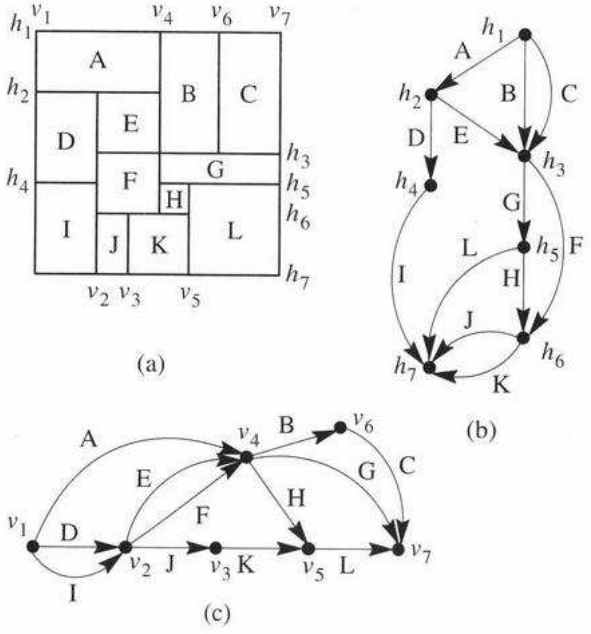


Figure 8.6 A floorplan with labeled horizontal and vertical line segments (a), its polar horizontal graph (b) and polar vertical graph (c).

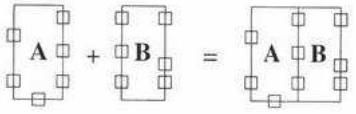


Figure 8.7 The creation of a composite cell by abutment.

incorporating them in the area estimations for the cells.

8.1.2 Optimization Problems in Floorplanning

There are a number of optimization problems related to floorplanning:

1. *Mapping of a structural description to a floorplan* (e.g. a slicing tree). In a true top-down design methodology, floorplanning will probably be performed manually or interactively as the number of children cells in which a parent cell is subdivided is relatively small and good decisions can be made based on designer experience. Another application of floorplanning is, however, the construction of a floorplan representation that is independent of the hierarchy in the structural domain. The circuit is then seen as the interconnection of many cells which have to be assigned a position. Floorplan construction is then just meant for

taking advantage of the flexible shapes of the leaf cells using floorplan sizing. The floorplan construction problem has similarities with the placement problem described in Chapter 7. The difference is that precise layout data that are known in placement are unknown in floorplanning. Anyhow, techniques known from placement like min-cut partitioning can also be used in floorplanning. At the same time, the terminal positions of abutting cells and the feedthrough routing patterns have to be determined. This problem is related to global routing and is sometimes called *abstract routing* in this context. These issues will not be discussed any further in this text.

2. *Floorplan sizing.* The availability of flexible cells implies the possibility of having different shapes for the same hardware unit. It is therefore possible to choose a suitable shape for each leaf cell such that the resulting floorplan is optimal in some sense (e.g. minimal area). This topic is discussed in the next section.

3. *Generation of flexible cells.* This task takes as input a cell shape, data on desired positions of terminals and a netlist of the circuit to be synthesized at some abstraction level (e.g. a netlist of transistors or standard cells) and uses a *cell compiler* to generate the layout that complies with the input. The problem is especially complex when the layout has to be composed of individual transistors because of the many degrees of freedom and the huge search space that is associated with it. Also, as this style of design amounts to full-custom design, quite some extra effort has to be spent in the *characterization* of the generated cells. Characterization is the process of determining all kind of electrical properties of a cell, such as parasitic capacitances and propagation delay, that will be necessary for an accurate simulation of the circuit containing the generated cell. Flexibility in a cell's shape can also be achieved using primitives belonging to a level higher than the transistor level. An example is a register file of 64 registers that can be laid out in many different ways, such as 8×8, 16×4, 4×16 or 1×64. Another possibility for cell compilation with higher-level primitives is to generate a macro cell out of standard cells and take information on cell shape and terminal positions into account during the layout generation process. The topic of cell compilation will not be discussed further in this text.

8.2 Shape Functions and Floorplan Sizing

A cell implements a certain function. When the cell is flexible, one could say that the realization needs an area A. Whichever shape the cell will have, its height h and its width w have to obey the constraint $hw \geq A$. The minimal height given as a function of the width is called the *shape function* of the cell. Figure 8.8(a) gives the shape function of a cell with constant area: $h(w) = \frac{A}{w}$. The shaded area, where $h \geq \frac{A}{w}$, contains all points (h, w) corresponding to feasible shapes of the cell. It is not realistic to have cells that are very elongated. Due to design rules neither the height nor the width will asymptotically approach zero. The shape function can be

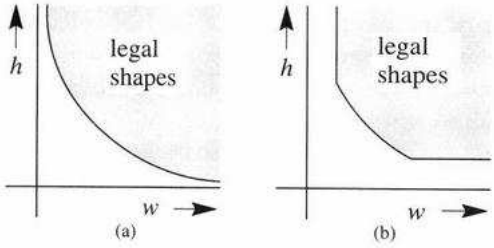


Figure 8.8 Shape functions for a cell without (a) and with (b) minimal width/height restrictions.

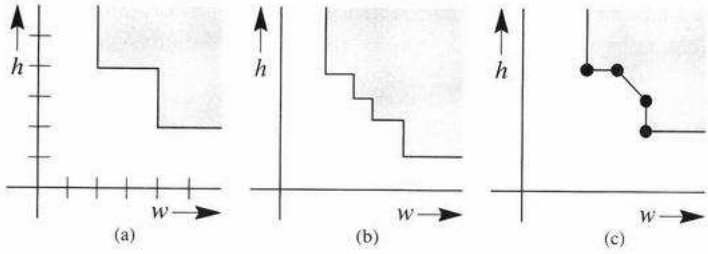


Figure 8.9 The shape function of an inset cell (a); the shape function of a cell with a discrete set of (h, w) pairs (b); a piecewise linear shape function (c).

modified such that the legal shapes have a minimal height and width as is shown in Figure 8.8(b).

Note: the concept of a function is used here in an informal sense. In order to define the shape function as a function in the mathematical sense, there should be at most one h value for each w value. It is left to the reader to refine the figures in this text and to solve the problems arising when computing the inverse of a shape function.

The shape functions mentioned so far refer to cells where height and width have continuous values. In reality, due to design rules, the height and width can only assume discrete values. Let us first consider an extreme case of a discrete-valued shape function: the *inset* or *rigid* cell. An inset cell, a predesigned cell residing in a library, has the possibility of rotations (only in multiples of 90°) and mirrorings as the only flexibility to be fit in a floorplan (note that such a cell will in general need routing channels around it). The shape function of a 2×4 inset cell is given in Figure 8.9(a). The figure has two horizontal segments, one corresponding to the horizontal orientation of the cell and the other to the vertical one. The shape function of a cell that can have a discrete number of (h, w) pairs is given in Figure 8.9(b). Each horizontal segment corresponds to a different realization of the cell.

The method for floorplan sizing, to be described below, can deal with any piecewise linear shape function. An example of such a function is given in Figure 8.9(c). A piecewise linear function is characterized by a partitioning of its input domain

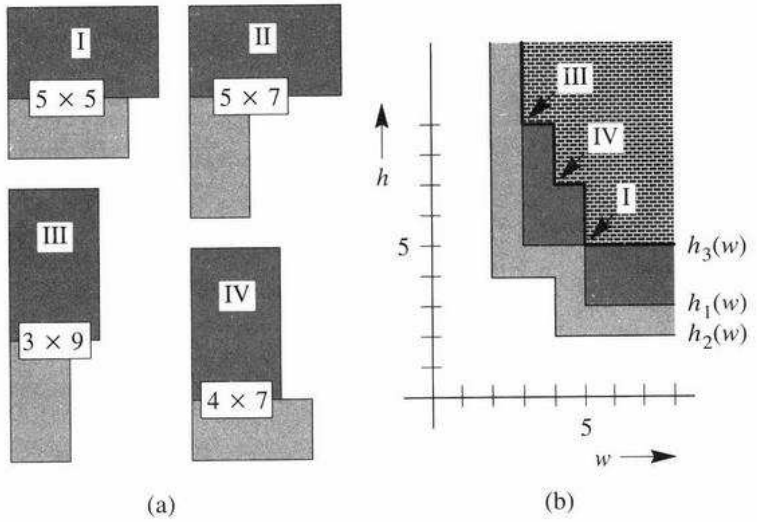


Figure 8.10 The four possible ways of vertical composition for two inset cells (a) and the shape functions related to this situation (b).

into intervals and a distinct linear behavior in each of these intervals. The points delimiting the intervals are called the function's *break points.*

The shape function of a composite cell in a slicing floorplan can be computed from the shape function of its children cells. First *vertical* composition is considered, i.e. the case where a cell c_1 is stacked on top of a cell c_2. If the shape function of c_1 is indicated by $h_1(w)$ and the one of c_2 by $h_2(w)$, then the shape function $h_3(w)$ of the composite cell can be expressed as: $h_3(w) = h_1(w) + h_2(w)$. As only piecewise linear functions are considered, h_3 can efficiently be computed by evaluating h_1 and h_2 at their break points.

This idea is illustrated in Figure 8.10 for a small example where both c_1 and c_2 are inset cells with respective sizes of 4×2 and 5×3. Clearly, there are four ways to stack the two cells vertically as shown in Figure 8.10(a); of these, the case labeled by 'I' has the smallest area. The shape functions $h_1(w)$ of c_1 and $h_2(w)$ of c_2 are given in Figure 8.10(b). $h_1(w)$ and $h_2(w)$ each have two break points: those of $h_1(w)$ are located at respectively $w = 2$ and $w = 4$, and those of $h_2(w)$ at $w = 3$ and $w = 5$. The shape function $h_3(w)$ of the composite cell can easily be constructed by evaluating $h_1(w)$ and $h_2(w)$ at these break points. It turns out that only three of the four break points are retained in $h_3(w)$ as the composite cell cannot be narrower than 3. The function values at the three break points of $h_3(w)$ in Figure 8.10(b) correspond to three of the four possible ways to realize a vertical composition shown in Figure 8.10(a), viz. the cases labeled with 'I', 'III' and 'IV'. The case labeled 'II' is a so-called *redundant* floorplan: one does not need to consider this case when searching for an optimal floorplan size.

In the case of *horizontal* composition, the shape function of a composite cell has

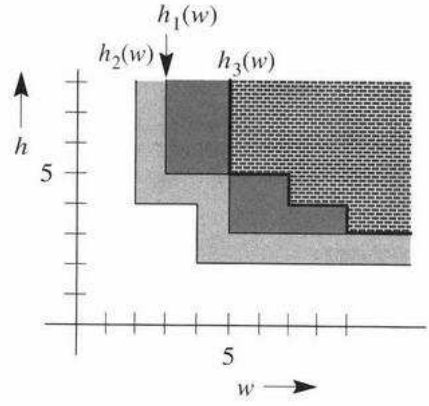


Figure 8.11 The shape functions of two inset cells and of the cell obtained by their horizontal composition.

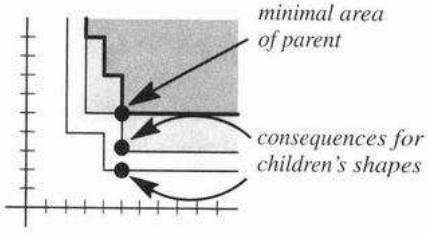


Figure 8.12 The consequence for the shape of children cells of fixing the shape of the parent cell.

to be computed using a detour via the inverses of the children's shape functions. The inverse of the composite cell's shape function is the sum of the inverses of its children cell's shape functions: $h_3^{-1}(w) = h_1^{-1}(w) + h_2^{-1}(w)$. Figure 8.11 shows the shape function resulting from the horizontal composition of the two inset cells used in the example of Figure 8.10.

Once the shape function of a composite cell is known, it is possible to choose a suitable shape for it from its legal shapes. Although one could consider other optimization goals, the most obvious entity to optimize is the area of the composite cell. A choice for the shape of the parent cell constrains the shapes of the children cells. If one chooses for a point on the boundary of the parent's shape function, the corresponding shapes of the children cells are uniquely determined. It is not so difficult to prove that the shapes of minimal area are always located on the boundary of the shape function and they will always be one of the break points if the shape function is piecewise linear.

Figure 8.12 illustrates how the children's shapes can be obtained from the optimal parent shape for the vertical composition example given in Figure 8.10. The evalu-

ation of the area at the break points of the shape function $h_3(w)$ gives $w = 5$ (and $h = 5$) as the one with minimal area. Tracing back how $h_3(w)$ was obtained from $h_1(w)$ and $h_2(w)$, one finds that the minimal area is obtained for the orientation of the inset cells shown as 'I' in Figure 8.10(a).

The procedure just described for vertical composition can easily be adapted for horizontal composition, which means that children shapes can be easily derived from the chosen parent shape for both types of composition. It is now possible to formulate the *sizing* algorithm for slicing floorplans:

1. Construct the shape function of the top-level composite cell in a bottom-up fashion starting with the lowest level and combining shape functions while moving upwards.
2. Choose the optimal shape of the top-level cell.
3. Propagate the consequences of the choice for the optimal shape down the slicing tree until the shapes of all leaf cells are fixed.

The sizing problem for slicing floorplans can be solved in polynomial time by the algorithm just mentioned. Assume that there are n cells with a piecewise linear shape function and that the total number of break points in all shape functions is q ($q \geq n$). Assume further that the depth of the slicing tree is d. Then, at each level of the tree the total number of computations for the computation of the new shape functions is $\mathcal{O}(q)$ which gives $\mathcal{O}(dq)$ for the time complexity of the complete algorithm, as Step 1 is the dominant part of the sizing algorithm. If the tree is balanced, i.e. $d = \mathcal{O}(\log n)$, then the time complexity becomes $\mathcal{O}(q \log n)$. At the other extreme, viz. a totally unbalanced tree, the time complexity becomes $\mathcal{O}(qn)$. Note that the slicing tree of the example in Figure 8.3 is rather unbalanced.

An interesting question to consider is the sizing of floorplans that are not slicing. The sizing problem is then NP-complete. Suppose that the floorplan has n leaf cells and that each leaf cell has at most k different shapes. The last assumption is quite realistic, but more restrictive than the piecewise linear shape functions that may allow an infinite number of shapes. One possible approach that could be used for any floorplan, is to the use of a general purpose optimization method such as genetic algorithms (see Section 5.8). A chromosome can consist of a string with n positions where each position indicates which of the possible shapes for each leaf cell has been included in the floorplan. The corresponding area can then be obtained by computing the height and width of the floorplan, which amounts to the computation of the longest paths in the vertical and horizontal polar graphs respectively. This can be done in $\mathcal{O}(n)$ time using the algorithm of Section 6.4.1, because the polar graphs have exactly n edges each.

The sizing of a hierarchical floorplan of order 5 can in principle be done in a similar way as the sizing of slicing floorplans. The only problem is how to find the shape function of a composite cell using an operator of order 5, given the shape function of the children cells. A brute force enumeration will lead to k^5 possibilities, if each child has k possible shapes. It turns out that one can obtain all nonredundant

floorplans using more clever algorithms having a time complexity of $\mathcal{O}(k^2 \log k)$. The use of such an algorithm in the bottom-up traversal of the floorplan tree with depth d will result in an algorithm of exponential complexity $\Omega(k^{2d})$ (disregarding the logarithmic factor for the sake of simplicity). This is not surprising given the NP-completeness of the problem.

8.3 Bibliographic Notes

Most of the textbooks on VLSI physical design automation, such as [Len90], [She93], [Sai95] and [Sar96], pay some attention to the floorplanning problem. The discussion in these books includes algorithms on how to obtain a slicing or nonslicing floorplan from a structural description, a topic that has been left out of this chapter. Floorplanning receives considerable attention in the review paper by Kuh and Ohtsuki [Kuh90] that covers VLSI layout in general.

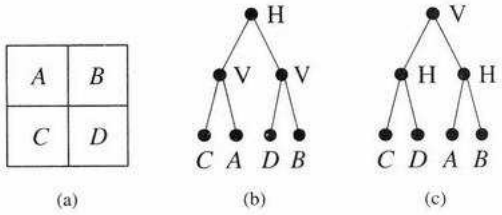
An interesting article on structured design for VLSI is [Séq83]. The floorplan-based design methodology has been advocated in many publications including [Gin84], [Zim86] and [Pig91].

Possible graph representations of floorplans that can be used both for the slicing as the nonslicing case, are discussed in detail in [Ott88].

An example of a system that uses a min-cut partitioning technique for floorplanning is [Hea85]. An abstract router to be used for floorplanning is described in [Hea87]. A system that includes floorplan construction, sizing and routing is presented in [Len93]. A review paper on cell compilation, including the generation of flexible cells, is [Gaj88a]. The example of the multiple shapes for a register file containing 64 registers originates from [Wim89].

Otten and Stockmeyer were the first to publish on floorplan sizing. The paper of Otten [Ott83] considers the sizing of slicing floorplans for cells with piecewise linear shape functions and shows that the evaluation of the shape function at its breakpoints is sufficient in order to get the minimal area. The article by Stockmeyer [Sto83] considers slicing floorplans in which all leaf cells are inset cells. It also proves that the sizing problem for floorplans in general is NP-complete. The analysis of the time complexity of the sizing algorithm presented in the text can be found in [Len90]. Shi has shown that the sizing of slicing floorplans can always be performed with a time complexity of $\mathcal{O}(n \log n)$ irrespective of the fact whether the slicing tree is balanced or not and that no faster algorithm can be designed [Shi95] (the assumption made is that each leaf cell is an inset cell).

The idea to use a genetic algorithm for floorplan sizing in a way similar to the one described in the text is presented in [Reb96]. A special-purpose algorithm is described in [Pan95]. It has the nice property that it reduces to the Otten/Stockmeyer algorithm when applied to slicing floorplans. Algorithms that find the nonredundant shapes in $\mathcal{O}(k^2 \log k)$ time when composing with an order 5 operator, are the topic of [Che93] and [Pan94].



(a) (b) (c)

Figure 8.13 A floorplan (a) and two alternatives for its slicing tree (b,c).

8.4 Exercises

8.1 Consider the floorplan of Figure 8.13(a), consisting of four leaf cells: A, B, C and D. The leaf cells are inset cells that can be rotated and mirrored. Their dimensions are:

$A : 2 \times 2$

$B : 1 \times 3$

$C : 1 \times 4$

$D : 2 \times 4$.

Give the shape functions of these cells. Compute the optimal shape of the circuit, applying the sizing algorithm given in Section 8.2 using the slicing tree of Figure 8.13(b). Repeat the computation, now using the slicing tree of Figure 8.13(c). Compare the results.

9

Routing

The *routing* problem consists of interconnecting the cells that have been assigned positions as a solution of the placement problem. The specification of a routing problem will consist of the position of the *terminals*, the *netlist* that indicates which terminals should be interconnected and the area available for routing in each layer.

Routing is normally performed in two stages. The first stage, called *global* or *loose routing*, determines through which wiring channels a connection will run. The second stage, called *local* or *detailed routing*, fixes the precise paths that a wire will take (its position inside a channel and its layer). In this chapter, the two problems will be discussed in the reverse order. Already within the class of local routing problems a multitude of specific problems can be distinguished as will become clear in the first section of this chapter. Two of these, viz. *area routing* (by means of Lee's maze routing algorithm) and *channel routing*, will then be discussed in more detail. Area routing techniques can be used by global routing algorithms as well. This is the reason why the presentation of global routing techniques is postponed until the second part of this chapter. The discussion on global routing pays considerable attention to the construction of minimum rectilinear Steiner trees.

9.1 Types of Local Routing Problems

There are many types of local routing problems. They can be characterized by some "parameters", the most important of which are listed below. Different "settings" of these parameters define different routing problems that may or may not require special techniques for solving them. The parameters are:

- The number of wiring *layers*. The number of layers available depends on the technology and the design style (some layers might already be used for other purposes: one cannot e.g. route across a cell in diffusion or polysilicon). Traditionally, most technologies offered only two wiring layers, which is a minimal requirement to realize crossing connections. Recent developments have led to technologies that make several more layers available. A contact cut that realizes a connection between two layers is often called a *via* in the context of routing.

- The *orientation* of wire segments in a given layer. *Reserved-layer* models of routing use either horizontal or vertical segments in one layer, but not both. This restriction does not always apply. Sometimes it is also allowed to use segments with an orientation that is a multiple of 45°.
- *Gridded* or *gridless* routing. In gridded routing, all wire segments run along lines of an orthogonal grid with uniform spacing between the lines. In gridless routing, wires of different widths as well as contacts (that normally are wider than minimum-width wires) are explicitly represented.
- The presence or absence of *obstacles*. Sometimes the complete *routing area* is available for routing, sometimes part of the area in one or more layers is blocked.
- The *position* of the terminals. Terminals are sometimes located on the boundary of the routing area, but might sometimes as well be located anywhere inside the area.
- Terminals with a *fixed* or *floating* position. In some problems the position of the terminals is fixed, but in other problems the router can move the terminal inside a restricted area.
- *Permutability* of terminals. Sometimes the router is allowed to interchange terminals because they are *functionally equivalent* (e.g. the two inputs of an NAND-gate are functionally equivalent).
- *Electrically equivalent* terminals. In some situations, a group of terminals belonging to the same net may already be connected to each other (e.g. in a layer not available for routing). Then the router should connect the rest of the net to only one of the terminals in this group, whichever is the most suitable.

The *area routing* problem to be discussed in the next section is characterized by a single wiring layer, a grid, the presence of obstacles, and fixed terminals in all the routing area. The *channel routing* problem to be discussed in Section 9.3 can be characterized by two reserved layers, a grid, no obstacles, a rectangular area, two rows of fixed terminals on parallel boundaries and floating terminals on the other boundaries.

9.2 Area Routing

Routing problems in which terminals are allowed anywhere in the area available for routing are normally classified as *area routing* problems. In this section, an algorithm will be presented for a simple version of area routing, namely the version for a single wiring layer. This algorithm is the "path connection" or "maze routing" algorithm published by Lee in 1961. It can be considered the first significant contribution to routing. It is mentioned here because many current-day algorithms for routing still incorporate this algorithm in some way.

The basic algorithm is meant to realize a connection between two points in a plane, in an environment that may contain obstacles. One of these points is called the

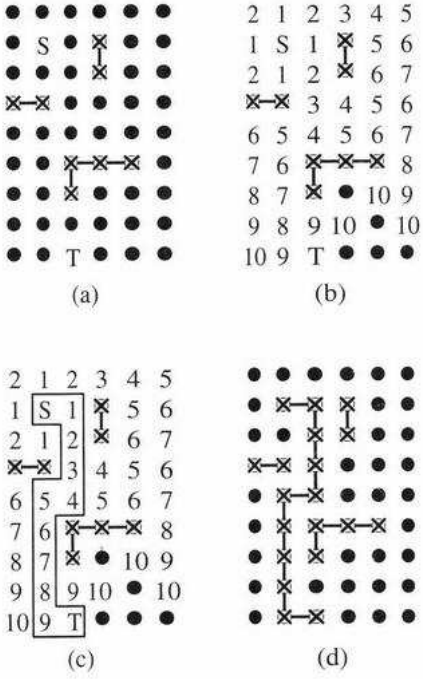


Figure 9.1 The basic version of Lee's algorithm: the points to be interconnected, S and T, together with the obstacles (a); the wave propagation from S to T (b); the path found (c); the situation to be used for new interconnections (d).

"source" terminal and the other the "target" terminal. If a path exists, the algorithm always finds the shortest connection, going around obstacles. It operates on a grid which means that the distance between two horizontally or vertically neighboring grid points corresponds to the shortest possible wire segment (the grid distance is normally derived from the design rules for a specific technology). Obstacles are modeled by grid points through which no wire segments can pass. Figure 9.1(a) shows an example problem that will be used for illustrating the different aspects of Lee's algorithm. In the figure, the dots indicate the grid points available for routing and the crosses the obstacles (e.g. wires resulting from earlier routing). The symbols "S" and "T" correspond to the source and target terminals respectively. The algorithm will try to find a connection from the source to the target.

The algorithm consists of three steps: *wave propagation*, *backtracing*, and *clean-up*. Its description in pseudo-code is given in Figure 9.2. Informally, in the first step a *wave front* of points is expanded, starting from the source terminal `S`. Points belonging to the same wave front receive the same integer-valued label, which is increased by one for subsequent wave fronts. This front goes around the obstacles in the routing area. The first step finishes when the front hits upon the target terminal `T`. This has been illustrated in Figure 9.1(b) that shows the successive wave fronts.

```
struct grid_point {
  int value; /* zero for unused, positive for a label, -1 for obstacle */
  ...
}

lee(struct grid_point S,T)
{
  set of struct grid_point wave_front, new_wave_front;
  struct grid_point neighbor, element, path_element;
  int label;
  /* Step 1: wave propagation */
  new_wave_front ← {S};
  label ← 0;
  while (T ∉ new_wave_front) {
     label ← label + 1;
     wave_front ← new_wave_front;
     new_wave_front ← ∅;
     for each element ∈ wave_front
       for each "neighbor of" element
         /* A neighbor is located above, below, at the left or at the right. */
         if (neighbor.value == 0) {
           neighbor.value ← label;
           new_wave_front ← new_wave_front ∪ {neighbor};
         }
  }
  /* Step 2: backtracing */
  path_element ← T;
  for (i ← label; i ≤ 1; i ← i − 1) {
    path_element ← "the neighbor of path_element such that
                    neighbor.value= i";
    /* In case of multiple possibilities use a heuristic to make a choice. */
    path_element.value ← -1
  }
  /* Step 3: clean up */
  for each point "on the grid"
    if (point.value > 0)
      point.value ← 0;
}
```

Figure 9.2 The pseudo-code of Lee's path-connection algorithm.

Because the ith wave front contains all points at distance i from the source, it is guaranteed that the label that the target receives equals the length of the shortest path (in this case: 10). The second step consists of determining the shortest path itself by selecting a sequence of points in the grid with decreasing labels starting from the target and going back to the source. Note that in this backtracing step, sometimes the neighbor with label i is not unique: a heuristic should be used to make a choice. An example of a heuristic is not to change the orientation of the path unnecessarily, minimizing the number of corners in the solution in this way. A possible path for the considered example is shown in Figure 9.1(c). Once a path has been found, it will act as an obstacle for the next connections to be made. In order to be able to run the algorithm for other pairs of source-target points, the labels of those points that did not take part in the solution have to be cleaned up. This is done in the third step of the algorithm. The situation for the example problem after the cleanup phase and after the conversion of the path found into obstacles is shown in Figure 9.1(d). It is easy to see that the worst-case time complexity of Lee's algorithm operating on an $n \times n$ grid is $\mathcal{O}(n^2)$. Its space complexity is also $\mathcal{O}(n^2)$.

Lee's algorithm has been adapted for different purposes, like operating with multiple layers, using various cost models, working with nets having more than two terminals, etc. Also many improvements have been made on the efficiency of the basic algorithm. In the case that there are multiple layers, the algorithm operates on a three-dimensional grid, where the size of the third dimension equals the number of layers available. Especially in this three-dimensional version of the algorithm, the use of a more sophisticated cost function becomes necessary. In the basic algorithm, the cost of a path is simply the path length: a connection between two neighboring points contributes a unity cost to the total cost. One can introduce higher than unity costs for vias, for horizontal wire segments in a layer that is meant for vertical connections, etc.

When a net has three or more terminals first a path between two terminals should be found and then a generalization of the algorithm has to be used where a path can either act as a source or a target for the wave propagation. In this way the terminals are added one by one to the routing of the net. As opposed to the version for two terminals, Lee's algorithm does not guarantee the shortest possible path, when more than two terminals have to be interconnected. Note: in the absence of obstacles, the shortest-interconnection problem for more than two terminals is equivalent to the minimum rectilinear Steiner tree problem, which has been mentioned to be NP-complete in Section 4.5.

As mentioned above, when routing several nets, the paths of the nets which already have been routed act as obstacles for those still to be routed. Actually, the fact that nets have to be routed sequentially is the weak point of Lee's algorithm. Routing the nets in a different order strongly influences the final result. For some problems, a careful ordering of the nets decides between failure and success. Heuristics have, for this reason, been proposed to determine an appropriate ordering. However, as problems are known that cannot be solved by routing in any possible order, the merit of maze routing (even with ordering heuristics) is limited. On the other hand,

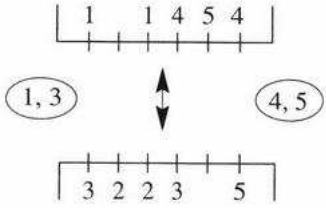


Figure 9.3 An instance of the channel routing problem.

it turns out that the principles of maze routing can be used with success within more sophisticated "iterative improvement" style algorithms where the routing of most nets is already known and a few nets are rerouted using the unoccupied grid points.

9.3 Channel Routing

Lee's algorithm is good in finding paths in an environment with many obstacles (when there are only a few paths with minimal length). However, it behaves poorly when there are few obstacles: it does not have the capability to choose a good path among the many possible paths. This section deals with such a situation without initial obstacles.

Channel routing occurs as a natural problem in standard cell and building block layout styles, but also in the design of printed circuit boards (PCBs). It consists of routing nets across a rectangular channel, as in Figure 9.3. In the figure, all terminals belonging to the same net have the same number as is customary in the field of routing. As opposed to Figure 9.1, the grid points are not shown; the grid distance is equal to the horizontal separation between the terminals. The nets have *fixed* terminals at the top and bottom of the channel and *floating* terminals at the "open" sides, at the left and right. A floating terminal is known to enter the channel on the left or on the right side, but it is up to the router to determine the exact position. In the example, the nets 1 and 3 have floating terminals at the left side and the nets 4 and 5 at the right. As a matter of fact, the height of the channel, the separation between top and bottom, is not fixed either. The main goal of channel routing is the minimization of the height, while a secondary goal is the minimization of the total wire length and the number of vias.

A routing problem that has some similarity with channel routing is *switchbox routing*. In this problem, fixed terminals can be found on all four sides of the rectangular routing area. So, opposite to channel routing, the minimization of the area is not an optimization goal. Switchbox routing is rather a *decision* problem: the goal is to find out whether a solution exists. When a solution can be found, a secondary goal is to minimize the total wire length and the number of vias.

In the rest of this section, first some more attention is paid to the channel routing models and then the problem is analyzed by means of the so-called *vertical* and

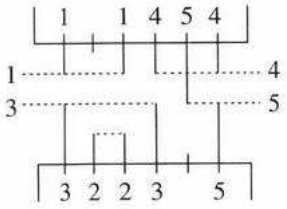


Figure 9.4 The solution of the channel routing problem of Figure 9.3: three rows are necessary.

horizontal constraints. Finally, an algorithm is presented that can solve the channel routing problem.

9.3.1 Channel Routing Models

Several models can be used for channel routing. The "classical" model is as follows:

- All wires run along orthogonal grid lines with uniform separation.
- There are two wiring layers.
- Horizontal segments are put on one layer and vertical segments on the other one.
- For each net, the wiring is realized by a single horizontal segment, with vertical segments connecting it to all terminals of the net. An exception is made when cycles in the *vertical constraint graph* (see later on) occur. In this case the problem cannot be solved unless at least one net in each cycle is realized with two horizontal segments.

The solution using the classical model of the example problem presented in Figure 9.3 is given in Figure 9.4. The vertical wire segments in the first wiring layer are shown as solid lines while the horizontal segments in the second layer are shown as dashed lines. Many variations on the classical model are possible. Routers have been designed for working without a grid. The so-called *gridless* routing model allows that each wire has a specific width (e.g. power wires that carry relatively large currents can be made wider to have less resistance). Other models use e.g. nonorthogonal grids, where 45° turns are allowed. As technology is progressing, more layers are becoming available for routing and algorithms are developed that can deal with more than two layers.

The model for routing where each layer has only wires in one direction is called the *reserved-layer* model. The use of this model can be motivated by referring to physical effects at the realization level: capacitive coupling between two wire segments running one on top of the other can lead to crosstalk. It cannot be denied, however, that this simple model with a relatively small solution space also simplifies the task of the designer of routing algorithms. Many modern algorithms work with a nonreserved layer model that has a larger solution space. The effects of using

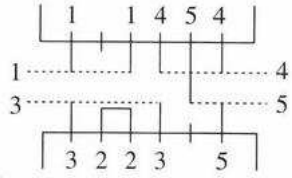


Figure 9.5 The solution of the channel routing problem of Figure 9.3 in the nonreserved layer model: two rows are sufficient.

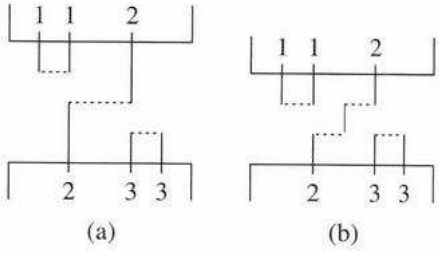


Figure 9.6 When the routing of Net 2 can only have one horizontal segment, the channel needs three rows (a); when doglegging is allowed, one row less is sufficient (b).

a nonreserved layer model are illustrated in Figure 9.5 where the solution of the problem in Figure 9.3 requires only two rows as compared to the three rows in the classical model. In the figure, the segment of Net 3 that runs on top of a segment of Net 2 has a slight offset with respect to the grid for the sake of clarity.

The requirement that all nets should be realized by a single horizontal segment, although again facilitating the task of the algorithm designer, may be too restrictive for the quality of the solution. The use of *doglegs*, i.e. of more than one horizontal segment per net, often offers the possibility of channel height reduction. Figure 9.6 shows an example of a channel routing problem where the introduction of a dogleg for Net 2 leads to a reduction of the channel height from 3 to 2.

9.3.2 The Vertical Constraint Graph

Consider a pair of terminals located in the same column and entering the channel in the same layer (see Figure 9.7(a)). It is obvious that in any solution of the problem, the endpoint of the segment coming from the top has to finish at a position higher than the endpoint of the bottom segment (otherwise, there would be a shortcircuit). This restriction is called a *vertical constraint*. Each column having two terminals in the same layer gives rise to a vertical constraint. The constraints are often represented in a *vertical constraint graph* (VCG). In this directed graph, the vertices represent the endpoints of the terminal segments and the directed edges represent the relation "should be located above" (see Figure 9.7(b)).

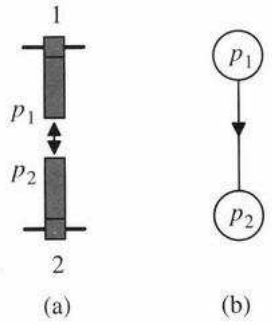


Figure 9.7 The vertical constraint between the interior endpoints p_1 and p_2 of terminal segments belonging to nets 1 and 2 (a) and its representation in the vertical constraint graph (b).

If nothing else is assumed, the VCG is rather uninteresting: it consists of pairs of vertices, one pair for each column that has two terminals in the same layer, each pair connected by a single directed edge from one vertex to the other, and unconnected vertices for the other columns. However, when the classical model for channel routing is used (one horizontal segment per net), for each net, all terminal segments end at the same row. This fact can be incorporated in the VCG by merging all vertices of the same net into a single vertex. Figure 9.8(a) shows an example problem that will be used to illustrate this. The vertical constraints considered separately for each of the columns a to h are shown in Figure 9.8(b). If one merges all vertices associated with the same net, the constraint graph of Figure 9.8(c) is obtained. This graph imposes a unique solution to the problem that is shown in Figure 9.8(d).

Up to now two extreme forms of the VCG have been presented, one of them is *fully merged* and the other one is *fully separated*. Many intermediate forms are also possible. They all have in common that one vertex corresponds to one row position for the horizontal segment that interconnects all interior endpoints of the terminal segments associated with the vertex. The main problem with the fully merged form is the possible existence of cycles, in which case the corresponding layout cannot be realized: a segment cannot be at the same time above and below another one. The problem can be solved by splitting one of the vertices in the cycle into two vertices (see Figure 9.9 for an example). Doglegging, which can lead to a reduction of the channel height as was shown in Figure 9.6, also involves splitting vertices in the VCG.

If the vertical constraints were the only constraints to be satisfied in the search of solution for channel routing, the problem would be relatively easy. In the absence of cycles in the VCG, a solution with a single horizontal segment per net would amount to finding the *longest path* in the graph. However, the presence of *horizontal constraints* that will be introduced below makes the problem considerably more complicated.

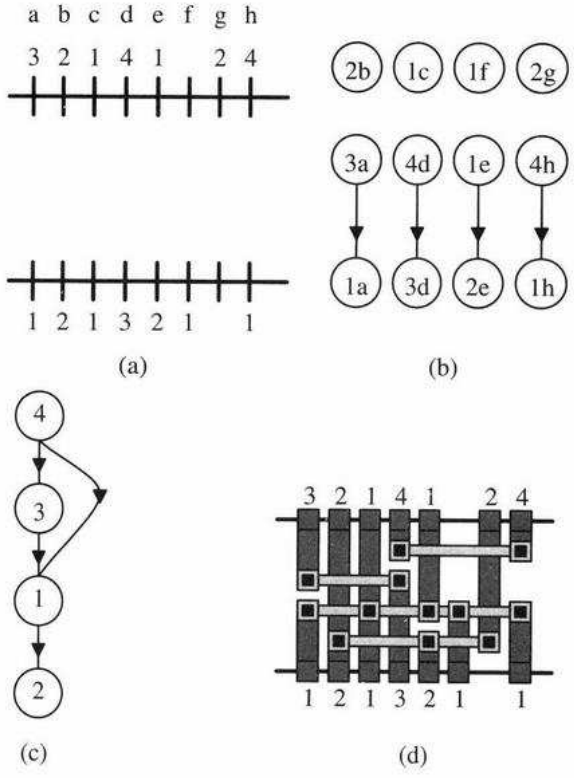


Figure 9.8 A simple channel routing problem (a), its VCG based on individual columns (b), the VCG when one horizontal segment per net is used (c) and the unique solution within the classical model (d).

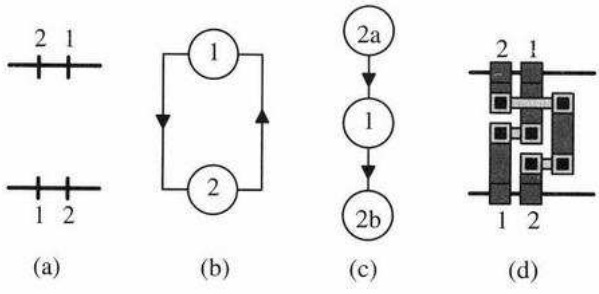


Figure 9.9 A simple problem (a) with a circular VCG (b); the cycle is broken by splitting one of the vertices (c), which makes it possible to solve the problem (d).

9.3.3 *Horizontal Constraints and the Left-edge Algorithm*

If, in the classical model for channel routing, horizontal segments belonging to different nets are put on the same row (implying that they will be in the same layer too), the segments cannot overlap (otherwise there would be a shortcircuit). This restriction is called a *horizontal constraint.*

If an instance of the channel routing problem does not have any vertical constraints (this happens e.g. when on each vertical grid line there is either a terminal at the top or at the bottom but not at both positions), the so-called *left-edge algorithm* solves the problem optimally. It was introduced by Hashimoto and Stevens for printed circuit board routing.

A net i in a channel routing problem without vertical constraints can be characterized by an interval $[x_{i_{min}}, x_{i_{max}}]$, corresponding to the left-most and right-most terminal positions of the net. The goal of channel routing is then reduced to assign a row position in the channel to each interval. No information is lost by the characterization of the net by an interval because it is known that there are no vertical constraints, and that the terminals of the net can therefore reach the horizontal segment corresponding to this interval irrespective of the row on which it will be located. An optimal solution combines those nonoverlapping intervals on the same row that will lead to a minimal number of rows.

The number of intervals that contain a specific x-coordinate x is called the *local density* at column position x and will be denoted by $d(x)$. The maximum local density in the range of all column positions is called the channel's *density* and is denoted by d_{max}:

$$d_{max} = \max_x d(x)$$

Obviously, the density is a lower bound on the number of necessary rows: all intervals that contain the same x-coordinate must be put on distinct rows.

The left-edge algorithm always finds a solution with a number of rows equal to the lower bound. Its pseudo-code is given in Figure 9.10. It is assumed that structures for the representation of intervals and linked lists of intervals, called `interval` and `list_of_interval` respectively, have been declared elsewhere. The code contains some standard "list processing" function calls: `first`(l) gives the first element of a list l; `rest`(l) gives the list that remains when the first element is removed from l; finally, `remove`(e, l) constructs a list of the elements of l in their original order leaving out those elements equal to e.

The algorithm operates on the list `i_list` that contains the intervals in order of increasing left coordinate. The main part of the algorithm consists of two nested loops. Each execution of the outer loop fills one row of the channel. The filling is done by the inner loop that scans `i_list` and removes from it the first interval that fits in the row. The algorithm is an example of a *greedy* algorithm, i.e. there is no sophisticated searching involved in the algorithm. The first "candidate" interval is directly inserted in the solution. Remarkably, the algorithm is always able to find a

```
left_edge(struct list_of_intervals i_list)
{
  /* the intervals in i_list are sorted by their left coordinate */
  set of set of struct interval solution;
  set of interval row;
  struct interval f;
  solution ← ∅;
  while ("i_list is not empty") {
      f ← first(i_list);
      i_list ← rest(i_list);
      row ← ∅;
      do { row ← row ∪ {f};
           f ← "first element in i_list nonoverlapping with f"
           i_list ← remove(f,i_list);
      } while ("such an f can be found");
      solution ← solution ∪ {row};
  }
  return(solution);
}
```

Figure 9.10 The left-edge algorithm.

global optimum, as opposed to most greedy algorithms that are likely to generate a local optimum (see Exercise 9.4).

Consider the instance of a channel routing problem without vertical constraints given in Figure 9.11(a). The seven nets can be represented by the following set of intervals: $i_1 = [1, 4]$, $i_2 = [12, 15]$, $i_3 = [7, 13]$, $i_4 = [3, 8]$, $i_5 = [5, 10]$, $i_6 = [2, 6]$ and $i_7 = [9, 14]$. The problem has a density of 3: $x = 3$ is for example contained in the intervals i_1, i_4 and i_6 (there are many more columns that are covered by three intervals). The solution found by the left-edge algorithm is: $\{\{i_1, i_5, i_2\}, \{i_6, i_3\}, \{i_4, i_7\}\}$. The routing pattern corresponding to this solution is shown in Figure 9.11(b).

The time complexity of the algorithm can easily be expressed in terms of the number of intervals n and the density of the problem d (the number of rows in the solution). Sorting the set of intervals by their left coordinate can be done in $\mathcal{O}(n \log n)$. The outer loop will be executed d times and at most n intervals from the sorted list will be inspected in the inner loop. This leads to a total worst-case time complexity of $\mathcal{O}(n \log n + dn)$. This complexity can be improved if the algorithm is slightly rewritten (see the Bibliographic Notes at the end of this chapter).

The problem of assigning nonoverlapping intervals to rows can also be described in graph-theoretical terms. A set of intervals defines a so-called *interval graph* $G(V, E)$: for each interval i, there is a vertex $v \in V$ and there is an edge (v_k, v_l) if the corresponding intervals i_k and i_l overlap. Conversely, a graph is an interval graph if it is possible to associate an interval with each vertex such that the intervals associated with adjacent vertices overlap. One should realize that a graph can be

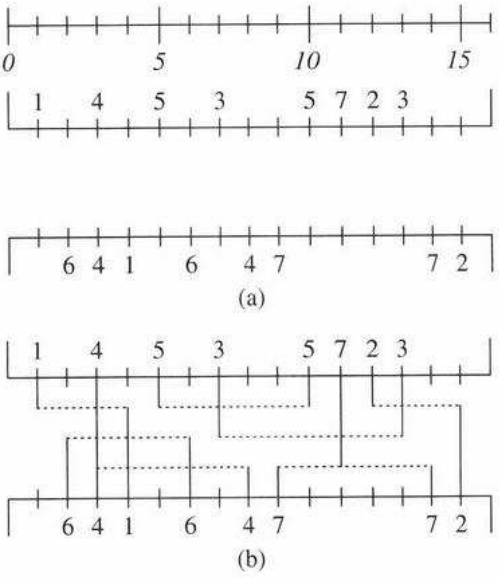


Figure 9.11 An instance of the channel routing problem without vertical constraints (a) and its solution as found by the left-edge algorithm (b).

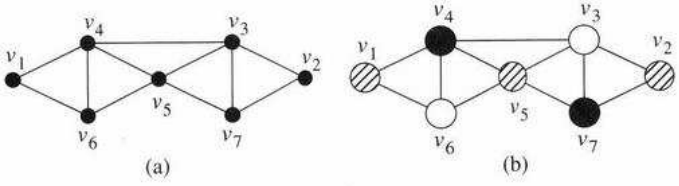


Figure 9.12 The interval graph constructed from the example problem instance in Figure 9.11 (a) and its coloring that corresponds to the solution (b).

constructed from any set of intervals, but that no set of intervals can be found for any graph. The set of interval graphs is therefore a strict subset of the set of all possible undirected graphs. The interval graph corresponding to the example of Figure 9.11 is given in Figure 9.12(a).

The problem of finding the minimum number of rows for the channel routing problem without vertical constraints is equivalent to finding a *vertex coloring* of the corresponding interval graph with a minimal number of colors. The vertex coloring problem for graphs is the problem of assigning a "color" to all vertices of the graph such that adjacent vertices have different colors and a minimal number of distinct colors are used. The problem is NP-complete for graphs in general. However, the existence of the left-edge algorithm clearly shows that the problem can be solved in polynomial time for the category of interval graphs. The coloring that corresponds

to the routing solution of Figure 9.11(b) is given in Figure 9.12(b).

9.3.4 *Channel Routing Algorithms*

The channel routing problem is NP-complete. As mentioned above, the problem can be solved optimally in polynomial time only in the case that there are no vertical constraints. One has, therefore, to be satisfied with heuristic algorithms in the more general case with vertical constraints.

In the course of time, many algorithms have been proposed for channel routing. Each of them has its strong and weak points. Below, one of these algorithms, viz. the *robust channel router* as published by Yoeli, will be presented in some detail. It has been selected here because it performs well in practice and is relatively easy to understand (there are more algorithms with similar properties).

A description in pseudo-code of the robust channel routing algorithm is given in Figure 9.13. The main loop of the algorithm iterates as often as there are rows in the channel (the number of rows is given by the variable `height`) and removes a group of nets from the problem description in each iteration. One can also say that channel routing problems of decreasing size (stored in the variable N) are solved in subsequent iterations. The selected nets will be located on the same row alternatingly either on the top or the bottom of the remaining channel (the top row is chosen when the variable `top` has a nonzero value). Each iteration consists of two parts: the assignment of weights to the nets and the selection of a maximal-weight subset of these nets. These two parts will be discussed below in more detail. It may be that the solution obtained contains vertical constraint violations. The algorithm tries to eliminate these by maze routing. Also this point will receive more attention later on.

The weight w_i of a net i expresses the desirability to assign the net to either the top or bottom row (depending on the value of the variable `top`). The side (top or bottom) that is selected in some point of the iteration will be called the "current side". Yoeli has chosen the following rules to compute the weights:

1. For all nets i whose intervals contain the columns of maximal density, add a large number B to the weights w_i. This stimulates the decrease of the density of the channel routing problem in the next iteration.

2. For each net i that has a current-side terminal at the column positions x, add to w_i the local density $d(x)$ for all x. This encourages the selection of nets with terminals at the current side.

3. For each column x for which an assignment of some net i to the current side will create a vertical constraint violation, subtract $Kd(x)$ from w_i; K is a parameter that should have a value between 5 and 10. This discourages the creation of vertical constraint violations.

Rules like these are typical "rules of thumb" that are characteristic for *heuristics*. It turns out that the performance of the algorithm is relatively insensitive to slight modifications of these rules, hence the name "robust" router.

```
robust_router (struct netlist N)
{
  set of int row;
  struct solution S;
  int total[channel_width + 1], selected_net[channel_width + 1];
  int top, height, c, r, i;

  top ← 1;
  height ← density(N);
  for (r ← 1; r ≤ height; r ← r + 1) {
    for all "nets i in netlist N"
      w_i ← compute_weight(N, top);
    total[0] ← 0;
    for (c ← 1; c ≤ channel_width; c ← c + 1) {
      selected_net[c] ← 0;
      total[c] ← total[c − 1];
      if ("some net n has a top terminal at position c")
        if (w_n + total[x_{n_min} − 1]) > total[c]) {
          total[c] ← w_n + total[x_{n_min} − 1]);
          selected_net[c] ← n;
        }
      if ("some net n has a bottom terminal at position c")
        if (w_n + total[x_{n_min} − 1]) > total[c]) {
          total[c] ← w_n + total[x_{n_min} − 1]);
          selected_net[c] ← n;
        }
    }
    row ← ∅;
    c ← channel_width;
    while (c > 0)
      if (selected_net[c]) {
        n ← selected_net[c];
        row ← row ∪ {n};
        c ← x_{n_min} − 1;
      }
      else
        c ← c − 1;
    solution ← solution ∪ {row};
    top ← !top;
    N ← "N without the nets selected in row"
  }
  "apply maze routing to eliminate possible vertical constraint violations"
}
```

Figure 9.13 The pseudo-code of the "robust" channel routing algorithm.

Once all nets have received a weight, the robust routing algorithm finds the maximal-weight subset of nets that can be assigned to the same row. Obviously, the nets selected for the subset should not have horizontal constraints. This problem can be formulated in graph-theoretical terms by constructing the interval graph implied by the intervals $[x_{i_{min}}, x_{i_{max}}]$ associated with the nets i as was discussed in Section 9.3.3.

For any graph, a set of vertices that does not contain pairs of adjacent vertices is called an *independent set*. The problem of finding the maximal-weight subset of the nets could therefore be formulated as the maximal-weight independent set problem of the corresponding interval graph. However, the maximal-weight independent set problem is NP-complete.

Fortunately, it is not necessary to solve the problem for general graphs here. An efficient algorithm is available for the maximal-weight independent set problem for interval graphs based on *dynamic programming* (see also Section 5.3). A characteristic of dynamic programming is the fact that the optimal solution of some problem instance can efficiently be defined in terms of problem instances of smaller size. In the case of the problem of obtaining the group of nonoverlapping intervals with maximal total weight, the subinstances can be identified by a single parameter γ, with $1 \leq \gamma \leq$ `channel_width`. To obtain the subinstance with $\gamma = c$, one should remove all intervals that extend beyond column position c from the original set of intervals. Consider e.g. the set of intervals $i_1 = [1, 4]$, $i_2 = [12, 15]$, $i_3 = [7, 13]$, $i_4 = [3, 8]$, $i_5 = [5, 10]$, $i_6 = [2, 6]$ and $i_7 = [9, 14]$. Then the subinstance for $\gamma = 0$ is empty, which is also true for the subinstances $\gamma = 1$, $\gamma = 2$ or $\gamma = 3$. The subinstances for $\gamma = 4$ and $\gamma = 5$ only contain i_1, while the subinstance for $\gamma = 6$ contains i_1 and i_6, and so on.

The costs of the optimal solutions for the subinstances with $\gamma = c$ are stored in the array locations `total`$[c]$. A crucial observation is then that the optimal cost for the subinstance with $\gamma = c$ can be derived from the optimal costs of the subinstances with $\gamma < c$ and the weights of the nets that have their right-most terminals at position c. There are at most two such nets (one with a terminal at the channel's top and the other at the bottom). Consider one of these nets identified with the integer n. Net n may or may not be part of the optimal solution for the subinstance with $\gamma = c$. It is part of the optimal solution if `total`$[c - 1] < w_n +$ `total`$[x_{n_{min}} - 1]$. Stated more informally, n is part of the optimal solution if n's weight added to the optimal solution for the subinstance that did not include any nets that overlapped with n, is larger than the optimal solution for the subinstance with $\gamma = c - 1$. In the pseudocode of Figure 9.13, such a test is performed twice: once for the net with a right-most terminal at the top and once at the bottom of column c. If a net is selected for some c, the net's identification is stored in the array `selected_net`.

Using the dynamic programming approach just described, it should be obvious that the arrays `total` and `selected_net` can be filled starting with the subinstance with $\gamma = 1$ (subinstances with an empty set of nets obviously have an optimal cost of zero) and moving from left to right in the channel. In order to find which nets are part of the optimal solution, one moves from right to left in the channel and makes

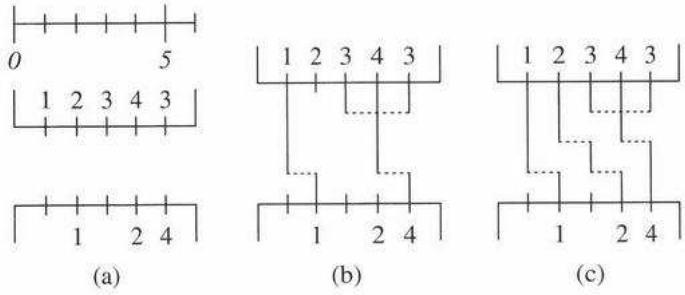


Figure 9.14 An example channel routing problem (a), the result of the robust router after the dynamic programming part (b) and after the rip-up and reroute part (c).

use of the array `selected_net` as described in the pseudo-code of Figure 9.13. Obviously, the right-most selected net is in the solution. Then one must skip any overlapping nets by jumping to the left and take again the right-most selected net, etc.

The selection of maximal-weight subsets of the nets is repeated as often as given by the variable `height` that is normally equal to the channel's density d_{max}. Although the weights are chosen such that vertical constraint violations should be avoided, it is not guaranteed that they will not occur. The robust channel routing algorithm uses a restricted maze routing algorithm to repair these violations by selectively undoing the assignments of nets to rows and rerouting these nets. Such a strategy is often called *rip up and reroute*. This will be illustrated in the example that follows after the next paragraph.

It may happen that applying maze routing in combination with rip up and reroute still does not succeed in finding a solution. In such a case, the variable `height` can be increased by one and a new attempt to solve the problem can be started. There also exist instances of the channel routing problem, like the one shown in Figure 9.9, where a solution can only be found by adding an additional *column* to the channel. This "outer loop" for height and width adjustment of the channel has not been included in the pseudo-code of Figure 9.13.

The rather lengthy description of the robust channel routing algorithm will now be illustrated by applying it to the problem instance with four nets and five columns given in Figure 9.14(a). The local densities are: $d(1) = 1$, $d(2) = 2$, $d(3) = 2$, $d(4) = 3$, and $d(5) = 2$. In the following weight calculations, the two constants in the heuristics mentioned above will be chosen as: $B = 1000$ and $K = 5$. The net weights in the first iteration ($r = 1$) are as follows (the contributions of the three heuristic rules are grouped by parentheses):

$$\begin{array}{lcrcrcrcr} w_1 & = & (0) & + & (1) & + & (-5 \times 2) & = & -9 \\ w_2 & = & (1000) & + & (2) & + & (-5 \times 3) & = & 987 \\ w_3 & = & (1000) & + & (2+2) & + & (0) & = & 1004 \\ w_4 & = & (1000) & + & (3) & + & (-5 \times 2) & = & 993 \end{array}$$

This results in the following values for the arrays `total` and `selected_net`:

$$\begin{aligned}
&\texttt{total}[1] = 0 && \texttt{selected_net}[1] = 0\\
&\texttt{total}[2] = \max(0, 0-9) = 0 && \texttt{selected_net}[2] = 0\\
&\texttt{total}[3] = 0 && \texttt{selected_net}[3] = 0\\
&\texttt{total}[4] = \max(0, 0+987) = 987 && \texttt{selected_net}[4] = 2\\
&\texttt{total}[5] = \max(987, 0+1004, 0+993) = 1004 && \texttt{selected_net}[5] = 3
\end{aligned}$$

Clearly, Net 3 is selected as the only net to occupy the top row.

In the second iteration, the current side switches to the bottom and Net 3 is left out of the new netlist N. In this situation, $d_{max} = 2$ ($d(1) = 1, d(2) = 2, d(3) = 1, d(4) = 2$, and $d(5) = 1$). This leads to the following net weights:

$$\begin{aligned}
w_1 &= (1000) + (2) + (0) = 1002\\
w_2 &= (1000) + (2) + (-5 \times 2) = 992\\
w_4 &= (1000) + (1) + (-5 \times 2) = 991
\end{aligned}$$

Making use of these, one finds:

$$\begin{aligned}
&\texttt{total}[1] = 0 && \texttt{selected_net}[1] = 0\\
&\texttt{total}[2] = \max(0, 0+1002) = 1002 && \texttt{selected_net}[2] = 1\\
&\texttt{total}[3] = 1002 && \texttt{selected_net}[3] = 0\\
&\texttt{total}[4] = \max(1002, 0+992) = 1002 && \texttt{selected_net}[4] = 0\\
&\texttt{total}[5] = \max(1002, 1002+991) = 1993 && \texttt{selected_net}[5] = 4
\end{aligned}$$

The conclusion after scanning the array `selected_net` from right to left is that Nets 4 and 1 should be put on the bottom row of the channel.

The third and final iteration is trivial: a single net, Net 2, is left for a single row. When combining the solutions of the three iterations, one discovers that the solution is not valid: putting Net 2 in the middle row of the channel would create a vertical constraint violation. This is shown in Figure 9.14(b). This means that maze routing in combination with rip up and reroute has to be applied. Clearly, Net 4 is preventing Net 2 from entering the channel and is ripped up. Net 4 can then be rerouted in the middle row. Now Net 2 can be routed with the maze router which is able to find a solution with a dogleg, as shown in Figure 9.14(c).

9.4 Introduction to Global Routing

As was mentioned at the beginning of this chapter, *global routing* is a design action that precedes local routing and follows placement. After the termination of placement, one will know the cell positions and the location of wiring channels

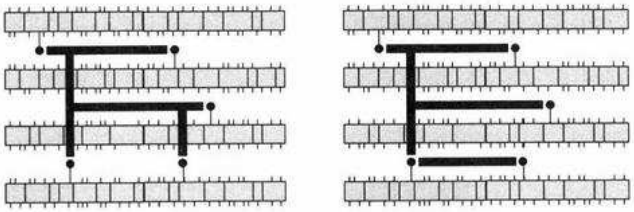


Figure 9.15 Rectilinear Steiner trees to determine a global route in standard-cell layout: a solution with minimal length (a) and one with a minimal number of cell-row crossings (b).

in between them. Global routing decides about the distribution across the available routing channels of the interconnections as specified by a netlist. Then, all required connections can be established by solving the local routing problem in each channel separately. Of course, global routing should contribute to the overall optimization goals, the most relevant of which are area minimization and observance of timing constraints.

In this section some issues that are relevant to global routing will be reviewed in general terms. In the next section, some techniques that are beneficial to the solution of global routing for standard-cell layout will be presented in more detail.

9.4.1 Standard-cell Layout

One of the most popular layout design methods for VLSI is standard cell layout. As was explained in Section 7.3, this type of layout is characterized by rows of cells separated by wiring channels. If all terminals of a net are connected to cells facing the same channel, the entire net can be routed by local routing only. If, on the other hand, the terminals of a net are connected to cells on more than two adjacent rows, the global router should split the net into parts that can be handled each by local routing.

Obtaining a wiring pattern that roughly (i.e. without performing detailed routing) interconnects all terminals of a net, amounts to constructing a *minimum rectilinear Steiner tree* (see Section 4.5). An example for a five-terminal net is shown in Figure 9.15(a). To illustrate that the exact positions within a channel are not known, the terminals have been positioned at the centers of the channels.

The rectilinear Steiner tree contains vertical segments that cross the rows of standard cells. They can be realized in different ways:

- By simply using a wiring layer that is not used by the standard cells.
- By making use of *feedthrough cells*; these are cells that are inserted between functional cells in a row of standard cells with the purpose of realizing vertical connections. As their insertion causes the cells to shift, the best results are

achieved by integrating placement and global routing and fixing the feedthrough-cell positions in this process.

- By making use of *feedthrough wires* that may be available within standard cells. These are wires that simply cross the cell without making a connection to the functional circuitry. They are sometimes incorporated in the cell layout with the purpose of facilitating the routing task.

All three variants will be called "feedthrough wires" in the following text whenever differentiation is not relevant.

In the same way that channel routing will be used to fix the exact wiring patterns in the channels, some kind of detailed routing should be used to fix the exact position of the vertical segments of the Steiner tree. First of all, it may be necessary to slightly shift the segments in order to align with feedthrough wire positions. Second, segments at approximately the same location can be permuted to reduce the densities in the channels above and below the row that they cross.

If feedthrough resources are scarce, their use can be minimized by building a Steiner tree for which vertical connections have a higher cost than horizontal ones. In the example of Figure 9.15(a), such an approach may lead to the alternative solution of Figure 9.15(b).

Another reason why a minimal-length Steiner tree may not be the optimal solution for a single net, is the existence of timing constraints. Connections that are part of a design's *critical path*, the path that determines the system's operation speed, receive special attention in *timing-driven* (or *performance-driven*) layout synthesis, both during placement and routing. Given the fact that longer wires roughly correspond to larger delays, cells connected to *critical nets* (nets that are part of the critical path) will receive a higher priority to be placed close to each other during placement.

It turns out that a more refined delay model than the simple total wire length of a net is required for the purpose of timing-driven global routing. Considering just the total wire length corresponds to a lumped electrical model composed of one resistance and one capacitance. In reality, a long wire in an IC behaves more like a *transmission line* and more accurate models should be used that partition the wire into multiple segments, each segment with its own resistance and capacitance. A widely-used model based on this principle, that shows sufficient accuracy for most purposes, is the *Elmore* delay model. It will not be further discussed here.

Assuming that the signal flow in a net is unidirectional starting from a *source* terminal and propagating to multiple *sink* terminals, signal changes will not arrive simultaneously at all sinks. Under such circumstances, properties of the chosen delay model will influence the Steiner-tree construction algorithms. It may e.g. be necessary to optimize the length of the connection from the source to the *critical sink* (this is a connection that is part of the critical path) rather than the overall tree length.

Of course, constructing Steiner trees alone is not enough to solve the global routing problem. As it is the case in local routing, the many nets in the layout interact. Finding the optimal pattern for one net may prevent another from being laid out

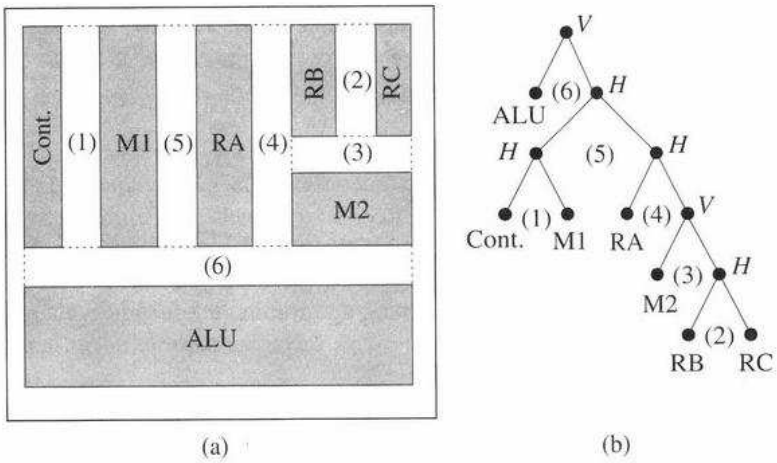


Figure 9.16 Channel definition and ordering for a building-block layout (a) and the corresponding slicing tree (b).

optimally. In one way or the other, the global router should chose the wiring patterns that minimize the overall cost function. This issue is discussed in more detail in Section 9.5.

In standard-cell layout, global routing minimizes the overall area if it minimizes the sum of all channel widths. It is not feasible to solve the channel routing problem for all channels for each tentative solution of global routing. Instead, the width of a channel is often estimated by computing the channel *density*.

9.4.2 *Building-block Layout and Channel Ordering*

Global routing for *building-block layout* is somewhat more complex than for standard-cell layout as a consequence of a higher degree of irregularity of the layout. One of the issues is the definition of routing channels. Area for routing is reserved around the cells, but it is not always obvious how this area can be partitioned into channels that can be handled by channel routers, and in which order these channels should be routed. These are respectively the *channel definition* and *channel ordering* problems. These problems are quite intricate in general, but almost trivial to solve if a *slicing floorplan* (see Section 8.1.1) of the layout is given. Consider e.g. the floorplan of Figure 8.2(b) and its slicing tree of Figure 8.3. If one assumes that the layout is composed of building blocks rather than flexible cells, one gets the layout of Figure 9.16(a). Figure 9.16(b) shows the corresponding slicing tree.

Both the layout and the tree are annotated with a number between parentheses that indicates a possible correct order for routing the channels. This order can be obtained by a *depth-first traversal* of the tree. This amounts to going down the tree starting from the root in a way similar to the depth-first search function discussed in Section 3.4.1. However, rather than processing a vertex before making the recursive

calls, the vertex is processed after the calls return. Then, a number one higher than the highest number used by the subtrees is assigned to the vertex. In Figure 9.16(b), this number is shown right below the vertex it belongs to, just between the roots of the subtrees.

The same number can be found in Figure 9.16(a) between the (composite or leaf) cells that correspond to the subtrees. The order also defines the channels. In the case of horizontal composition in the slicing tree, the channels are delimited by the top and bottom edges of the two cells involved in the composition. In the case of vertical composition, the left and right edges determine the channel borders. Note that the cells in Figure 9.16(a) always have matching top/bottom or left/right borders; this is not true in general, in which case the cell with the most extreme edges determines the channel size.

All detailed routing for a slicing floorplan can be performed with channel routing only by processing the channels in the right order. In the example of Figure 9.16(a), once Channel (2) has been routed, its floating terminals at its "bottom" side are fixed by the channel router and become fixed terminals for the top side of Channel (3). The floating terminals at the left side of Channel (3), on their turns, receive a fixed position after completing the routing of the channel and become fixed terminals for the right side of Channel (4), etc. If the floorplan is nonslicing, channel routers alone cannot complete all local routing. Switchbox routers or so-called *three-sided channel routers* are necessary for some parts of the routing (see also Exercise 9.5). A three-sided channel is a channel that has floating terminals only on one of the four borders of the rectangular routing area. It is comparable to a switchbox because the routing area cannot be stretched by the router if necessary.

Once the channels have been defined, global routing can take place. Global routing for building-block layout is not much different from the variant for standard-cell layout. However, if no additional wiring layer is available to route on top of the cells, all connections should run along the wiring channels. This means that rectilinear Steiner trees should be constructed that are entirely embedded in the routing area. The structure of the routing area, consisting of channels and the locations where they meet, can be modeled by a graph (an obvious choice is to use vertices for the meeting points and edges for the channels). Instead of looking for Steiner trees in the plane, one then needs to solve the *graph version* of the Steiner tree problem. In this version a subset of the vertices of the graph have to be interconnected by a tree using edges of the graph. Vertices of the graph not included in the subset may act as Steiner points.

9.5 Algorithms for Global Routing

This section presents a possible approach for the global routing of standard-cell layouts. The main ideas are based on the work by Lee and Sechen on sea-of-gates global routing, although the algorithm for rectilinear Steiner trees originates from Griffith *et al.* (see the Bibliographic Notes at the end of this chapter). Sea-of-gates layout has many similarities with standard-cell layout; the main differences are that the number of routing tracks in channels is fixed and that vertical routing channels

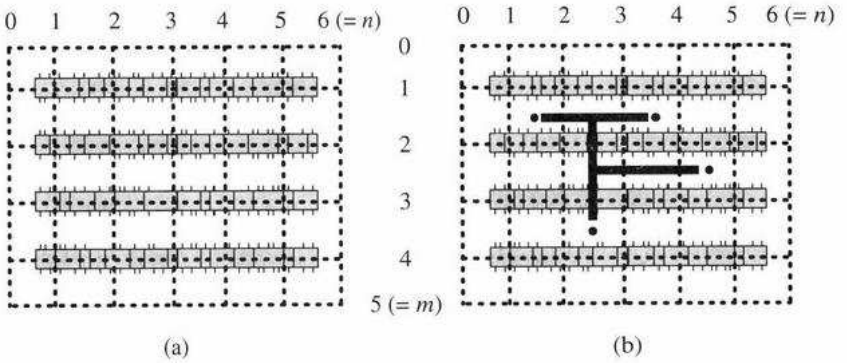


Figure 9.17 The partitioning of the standard-cell layout area by a grid for the sake of global routing (a) and a rectilinear Steiner tree embedded in this grid (b).

can be used instead of feedthrough wires. The ideas are, therefore, presented below in the context of standard-cell routing.

9.5.1 Problem Definition and Discussion

In order to achieve the appropriate degree of abstraction, the layout is covered by a grid. The horizontal grid lines are chosen such that they run across the centers of the cell rows. Vertical grid lines can be chosen such that the horizontal and vertical resolutions are roughly equal. Note that the exact distance between horizontal lines is not known in advance and depends on the results of channel routing. This idea is illustrated in Figure 9.17(a). The grid divides the routing area into elementary rectangles. The abstraction effected by the use of the grid implies that all terminals located in such a rectangle will be thought of as having the same coordinates. The points to be interconnected by rectilinear Steiner trees will then all be considered to lie at the center of these unit rectangles as is shown in Figure 9.17(b). It is, of course, the task of local routing to refine the global solution results by assigning exact positions to all wires.

Suppose that there are $m + 1$ horizontal grid lines numbered $0, 1, \ldots, m$ from top to bottom and $n + 1$ vertical grid lines numbered $0, 1, \ldots, n$ from left to right. The *local vertical density* $d_v(i, j)$ $(1 \leq i \leq m;\ 1 \leq j \leq n - 1$; the border of the layout is left out of consideration) is then defined as the number of wires crossing the vertical grid segment located on vertical grid line j between the horizontal lines $i - 1$ and i. Analogously, the *local horizontal density* $d_h(i, j)$ $(1 \leq i \leq m - 1;\ 1 \leq j \leq n)$ is defined as the number of wires crossing the horizontal grid line i between the vertical grid lines $j - 1$ and j. The density $D_v(i)$ $(1 \leq i \leq m)$ of the channel between grid lines $i - 1$ and i is then given by:

$$D_v(i) = \max_{j=1}^{n-1} d_v(i, j)$$

The goal of global routing is to minimize the *total channel density* given by:

$$\sum_{i=1}^{m} D_v(i)$$

subject to $d_h(i, j) \leq M_{ij}$ $(1 \leq i \leq m-1;\ 1 \leq j \leq n)$, where the M_{ij} are the parameters that give the maximum number of feedthroughs that can be accommodated per horizontal grid segment. All the values M_{ij} may be equal and depend on the technology and the properties of the available standard-cell library. They may also be distinct to reflect the number of feedthrough cells assigned by a placement tool (as part of integrated placement and global routing).

There are many ways to solve the problem as defined above. One could follow a sequential approach and process the nets one after the other in a similar way as was explained for local area routing (see Section 9.2). One could e.g. use a variant of Lee's algorithm that increases the segment weights crossed by a wire before routing the next segment. Such an approach has the same disadvantages that maze routing has for local routing (the dependency on net ordering, the wrong choices made when there are many equivalent solutions, etc.).

One could sequentially construct rectilinear Steiner trees for all nets. This would require a Steiner tree algorithm that is able to avoid congested areas, e.g. by using some weighted distance where the weight is derived from the connections that have already been established. One could, however, construct Steiner trees for all nets independently, examine the result for congested areas and try to modify the shapes of those trees that are the cause of overcongestion, or those trees that contribute to the reduction of the total channel density after reshaping. The algorithms presented here follow such an approach.

Instead of using the same grid during the complete routing process, one could start with a very coarse grid, say a 2×2 grid, perform global routing on this grid by assuming that all terminals covered by an elementary rectangle are located at the rectangle's center, and construct Steiner trees that evenly distribute the wires crossing the grid segments. One then gets four smaller routing problems that can be solved recursively following the same approach. The recursion stops when a sufficient degree of detail has been reached for handing the problem over to a local router. One can also continue and directly solve the local routing problem in this way. Such a *divide-and-conquer* algorithm has the advantage of limited problem size at each level of recursion. On the other hand, the subproblems to be solved are not completely independent. The decision on the ordering of wires crossing a boundary for one subproblem will constrain the search space of the neighboring one. Such a *hierarchical routing* approach may be especially attractive when it is combined with min-cut placement (see Section 7.4.1) and the floorplan-based design methodology (see Chapter 8).

Below, an algorithm for the construction of Steiner trees is presented that can operate in the (nonhierarchical) grid for the global-routing of standard cells as presented above. The algorithm constructs a Steiner tree based on a rectilinear

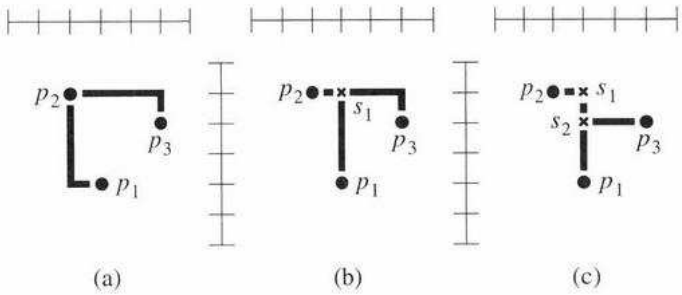


Figure 9.18 The interconnection of three points based on the minimal spanning tree (a), the introduction of a Steiner point after flipping and merging (b), and the final Steiner tree (c).

distance metric. Timing issues that may influence the tree shape are not taken into consideration. The trees are independently constructed for each net. The interaction of nets is discussed later on.

9.5.2 *Efficient Rectilinear Steiner-tree Construction*

The input of the *rectilinear Steiner-tree problem* is a set of n points P located in the two-dimensional plane: $P = \{p_1, p_2, \ldots, p_n\}$. The rectilinear distance between a pair of points $p_i = (x_i, y_i)$ and $p_j = (x_j, y_j)$ is equal to $|x_i - x_j| + |y_i - y_j|$. The goal of the problem is to find a minimal-length *tree* that interconnects all points in P and makes use of new points in the plane not included in P if these new points contribute to the tree-length reduction. The set of new points will be denoted by S.

It was already mentioned in Section 4.5 that the rectilinear Steiner tree problem is NP-complete, but that the optimal tree can be approximated by a spanning tree that can e.g. be computed in polynomial time by Prim's algorithm. It is known that the length of the tree approximated in such a way is at most a factor of $\frac{3}{2}$ longer than the minimum Steiner tree. This fact has inspired many researchers to design heuristics that improve a given minimum spanning tree. One can e.g. exploit the two possible L-shaped connections between two points that are not located on the same horizontal or vertical line and choose for those shapes that allow the merging of wire segments. Consider e.g. the situation shown in Figure 9.18(a)[1] that shows a spanning tree for the three points p_1, p_2 and p_3. "Flipping" the L-shaped connection between p_1 and p_2 leads to the merging with the L-shaped connection between p_2 and p_3 and the creation of Steiner point s_1 as is shown in Figure 9.18(b). A similar "flipping" of the newly created connection between s_1 and p_3 creates a second Steiner point s_2 as shown in Figure 9.18(c). In the new situation, s_1 is no longer a Steiner point: the connections (p_1, s_1) and (s_1, s_2) can be replaced by the direct connection (p_1, s_2)

[1] For the discussion of rectilinear Steiner trees it is more convenient to draw the net terminals on the crossings of the grid lines rather than at the centers of the elementary grid rectangles as was done in the problem definition; clearly, this does not affect the validity of the presented theory.

```
(set of struct vertex, set of struct edge) steiner(set of struct vertex P)
{
  set of struct vertex T;
  set of struct edge E, F;
  int gain;

  E ← prim(P);
  (T, F, gain) ← 1-steiner(P, E);
  while (gain > 0) {
     P ← T;
     E ← F;
     (T, F, gain) ← 1-steiner(P, E);
  }
  return (P, E);
}
```

Figure 9.19 The iterated 1-Steiner heuristic for Steiner-tree construction.

without affecting the tree length. Stated more generally, s_1 is no longer a Steiner point because its *degree* is two (it has two incident edges). This phenomenon of a point acting only temporarily as a Steiner point also occurs in the algorithm explained below.

The algorithm to be presented here also makes use of a spanning-tree construction algorithm, but does this in a different way than the heuristic approach sketched above. It is guaranteed that the tree lengths of the solutions produced by it are strictly less than $\frac{3}{2}$ times the optimal length. This means that the algorithm outperforms a spanning-tree algorithm in solution quality for all cases when a spanning-tree algorithm would have worst-case performance.

In order to explain the algorithm, first the *1-Steiner tree problem* should be defined. This is the problem of finding a spanning tree in the set of points $P \cup \{s\}$ where the point s is chosen such that the length of the resulting spanning tree is minimal. One may say that this is a version of the general minimal Steiner-tree problem with the restriction $|S| = 1$. As will be explained below, efficient algorithms exist to find such a point s. The actual Steiner-tree heuristic amounts to repetitively solving the 1-Steiner problem. The pseudo-code of this procedure is shown in Figure 9.19. The algorithm makes use of graph-theoretical data structures: a point is represented by a `vertex` and a connection between two points by an `edge`. Apart from the information required for graph-theoretical computations, the structure `vertex` should, of course, contain information on the coordinates of its location for the purpose of distance computation.

The function `prim` computes the minimal spanning tree using Prim's algorithm (see Section 3.4.4) and is supposed to return the edge set that represents the tree. The function `1-steiner` takes the vertex and edge sets of a spanning tree as input and returns three values corresponding to the vertex and edge sets of the constructed 1-Steiner tree, and the decrease in tree-length that was the result of adding one Steiner

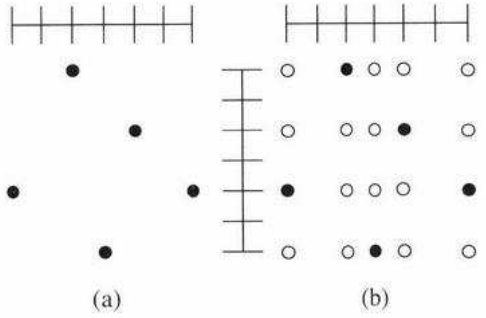


Figure 9.20 A set of points P for which the Steiner-tree should be constructed (a) and its Hanan points (b).

point. If no Steiner point could be found that improves the existing tree length, the third value is zero. This situation indicates to the calling environment that its main loop can terminate.

The principle for solving the 1-Steiner problem is quite simple: all candidate points s are visited and the spanning tree for the points in $P \cup \{s\}$ is computed each time. The point that leads to the cheapest tree is then selected. Two issues are important here. The first is to identify the set of candidate points and the second is to find a method for the spanning-tree construction that is more efficient than calling the function `prim` each time a new s is considered in the set $P \cup \{s\}$.

The first issue was settled by Hanan in 1966. He proved that an optimal rectilinear Steiner tree can always be embedded in the grid composed of only those grid lines that carry points of the set P. So, all grid lines on which no point of P is located can be left out of consideration. The candidate points are commonly called *Hanan points.* Figure 9.20(a) shows an example set of points P (the same set as in Figure 4.5(a)) and Figure 9.20(b) the Hanan points associated with the problem instance (white circles).

The best-known solution for the second issue was proposed by Griffith *et al.* It involves the *incremental* computation in linear time of the minimum spanning tree for the set $P \cup \{s\}$ given the minimum spanning tree for the set P. This is done by the function `spanning_update` in the pseudo-code of the function `1-steiner` shown in Figure 9.21. The code of the function `spanning_update` itself is shown in Figure 9.22.

The linear-time update algorithm is based on the following principles that are presented without proof. Restricting all points in a spanning tree to have at most degree four does not prevent the tree from having minimal length. The four points to which point s may be connected are the closest ones in each of the four regions obtained by partitioning the plane by two lines crossing s at angles of $+45$ and -45 degrees. In the pseudo-code, these four regions are called `north`, `east`, `south` and `west`, while the closest point to s from a point set V (excluding s itself) in a region r is computed by the function `closest_point`.

```
(set of struct vertex, set of struct edge, int)
  1-steiner(set of struct vertex V, set of struct edge E)
{
  set of struct vertex W;
  set of struct edge F;
  struct vertex maxpoint;
  int gain, maxgain;

  maxgain ← 0;
  for each s ∈ "Hanan points of V" {
    (W, F, gain) ← spanning_update(V, E, s);
    if (gain > maxgain) {
      maxgain ← gain;
      maxpoint ← s;
    }
  }
  if (maxgain > 0) {
    (W, F, gain) ← spanning_update(V, E, s);
    return (W, F, maxgain);
  }
  else return (V, E, 0);
}
```

Figure 9.21 The pseudo-code of the 1-Steiner algorithm.

```
(set of struct vertex, set of struct edge, int)
  spanning_update(set of struct vertex V, set of struct edge E, struct vertex s)
{
  int delta;
  struct vertex u, v, w;

  delta ← 0;
  V ← V ∪ {s};
  for each d ∈ {north, east, south, west} {
    u ← closest_point(V, s, d);
    delta ← delta − distance(s, u);
    E ← E ∪ {(s, u)};
    if (cycle(V, E)) {
      (v, w) ← largest_cycle_segment(V, E);
      E ← E \ {(v, w)};
      delta ← delta + distance(v, w);
    }
  }
  return (V, E, delta);
}
```

Figure 9.22 The function that incrementally computes a spanning tree when a new point is added to the original point set.

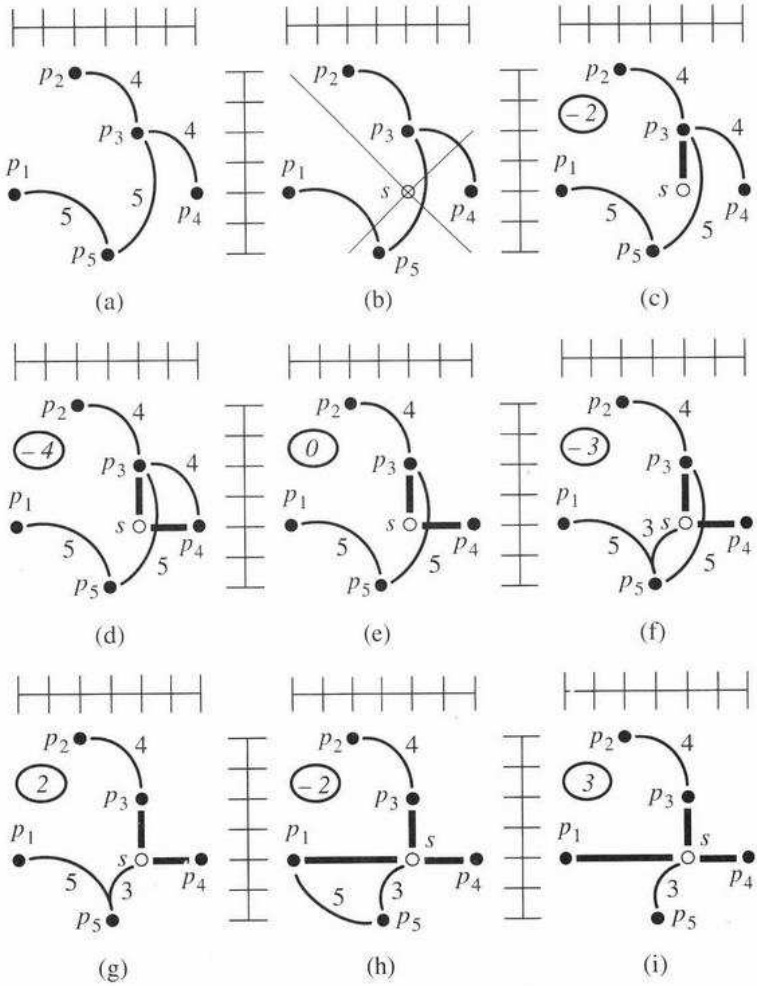


Figure 9.23 The different steps in the incremental computation of a spanning tree.

Figure 9.23 illustrates the functions `1-steiner` and `spanning_update`. The initial spanning tree for the point set P is shown in Figure 9.23(a). The connections between the points are shown as arcs rather than rectilinear segments to emphasize that no decision has been taken on the actual path that the connection follows (such as either of the L-shaped patterns). The length of the connection is shown by the label next to the arc. Figure 9.23(b) shows the new point s for which the spanning tree $P \cup \{s\}$ should be computed incrementally and the two lines that divide the area into four regions. Following the pseudo-code of Figure 9.22 the closest point to s in the `north`, `east`, `south` and `west` regions are p_3, p_4, p_5 and p_1 respectively.

The spanning tree for $P \cup \{s\}$ is constructed by connecting the point s with the

four points in each region. If the connection gives rise to a cycle (this condition is checked by the function `cycle` in the pseudo-code), the cycle should be identified and its largest segment removed. The identification is done by the function `largest_cycle_segment`. By definition of a tree, a cycle will be created for all connections except the first one. Figures 9.23(c)–(i) show how the algorithm progresses. In all figures, straight line segments are used whenever a connection between points on the same grid line is made (they are guaranteed to have the unique shortest pattern) and arcs otherwise. The value of the variable `delta` that indicates how much shorter the tree has become is shown in an ellipse at the upper left side (the function `distance` obviously computes the rectilinear distance between two points).

It is not difficult to see that the function `spanning_update` operates in linear time. Finding the four closest points can simply be done by visiting all points and saving the closest ones in each direction. The same is true for cycle detection (use depth-first search and signal detection of a cycle when coming back to a vertex that was visited earlier) and the identification of the longest segment in a cycle (keep track from where you arrive during depth-first search in order to trace the cycle). Given the fact that the number of Hanan points is $\mathcal{O}(n^2)$, the worst-time complexity of the function `1-steiner` becomes $\mathcal{O}(n^3)$.

Because the function `1-steiner` will be called at most $\mathcal{O}(n^2)$ times, the time complexity of the main function `steiner` can be stated to be $\mathcal{O}(n^5)$. This is, however, a very pessimistic estimation because it is known from the theory that a point set with n points has at most $n - 2$ Steiner points (this result follows from the fact that a Steiner point has at least degree three while the number of edges in the tree is bounded by the fact that a tree cannot have cycles by definition). On the other hand, points that have been selected as Steiner points may later become obsolete as was the case in Figure 9.18. A significant speedup can be obtained for large point sets using a *batched* version of the function `steiner`. This version takes advantage of the fact that the function `1-steiner` visits all candidate Hanan points and constructs a spanning tree for each point. It accepts a batch of incremental updates rather than a single one. This is allowed if the incremental updates are independent. Practical experiments show that the number of calls of `1-steiner` is a small constant (with an average value of about 2 for $n = 40$) that hardly grows with the problem size which justifies the claim that the overall average time complexity for constructing Steiner trees in this way is $\mathcal{O}(n^3)$.

Coming back to the example problem of Figure 9.20(a), it turns out that point s in Figure 9.23 is actually the optimal solution of the 1-Steiner tree problem (exercise: check this). In a second iteration, a second Steiner point can be found, after which no further improvement is possible. The final solution, which is optimal, is shown in Figure 9.24. The solution differs from the one of Figure 4.5(b) that is also an optimal solution for the same problem instance. In fact, it happens very often that a minimum rectilinear Steiner tree problem instance has many distinct optimal solutions. It may also happen that solutions exist with Steiner points that are not Hanan points. The next section shows how these facts can be exploited.

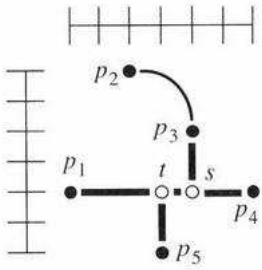


Figure 9.24 The solution found by the iterated 1-Steiner heuristic for the problem instance of Figure 9.20.

9.5.3 Local Transformations for Global Routing

Once Steiner trees for all nets have been generated independently, congested areas in the grid introduced in Section 9.5.1 can be identified. The trees of the nets contributing to the congestion can be reshaped by applying local transformations. Examples of such transformations are shown in Figure 9.25. Figure 9.25(a) shows the flip of an L-shaped connection discussed earlier. The more refined transformation of Figure 9.25(b) allows any Z-shaped connection between two points that are not on the same grid line. The transformation of Figure 9.25(c) replaces straight-line connections by U-shaped connections. Clearly, such a transformation increases the tree length but may decrease the global cost function because it reduces congestion. Figure 9.25(d) finally shows a much more sophisticated "segment shift" transformation which cannot be reversed with a single segment shift.

Algorithms for global routing may choose to use any of the types of transformation just mentioned. These transformations may be controlled by a general purpose optimization strategy such as local search or simulated annealing.

If the chosen optimization strategy fails to find a solution, maze routing may be used as a last resort. The nets crossing overcongested grid segments are then ripped up and rerouted using an appropriate variant of Lee's algorithm.

9.6 Bibliographic Notes

The textbooks [Len90], [She93], [Sai95] and [Sar96], that deal with physical design automation in general, pay quite some attention to both global and local routing. A book that is dedicated to to multilayer routing is [She95].

The original publication of Lee on the path connection algorithm is [Lee61]. Overviews of work that elaborate further on this algorithm concerning generalizations and techniques to make it more efficient can be found in [Ake72], [Rub74] and [Oht86]. The importance of Lee's algorithm for routing can also be seen from the fact that special hardware has been constructed to implement it (see e.g. [Hon83] and [Suz86]).

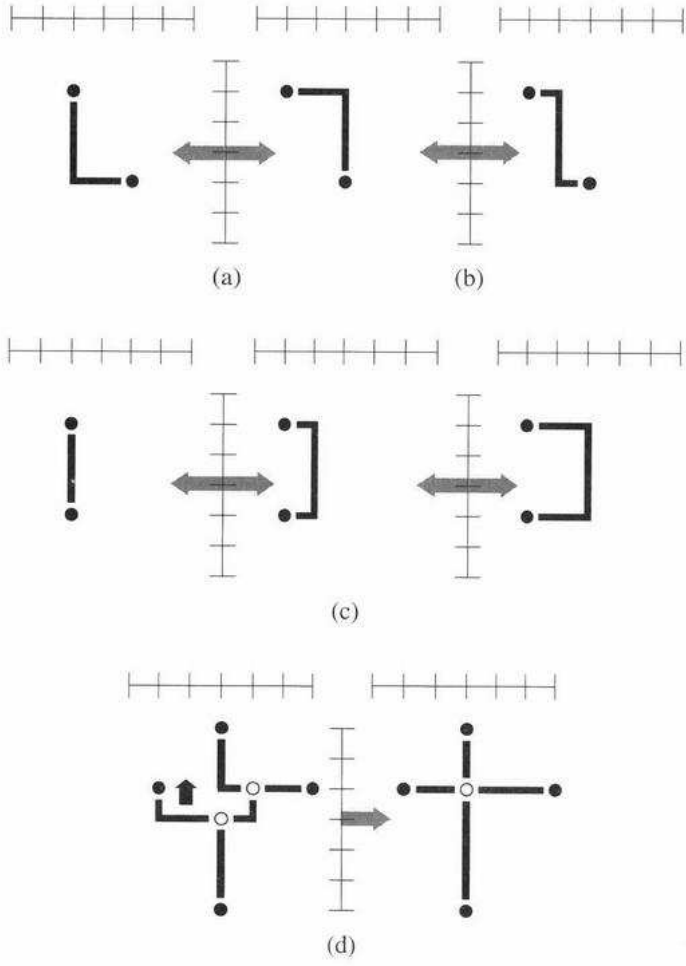


Figure 9.25 Local transformations for Steiner trees.

An example of a problem that cannot be solved by Lee's algorithm irrespective of the net ordering is given in [Oht86]. Examples of "iterative improvement" style algorithms that make use the Lee's algorithm as a subroutine are described in [Shi87] and [Lin89].

There exist review papers on channel routing by Burstein [Bur86] and on switchbox routing by Marek-Sadowska [MS92]. A famous early paper on channel routing for printed circuit board layout is [Has71]. One of the pioneering papers on channel routing that makes use of the "classical model" described in Section 9.3.1 is [Ker73]; it proposes a branch-and-bound style algorithm. An example of a gridless channel router is described in [Che86] and an example of a router using 45° segments in [Enb87]. Examples of channel routers that can deal with more than two

wiring layers are presented in [Bra88] and [Con88]. Examples of algorithms that use the nonreserved-layer model are [Joo86] and [Shi87]. The use of "doglegs" was introduced by Deutsch [Deu76].

The left-edge algorithm was originally presented in [Has71]. The paper also contains a proof that the algorithm always finds an optimal solution (with a number of rows equal to the density). The NP-completeness of graph vertex coloring for general graphs is mentioned in [Gar79]. The time complexity of the left-edge algorithm can be improved by sorting both endpoints of all intervals in increasing order and putting them in a list. The assignment to rows of the intervals can then be accomplished in a single scan of this list [Gup79]. The sorting step determines the overall time complexity in such a case, which means that the time complexity becomes $\Theta(n \log n)$ or even $\Theta(n)$ if *radix sort* (see e.g. [Aho74] or [Sed88]) is used.

The NP-completeness of channel routing is proved in [Szy85]. An example of an algorithm that uses a combination of the left-edge algorithm and maze routing is described in [Ree85]. A description of Yoeli's "robust router" can be found in [Yoe91]. The NP-completeness of the maximal-weight independent set problem follows from the NP-completeness of the maximal independent set problem which is mentioned in [Gar79]. The latter is a special case of the first in which all vertex weights are equal to one.

A review paper on global routing is [Kuh86]. It was mentioned in the text that a local reordering of feedthrough wires can affect the densities in neighboring channels. Optimization algorithms for this problem are discussed in [Che97].

A key paper on algorithms for rectilinear Steiner trees in the context of timing-driven layout is [Boe95]. It gives a justification of the Elmore delay model based on appropriate experiments and then presents Steiner-tree heuristics that directly optimize the tree cost for the Elmore delay. Some other sources for timing-driven layout are [Kuh91], [Sap93] and [Kah95].

A detailed study of the channel definition and ordering problem can be found in [Cai89].

The two papers that are the basis for Section 9.5 are [Lee91b] (for the problem formulation and the local transformations) and [Gri94] (for the Steiner-tree construction).

The hierarchical routing approach was originally proposed by Burstein for local routing [Bur83b, Bur83c] and for simultaneous placement and routing [Bur83a]. Generalizations of these ideas have been elaborated in [Hac89].

At the end of Chapter 4 a number of references on rectilinear Steiner trees were already provided. Hanan's paper in which he shows that the Steiner points in a minimum rectilinear Steiner tree always can be chosen to have a coordinate in common with the original point set, is [Han66]. The result that a minimum Steiner tree (not only a rectilinear one) for a set of n points has at most $n - 2$ Steiner points is proved in [Gil68].

Recently, new approximation algorithms have been proposed that guarantee a solution that has a tree length which is at most $\frac{11}{8}$ times longer than the optimal one. One can consult [Fös97] and its reference list for more information on this topic.

As mentioned in the main text of this chapter, many algorithms have been published for the transformation of a rectilinear spanning tree into a rectilinear Steiner tree. Polynomial-time algorithms that find the optimal Steiner tree under the restriction that a spanning tree has been taken as a starting point are presented in [Ho90].

The idea to construct a rectilinear Steiner tree based on a heuristic that repetitively solves the 1-Steiner problem is presented in [Kah92b]. This paper also proves that the heuristic deviates at most by a factor of $\frac{4}{3}$ from the optimum length for all cases that a spanning-tree would be exactly $\frac{3}{2}$ times longer. The contribution of [Gri94] is the improvement in the time complexity for the 1-Steiner problem leading to the algorithm that has been presented in the text. It contains the proofs of many statements made in this text without proof.

The local transformations for global routing mentioned in [Lee91b] are quite similar to those proposed in [Ger89] for local routing.

9.7 Exercises

9.1 It was mentioned in Section 9.2 that Lee's algorithm can be generalized for nets with more than two terminals. Indicate how the code of Figure 9.2 should be modified for this goal. Invent an example problem with three terminals for which the solution found by the modified Lee algorithm is suboptimal.

9.2 Lee's algorithm and Dijkstra's algorithm discussed in Section 3.4.3 both compute shortest paths. Show how Lee's algorithm can be interpreted to be a version of Dijkstra's algorithm.

9.3 Consider the following intervals: [1,3], [4,8], [2,5], [10,12], [11,12], [7,9], [7,11], and [3,8]. What is the density of this set of intervals? Use the left-edge algorithm (see Figure 9.10) to group these intervals in rows, such that intervals on the same row do not overlap. Draw also the interval graph corresponding to this set of intervals and give the vertex coloring of this graph corresponding to the solution obtained by the left-edge algorithm.

9.4 Show that the left-edge algorithm (see Figure 9.10) produces a solution with a number of rows exactly equal to the density of the problem. Hint: show that in each iteration of the `while` loop the density of the subproblem formed by the intervals remaining in the list `i_list` decreases by one.

9.5 Consider a building-block layout with a "wheel" floorplan (see Figure 8.4) and with routing regions around each cell. Partition the routing region in such a way that the area to be routed by conventional channel routers is maximized and identify the areas that should be handled by a switchbox and/or three-sided-channel router.

12

High-level Synthesis

High-level synthesis is the process of mapping a behavioral description at the algorithmic level to a structural description in terms of functional units, memory elements and interconnections (e.g. multiplexers and buses). The functional units normally implement one or more elementary operations like addition, multiplication, etc. This step in the design process can be made visible in Gajski's Y-chart (see Figure 1.2) as illustrated in Figure 12.1.

This chapter pays attention to several aspects of high-level synthesis (but not all of them). Section 12.1 is dedicated to *hardware models:* before mapping an algorithm to hardware, one should define the type of the target hardware. Another important point in high-level synthesis is the formal description of the algorithm to be mapped. Most high-level synthesis systems use some sort of *data-flow graph* for this purpose. These graphs are discussed in Section 12.2. Prior to the actual mapping, transformations can be applied to the input description. However, they are better understood with more knowledge of the mapping process and are therefore discussed at the end of the chapter in Section 12.6. The actual mapping consists of the tasks *allocation, assignment* and *scheduling*. The terms are introduced in Section 12.3. A number of popular scheduling algorithms are presented in Section 12.4 while a short introduction to the assignment problem is given in Section 12.5.

12.1 Hardware Models for High-level Synthesis

It is assumed that the reader is familiar with the basic notions of digital hardware design. Here only those aspects that are relevant for high-level synthesis are briefly reviewed.

Logic circuits interact with their environments by means of input and output signals. If the outputs of a circuit are observed for different patterns of input signals and it turns out that the outputs only depend on the current inputs (but not on the previous inputs), the circuit is called *combinational*. The function of such a circuit can be completely described by a *truth table* that presents the value of the output signals for each possible combination of input signal values. If, on the other hand, the output signals are not uniquely defined for a combination of input signal values, the circuit is called *sequential*. This means that the *state* of the circuit influences

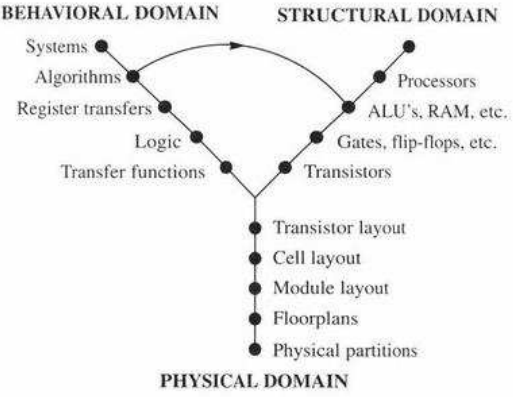


Figure 12.1 High-level synthesis as a transition in Gajski's Y-chart.

the output values, implying that the circuit has an internal memory. Sequential circuits are divided into two groups: *synchronous* or *clocked* circuits, where the state transitions can only happen on regular moments defined by one or more *clock* signals and *asynchronous* circuits, where state transitions can occur on arbitrary moments.

If no restrictions are put on the type of hardware, it becomes difficult, if not impossible, to clearly define the high-level synthesis problem. In addition, even if one succeeded to define the problem, the search space for a solution would become too large. Therefore, almost all high-level synthesis systems restrict the hardware to synchronous circuits. Also, restrictions are put on the type of hardware components used, the way they are interconnected, and the clocking strategy. All restrictions together define a high-level synthesis system's *hardware model*, the type of hardware that the system is able to generate.

12.1.1 Hardware for Computations, Data Storage, and Interconnection

A very essential hardware component is the *functional unit* (FU). This is a combinatorial or sequential logic circuit that realizes some Boolean function, such as an adder, a multiplier or an arithmetic logic unit (ALU). Figure 12.2(a) presents the symbol to be used for an FU throughout this chapter. Another essential component inherent to synchronous logic is the *register* which makes it possible to store data in the circuit (see Figure 12.2(b)). Both FUs and registers normally operate on *words*, which means that each input or output is actually realized by a number of signals carrying a *bit*. The number of bits in a word is called the *word length*.

A register is the simplest form of a *memory* element. Sometimes, using many separate registers is not very efficient, as individual wires have to be connected to each register. Registers are then combined to form so called *register files*. Registers in a register file share their input and output signals. *Address* signals indicate to which of the registers in the file the input should be written to or from which register

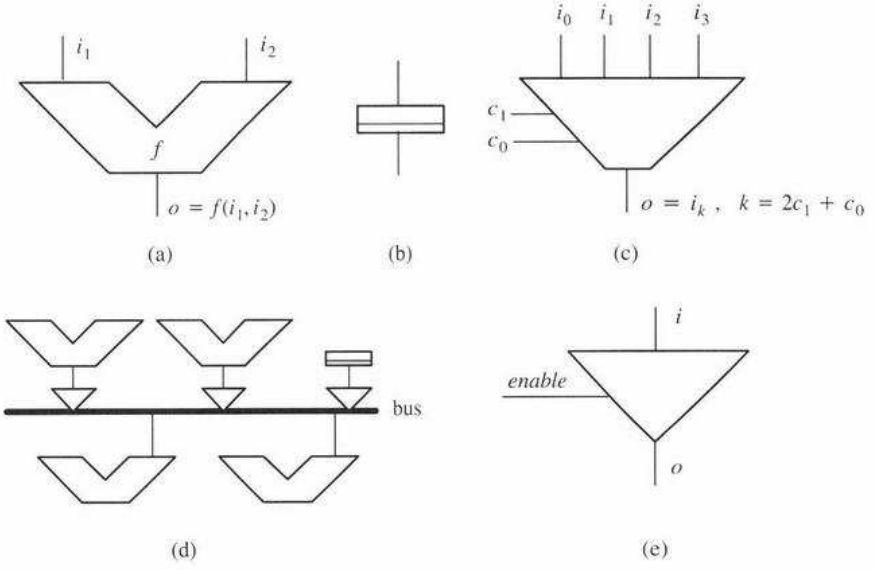


Figure 12.2 Hardware components that can be used by a high-level synthesis system: a functional unit (a), a register (b), a 4-input multiplexer (c), a configuration with a bus (d), and a three-state driver (e).

the output should be read. A register file is, in a sense, a small RAM (random access memory). In some applications, constant data are required to be present in the circuit. They can be stored in a ROM (read-only memory).

The simplest way to make connections between hardware components is by using *wires*. It often happens, however, that hardware can be used much more efficiently if it is possible to change the connections between components during a computation (e.g. an adder taking one of its inputs from register x at some moment and from register y at a later moment). One way to realize this is to use *multiplexers:* depending on the value of control signals, the output becomes equal to one of the inputs. In general n control signals can select one out of 2^n inputs. Figure 12.2(c) shows a 4-input multiplexer that requires 2 control signals. Even more efficient hardware design is possible if *buses* and *three-state drivers* (also called *tri-state* drivers) can be used. As can be seen in Figure 12.2(d), many components are connected to a bus for reading and writing. The use of buses becomes especially interesting when the word length increases. Of course, only one component at a time can write on a bus. This is accomplished by using three-state drivers to connect a component that needs to write to the bus. Depending on an *enable* signal, the three-state driver either connects its input to its output or puts its output in *high-impedant* state, which means that effectively the input and output become electrically disconnected (see Figure 12.2(e)).

12.1.2 Data, Control, and Clocks

It is a common practice to divide signals in a logic circuit into two groups: *data* signals and *control* signals. Informally, data signals carry the "operands" of the functional units. Control signals regulate the transfer of data signals between hardware components: the enable input of a three-state buffer, the control inputs of a multiplexer, the signals that select the function of an ALU, the address signals for a register file, etc. The hardware components interconnected by wires carrying data signals form the so-called *data path*. One can say that the data path is the part of the logic circuit where the actual computations are performed and their results are stored. Of course, it is also necessary to have hardware that generates the correct control signals at the correct moment. This part of the logic circuit is called the *control circuitry* or *controller*.

Most high-level synthesis systems concentrate on the generation of a data path: the term *data-path synthesis* is used in this context. The automatic synthesis of the control circuitry is normally not considered to be part of high-level synthesis. *Controller synthesis* is a field of its own that is often seen as part of logic synthesis (see also Section 2.2).

In synchronous circuits, the notion of the *system clock* essential. The duration of a computation on an FU can be expressed in multiples of the system clock period. Such a period is also called a *control step* or a *cycle*. Some high-level synthesis systems work with a hardware model where all computations are done in one clock period. In a more general hardware model, computations are allowed to last several clock periods. These are so-called *multicycle* operations. The opposite, performing more than one computation in one clock period, is called *(operation) chaining*. Multicycle operations normally occupy an FU for the total duration of the computation. However, if the FU is *pipelined,* i.e. it has internal registers, the FU can receive a new input before the output belonging to the previous input has been produced. Clearly, pipelined FUs are not combinatorial circuits. Note that the designer often has the freedom of selecting the clock frequency but is limited in his or her choice of FUs. The FUs are included in some given library. Each element of the library is characterized by some operation delay expressed in absolute time (e.g. in nanoseconds). So, the choice of the clock frequency will influence the presence or absence of multicycle operations.

In the above description, it was assumed that all computations are performed during a single clock phase, which means that all computations and data transfers take place in one clock phase, while only the master-slave updates of the registers take place in the other. Sometimes *two-phase* clocks are part of the hardware model of a high-level synthesis system. Then, one part of the data transfers is performed during one clock phase and the other part is performed in the second clock phase.

Above, many possibilities have been sketched for the hardware model of a high-level synthesis system. When designing such a system, one should make motivated choices, being conscious that these choices have consequences for the actual synthesis algorithms. Examples of choices to be made are: to allow or disallow pipelined

FUs, to only have registers or register files as well, to make interconnections with or without buses, etc.

12.2 Internal Representation of the Input Algorithm

The algorithm to be synthesized by the high-level synthesis system has to be described in some way. The usual way of doing this is in textual form, by means of a formal language. This could be in a conventional programming language or a *hardware description language.* Researchers in the field have very different opinions about the language most suitable for high-level synthesis. VHDL, Verilog and Silage are examples of such languages, just to name some popular ones. It is, however, outside the scope of this text to pay attention to this point. Whatever language is used, the textual form is not appropriate for the representation of the algorithm during the process of synthesis. Clearly, text is simply a long string of symbols and the string representation does not contain any explicit structure. One would especially be interested in the representation of the parallelism present in the algorithm. It is therefore necessary to *parse* the text and transform it into a structured internal representation. It is almost generally agreed that this representation should be graph based. The graph that is used to represent an algorithm is called a *data-flow graph* (DFG).

The main goal of this section is to introduce the notion of a DFG. Before explaining the properties of a DFG, it is necessary to mention that many definitions of such a graph exist, although all of them roughly amount to the same. The different definitions may or may not be indicated by a specific name.

The distinction made in hardware between data path and control (see Section 12.1) has made some researchers define two graphs: *data-flow* and *control-flow* graphs. However, the term "data-flow graph" is also widely used when information on control is included in the graph. The latter will be the case in this text.

Data-flow graphs are closely related to *signal-flow graphs,* which are traditionally used in the field of digital signal processing (DSP). The special requirements of DSP, such as the repetitive application of the same algorithm to data arriving at fixed intervals, have led to the development of specific DFGs for the synthesis of DSP algorithms. An important class of DSP algorithms, characterized by the absence of computations that are controlled by data-dependent conditions (such as if-then-else and while constructs), is formed by the *synchronous data-flow graphs.*

12.2.1 Simple Data Flow

A data-flow graph is a directed graph $G(V, E)$. The set of nodes[1] V is subdivided in *computational* nodes, where actual computations are performed, *input* and *output*

[1] In this chapter, the term *node* will be used for data-flow graphs, whereas the term *vertex* will be used for other type of graphs.

$$x \leftarrow a * b$$
$$y \leftarrow c + d$$
$$z \leftarrow x + y$$

Figure 12.3 A short program.

nodes for the communications with the outer world, and *conditional* nodes, where data-dependent decisions are taken. Computational nodes are either *atomic* or *composite*. Atomic nodes perform elementary computations (e.g. additions), composite nodes are DFGs themselves and therefore allow the representation of hierarchy. If one wants to allow recursively defined computations at the data-flow level, recursive computations can be expressed using composite nodes.

An edge $(v_i, v_j) \in E$ transports a stream of so-called *tokens* from node v_i to v_j. One says that the edge indicates a *data dependency* between nodes v_i and v_j which means that v_j needs the result of the computation performed on v_i before it can start its own computation. A token carries data of some specific type. In the application area of high-level synthesis, data types such as bits, n-bit integers, or floating point numbers of a given precision are carried by a token. The principle of data-flow computing is that a computational node can only be active if tokens are present at each of its inputs. It *consumes* the input tokens, applies a specific computation on them and *produces* output tokens that carry the result of the computation. One says that a node *fires* when it consumes all required input tokens.

In a general data-flow model, a node may require multiple tokens to be present on an input edge before it can fire, and may produce multiple tokens at its output after firing. When multiple tokens are present on an edge, the ordering of the tokens has to be respected: a token that was produced first at one end of an edge is consumed first at the other end. The implementation of such a graph then requires *queues* or *buffers* for the edges. An important issue is that a buffer used for an edge has the right size; it certainly has to be avoided that an unbounded number of tokens are accumulated on an edge. For purposes of simplicity, it will be assumed in the rest of the text that all nodes consume a single token from each input edge and produce a single token on each output edge. Input nodes can only produce tokens, whereas output nodes can only consume tokens.

As an example, consider the DFG representation of the program given in Figure 12.3. There, the variables `a`, `b`, `c` and `d` are inputs, `x` and `y` are the labels of intermediate edges and `z` is an output. The DFG is shown in Figure 12.4(a). The rest of the figure illustrates the token flow. All inputs produce their tokens simultaneously at, say, time $t = 0$ (Figure 12.4(b)). As they arrive simultaneously, they are immediately consumed by the addition and multiplication nodes that will compute `x` and `y` respectively. Assuming that an addition is ready earlier than a multiplication, the input tokens for the addition node that will compute `z` arrive at distinct moments in time (Figure 12.4(c,d)). When both are present, they will be consumed and some time later the output token `z` will be computed (Figure 12.4(e)).

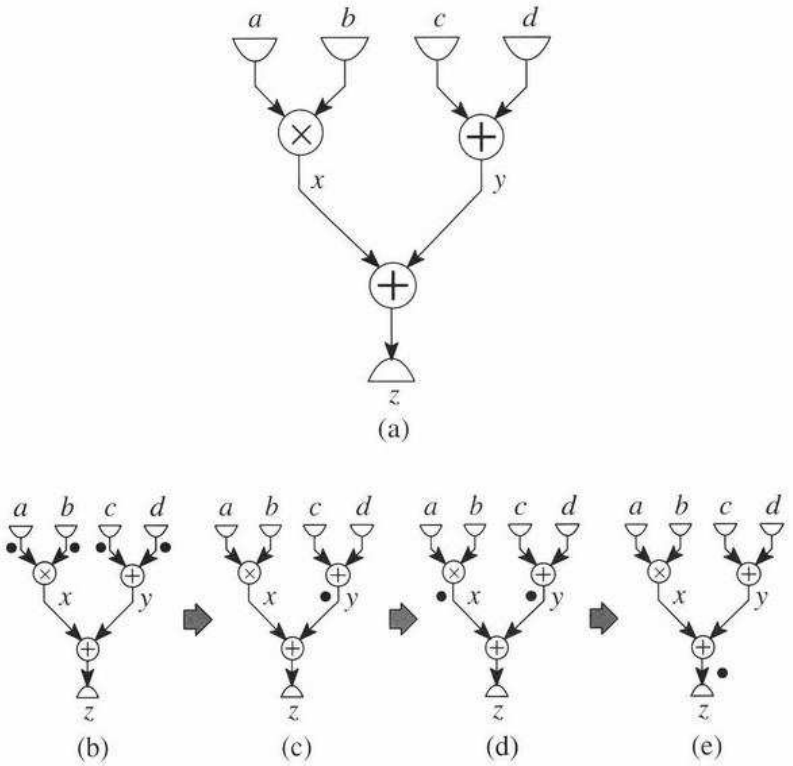


Figure 12.4 A simple DFG (a) and different stages of its execution (b-e).

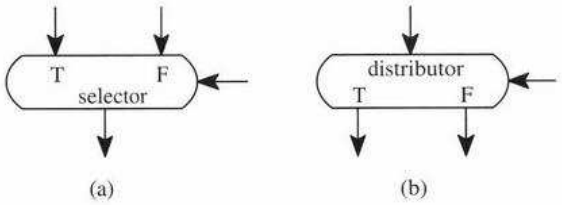


Figure 12.5 The selector node (a) and the distributor node (b).

12.2.2 Conditional Data Flow

Conditional computations require the use of two special *conditional* nodes: the *selector* node and the *distributor* node. They are shown in Figure 12.5. Both of them are characterized by a *horizontal* input that can only carry Boolean tokens. Boolean tokens can be produced by computational nodes that e.g. perform a comparison such as "less than or equal to" ($\leq$). A selector node has two inputs labeled *true* and *false*,

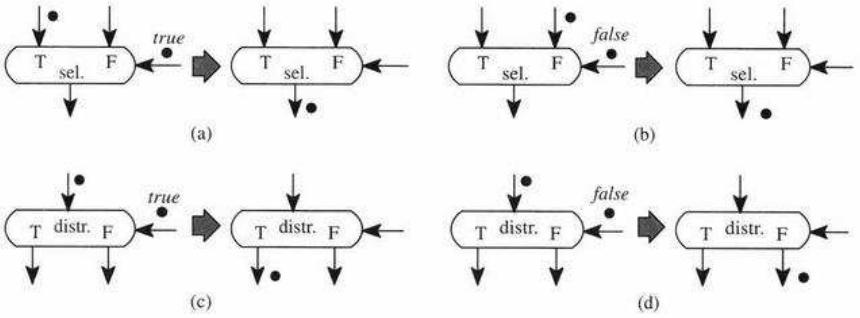


Figure 12.6 The firing rules for a selector node (a,b) and a distributor node (c,d).

```
if (a > b)
  c ← a − b;
else
  c ← b − a;
```

Figure 12.7 An example of a conditional construct.

one of which will be selected depending on the value of the token on the horizontal input. In a similar way, the horizontal input selects one of the two outputs labeled *true* and *false* of a distributor node.

Conditional nodes have firing rules that slightly differ from the firing rule for a computational node. In the case of a selector node, a node can fire if:

- there is a token with value *true* on the horizontal input and a token at the input labeled *true;* in this case, the latter token will be propagated to the output.
- there is a token with value *false* on the horizontal input and a token at the input labeled *false;* in this case, the latter token will be propagated to the output.

A distributor node can fire when both its horizontal and "vertical" inputs contain a token. However, a token is produced at only one output: if the value of the token at the horizontal input is *true* the input token is gated to the output labeled *true*; it is gated to the output labeled *false,* otherwise. The firing rules for conditional nodes have been illustrated in Figure 12.6.

Conditional nodes can be used to represent if-then-else constructs, by using combinations of distributor and selector nodes that receive the same horizontal input. An example of a conditional program fragment is shown in Figure 12.7. A corresponding DFG is shown in Figure 12.8(a). The same program fragment can also be represented in a DFG by using selector nodes only, as shown in Figure 12.8(b). The first graph makes it easier for the synthesis program to identify the conditional part of the DFG: the subgraph corresponding to the if-then-else construct is delimited by distributor nodes on one side and selector nodes on the other. Therefore, many

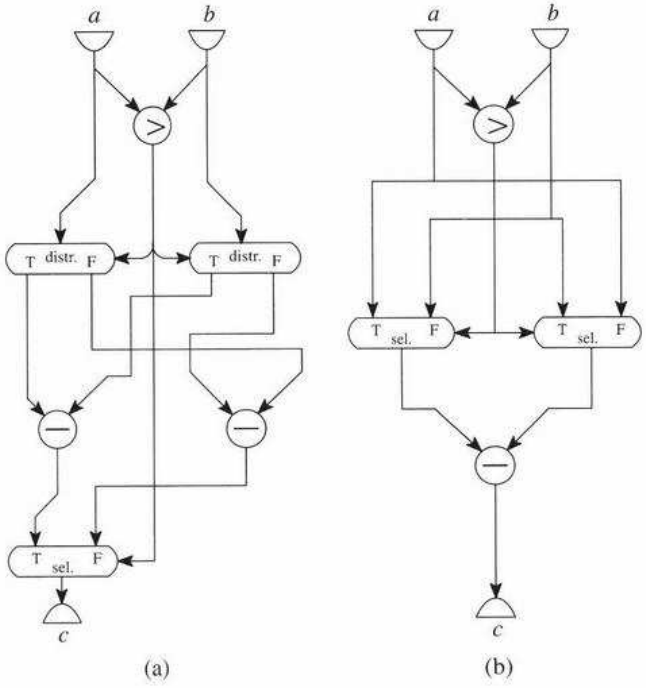


Figure 12.8 Two different DFG representations of the program fragment in Figure 12.7.

high-level synthesis systems may restrict the use of conditional nodes to structures as the one in Figure 12.8(a).

Note: as opposed to the simple DFG of Figure 12.4, the two DFGs of Figure 12.8 have nodes whose outputs are used by multiple nodes (the two input nodes and the "greater than" operator). The easiest way to represent such a situation is to have multiple edges leaving the node concerned. The consequence for the token flow is that as many tokens per computation and per output are produced as there are edges connected to the output. Each of these tokens carries identical values. For the sake of readability, the multiple outgoing edges in Figure 12.8 have been partially drawn on top of each other.

12.2.3 Iterative Data Flow

Combinations of selector and distributor nodes can also be used for the representation of iterative constructs. This can be done in many ways. However, a structured representation makes it easier to recognize these constructs in the DFG. A simple while loop is shown in Figure 12.9 as an example of an iterative construct. Its possible representation as a DFG is given in Figure 12.10. Note that an initial token with value *false* has to be present at the horizontal input of the selector node in order to allow

```
while (a > b)
    a ← a − b;
```

Figure 12.9 An example of an iterative construct.

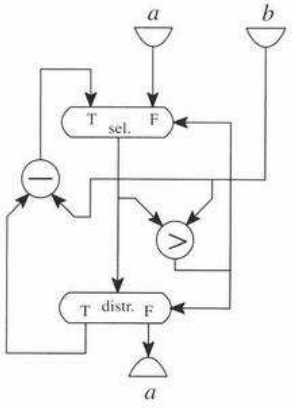


Figure 12.10 A DFG representation of the program fragment given in Figure 12.9.

the computation to start.

Note also that the representation of iteration in this way introduces cycles in the DFG. A cycle in a DFG can give rise to *deadlock:* none of the nodes in the cycle can fire as each node is waiting for its predecessor in the graph to produce a token. In the example here, the initial conditions for the conditional nodes have been chosen such that the computation can start and terminate properly. In a DFG without conditional nodes, a necessary condition for the graph to be free of deadlocks is that each cycle contains at least one edge carrying an initial token.

Apart from explicitly describing iteration as part of a DFG, there is another way of specifying a repetitive computation. This is done by having the input nodes fire repetitively: the computation is performed again each time that new inputs are available. This type of iteration is restricted to the outer loop of a computation. In spite of this restriction, it is often used in the context of digital signal processing. In many applications in this field, there is an infinite stream of input values to which the same computations should be applied yielding an infinite stream of output values.

Input values that arrive at regular time intervals introduce synchronous behavior in DFGs that are asynchronous by nature. For each set of input tokens that arrive at the start of a new iteration a corresponding set of output tokens is produced after some time. In such a synchronous system, tokens initially present on edges function as buffers that make that the output values not only depend on the current input values but also on previous ones. This is illustrated in Figure 12.11. In this figure the variables have an index related to the iteration number which they belong to. In

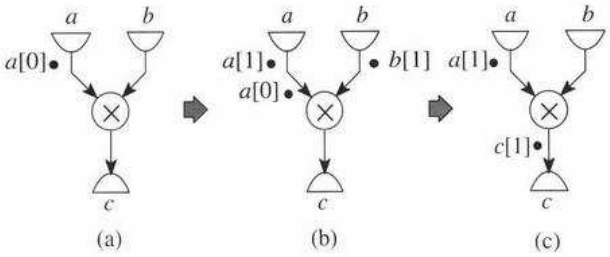


Figure 12.11 The first stages of iterative synchronous token flow in the presence of an initial token.

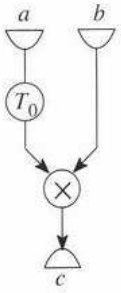


Figure 12.12 The representation of the DFG of Figure 12.11 by using a separate delay node.

DSP, it is sometimes customary to use separate *delay* nodes instead of initial tokens on the edges as shown in Figure 12.12. Such a node stores an incoming token for a period T_0, the *iteration period* of the graph (the time interval between subsequent inputs). After this period the token is passed to the output. A delay node may be associated with a *state variable* most of the time. A small example of a DFG that is often used in the field of digital signal processing, a "second-order digital filter section", is shown in Figure 12.13. In this example, all multiplications take one of their inputs from an input node. These input nodes are special in the sense that they should provide the same values in each iteration as they model multiplications with constant coefficients.

12.2.4 Data-flow Graph Representation

Above, it was stated that DFGs were very suitable to be used as the internal representation of the algorithm to be synthesized. It is therefore appropriate to consider the data structures by which DFGs can be modeled.

The first issue to note is that DFGs are more than ordinary graphs, although they are composed of nodes (vertices) and directed edges. The additional information to be modeled is the fact that the inputs and outputs of a node need to be distinguished.

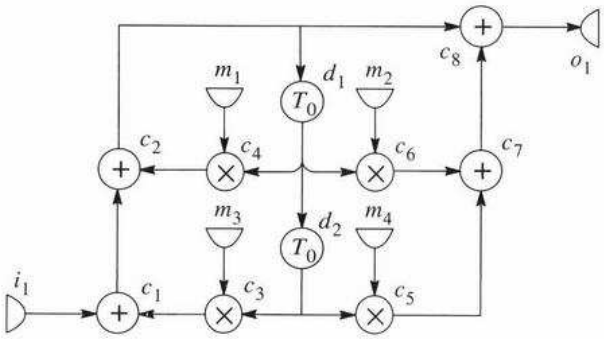


Figure 12.13 The DFG of a second-order digital filter section.

An arithmetic node like "subtraction", for example, has two inputs that cannot be interchanged without changing the algorithm represented by the DFG. Similar remarks can be made for the *true* and *false* inputs of a selector node and the *true* and *false* outputs of a distributor node. This issue can be handled by the introduction of *ports* as was done in the "cell-port-net" model for circuit representation discussed in Section 7.1. On the other hand, the "cell-port-net" is not very suitable either because edges in a DFG always connect two ports as opposed to nets that can connect to multiple ports. So, the best representation is a graph structure with two types of vertices: one for the nodes and the other for the ports respectively. Directed edges indicate the data flow between nodes as well as within the nodes. To illustrate this representation a part of the DFG of Figure 12.8(a) has been reproduced in Figure 12.14(a) while the corresponding "node-port" representation is shown in Figure 12.14(b).

As can be seen in the figure, each node can be identified by its type (input, distributor, "greater than" operation). Each port has a label (e.g. "left", "right", or "out") for unique identification. The data model should also contain additional information that is not shown in the figure. Essential information is e.g. the data type carried by an edge such as "two's complement 16-bit integer". Another thing that is useful is to internally represent "reverse edges" (for each edge a companion edge opposite to the data flow direction) such that the data structure can be traversed in any desired direction.

Note that this data model can easily be extended to deal with *hierarchical* DFGs, i.e. DFGs in which nodes can be complete DFGs themselves. Similarly to the "cell-port-net" model of Section 7.1, this can be achieved by contracting a DFG to a single "master" node. In this process, the input and output nodes of the DFG are converted into the ports of the new node.

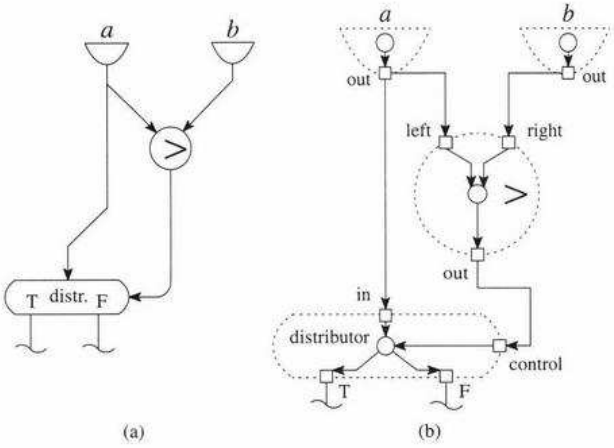


Figure 12.14 A fragment of a DFG (a) and its model for representation with data structures (b).

12.3 Allocation, Assignment and Scheduling

12.3.1 Goals and Terminology

The main issue in high-level synthesis is the mapping of the internal description of some algorithm to a hardware configuration that obeys the hardware model of the synthesis system. In the rest of this chapter, it will be assumed that the internal representation consists of a DFG. Also, it will be assumed that there are no conditional computations. Such applications are sometimes called *data dominated* as opposed to *control-dominated* applications that are characterized by conditional computations. The algorithms that are used for the synthesis of control-dominated problems are quite different from those used for data-dominated ones. They will not be discussed in this chapter.

In data-dominated problems, each operation in the DFG has to be assigned at least two entities: the functional unit (FU) on which it will be executed and the time step in which its execution will start. In addition, the remaining part of the hardware configuration has to be specified, viz. the memory elements with the values they will store at specific instants and the interconnections between all hardware units.

High-level synthesis is often divided into a number of subtasks. These tasks are normally interdependent. However, considering them as independent tasks makes it easier to define optimization problems and to design algorithms to solve them. Below, the most important subtasks are defined (actually, there is no general agreement by the community of researchers in this field on the terminology; the meanings of "assignment" and "allocation" are e.g. often interchanged with respect to those given below).

Scheduling is the task of determining the instants at which the execution of the operations in the DFG will start.

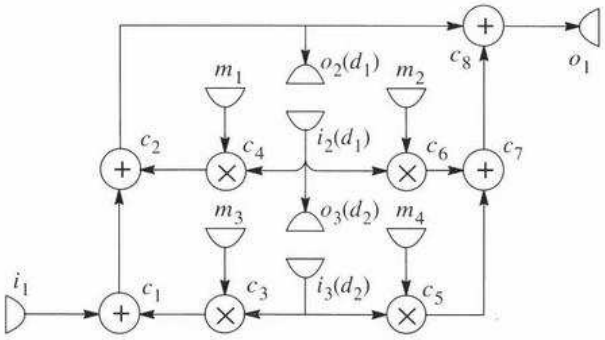


Figure 12.15 The acyclic DFG corresponding to the second-order filter section of Figure 12.13.

Assignment maps each operation in the DFG to a specific functional unit on which the operation will be executed. Assignment is also concerned with mapping *storage values* to specific memory elements and of data transfers to interconnection structures. A storage value is an intermediate result produced by the data path that needs to be stored until no operation will make use of the value anymore. Assignment is also called *binding*.

Allocation (or "resource allocation") simply reserves the hardware resources that will be necessary to realize the algorithm. So, it determines that x units of resource type A, y units of resource type B, etc. will be used, without specifying which unit will execute which operation. Another term used for this task is *module selection*.

Before continuing with the optimization issues associated with these subtasks, the issues are introduced informally. This is done in the next subsection by means of the synthesis of a small example.

12.3.2 A Detailed Example

The example used here is the second-order filter section of Figure 12.13. Although it is possible to use the DFG as such for high-level synthesis, the synthesis becomes easier if the DFG is first transformed into an acyclic graph. To this end, a delay element is regarded as a pair of output and input nodes: the input of the delay element is an output node for the DFG and the output of the delay element is an input node. The assumption behind this transformation is that the output produced in an iteration will be stored in a register which will be read in the next one. The data is then delayed for the duration of one iteration, which is exactly the functional behavior of a delay element. The acyclic DFG obtained in this way from the DFG of Figure 12.13 is shown in Figure 12.15. Note that the acyclic graph actually models a single iteration whereas the original graph models all iterations. One says that only the *intra-iteration* parallelism, i.e. the parallelism between operations belonging to the same iteration, is represented in the acyclic DFG. In the original graph, however,

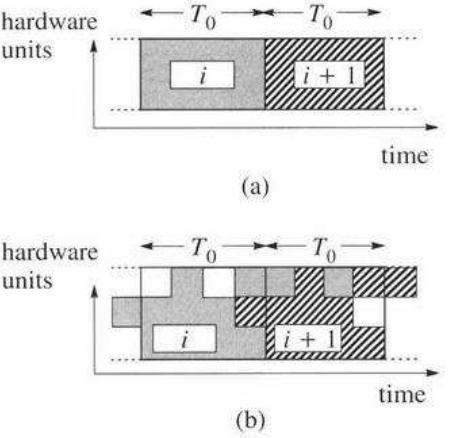


Figure 12.16 A nonoverlapped (a) and an overlapped (b) schedule for an iterative algorithm.

the *inter-iteration* parallelism or the parallelism between operations belonging to different iterations is also present.

Disregarding inter-iteration parallelism makes synthesis easier (the search space is smaller and the optimization algorithms can be simpler), but may lead to poorer results because part of the search space is not accessible to the optimization algorithm. When inter-iteration parallelism is taken into account, the search space is larger because so-called *overlapped* schedules can be generated. These are schedules where not all operations of a certain iteration need to have been executed before the next iteration can start. This is illustrated in Figure 12.16. Other names for overlapped scheduling are *loop folding* and *software pipelining*.

The synthesis of the second-order filter section based on a nonoverlapped schedule is considered first. It is assumed that only a single multiplier and a single adder are available as functional units. One could say that the decision to use this quantity of hardware was the result of the allocation subtask. The properties of the hardware are such that a multiplication takes two cycles of the internal clock and that an addition takes one cycle. Because there is only one FU of each type, the assignment task is trivial: all multiplications have to be assigned to the only multiplier present and all additions to the only adder.

One of the main issues in scheduling is that the *precedence relations* in the DFG should be respected. This means that for each edge $(v_i, v_j) \in E$ between the computational nodes v_i and v_j, v_j cannot be scheduled earlier than the time when the execution of v_i has been completed.

A schedule for the graph of Figure 12.15 is given in Figure 12.17. The figure shows a time axis of 10 clock cycles and the operations that are executed on those cycles on each functional unit. (Exercise: check that the graph cannot be scheduled in less that 10 cycles; check also that all precedence relations are respected.)

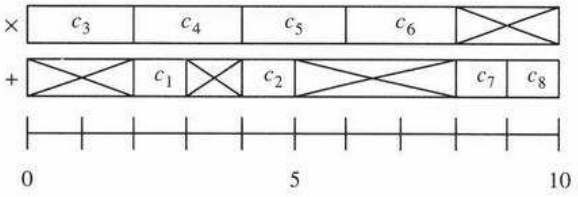


Figure 12.17 The scheduling of the additions and multiplications of the DFG of Figure 12.15.

operation	first input	second input	output
c_1	i_1	r_3	r_1
c_2	r_1	r_3	r_1, d_0
c_3	ROM	d_2	r_3
c_4	ROM	d_1	r_3
c_5	ROM	d_2	r_3
c_6	ROM	d_1	r_2
c_7	r_2	r_3	r_4
c_8	r_1	r_4	o_1

Figure 12.18 The register assignment for the example DFG given in Figure 12.15.

In order to complete the synthesis, the memory elements have to be specified and the design has to be completed by interconnecting all FUs and memory elements. In this example, it is assumed that the multiplier coefficients are stored in a ROM and that other intermediate results are stored in registers. So, specifying the memory elements amounts to performing the "register assignment" subtask. A solution is given in Figure 12.18. Apart from the registers representing the input (i_1), the output (o_1), the delay elements (d_1 and d_2), and the ROM, five registers have been introduced to store intermediate results (r_1 through r_4 and d_0). Register d_0 receives the new value to be stored in d_1 (exercise: check that the old value of d_1 still needs to be used when its new value is produced). At the end of the iteration, the value of d_0 should be copied to d_1 and the value of d_1 to d_2.

After realizing the interconnections between the registers and the FUs and introducing multiplexers where necessary, one obtains the network as shown in Figure 12.19. (Verify the correctness of the register assignment and the network.) The actual result of the synthesis is this network augmented with the specification of the required control signals in this network. The control signals can be derived directly from the schedule and the register assignment. (Specify for each clock cycle the control signals: the control signals for the multiplexers, the write-enable signals for the registers, and the ROM addresses).

Consider now the synthesis of the second-order filter section based on an overlapped schedule. It should be based on the DFG of Figure 12.13 (the delay elements

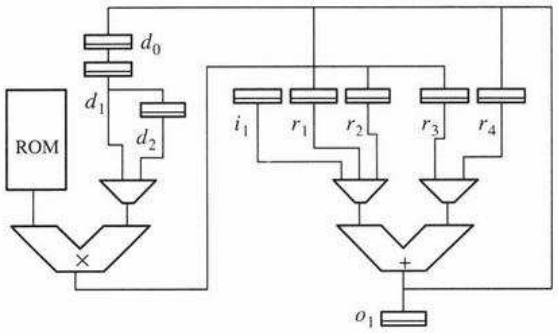


Figure 12.19 The network resulting from the synthesis of the DFG of Figure 12.15.

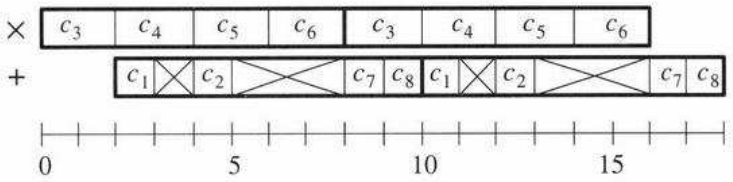


Figure 12.20 Two iterations of an overlapped schedule for the DFG of Figure 12.13.

should not be transformed into pairs of input and output nodes). Assuming that again only one multiplier and one adder are available, the solution to the scheduling problem will be as given in Figure 12.20. The figure actually shows two iterations to emphasize the overlapped nature of the schedule. From the schedule it can be seen that the computation can be repeated every 8 clock cycles compared to an iteration period of 10 in the case of the nonoverlapped schedule. The generation of the register assignment and interconnection hardware is similar to the nonoverlapped case and is not further considered here (see also Exercise 12.3).

12.3.3 Optimization Issues

The function to be optimized in high-level synthesis is in principle the same as for a VLSI design problem as a whole (see Section 1.1). In order to simplify the problem, however, all hardware units are characterized by two parameters only: *speed* and *cost*, where the cost normally mainly reflects the *area*. Another entity, the optimization of which is becoming more and more important, is *power consumption*. However, this optimization issue will be kept outside the discussion in this chapter.

By making use of these two parameters, two problems can be investigated:

1. Given the time within which the hardware has to complete its computation, find the hardware configuration with minimal overall cost. The overall cost is simply

computed by adding the cost parameters of all hardware elements used. This problem is called *time-constrained* synthesis.

2. Given an allocation of hardware, find a scheduling and assignment such that the total computation is completed in minimal time. This problem is called *resource-constrained* synthesis. In another version of this problem an allocation is not given, but a limit to the overall cost is specified.

There is actually a third problem that amounts to the decision version of the two optimization problems mentioned above: check whether a network can be constructed to execute an input algorithm within a given time, while limitations are given as well on either the use of individual hardware elements or the overall cost.

The problem is often further simplified by considering FUs as the only hardware elements contributing to the cost. This means that the use of memory elements and interconnection is not optimized in the first place. The optimization of these resources is done as a *postprocessing* step after a synthesis step in which only the FU cost is taken into account.

In the rest of this chapter, the following scheduling problem is given some deeper attention:

- The schedule should be nonoverlapping.
- The input for the scheduler consists of a DFG $G(V, E)$ and a *library* $\mathcal{R}$ of FUs ($\mathcal{R}$ is also called the set of resource types). $G(V, E)$ does not have conditional nodes. For each node $v \in V$, the input description also states on which library element $r \in \mathcal{R}$ this node will be executed. This mapping is denoted by a function $\rho : V \to \mathcal{R}$. Because of this unique mapping and the known characteristics of the library, a time $\delta(v)$ can be associated with each $v \in V$ indicating the number of control steps required to execute v. The cost of an element $r \in \mathcal{R}$ is given by $\omega(r)$.
- The problem is time-constrained. The hardware resulting from synthesis should complete the required computation at time T_0 at latest. This means that the time instants from the set $\mathcal{T} = \{0, 1, 2, \ldots, T_0 - 1\}$ are available for an operation to start its execution (of course, an operation $v \in V$ with duration $\delta(v) > 1$, can start at time $T_0 - \delta(v)$ at latest).
- A schedule σ is a function that gives the starting time for the execution of each operation: $\sigma : V \to \mathcal{T}$. Obviously, a schedule can only be legal if the following inequality is satisfied for each edge $(v_i, v_j) \in E$ between computational nodes v_i and v_j:

$$\sigma(v_j) \geq \sigma(v_i) + \delta(v_i) \tag{12.1}$$

- Assuming that there is a "requirement function" $N_r(\sigma)$ that gives the minimal number of resources of type r that are necessary to execute the schedule σ, the goal of this problem is to find a σ such that the overall cost given by:

$$\sum_{r \in \mathcal{R}} \omega(r) N_r(\sigma) \tag{12.2}$$

is minimized.

N_r is actually a function that has to perform the assignment for each resource type separately. Depending on the variant of the synthesis problem involved, the assignment problem may be tractable or intractable, as will be explained in Section 12.5. As the scheduling problem preceding the assignment is NP-complete, the synthesis problem defined here is intractable in any case. One therefore has to be satisfied with heuristic scheduling algorithms. A number of them are discussed in the next section.

12.4 Some Scheduling Algorithms

12.4.1 ASAP Scheduling

ASAP (an abbreviation of "as soon as possible") scheduling is one of the simplest ways to find a solution to a scheduling problem with precedence constraints. An operation in the DFG is scheduled exactly at the moment that all its predecessor nodes in the graph have completed their computations.

Finding the ASAP schedule of an acyclic DFG is equivalent to finding the longest path in a directed acyclic graph. An algorithm to find such a longest path was presented in Figure 6.8 in the context of layout compaction. In order to be able to apply this algorithm, a simple directed graph $H(\{v_0\} \cup V, F)$ derived from the DFG $G(V, E)$ should be constructed as follows:

- All edges $(v_i, v_j) \in E$ are also member of F.
- Each edge (v_i, v_j) such that v_i is a computational node, has to be assigned a weight $w((v_i, v_j)) = \delta(v_i)$, the computational delay of the computational node v_i that puts a token on the edge. Note that in this way all inequalities obeying Inequality (12.1), are represented in the graph in the same way as the inequalities of the layout compaction problem.
- An edge (v_i, v_j), where v_i is an input node, receives weight $w((v_i, v_j)) = 0$. This actually models that all inputs are available at time 0. Inputs that arrive later can be represented by edges with an appropriate weight. Also, new edges (v_0, v_i) with weight $w((v_0, v_i)) = 0$ are added to F for each input node v_i. v_0 is a dummy source node that is required by the algorithm presented in Figure 6.8.

The values x_i found by the algorithm denote the ASAP scheduling times $\sigma(v_i)$ of the computational nodes v_i. The transformed graph of the example in Figure 12.15 is shown in Figure 12.21.

Note: the scheduling times found by ASAP scheduling are the earliest possible; the scheduling problem therefore only has a solution if for all computational nodes $\sigma(v_i) + \delta(v_i) < T_0$ (see also Subsection 12.3.3).

ASAP scheduling has the disadvantage that the algorithm nowhere refers to the resource usage. Therefore, no attempt is made to minimize the cost function of Formula (12.2). However, ASAP scheduling can be used as part of more sophisticated algorithms such as those discussed in the next subsections.

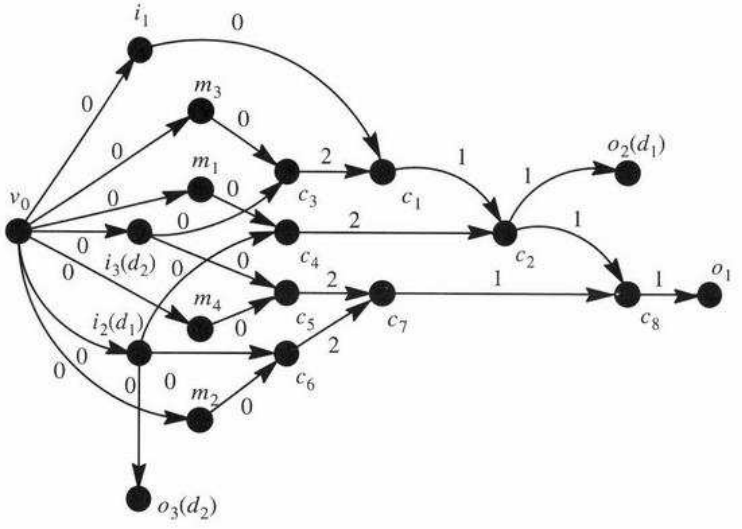


Figure 12.21 An adaptation of the graph of Figure 12.15 such that the longest-path algorithm of Figure 6.8 can operate on it.

12.4.2 Mobility-based Scheduling

In ASAP scheduling one computes the earliest time at which an operation can be scheduled. One can also do the opposite and compute the latest time at which an operation can be scheduled, knowing that all operations should be ready at time T_0. It is not difficult to adapt the longest-path algorithm described above to work from the outputs backwards and find the "as late as possible" (ALAP) times in this way (see Exercise 12.4).

ALAP scheduling, as such, has the same disadvantages as ASAP scheduling. However, combining the information obtained by both ways of scheduling gives rise to more powerful heuristics. If the ASAP scheduling time of node v_i is denoted by $\sigma_S(v_i)$ and the ALAP time by $\sigma_L(v_i)$, the interval $[\sigma_S(v_i), \sigma_L(v_i)]$ contains all possible time instants at which v_i can be scheduled. This interval is sometimes called the *time frame* or the *scheduling range* of the operation. The length of the interval, i.e. $\sigma_L(v_i) - \sigma_S(v_i)$, is called the operation's *mobility*.

Mobility can be used as the basis of several scheduling heuristics. Such a heuristic investigates the time instants within the scheduling range of an operation and chooses an instant in such a way that the usage of resources is optimized. Fixing the time of an operation can affect the mobility of the yet unscheduled operations. Their mobilities may decrease. Consider e.g. the simple DFG of Figure 12.22 in which the input and output nodes are connected by three nodes in series, each with a delay of 2 clock cycles and $T_0 = 10$. The initial scheduling ranges for the nodes c_1, c_2, and c_3 respectively are: [0,4], [2,6], and [4,8]. If the scheduling algorithm decides to fix c_2 at time 5, the ranges become: [0,3], [5,5], and [7,8].

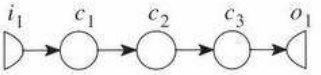


Figure 12.22 A simple DFG.

```
"determine σ̃^(0) by computing σ_S and σ_L";
k ← 0;
while ("there are unscheduled operations") {
    v ← "one of the nodes with lowest mobility";
    "schedule v at some time that optimizes the current resource utilization";
    "determine σ̃^(k+1) by updating the scheduling ranges
     of the unscheduled nodes";
    k ← k + 1
}
```

Figure 12.23 A simple mobility-based scheduling algorithm.

When talking of scheduling ranges that are decreased in the course of an algorithm, the concept of a *partial schedule* is very useful. A partial schedule $\tilde{\sigma}$ is a function that gives a scheduling range for each computational node in a DFG: $\tilde{\sigma} : V \rightarrow [\mathcal{T}, \mathcal{T}]$. Let the lower bound of the interval for a node v be denoted by $\tilde{\sigma}_{min}(v)$ and the upper bound by $\tilde{\sigma}_{max}(v)$. Many mobility-based scheduling algorithms generate a sequence of partial schedules: $\tilde{\sigma}^{(0)}, \tilde{\sigma}^{(1)}, \ldots, \tilde{\sigma}^{(n)}$. $\tilde{\sigma}^{(0)}$ gives the ASAP and ALAP schedules. In each step the mobility of at least one operation decreases, while no operation's mobility increases. The final partial schedule $\tilde{\sigma}^{(n)}$ is actually a complete schedule σ, as for all nodes $v \in V$ the following should be true: $\tilde{\sigma}^{(n)}_{min}(v) = \tilde{\sigma}^{(n)}_{max}(v) = \sigma(v)$.

The pseudo-code of a general algorithm based on these ideas is presented in Figure 12.23. This is a very general algorithm. Many details have to be elaborated before the algorithm can be applied in practice. The algorithm uses the heuristic of starting with operations of lowest mobility. This seems to be a good rule as it is easier to find a suitable scheduling time for operations of higher mobility later, when the resource usage of low-mobility operations is known. If, on the other hand, one starts with high-mobility operations, it is easier to make the wrong choice in the presence of more freedom. More criteria are needed to make a selection among the nodes with equal lowest mobility. Also criteria are needed to chose at which time an operation should be scheduled. Updating the ranges can be done using the longest-path algorithm, in a similar way as was done for computing the initial ranges (note that in Figure 6.8 the initialization occurs outside the procedure `longest-path`).

12.4.3 *Force-directed Scheduling*

A more sophisticated scheduling algorithm based on mobility is the *force-directed* method introduced by Paulin and Knight. It provides a heuristic solution to a problem already mentioned in Figure 12.23: how to find the best position in time for a node such that the overall resource utilization is optimized, while still many other nodes have to be scheduled. In force-directed scheduling, the resource utilizations of the nodes yet to be scheduled are estimated by assuming an "average" utilization within their scheduling ranges. For example, a unit-time operation with a mobility of three is assumed to use one third of a resource for the three time instants within its scheduling range. The combination of these averages results in a *resource requirement distribution function* that is extensively used by the scheduling algorithm. In this way, the unscheduled nodes also participate in the decision made during scheduling.

More precisely, the resource requirement distribution function for a resource r at time t, $\theta_r(\tilde{\sigma}, t)$ is computed making use of all the information present in the partial schedule $\tilde{\sigma}$. It is computed as follows:

$$\theta_r(\tilde{\sigma}, t) = \sum_{v \in V} \alpha(v, r, \tilde{\sigma}, t)$$

In the formula, the contributions of individual operations to the resource requirement distribution, given by the auxiliary function α, are added together. When $\rho(v) \neq r$, obviously $\alpha(v, r, \tilde{\sigma}, t) = 0$. Otherwise, α is defined as follows:

$$\alpha(v, r, \tilde{\sigma}, t) = \sum_{i=\tilde{\sigma}_{min}(v)}^{\tilde{\sigma}_{max}(v)} \sum_{j=0}^{\delta(v)-1} \xi(i + j, t) \times \frac{1}{\tilde{\sigma}_{max}(v) - \tilde{\sigma}_{min}(v) + 1}$$

where:

$$\xi(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{if } a \neq b \end{cases}$$

The function α can be understood as follows: for a unit-delay operation, its requirement distribution is uniform within its scheduling range; for an operation of longer duration, the requirement distribution is computed as if the operation was composed of a concatenation of unit-delay operations. Figure 12.24 shows a simple DFG and the computation of the resource requirement distribution functions for the initial schedule $\tilde{\sigma}^{(0)}$ when $T_0 = 4$, $\delta(+) = 1$, and $\delta(\times) = 2$.

Given a partial schedule $\tilde{\sigma}^{(k)}$, force-directed scheduling makes a transition to schedule $\tilde{\sigma}^{(k+1)}$ by selecting a node v which is scheduled at a time t, in a way similar to what is done in the simple algorithm of Figure 12.23. In order to find the best choices for v and t, force-directed scheduling evaluates all possibilities. This is done by computing the "force" associated with a possible transition.

Let $\tilde{\sigma}|_{v \to t}$ denote the schedule obtained from $\tilde{\sigma}$ by fixing operation v at a time t within its scheduling range. Then the force associated with the transition is given by:

$$F(\tilde{\sigma}|_{v \to t}, \tilde{\sigma}) = \sum_{s \in \mathcal{T}} \theta_r(\tilde{\sigma}, s)[\alpha(v, \rho(v), \tilde{\sigma}|_{v \to t}, s) - \alpha(v, \rho(v), \tilde{\sigma}, s)] \qquad (12.3)$$

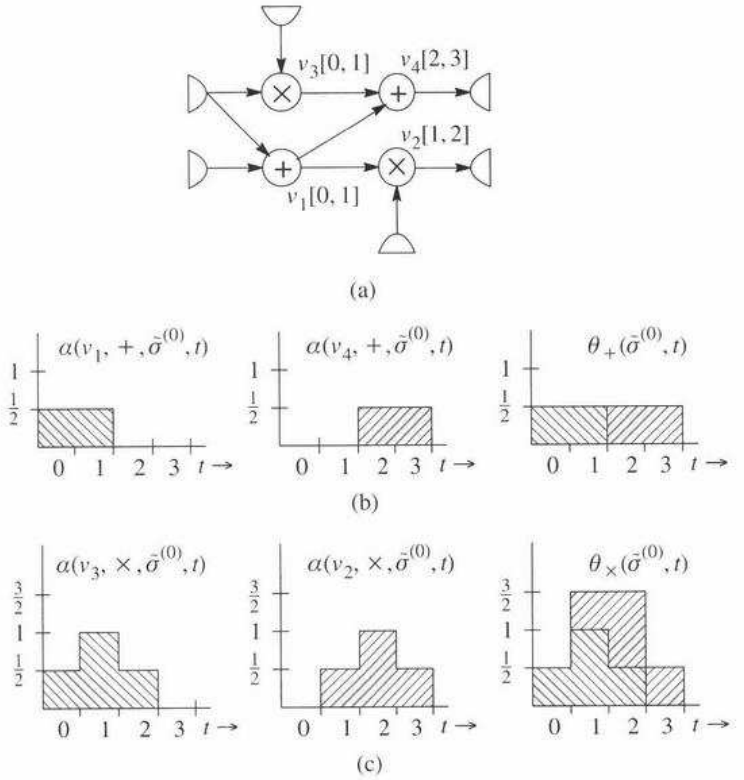


Figure 12.24 A simple DFG (a) and its initial resource requirement distribution functions for the resources + (b) and × (c).

The transition chosen is the pair (v, t) that results in the minimal force.

Inspection of the terms that are summed in Equation (12.3) shows that, for each instant s, the force is composed of the product of the current distribution and the change of the distribution for the operation that is tentatively scheduled at time t. One could interpret each term as the product of a spring constant (the distribution) with a displacement (the change of the distribution for the operation under investigation). In analogy with Hooke's Law, one could call the product a force, hence the name of the method.

One can understand Equation (12.3) as follows: the change in the distribution of the operation v due to its fixing at time t is weighted with the total distribution. For this reason, the contribution to the force of those time instants s with a higher distribution value will be more significant: a decrease in the distribution of v at s due to the fact that v is fixed at a time instant $t \neq s$, will be more strongly rewarded, while an increase due to $t = s$ will be more strongly penalized. In this way, nodes will be scheduled at instants where the expected resource utilization is relatively low.

Consider the scheduling of the multiplications in the example of Figure 12.24,

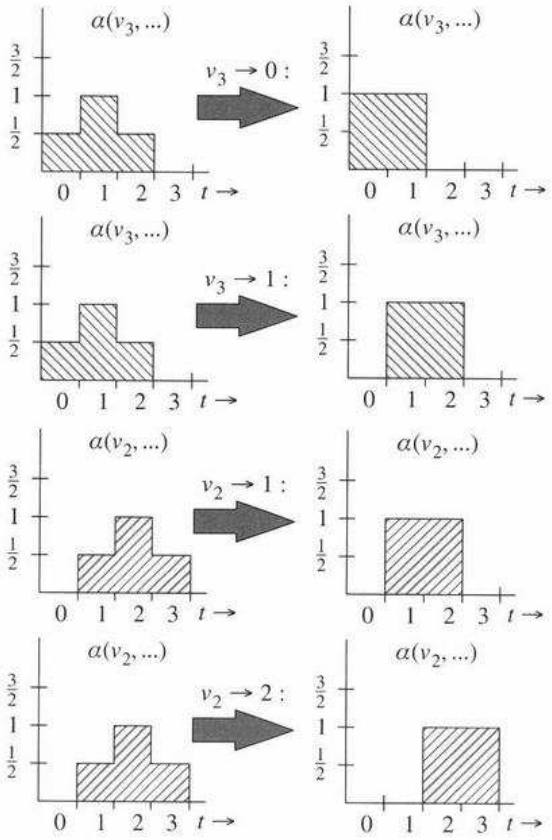


Figure 12.25 The changes in the values of the function α for the tentative schedulings to be considered when applying the force-directed method to the situation in Figure 12.24.

in which the following (v, t) pairs should be investigated: $(v_3, 0)$, $(v_3, 1)$, $(v_2, 1)$, and $(v_2, 2)$. The changes to the distributions of the nodes concerned are shown in Figure 12.25. The forces associated with the four tentative schedulings are given below:

$$F(\tilde{\sigma}^{(0)}|_{v_3 \to 0}, \tilde{\sigma}^{(0)}) = \frac{1}{2} \times \frac{1}{2} + \frac{3}{2} \times 0 + \frac{3}{2} \times -\frac{1}{2} + \frac{1}{2} \times 0 = -\frac{1}{2}$$

$$F(\tilde{\sigma}^{(0)}|_{v_3 \to 1}, \tilde{\sigma}^{(0)}) = \frac{1}{2} \times -\frac{1}{2} + \frac{3}{2} \times 0 + \frac{3}{2} \times \frac{1}{2} + \frac{1}{2} \times 0 = \frac{1}{2}$$

$$F(\tilde{\sigma}^{(0)}|_{v_2 \to 1}, \tilde{\sigma}^{(0)}) = \frac{1}{2} \times 0 + \frac{3}{2} \times \frac{1}{2} + \frac{3}{2} \times 0 + \frac{1}{2} \times -\frac{1}{2} = \frac{1}{2}$$

$$F(\tilde{\sigma}^{(0)}|_{v_2 \to 2}, \tilde{\sigma}^{(0)}) = \frac{1}{2} \times 0 + \frac{3}{2} \times -\frac{1}{2} + \frac{3}{2} \times 0 + \frac{1}{2} \times \frac{1}{2} = -\frac{1}{2}$$

The force-directed scheduling algorithm will choose for either the pair $(v_3, 0)$ or the

```
"determine σ̃^(0) by computing σ_S and σ_L";
k ← 0;
while "there are unscheduled operations" {
    for each r ∈ R
        "compute θ_r(σ̃^(k), t)";
    for each "unscheduled" v ∈ V {
        for (t ← σ̃^(k)_min(v); t ≤ σ̃^(k)_max(v); t ← t + 1) {
            "compute F(σ̃^(k)|_{v→t}, σ^(k))";
            if ("this force is smaller than previous ones") {
                v_best ← v;
                t_best ← t;
            }
        }
    σ̃^(k+1) ← σ̃^(k)|_{v_best→t_best};
    k ← k + 1;
}
```

Figure 12.26 The force-directed scheduling algorithm.

pair $(v_2, 2)$, which in both cases will avoid the allocation of a second multiplier.

The main ideas of force-directed scheduling, as presented above, have been summarized in the pseudo-code of Figure 12.26. From the code, the worst-case time complexity of the algorithm can be derived. The force computations dominate the calculation and require $\mathcal{O}(n^2 t_{max}^2)$ time, where n is the number of nodes in the DFG.

In Equation (12.3), only the change in the distribution of v resulting from the tentative scheduling of v at t is taken into account. This is only a rough approximation of changes in the resource requirement distribution function. In reality, such a tentative scheduling will also affect the scheduling ranges of other unscheduled operations. The original description of Paulin and Knight therefore not only considers the force as given in Equation (12.3), called by them the *self-force*, but also the changes in the distributions of the nodes incident to and incident from v. These changes result in the so-called *predecessor* and *successor* forces. It is beyond the scope of this text to discuss the details of these force computations.

Although force-directed scheduling has been used by many researchers, it has a higher time-complexity than other heuristics without guaranteeing better results. In particular, heuristics based on *list scheduling* should be considered as an alternative to force-directed scheduling. A short description of list scheduling follows in the next subsection.

12.4.4 List Scheduling

The scheduling methods described until now (ASAP, mobility-based and force-directed scheduling) are meant for time-constrained problems, although they could be adapted for resource-constrained problems as well. *List scheduling*, on the other

hand, is meant for resource-constrained problems in the first place, but can also be adapted for time-constrained problems.

The main idea is to process the available time instants in increasing order (starting at zero) and schedule as many as possible operations at a certain instant before moving to the next. This continues until all operations have been scheduled. In this way an attempt is made to minimize the total execution time of the operations in the DFG. Of course, the precedence relations also need to be respected in list scheduling. All operations, whose predecessors in the DFG have completed their executions at a certain time instant t are put in the so-called *ready list* L_t. The list contains exactly those operations that are available for scheduling.

If there are sufficient unoccupied resources at time t for all operations in L_t, these operations can be scheduled trivially. However, if an appropriate resource for each operation in L_t is not available, a choice has to be made on which operations will be scheduled at time t and which operations will be deferred to be scheduled at a later time. This choice is normally based on heuristics, each heuristic defining a specific type of list scheduling.

A popular version is *critical-path* list scheduling. In this context, the longest path from a node $v \in L_t$ to an output node of the DFG is called its critical path (note that the critical path of the DFG as a whole is the longest path starting from any of the input nodes and ending in any of the output nodes). The maximum over all $v \in L_t$ of the critical-path lengths gives a lower bound on the total time necessary to execute the remaining part of the schedule. Therefore, in critical-path list scheduling, nodes with the greatest critical-path lengths are selected to be scheduled at time t. The method is illustrated in Figure 12.27, where the DFG shown should be scheduled on two identical ALUs that require one time unit to compute an addition or subtraction and two time units to compute a multiplication. The critical-path length for each node is given between parentheses behind the node name. The intermediate values of L_t during the execution of the algorithm are given in Figure 12.28. Note that the elements in the lists are sorted according to their critical-path lengths and that therefore the first two elements are selected for scheduling whenever the list contains more than two elements.

A reason for the popularity of critical-path list scheduling is that often satisfactory results are obtained while the algorithm has a low time complexity. The critical-path lengths for each node need only to be computed once, which can be done in linear time traversing the graph from the output nodes to the inputs in a way similar to the longest-path algorithm presented in Figure 6.8, resulting in a time complexity of $\mathcal{O}(|E|)$. Sorting the list of all nodes by critical-path length can also be done only once, in $\mathcal{O}(n \log n)$ time. Composing the ready list and selecting its first elements requires $\mathcal{O}(nT_0)$ time in the worst case when implemented in a straightforward way. So, the overall worst-case time complexity is $\mathcal{O}(|E| + n \log n + nT_0)$.

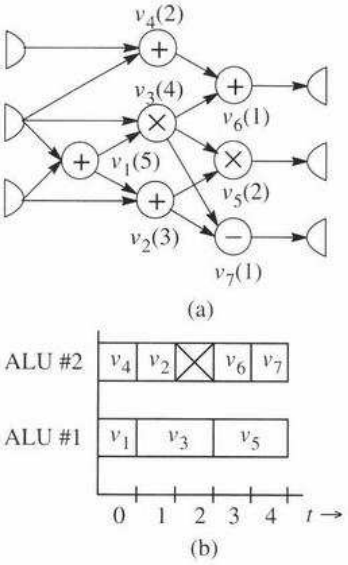


Figure 12.27 A DFG (a) and its schedule obtained by critical-path list scheduling when two ALUs are available (b).

t	L_t
0	$\{v_1(5), v_4(2)\}$
1	$\{v_3(4), v_2(3)\}$
2	$\emptyset$
3	$\{v_5(2), v_6(1), v_7(1)\}$
4	$\{v_7(1)\}$

Figure 12.28 The evolution of the set L_t in the critical-path list scheduling of the problem from Figure 12.27.

12.5 Some Aspects of the Assignment Problem

12.5.1 Optimization Issues

As mentioned earlier, the scheduling and assignment tasks are interrelated. For an optimal design they should be solved simultaneously. Some synthesis systems use algorithms that do so. However, most systems first solve the scheduling problem and then try to find a good assignment given a certain schedule. There also exist systems that first solve the assignment problem and schedule afterwards. In the rest of this text, it will be assumed that assignment follows scheduling.

The assignment problem itself consists of several subproblems:

- *operation-to-FU assignment:* this is the problem of mapping a computation to an FU of an appropriate type.

- *value grouping:* this is the problem of partitioning all storage values in such a way that a subset does not contain values that are read or written simultaneously. Then each subset can be realized as a register bank. In the case of *multiport memories*, the conditions for grouping should be adapted accordingly (a multiport memory has several ports that allow to perform more than one read or write action in a single clock cycle).
- *value-to-register assignment:* this is the problem of assigning a memory location to storage values in the same group. Values with nonoverlapping *life times* can share the same location. The life time of a storage value is the time interval starting at the instant that it is created, and ending at the moment that it no longer is required.
- *transfer-to-wire assignment:* a transfer is the actual transport of data from one hardware unit to another. In a bus-based architecture, one has the choice of which bus to write. The choice affects the number of three-state drivers connected to the unit from which the transfer originates and the type of multiplexers connected to the unit receiving the transfer.
- *wire to FU-port assignment:* in the case of commutative operations, one can choose one of the two equivalent input ports to feed the data to the functional unit. More in general, the problem exists when a functional unit has ports that are functionally equivalent.

Again, the complete solution space can only be explored by considering all problems simultaneously. The problem is quite complex, however, so that only heuristic methods can generate solutions for problem instances of practical size. The alternative of considering the different subproblems individually and possibly solving them exactly, may, therefore, be a better alternative.

Assuming that assignment is performed after scheduling, the number of clock cycles that the algorithm needs cannot be influenced anymore. The optimization goal is now primarily the total chip area. Another goal could be the performance (the length of the interconnections, the number of multiplexers connected to a bus, etc. can affect the length of the clock cycle). Interconnection cost is one of the items to be minimized during assignment. It includes the number of multiplexers and three-state drivers, but certainly also the bus length. As the length is difficult to estimate, most systems do not take it into account. There exist a few systems, however, that do some kind of *floorplanning* (see Chapter 8) during assignment.

12.5.2 Graph Theoretical Problem Formulation

Problems like operation-to-FU assignment and value-to-register assignment, when looked upon in a more abstract way, are variants of the same problem. This problem will be called *task-to-agent* assignment, where a *task* can be an operation or a value and an *agent* can be an FU or a register. Tasks are called *compatible* if they can be executed on the same agent. In case of values, for example, this means that they are compatible when their life times do not overlap. The set of tasks can be used as

the vertex set of a so-called *compatibility graph* $G_c(V_c, E_c)$. The graph has edges $(v_i, v_j) \in E_c$ if and only if the tasks corresponding to $v_i \in V_c$ and $v_j \in V_c$ are compatible.

Alternatively, one can say that two tasks are in *conflict* if they cannot be executed on the same agent. The set of tasks is then used as the vertex set of a *conflict graph* that has edges for those vertex pairs that are in conflict. The conflict graph is the *complement graph* of the compatibility graph. This means that the conflict graph can be obtained from the compatibility graph $G_c(V_c, E_c)$ by taking the complete graph of the vertex set V_c and removing all edges in E_c from this graph. Clearly, taking the complement of the conflict graph gives back the compatibility graph.

The goal of the assignment problem is to minimize the number of agents for the given set of tasks. This goal can be formulated either in terms of the compatibility graph or the conflict graph as is explained below.

The vertices of any complete subgraph of a compatibility graph correspond to a set of tasks that can be assigned to the same agent. The goal of the assignment problem is then to partition the compatibility graph in such a way that each subset in the partition forms a complete graph and the number of subsets in the partition is minimal. The subsets are pairwise disjoint and the union of the subsets forms the original set by definition of a partition. In the literature such a partitioning is called a *clique partitioning* even when the subsets in the partition are not necessarily cliques, but only complete subgraphs. A term that is mathematically more precise is a "covering by disjoint completely connected sets".

In a conflict graph, on the other hand, adjacent vertices in the graph have to be assigned to different agents. Minimizing the number of agents then amounts to finding the *minimal coloring* of the conflict graph. Vertices receiving the same color can be assigned to the same agent.

The notions presented in this section are illustrated in Figure 12.29. Figure 12.29(a) shows an example of a compatibility graph, while the corresponding conflict graph is shown in Figure 12.29(b). A possible minimal clique covering of the compatibility graph consists of the three vertex sets $\{v_1, v_2, v_6\}$, $\{v_3, v_4\}$ and $\{v_5\}$ as is shown in Figure 12.29(c). Not surprisingly, the minimal coloring of the conflict graph needs three colors as shown in Figure 12.29(d).

At first sight it may seem that there is no advantage of formulating one and the same problem as two different graph theoretical problems, especially because solving one of the problems automatically implies a solution for the second as well. Both the minimal clique covering problem and the graph vertex coloring problem are NP-complete in the case that the graphs concerned do not belong to special families. It may, however, be that the conflict graph is an interval graph, which leads to an optimal solution in polynomial time as explained in Section 12.5.3 following next. On the other hand, an algorithm such as the one presented in Section 12.5.4 can better be explained in terms of the compatibility graph.

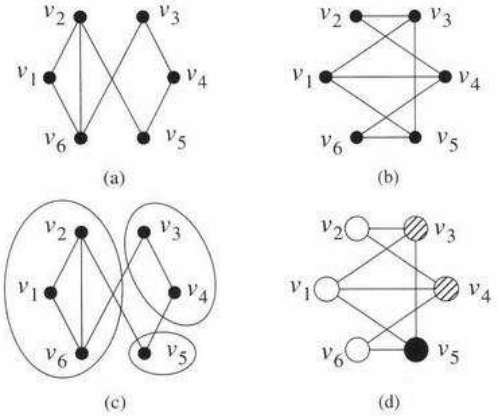


Figure 12.29 A compatibility graph (a), its corresponding conflict graph (b), a minimal clique partitioning of the compatibility graph (c) and a minimal coloring of the conflict graph (d).

12.5.3 Assignment by Interval and Circular-arc Graph Coloring

Consider the operator-to-FU assignment subproblem. If the goal of this subproblem were to minimize the number of FUs *without* considering the impact on register and interconnect cost, then the problem can optimally be solved in polynomial time in the case of *nonoverlapped* scheduling. As the times that an operation starts and terminates are known after scheduling and each operation thus has an execution interval, the problem amounts to "packing intervals" in as few as possible rows. This problem can be solved optimally in polynomial time by the *left-edge algorithm*, first proposed for the channel routing problem (see Section 9.3.3). Stated in graph theoretical terms, the intervals can be thought as vertices in an *interval graph*. This interval graph is a conflict graph, which means that overlapping intervals give rise to an edge in the graph. Finding the minimal number of FUs then amounts to coloring the graph with a minimal number of colors. The left-edge algorithm guarantees an optimal solution in polynomial time but cannot take into account "weights" that might be used to express the impact on interconnection of a specific assignment.

In the case of *overlapped* scheduling, however, the time axis is folded back on itself. This means that execution intervals at the end of the iteration period may overlap with those at the beginning (of the next iteration period). Intervals of this type can be visualized as arcs of a circle as has been done in Figure 12.30(a). In order to keep the figure readable, the arcs have been drawn on separate concentric circles. Again a conflict graph can be constructed based on the overlapping relation (in the figure, one should project the arcs radially on the inner circle to detect whether two arcs overlap). Such a graph is called a *circular-arc graph*. Figure 12.30(b) shows the circular-arc graph derived from the set of arcs in Figure 12.30(a). The minimal coloring of this graph is given in Figure 12.30(c). The minimal solution requires three agents: one for the tasks corresponding to v_1 and v_4, one for those corresponding to

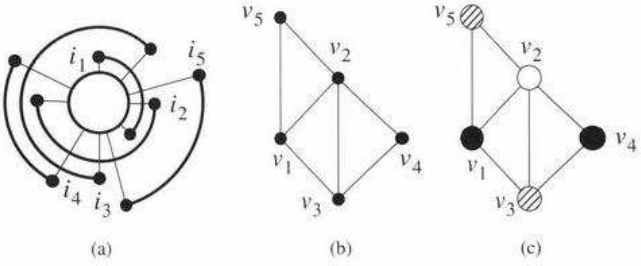


Figure 12.30 A set of circular arcs (a), its corresponding circular-arc graph (b) and the minimal coloring of this graph (c).

v_3 and v_5 and one for v_2.

Note that a graph can be constructed from any set of circular arcs, but that the reverse is not true in general: a given graph may or may not have a set of corresponding circular arcs. A similar statement can be made about interval graphs. So, the set of all circular-arc graphs and the set of all interval graphs are strict subsets of the set of all possible graphs (and the set of all interval graphs is a subset of the set of all circular-arc graphs). Because of their specific properties it may be that, for some graph problems, algorithms can be designed that are more efficient than algorithms meant for all possible graphs.

Even when the coloring problem for circular-arc graphs looks similar to the coloring problem for interval graphs at first sight, it is an NP-complete problem. No specific algorithm for coloring circular-arc graphs will be discussed here. One possibility is to take the complementary compatibility graph and then use the "clique partitioning" approach presented in the next section.

The problem of value-to-register assignment is similar to the operator-to-FU assignment problem. However, one should be aware that a nonoverlapping schedule of an iterative algorithm still will require that some values will be alive across loop boundaries. So, the problem will still be a circular-arc graph coloring problem.

12.5.4 Assignment by Clique Partitioning

This section explains a technique called *clique partitioning* that can be applied to almost any subproblem of the assignment task (see Section 12.5.1). The problem of finding a clique partitioning of a graph is NP-complete. A heuristic algorithm to solve the problem was proposed by Tseng and Siewiorek and will be presented here. It is based on combining vertices in the compatibility graph step by step. The resulting vertices are called *supervertices*. The index i of a supervertex v_i represents the set of indices of the vertices from which the supervertex was formed. For example, combining vertices v_1, v_3 and v_7 gives a supervertex $v_{1,3,7}$. By definition, the original vertices of the compatibility graph will also be called supervertices. If the intermediate graph after k steps of the algorithm is called $G_c^k(V_c^k, E_c^k)$, the initial graph $G_c^0(V_c^0, E_c^0)$ is equal to the compatibility graph.

```
k ← 0;
G_c^k(V_c^k, E_c^k) ← G_c(V_c, E_c);
while (E_c^k ≠ ∅) {
   "find (v_i, v_j) ∈ E_c^k with largest set of common neighbors";
   N ← "set of common neighbors of v_i and v_j";
   s ← i ∪ j;
   V_c^{k+1} ← V_c^k ∪ {v_s} \ {v_i, v_j};
   E_c^{k+1} ← ∅;
   for each (v_m, v_n) ∈ E_c^k
     if (v_m ≠ v_i ∧ v_m ≠ v_j ∧ v_n ≠ v_i ∧ v_n ≠ v_j)
       E_c^{k+1} ← E_c^{k+1} ∪ {(v_m, v_n)};
   for each v_n ∈ N
     E_c^{k+1} ← E_c^{k+1} ∪ {(v_n, v_s)};
   k ← k + 1;
}
```

Figure 12.31 A heuristic to compute the clique partitioning of a graph.

The algorithm looks for a pair of supervertices with the largest number of *common neighbors*. A supervertex $v_n \in V_c^k$ is a common neighbor of the supervertices $v_i, v_j \in V_c^k$, if both edges (v_i, v_n) and (v_j, v_n) are included in E_c^k. The two supervertices are combined into a new supervertex as described in the pseudo-code of Figure 12.31. The new supervertex remains connected to the common neighbors only. The algorithm goes on to combine supervertices until the graph $G_c^k(V_c^k, E_c^k)$ has an empty edge set.

A problem here is how to break ties when there are several pairs of supervertices with the largest set of common neighbors. One could work with priorities or edge weights to express a preference of one combination above another. Such a preference scheme is based on the actual problem that one wants to solve: a specific combination of tasks on the same agent might e.g. require more multiplexers than another combination. In the example shown in Figure 12.32, which illustrates the application of the algorithm to some example graph, ties have been broken by giving preference to the lowest vertex index.

Clique partitioning can be used for both nonoverlapped and overlapped scheduling. This is due to the global nature of the method: it simply considers conflicts between tasks, it does not process the resources in increasing order of time instants as e.g. the left-edge algorithm.

12.6 High-level Transformations

Until now, it has been assumed in this chapter that the DFGs used for the input description of high-level synthesis could not be manipulated. Although it was stated in Section 12.2 that a DFG explicitly represents *all* parallelism present in a computation, this does not mean that there is a unique DFG for each computation (an example

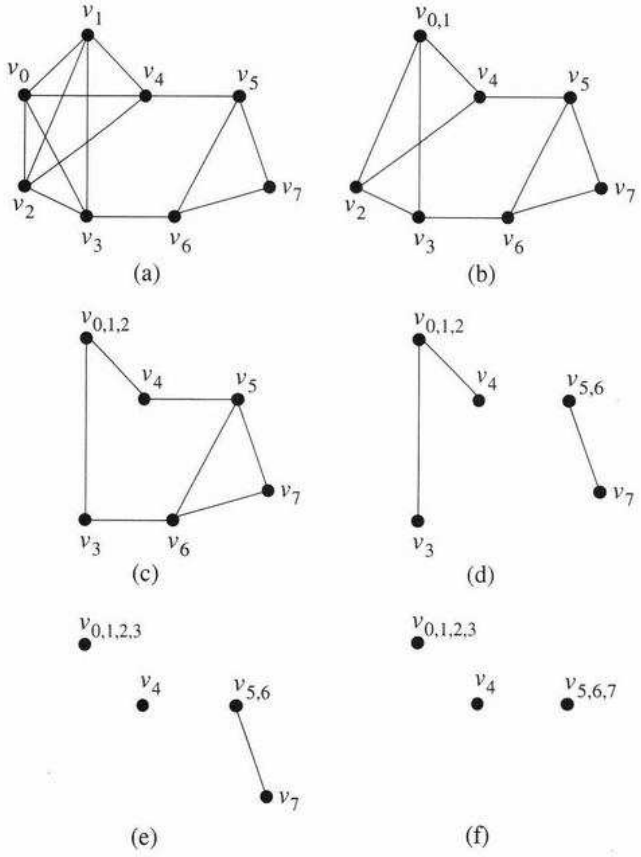


Figure 12.32 The successive graphs obtained in the application of the clique partitioning heuristic to an example graph.

of two different DFGs for the same computation was given in Figure 12.8). On the other hand, it should be clear from the discussion in the previous sections that modifications of the DFG may affect the result of scheduling and assignment and therefore the quality of the final design. For this reason, *transformations* are normally applied to a DFG before performing steps like scheduling and assignment.

There are many different types of transformation, of which a few that are applicable to data-dominated applications will be discussed here. First of all, there are simple arithmetic transformations based on "commutativity", "associativity" and "distributivity". Addition and multiplication are examples of *commutative* operators, which e.g. means that $a + b = b + a$. Such a transformation may lead to savings in interconnection hardware like multiplexers during assignment.

For an illustration of *associativity*, consider the addition of 8 operands given

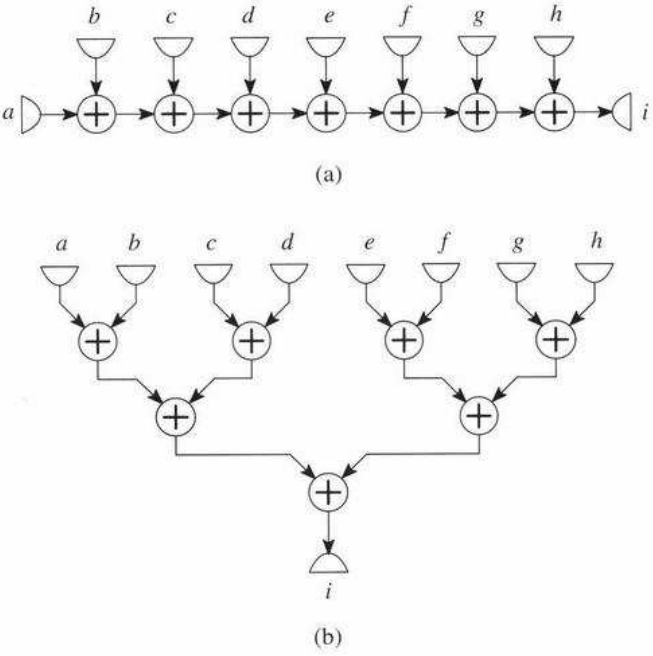


Figure 12.33 The most serial (a) and most parallel (b) DFG for an eight operand addition.

below:

$$a + b + c + d + e + f + g + h$$

When translating this addition into a DFG using addition nodes with two operands, one could interpret the expression as:

$$((((((a + b) + c) + d) + e) + f) + g) + h$$

or as:

$$((a + b) + (c + d)) + ((e + f) + (g + h))$$

This is the consequence of the associativity of addition. The DFGs corresponding to the two interpretations are shown in Figure 12.33. The "most serial" version of the computation shown Figure 12.33(a) has a critical path of seven additions whereas the "most parallel" one of Figure 12.33(b) has a critical path of three additions. However, the latter form is not always the most preferable one. This is especially the case if the DFG of this addition is embedded in a DFG of a larger computation. It could then happen that the critical path of the overall computation becomes longer when transforming the serial form into the parallel form (suppose for example that the data at input h are available much later than the other inputs). In another situation, e.g.

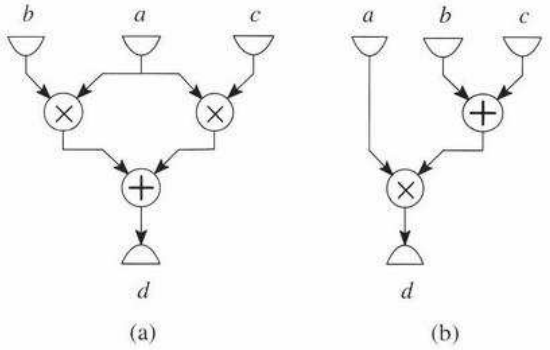


Figure 12.34 A DFG transformation corresponding to the distributivity law.

one with a resource constraint of a single adder, the parallel version will require more registers for the storage of intermediate results. The conclusion is that the application of associativity can lead to improvements in the quality of the final result, but that the choice between the most serial, the most parallel and any of the intermediate forms strongly depends on the precise synthesis circumstances.

Distributivity states that:

$$a \times b + a \times c = a \times (b + c) \tag{12.4}$$

as illustrated in Figure 12.34. At first sight the situation of Figure 12.34(b), that has one multiplication less than the situation of Figure 12.34(a), seems to be preferable. However, also here the embedding of this DFG in a larger one may make the DFG with the two multipliers a better choice (because no multiplier is available at the end of the iteration period whereas there is time and hardware to perform the two multiplications at the beginning).

Although Equation (12.4) is perfectly valid from a mathematical point of view, it does not necessarily hold at the moment that the computations occurring in the expressions are performed on hardware. This is due to the fact that numbers have to be represented by a finite number of bits and that the outputs of a multiplication or addition require more bits than the inputs if no precision is to be lost. In applications like digital signal processing where "state variables" are repetitively recomputed by additions and multiplications, it is inevitable to perform some kind of rounding or truncation after each arithmetic operation. It may therefore be that the so-called "finite word-length effects" are different before and after the application of the distributivity transformation (because the additions and multiplications are performed in a different order).

Because of the effects just mentioned, it is even doubtful whether distributivity belongs to the category of high-level transformations. It is rather an *algorithmic* transformation, a type of transformation that can be applied by the algorithm designer. Of course, some interaction between high-level design and algorithm design

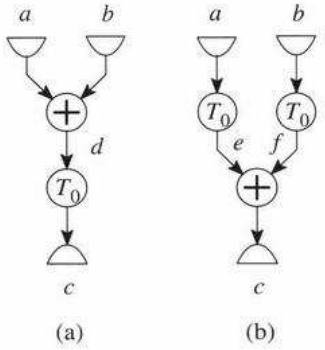


Figure 12.35 The retiming transformation.

is always possible to see whether a specific transformation like distributivity is acceptable. There exist many more algorithmic transformations that e.g. can be used to increase the parallelism in the computation in order to get faster implementations or to reduce the dissipated power. They are, however, outside the scope of this chapter.

In Section 12.3.2 it was explained that delay nodes should be split into pairs of input and output nodes when generating a nonoverlapped schedule for the algorithm. The transformation to be discussed next changes the position of the delay nodes in the DFG and therefore may affect the length of the critical path of the DFG (the longest path from any input to any output node). Clearly, the critical path gives the minimal iteration period of a nonoverlapped schedule in the absence of resource constraints. The transformation is called *retiming*. Simply stated, the transformation is based on the fact that performing some operation in the DFG and then delaying its output is equivalent to performing the same operation on the delayed inputs. This is illustrated in Figure 12.35, where the graphs used are in principle parts of larger graphs. It is not difficult to see that the transformation is valid. In the derivation, the signals will be indexed by a time instant as was done in Section 12.2.3. The following holds in Figure 12.35(a): $c[n] = d[n-1]$ and $d[n] = a[n] + b[n]$, which means that $c[n] = a[n-1] + b[n-1]$. In Figure 12.35(b), the situation is: $c[n] = e[n] + f[n]$, $e[n] = a[n-1]$, $f[n] = b[n-1]$, and therefore $c[n] = a[n-1] + b[n-1]$. A similar derivation could be given for any node in the DFG with any number of inputs or outputs.

As stated before, retiming can be used to optimize the critical path of a DFG for nonoverlapping schedules. It can also be used to achieve a better distribution of the resources over the iteration period (e.g. to improve a situation where all multiplications are performed at the beginning of the iteration period and all additions at the end). Actually, retiming is only useful for nonoverlapped scheduling. Most overlapped scheduling methods (not discussed here) in a sense consider all retimed versions of a DFG simultaneously, because they do not split delay nodes into pairs of input and output nodes.

A final issue to discuss is the strategy for applying the transformations. Some

systems are interactive and let the user choose which transformations should be applied on which part of the DFG. If one wants to apply transformations without user interaction, one should first be aware of the computational complexity of the problems involved. An example of a problem that can be solved in polynomial time is "retiming for the optimization of the critical path". However, "retiming for the optimization of resources" turns out to be NP-complete. In such a case, one could use a method like simulated annealing (see Section 5.6) to search for a good solution. The "moves" for simulated annealing can be chosen from a set of transformations that can be as large as wanted. The cost function could e.g. be based on the resource requirement distribution function discussed in Section 12.4.3. It is relatively easy to calculate compared to the effort involved in complete scheduling and assignment.

12.7 Bibliographic Notes

[McF90] is a review article that is a good introduction to the topic of high-level synthesis and covers all important aspects. Less detailed but more recent review articles are [Gaj94] and [Lin97]. [Gaj92] and [DM94] are introductory books, mainly written with an educational goal. Books more oriented to the presentation of research results in the field include [Cam91b], [Wal91], [Ku92], [Mic92b], [Cat93], [Van93] and [Bay94].

This chapter assumes a basic familiarity with computer architecture. Those readers that lack this knowledge, are referred to [War90], one of the many good books on this topic. In the context of the hardware models a distinction was made between data and control signals. It is not always easy to make this distinction; a formal definition of control signals is given in [Gho92]. A clear explanation of "two-phase clocking" can be found in [Mea80]. An example of a high-level synthesis system that uses a two-phase hardware model is [Har89].

A topic that is just mentioned in this chapter, but not further covered, concerns the description language to be used as a starting point for high-level synthesis. One of the most used languages is VHDL, even when the language is primarily suitable for simulation rather than for synthesis. This language spans many levels of the hierarchy of design descriptions including the algorithmic one. [Lip89] and [Nav93] are two of the many books available on VHDL. Another popular language that is somewhat comparable to VHDL is Verilog; it is explained in [Tho91] and [Gol96]. A totally different language that was especially designed for the description and synthesis of digital signal processing algorithms is Silage; information on Silage can be found in [Hil85] and [Hil92].

A good introduction to the topic of data-flow graphs, including a section on the history of data-flow models for parallel computing, is given in [Dav82]. Similar ideas can be found in the work of Lee [Lee88, Lee91a, Lee95], and in the proposal for a DFG exchange standard for the purpose of synthesis from Eindhoven University of Technology [Eij92]. Examples of publications that use separate data and control flow graphs are [McF90] and [Cam91a]. Other researchers combine the two in one graph, calling the graph *control-data flow graph* [Pau89] or *data-control flow graph* [Pot92].

More on *signal-flow graphs* can be found in many publications in the field of digital signal processing, including [Fet76] and [Rob87]. Synchronous data flow graphs are explained in [Lee87]. They have for example been used in the Gabriel system as a graphic input language [Lee89]. The consistency problem for data-flow graphs (in a consistent graph, it is ensured that all nodes receive sufficient tokens to keep on computing, while no unbounded number of tokens is accumulated on any of the edges) has been studied by Lee both for synchronous [Lee87] and asynchronous [Lee91a] graphs.

Examples of high-level synthesis systems for control-dominated applications, a topic not covered in this text, are: [Cam91a], [Wol92] and [O'B93].

The definitions in the text for "scheduling", "assignment" and "allocation" follow those given in [Pot92]).

More on the principles of "overlapped scheduling" can be found in the review paper [Ger98]. Key articles on the topic are [Par91] and [HdG92]. Examples of high-level synthesis systems that generate overlapped schedules are described in [Goo90], [Olá92], [Lee94], [Kos95] and [Wan95]. Overlapped scheduling can be performed for computations with multiple nested loops such as found in image processing systems. The Phideo high-level synthesis system can deal with the overlapping of nested loops [Mee95, Ver97].

A review paper dedicated to the scheduling problem only is [Wal95]. The notation used here for the description of the scheduling problem has been adapted from [Olá92] which in its turn was based on the one of [Ver91]. Many different variants of the scheduling problem exist and most of them are NP-complete. Those that have been discussed in this chapter can easily be reduced to the NP-complete problems listed in [Gar79].

Mobility-based scheduling has been investigated by many researchers. One of the earliest publications of this approach applied to high-level synthesis is [Par86] where "mobility" is called "freedom". The terms "time frame" and "scheduling range" originate respectively from [Pau89] and [HdG92]. Descriptions of Paulin's force-directed scheduling algorithm can be found in [Pau89] and [Pau91]. This algorithm has been used and adapted by many researchers as can e.g. be seen from the reference list of [Ver92]. A comparison between force-directed and list scheduling can be found in [Jai91].

Regarding the assignment problem, [Sto94] is a detailed review paper on different assignment problems occurring in data-path synthesis. An example of a system that aims at optimizing the performance is [Jia94], while [Rim94] is an example of a system that takes wire length into account by performing some kind of floorplanning. The term "task-to-agent" assignment originates from [Rim94].

A discussion on the use of compatibility and conflict graphs in high-level synthesis and their respective subclasses such as interval graphs is presented in [Spr94]. More on interval graphs, other special families of graphs, and their colorings can be found in [Gol80]. The NP-completeness of circular-arc graph coloring is proved in [Gar80]. An algorithm requiring exponential time in the worst-case for finding the optimal solution for the value-to-register assignment problem (which is equivalent to

```
gcd (int a, b)
{
  while (a ≠ b)
    if (a > b)
      a ← a − b;
    else
      b ← b − a;
  return a;
}
```

Figure 12.36 Euclid's Algorithm to compute the greatest common divider (GCD) of two integers.

circular-arc-graph coloring) is presented in [Sto92]. If special addressing hardware is available, also the polynomial-time algorithm proposed in [Mee93] could be used.

Tseng and Siewiorek were the first to propose the use of a "clique partitioning" heuristic to assignment [Tse86]. A description of their algorithm can also be found in [Gaj92]. More on the NP-completeness of the clique partitioning problem can be found [Gar79].

The application of transformations is often one of the tasks performed by a high-level synthesis system as is reported in e.g. [Har89]. Publications that deal with transformations as their main topic include [Pot94] and [Cha95b]. The NP-completeness of the optimization of resource utilization by retiming is discussed in [Pot94]. Retiming and algorithms to achieve an optimal retiming for the shortest critical path are presented in [Lei83] and [Lei91]. More on algorithmic transformations for digital signal processing can be found in the review papers [Par89] and [Par95].

12.8 Exercises

12.1 Suppose that the program of Figure 12.3 is written as "$x \leftarrow a * b$; $z \leftarrow c+d+x$" and that the nodes allowed in the DFG are either two-input additions or two-input multiplications but not three-input additions. Give the different DFGs corresponding to this program.

12.2 Figure 12.36 shows *Euclid's Algorithm* to compute the greatest common divider of two integers. Give a DFG that represents this computation; structure the graph in such a way that the while-loop and if-statement are clearly delimited by conditional nodes.

12.3 Find a register assignment and a data path for the overlapped schedule given in Figure 12.20.

12.4 Describe how an acyclic DFG should be modified such that the longest-path algorithm of Figure 6.8 computes the ALAP scheduling times of all computational nodes.

12.5 Provide simple criteria for the missing details of the scheduling procedure in Figure 12.23. Use the algorithm obtained to find a schedule for the DFG of Figure 12.15, with $T_0 = 10$. Do you find a solution that only uses one multiplier and one adder?

Index

abstract
  data type, 177, 206
  routing, 125
abstraction, 4, 104, 155, 167
  level, 167
abutment, 106, 123
acyclic data-flow graph, 248
adder, 14, 236
adjacency
  list, 25, 187
  matrix, 24
adjacent, 22, 199
agent, 262
ALAP scheduling, 254
algorithmic
  graph theory, 21
  transformation, 269
allocation, 248
analog, 167, 277
analysis tool, 8
application-specific integrated circuit, 3
approximation algorithm, 49, 53
arc, 22
area, 4, 125, 251
  routing, 134, 156
arithmetic logic unit, 236, 238, 260
ASAP scheduling, 253, 254
assignment, 247, 261, 267
  register, *see* register assignment
associativity, 267
asynchronous circuit, 236
atomic node, 240
automatic test-pattern generation, 14
average-case time complexity, 28, 67

backtracking, 56
backward edge, 93
BDD package, 206, 214, 216
behavior, 168, 196, 217
behavioral domain, 5, 119
Bellman-Ford algorithm, 95, 96
bidirectional signal flow, 180, 279
big
  O ($\mathcal{O}$), 28
  Omega (Ω), 28
  Theta (Θ), 28
binary
  decision diagram, 195
    ordered, *see* ordered *etc.*
    reduced ordered, *see* reduced *etc.*
  signal, 170
binate covering, 222
binding, 248
bipartite graph, 22, 104
bipartitioning, 108
Blake's canonical form, 198
blocking (in switch-level simulation), 184
Boltzmann distribution, 72
Boole, 202
Boolean
  algebra, 5, 201, 281
  function, 196, 201
  signal, 170, 174
  value, 196, 277
  variable, 196
bottom-up design, 7, 119
branch, 22
branch-and-bound, 59, 64, 225
breadboarding, 17
breadth-first search, 32, 62, 91
break point, 127
buffer, 240, 244
building block
  layout, 107, 138, 153
  placement, 106, 111

bus, 237, 262

C (programming language), *see* pseudo-code
canonical form, 66, 197, 198, 202
  strong, *see* strong canonicity
capacitance, 15, 152, 168, 172, 181, 191
  parasitic, *see* parasitic capacitance
capacitive coupling, 139
capacitor, 168
cell, 6, 54, 98, 102, 119, 121, 150
  compiler, 16, 125
  generation, 125
center of gravity, 111
chaining, 238
channel, 106, 123, 133, 150
  definition, 153
  density, *see* density
  height, 138
  of CMOS transistor, 283
  ordering, 153
  routing, 134, 138
    model, 139
channel-connected component, 184
characterization, 15, 125
charge sharing, 183
charged net, 186
checking
  solution, *see* solution checking
child, 58, 75, 121, 203
chromosome, 75, 78
circuit, *see* cycle
  extraction, 15, 17
  representation, 101, 173, 246
circuit-level simulation, 15, 168, 181
circular-arc graph, 264
clique, 22, 104, 106, 108, 112
  partitioning, 265
clock, 236, 238
  phase, 238
  two-phase, *see* two-phase clock
clustering, 108, 109
CMOS
  complex gate, 279
  domino logic, *see* domino logic
  inverter, 279
  static, *see* static CMOS
  technology, 9, 183, 222, 277
co-design, *see* hardware-software co-design
co-NP-complete, 215
co-simulation, *see* hardware-software co-simulation
code generation, 13
cofactor, 202, 213
coloring, *see* graph coloring
column dominance, 224
combinational logic, 168, 173, 195, 213, 217, 235
combinatorial optimization problem, 42, 53
  decision version, 44
  evaluation version, 43
  optimization version, 43
commutativity, 267
compaction, 17, 83, 86
  one-dimensional, *see* one-dimensional compaction
  two-dimensional, *see* two-dimensional compaction
compatibility graph, 263
compiler-driven simulation, 173, 180
complement, 196
  graph, 263
complementary network, 182, 279
complete graph, 22, 49, 263
complex gate, *see* CMOS complex gate
complexity
  class, 45
    NP, *see* NP
    NPC, *see* NPC
    P, *see* P
  computational, *see* computational complexity
  space, *see* space complexity
  time, *see* time complexity
component
  channel-connected, *see* channel-connected component
  connected, *see* connected component
  strongly connected, *see* strongly *etc.*
  weakly connected, *see* weakly *etc.*
composite
  cell, 121, 127
  node, 240
composition problem, 213
computational
  complexity, 21, 26
  geometry, 17
  node, 239
  step, 28
conditional node, 240, 241

conductance, 181
conflict graph, 263, 264
connected
  channel, *see* channel-connected component
  component, 23
  graph, 23
  strongly, *see* strongly connected *etc.*
  vertices, 23
  weakly, *see* weakly connected
connectivity modeling, 172, 181
constant time, 29
constraint
  explicit, *see* explicit constraint
  generation, 98
  graph, 89, 93, 97, 98
    vertical, *see* vertical *etc.*
  implicit, *see* implicit constraint
constructive placement, 108
contact cut, 86, 133, 283
continuous optimization problem, 42
control
  circuitry, 238
  signal, 238
  step, 238, 252
control-dominated application, 13, 247
control-flow graph, 239
controller, 238
  synthesis, 238
Cook's theorem, 48
cooling schedule, 73
correct by construction, 8, 18
cost, 251
  function, 42, 56, 71
    for VLSI, *see* VLSI cost function
cover
  irredundant prime, *see* irredundant *etc.*
covering
  binate, *see* unate covering
  matrix, 221
  set, *see* set covering
  unate, *see* unate covering
critical
  path, 97, 152, 260, 268, 270
    list scheduling, 260
  sink, 152
crossover, 75, 78
  order, *see* order crossover
crosstalk, 139
cube, 197, 222
current source, 168
cycle, 23, 24, 238
  directed, *see* directed cycle
  Hamiltonian, *see* Hamiltonian cycle
  negative, *see* negative cycle
  positive, *see* positive cycle
  simple, *see* simple cycle
cyclic core, 225

data
  dependency, 240
  management, 18
  model, 102
  path, 238
    synthesis, 238
  signal, 238
  structure (for graphs), 24
data-dominated application, 247, 267
data-flow graph, 239, 266
  acyclic, *see* acyclic *etc.*
dc-set, 196, 217, 223, 226
De Morgan's Rule, 201, 222, 281
deadlock, 244
decision
  problem, 44, 48, 138
  version, 44
decomposition tree, 5
degree, 22, 158
delay, 238
  modeling, 152, 171
  node, 245, 270
demand-driven simulation, 180
density, 143, 152, 153, 155
  local, *see* local density
depletion nMOST, 183
depth-first search, 30, 62, 153, 162, 184, 188
description
  structural, *see* structural description
design
  action, 7
  constraint, 4
  domain, 5, 119
  management tool, 8, 18
  methodology, 3, 119, 124
    floorplan-based, *see* floorplan-based *etc.*
  rule, 17, 83, 125, 135
    checker, 17, 85
  time, 4

verification, *see* verification
detailed routing, *see* local routing
device-level simulation, 167
diffusion, 282
digital
abstraction, 277
domain, 167
signal
processing, 239, 244, 269
processor, 3
digraph, 23
Dijkstra's algorithm, 34, 49, 63, 97
directed
acyclic graph, 89, 96
cycle, 24, 90
graph, 23
path, 24
disconnected
graph, 23
discrete optimization problem, 42
distance
Euclidean, *see* Euclidean distance
maximum, *see* maximum distance
minimum, *see* minimum distance
rectilinear, *see* rectilinear distance
distributivity, 201, 267, 269
distributor node, 241
divide-and-conquer algorithm, 156
dogleg, 140, 150
domino logic, 183
don't care, 196
dopant, 282
drain, 181, 278
driven net, 186
dual network, 282
dynamic
partitioning, 184
programming, 62, 148
search tree, 62
switch-level simulation, 191

edge, 22
parallel, *see* parallel edge
edge-weighted graph, 24, 34, 220
EDIF, 19, 117
ellipsoid algorithm, 67
Elmore delay, 152
endpoint, 22
energy, 71
enhancement nMOST, 183
equivalent terminal, 134
essential prime implicant, 224
estimation
wire-length, *see* wire-length estimation
Euclidean
distance, 43, 105
squared, *see* squared Euclidean distance
traveling salesman, 43, 76
evaluation version, 43
event, 176
queue (or event list), 176
event-driven simulation, 173, 176, 180
EXCLUSIVE-OR, 205
exhaustive search, 53, 56
explicit constraint, 56
exponential order, 29

fanout, 172
fault simulation, 14
feasible solution, 42, 56, 69, 71, 75, 78
feedthrough
cell, 151
wire, 121, 123, 152
field-programmable gate array, 9, 17
FIFO queue, 32, 73
firing, 240
first improvement, 70
first-in-first-out order, 62
fixed terminal, 134, 138, 154
flexible cell, 123, 125, 153
floating terminal, 134, 138, 154
floorplan
of order 2, 122
of order 5, 122, 129
sizing, *see* sizing
tree, 122, 130
floorplan-based design methodology, 119, 156
floorplanning, 16, 120
force computation, 256
force-directed
placement, 111
scheduling, 256
formal
description, 12
verification, 12, 18
forward edge, 93
framework, 19
freedom, 97, 272

full-custom design, 8, 15, 16, 125
functional unit, 236, 262

garbage collection, 214, 216
gate, 181
  array, 9
  complex, *see* CMOS complex gate
  logic, *see* logic gate
  modeling, 170
  NAND, *see* NAND gate
  NOR, *see* NOR gate
  of transistor, 278, 282
gate-level simulation, 168, 169, 173, 181
general-cell layout, 107
general-purpose
  integrated circuit, 3
  optimization method, 53, 220
genetic algorithms, 75, 78, 129
genotype, 75
geometric layout, 85
geometry, 85
glitch, 174
global routing, 133, 150
graceful degradation, 213
graph, 21
  algorithms, 29
  coloring, 145, 263, 264
  data structure, 24
  model, 103
  theory, 21
greedy algorithm, 143, 230
gridded routing, 134
gridless routing, 134, 139

half-perimeter metric, 105
halting problem, 45
Hamiltonian cycle, 47
Hanan point, 159
hardware
  description language, 11, 168, 169, 239
  model, 236
hardware-software
  co-design, 13
  co-simulation, 13, 169
hash table, 206, 208
heuristics, 50, 53, 78, 146, 225
hierarchical routing, 156
hierarchy, 4, 98, 104, 121, 122, 124, 246
high-level
  synthesis, 12, 102, 235
  transformation, 267
horizontal
  composition, 127
  constraint, 143, 148
hybrid algorithm, 78
hyperedge, 104
hypergraph, 104

if-then-else operator, 210
implementation, 217
implicant, 198
  prime, *see* prime implicant
implicit constraint, 56
in-degree, 23
incident
  from, 23
  to, 23
  with, 22
independent set, 148
inertial delay, 172
initial placement, 108
input
  cell, 103
  net, 181, 184
  node, 239
  size, 26, 65
  terminal, *see* terminal
inset cell, 126
instance, 41, 42, 102
integer linear programming, 65, 67
inter-iteration parallelism, 249
interacting layer, *see* layer interaction
interactive tool, 8
interval graph, 144, 148, 264
intra-iteration parallelism, 248
intractable problem, 41, 53
inverter, *see* CMOS inverter
irredundant prime cover, 198, 223
iteration period, 245, 251, 270
iterative
  data flow, 243
  improvement, 108, 109, 138
  placement, 108

jog insertion, 97

Kernighan-Lin partitioning algorithm, 73, 112
kill, 59

lambda (λ), 84

last-in-first-out order, 62
layer, *see* wiring layer
  interaction, 84
layout, 15, 119
  compaction, *see* compaction
  domain, *see* physical domain
  editor, 16
  language, 85
leaf
  cell, 121
  node, 58
  vertex, 203
least-cost order, 62
Lee's algorithm, 134, 156, 163
left-edge algorithm, 143, 264
length
  of path, *see* path length
  of transistor, *see* transistor length
  of tree, *see* tree length
level, 181
leveling, 173
library, 9, 14, 15, 102, 106, 119, 238, 252
life time, 262
LIFO queue, 32
linear
  order, 29
  programming, 65, 220
list
  adjacency, *see* adjacency list
  scheduling, 259
literal, 196, 222
local
  density, 143
  minimum, 71
  routing, 133, 150
  search, 69, 78, 111, 117, 163
  transformation, 70, 163
logic
  combinational, *see* combinational logic
  gate, 279
  multiple-valued, *see* multiple-valued logic
  signal, 106
  synthesis, 14, 102, 195
    multilevel, *see* multilevel *etc.*
    two-level, *see* two-level *etc.*
  verification, 195, 217
logic-level simulation, *see* gate-level simulation
logistic signal, 106
longest path, 49, 96, 141, 253, 255
  in a DAG, 89, 91, 93, 96, 129, 173
loop, *see* cycle
  folding, 249
loose routing, *see* global routing
low power, 4, 10, 277

macro, 174
  cell, 106, 125
macro-level simulation, 168
Manhattan distance, *see* rectilinear distance
mask
  layout, *see* geometric layout
  programmable, 9
mask-to-symbolic extraction, 86
master, 102, 105
matrix
  adjacency, *see* adjacency matrix
  covering, *see* covering matrix
maximum distance, 90, 93
maze routing, 134, 137, 149, 150, 163
memory, 3
  element, 6, 236
  multiport, *see* multiport memory
  random-access, *see* RAM
  read-only, *see* ROM
metal layer, 283
methodology management, 19
metric
  wire-length, *see* wire-length metric
microcell, 16
microprocessor, 3
min-cut
  partitioning, 108, 125
  placement, 112, 156
minimum
  distance, 84, 93
  feature size, 17, 84
  spanning tree, 37, 60
    rectilinear, 49, 105
  Steiner tree
    rectilinear, 49, 105, 137, 151
minimum-cost satisfiability, 219
minterm, 196, 223
mixed integer linear programming, 69
mixed-level simulation, 169
mixed-mode simulation, 169
mobility, 97
mobility-based scheduling, 253
module, 9

generation, 9, 16
selection, 248
MOS, 282
transistor, 181, 278, 282
move, 70, 71, 78, 107
uphill, *see* uphill moves
multicycle operation, 238
multigraph, 22, 185
multilevel logic synthesis, 195, 200, 222
multiple layers, 137
multiple-valued logic, 170, 174
multiplexer, 237, 262
multiplier, 14, 236
multiport memory, 262
multiprocessor scheduling, 80
mutation, 77

n-channel transistor, 183, 278
NAND gate, 6, 102, 172, 195, 279
negation, 196
negative cycle, 92
neighbor, 69
neighborhood, 69, 73
net, 54, 102, 121, 173, 181
netlist, 54, 125, 133, 151
neural network, 78
nMOS technology, 183, 222
nMOST, 183, 278
node, 22, 181, 239
nondeterministic polynomial time complexity, 45
nonoverlapped scheduling, 264, 270
nonplanar graph, 23
nonreserved-layer model, 139
NOR gate, 195, 279
NP, 45, 46, 48
NP-complete, 45
problem, 41, 45, 46
proof, *see* proving NP-completeness
NP-hard, 49
NPC, 45, 46, 48

O
big ($\mathcal{O}$), *see* big O
object-oriented programming, 105, 206
obstacle, 134, 135
off-set, 196
Omega
big (Ω), *see* big Omega
on-set, 196, 218, 223, 225
one-dimensional compaction, 86, 97
operation chaining, *see* chaining
optimization
combinatorial, *see* combinatorial *etc.*
continuous, *see* continuous *etc.*
discrete, *see* discrete *etc.*
problem, 42
tool, 8
version, 43
order, 28, 29
crossover, 76
of floorplan, *see* floorplan, *etc.*
of variables, *see* variable ordering
ordered binary-decision diagram, 203
out-degree, 23
output
cell, 103
node, 239
terminal, *see* terminal
overconstrained layout, 93
overflow list, 178
overlapped scheduling, 249, 264, 270

P, 45, 48
p-channel transistor, 278
parallel
composition, 281
edge, 22
parallelism, 248
parasitic capacitance, 15, 84, 119, 125, 170, 172, 283, 285
parent, 58, 75, 121
parity function, 198
partial
schedule, 255
solution, 56
partitioning, 16, 73, 101, 108, 112
dynamic, *see* dynamic partitioning
static, *see* static partitioning
pass, 113
path, 23, 24, 56
connection algorithm, 134
directed, *see* directed path
length, 23, 34
longest, *see* longest path
shortest, *see* shortest path
simple, *see* simple path
performance-driven layout, *see* timing-driven layout
permutable terminals, 134

phenotype, 75
physical domain, 6, 98, 119
pipelining, 238
placement, 15, 16, 54, 73, 78, 101, 102, 119, 125, 133, 150
  building-block, *see* building-block placement
  standard-cell, *see* standard-cell placement
planar graph, 23, 49
pMOST, 278
point
  Steiner, *see* Steiner point
polar graph, 122, 129
polynomial
  order, 29
  reduction, 46
  time complexity, 45
polysilicon, 282
population, 75, 78
port, 102, 121, 246, 262
positive cycle, 93
power, 4, 119, 251, 270
  low, *see* low power
  set, 69
precedence relation, 249
precharging, 183
predecessor force, 259
Prim's algorithm, 37, 49, 157
prime implicant, 198, 222, 225
principle of optimality, 62
printed circuit board, 138
priority queue, 178
problem, 41
  decision, *see* decision problem
  instance, *see* instance
  intractable, *see* intractable problem
  NP-complete, *see* NP-complete problem
  optimization, *see* optimization problem
  size, *see* input size
  tractable, *see* tractable problem
  undecidable, *see* undecidable problem
product of sums, 201
programmable logic array, 14, 222
programming, 53
propagation delay, 125, 171
prototyping, 17
proving NP-completeness, 46
pruning, 59, 225
pseudo-cell, 103
pseudo-code, 287
  array, 288
  assignment statement, 290
  block, 290
  compound statement, 290
  conditional statement, 290
  data type
    Boolean, 288
    primitive (`int`/`float`/`char`), 288
    set, 293
    unspecified, 292
  `do` statement, 291
  double quotes, 292
  expression, 289
  `for each` statement, 293
  `for` statement, 291
  function
    call, 291
    definition, 291
    unspecified, 292
  `if` statement, 290
  iteration statements, 290
  `main`, 292
  pointer, 288, 289
  `return` statement, 292
  string, 288
  structure (`struct`), 288
  typography, 287
    mathematical, 293
  `while` statement, 290

quadratic order, 29
queue, 62, 240
  FIFO, *see* FIFO queue
  LIFO, *see* LIFO queue
Quine-McCluskey algorithm, 223

radix sort, 165
RAM, 237
random
  logic, 14
  number, 72, 75
  perturbation, 72, 111
rapid system prototyping, 17
ready list, 260
real time, 17
reasonable encoding, 27
rectilinear
  distance, 49, 105, 156
  polygon, 83
  segment, 49

spanning tree, 49
minimum, *see* minimum spanning tree
Steiner tree, 49, 157
minimum, *see* minimum Steiner tree
reduced ordered binary-decision diagram, 201, 203, 223, 225
reduction, *see also* polynomial reduction, 65
redundant
floorplan, 127
solution, 221
register, 173, 236
assignment, 250
file, 236
register-transfer level, 6, 120
simulation, 168
requirement function, 252
reserved-layer model, 134, 139
resistance, 86, 152, 168, 181, 191, 284
resistor, 15, 168
resource
allocation, *see* allocation
requirement distribution function, 256, 271
type, 252
resource-constrained synthesis, 252, 259
restriction, 201, 208, 210, 213, 228
retiming, 270
rigid
cell, 126
rectangle, 86, 97
rip up and reroute, 149, 150, 163
rise/fall delay model, 172
robust channel router, 146
ROM, 237
root, 58, 203
routing, 15, 49, 55, 101, 119, 133
channel, *see* channel
track, *see* track
row dominance, 224
RS-latch, 102

satisfiability, 42, 48, 78, 219
minimum cost, *see* minimum-cost satisfiability
schedule
cooling, *see* cooling schedule
partial, *see* partial schedule
scheduling, 247, 261, 267
overlapped, *see* overlapped scheduling
range, 254
schematic
editor, 13
entry, 169
sea of gates, 9, 154
search
breadth-first, *see* breadth-first search
depth-first, *see* depth-first search
space, 42, 56
tree, 58, 59, 62
selector node, 241
self-force, 259
selfloop, 22
semicustom design, 8, 15
sequential logic, 6, 235
synthesis, 195
series composition, 281
series-parallel network, 281
set, 219
covering, 220, 223
Shannon expansion, 202, 213
shape function, 125
shortest path, 34, 42, 49, 63, 96
Dijkstra's algorithm, *see* Dijkstra's algorithm
in a DAG, 96, 220
signal, 167, 181
level, 170
modeling, 170, 181
strength, 170
transition, 170
signal-flow graph, 239
Silage, 239
silicon
compiler, 12
oxide, 282, 283
polycrystalline, *see* polysilicon
substrate, 282
simple
cycle, 23, 43, 47
graph, 22
path, 23
simplex algorithm, 67
simulated
annealing, 71, 78, 99, 111, 163, 271
evolution, 78
simulation, 12–14, 17, 102, 167
fault, *see* fault simulation
simulator
kernel, 169, 173

module, 169
sink terminal, 152
sizing, 125, 129
slack variable, 67
slicing
floorplan, 121, 153
tree, 111, 121
software pipelining, 249
solution
checking, 44–46
feasible, *see* feasible solution
space, *see* search space
source, 181, 278
terminal, 152
vertex, 89, 97
space complexity, 27
spanning tree, 37
minimum, *see* minimum spanning tree
rectilinear, *see* rectilinear spanning tree
specification, 217
speed, 4, 251
spiral floorplan, *see* wheel floorplan
squared Euclidean distance, 106
standard
cell, 9, 12, 14, 15, 102, 106, 125, 222
layout, 138, 151
placement, 106, 111
form, 66, 220
state, 14, 173, 235
static
CMOS, 182
partitioning, 184
search tree, 62
switch-level simulation, 190
statistical cooling, *see* simulated annealing
steepest descent, 70
Steiner
point, 49, 154
tree
graph version, 154
rectilinear, *see* rectilinear Steiner tree
step
computational, *see* computational step
stimuli, 169, 174, 179
storage
net, 181
value, 248
strength, *see also* signal strength, 181
stretchable rectangle, 86, 97
strong canonicity, 217
strongly connected
component, 24
vertices, 24
structural
description, 101, 102
domain, 6, 119, 124
subgraph, 22, 69
sublinear order, 29
subroutine, 171
substrate, *see* silicon substrate
subtree, 58
successor force, 259
sum of minterms, 197, 202
supervertex, 265
switch-level
simulation, 15, 168, 180
dynamic, *see* dynamic *etc.*
static, *see* static *etc.*
timing simulation, 191
switchbox routing, 138, 154
symbolic
computation, 207, 229
layout, 17, 85
editor, 85
synchronous
circuit, 168, 173, 236
data flow, 239
synthesis, 12
high-level, *see* high-level synthesis
logic, *see* logic synthesis
tool, 8
system-level simulation, 168

tabu
list, 73
search, 73, 111
task, 262
tautology, 218
technology, 9, 133
CMOS, *see* CMOS technology
file, 85
mapping, 14
temperature, 71
terminal, 102, 103, 121, 133, 151
test pattern, 14
testability, 4
Theta
big (Θ), *see* big Theta
three-sided channel, 154
three-state driver, 237, 262

time
  complexity, 27
    average case, *see* average-case *etc.*
    nondeterministic polynomial, *see* nondeterministic *etc.*
    polynomial, *see* polynomial *etc.*
    worst case, *see* worst-case *etc.*
  frame, 254
  wheel, 178
time-constrained synthesis, 252, 259
timing, 119
  analysis, 14
timing-driven layout, 16, 152
timing-level simulation, 15, 168
token, 240
  firing, *see* firing
tool integration, 19
top-down design, 7, 16, 119
topological layout, *see* symbolic layout
topology, 17, 85
tour, 47
track, 55
tractable problem, 41
transfer, 262
transformation
  algorithmic, *see* algorithmic transformation
  high-level, *see* high-level transformation
transistor, 15, 86, 125, 168
  length, 284
  MOS, *see* MOS transistor
  width, 284
transition, *see* signal transition
transmission line, 152
traveling salesman, 43, 76
  Euclidean, *see* Euclidean traveling salesman
  in graphs, 43, 46, 57, 60, 63, 67
tree, 37, 49, 58, 157
  decomposition, *see* decomposition tree
  length, 37, 49
  search, *see* search tree
  spanning, *see* spanning tree
  Steiner, *see* Steiner tree
tri-state driver, *see* three-state driver
tripartite graph, 103
truth table, 6, 171, 197, 201, 202, 235
Turing machine, 48
two-dimensional compaction, 87, 97
two-level logic synthesis, 195, 222
two-phase clock, 238

unate covering, 222
undecidable problem, 45
undirected graph, 23
unidirectional signal flow, 152, 168, 180, 183, 279
uninitialized (signal value), 170
unique table, 206
unit-delay model, 172, 174
unit-size placement, 54, 58, 61, 65, 69, 70, 74, 75, 101, 107, 111
unknown (signal value), 170, 181
uphill move, 71, 73
user interface, 19

value grouping, 262
variable ordering, 203, 210, 215, 230
V_{dd}, 279
verification, 17, 167, 197
  logic, *see* logic verification
  tool, 8
Verilog, 11, 239
version management, 18
vertex, 22
  connected, *see* connected vertices
vertex-weighted graph, 24
vertical
  composition, 127
  constraint, 140, 143, 146
    graph, 139, 140
VHDL, 11, 168, 170, 199, 239
via, 133, 137, 138
virtual-grid compaction, 99
VLSI, 3
  cost function, 4
voltage, 167
  source, 168
V_{ss}, 279

weakly connected
  component, 24
  vertices, 24
weighted graph, 24
well, 283
wheel
  floorplan, 121
  time, *see* time wheel
width of transistor, *see* transistor width
wire, 86, 237
  feedthrough, *see* feedthrough wire

length, 138
estimation, 105, 119
metric, 105
wiring, *see* routing
layer, 133
word length, 9, 236, 269
worst-case time complexity, 28

Y-chart, 6, 11, 119, 235
yield, 4

zero-delay model, 172, 173
zero-one integer linear programming, 67, 220

Algorithms for VLSI Design Automation